



Bilgisayar ve Bilişim Bilimleri

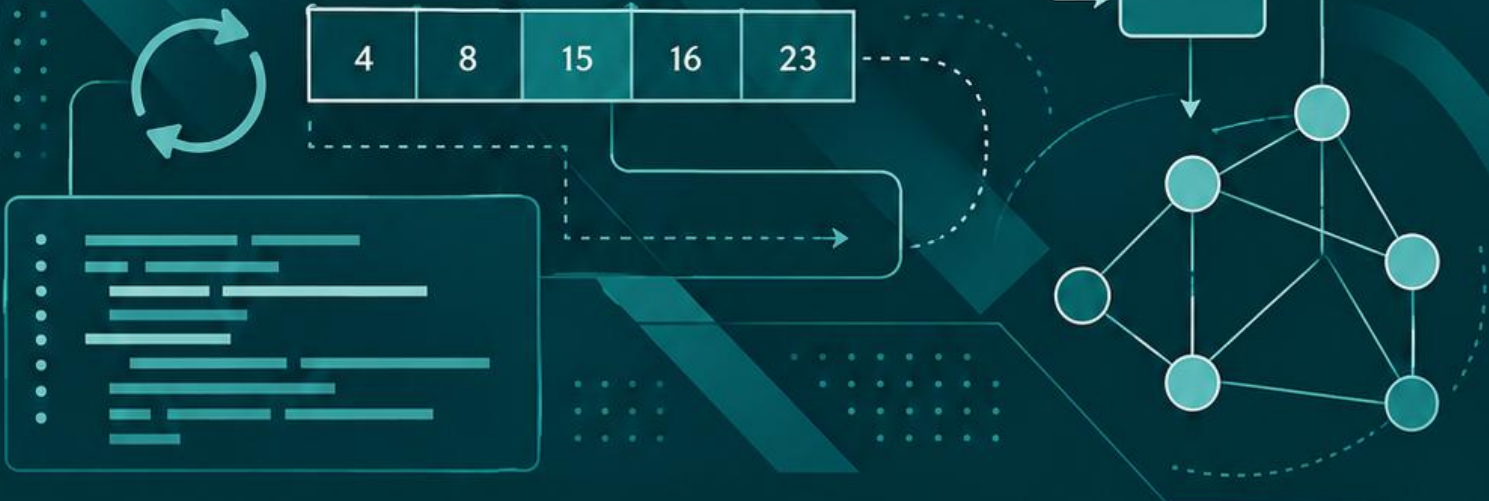
C#

Prosedürel Programlama

Çalışma Kitabı

100 Çözümlü Problem

Nesneye yönelik programlamaya hazırlık



Prof. Dr. Zafer CÖMERT

Önsöz

Programlama öğrenmek, bir dilin sözcüklerini ve yazım kurallarını öğrenmenin ötesinde, bir problemi anlayabilme ve çözümünü açık, tutarlı adımlarla ifade edebilme becerisi kazanmaktır. Bir programın nasıl yazıldığını bilmek kadar, neden o şekilde çalıştığını açıklayabilmek de önemlidir. Bu çalışma kitabını, öğrencilerimizin nesneye yönelik programlamaya geçmeden önce bu düşünme alışkanlığını geliştirmelerine yardımcı olmak amacıyla hazırladım. Kitabın temel yaklaşımı, kavramları yalnız tanımlarıyla sunmak yerine, her kavramı çözülmesi gereken somut bir problem içinde ele almaktır.

Nesneye yönelik programlama; veriyi ve davranışı birlikte modellemeyi, sorumlulukları belirlemeyi ve yazılımı birbirleriyle ilişkili bileşenler halinde düzenlemeyi gerektirir. Bu yaklaşımı anlamak için değişkenlerin nasıl değiştiğini, işlemlerin hangi sırayla yürütüldüğünü, koşulların hangi kararı verdiğini ve döngülerin neden sonlandığını kavramak gerekir. Prosedürel programlama bu temeli kurmak için elverişli bir öğrenme alanı sunar. Burada edinilen beceriler, ilerleyen aşamalarda bir sınıfın sorumluluğunu, bir metodun sözleşmesini ve nesneler arasındaki ilişkinin anlamını daha bilinçli değerlendirmeyi sağlar.

Kitapta C# diliyle çözülen 100 bağımsız problem, kolaydan zora ilerleyen on grup halinde düzenlenmiştir. İlk örneklerde girdi, işlem ve çıktı ilişkisi üzerinde durulmaktadır. Ardından karar yapıları, döngüler, sayı algoritmaları, diziler, sıralama, metin işleme, metotlar ve özyineleme ele alınmaktadır. Dosyalar ve birden çok veri kümesiyle yapılan çalışmalar, öğrenilen işlemleri kalıcı veriler üzerinde bir araya getirmektedir. Son grupta ise zaman aralıkları, paylaşılan kaynaklar, dinamik programlama, çizgeler ve kısıtlı optimizasyon aracılığıyla daha kapsamlı problemler çözülmektedir. Zorluk artışı, yalnız kodun uzamasından değil, birlikte korunması gereken kuralların ve kurulması gereken ilişkilerin çoğalmasından doğmaktadır.

Her problem; tanımı, girdi ve çıktı sözleşmesi, algoritması, tam C# çözümü, örnek çalıştırmaları, sınır durumları, kazanımları ve alıştırmalarıyla birlikte sunulmuştur. Bu düzenin amacı, öğrencinin çözümü bir bütün olarak inceleyebilmesidir. Geçerli bir örnekte doğru sonuç üretmek tek başına yeterli değildir. Boş veri, geçersiz girdi, kapasite sınırı, sayısal taşma veya eksik ilişki karşısında programın nasıl davranacağı da çözümün parçasıdır. Bu nedenle doğrulama, hata bildirme ve kaynakları doğru kullanma konuları temel hesaplamalarla birlikte ele alınmıştır.

Çözümlerin verilmiş olması, öğrencinin kendi çözümünü kurma çabasını gereksiz kılmaz. Önerim, önce problemi okuyarak beklenen sonucu tahmin etmeniz, ardından algoritmayı kendi cümlelerinizle yazmanızdır. C# çözümünü incelemekten önce değişkenlerin anlamlarını ve işlem adımlarını belirleyin. Kodu çalıştırdıktan sonra ara değerleri izleyin; örnekleri değiştirin ve sınır durumlarını deneyin. Bir sonucun neden doğru olduğunu açıklayamadığınızda, yalnız o sonucu elde etmekle yetinmeyin. Alıştırmalar, aynı fikri farklı koşullara uyarlamanız ve öğrendiğiniz kavramın sınırlarını görmeniz için hazırlanmıştır.

Prosedürel yaklaşım, programlama öğreniminin değerli bir parçasıdır; bütün yazılım problemlerinin tek ve değişmez çözüm biçimi değildir. Farklı paradigmaları tanımak, bir problemi hangi açıdan ele alacağımıza ilişkin seçeneklerimizi genişletir. Bu kitapta tekrar eden veri ve davranışları fark etmeniz, sonraki nesneye yönelik derslerde bunları neden ve nasıl birlikte modelleyebileceğinizi tartışmanız beklenmektedir. Böylece bir paradigmadan diğerine geçiş, yeni sözcükleri öğrenmenin yanında, tasarım kararlarının gerekçelerini anlayarak ilerlediğiniz bir öğrenme sürecine dönüşebilir.

Bu çalışma kitabının, programlama yolculuğunuzda düzenli çalışma, dikkatli sorgulama ve çözümünüzü gerekçelendirme alışkanlıklarını güçlendirmesini diliyorum. Her problemi, yalnız çalışan bir program yazma fırsatı olarak değil, düşüncenizi daha açık ve sistematik hale getirecek bir çalışma olarak değerlendirmenizi öneriyorum.

Prof. Dr. Zafer CÖMERT

İçindekiler

Önsöz.....	2
Giriş.....	7
Prosedürel Programlama Nedir?	7
Neden Öğrenilmelidir?	8
Farklı Programlama Paradigmaları Nelerdir?	9
Paradigmalara Hakim Olmak Neden Önemlidir?	10
Bu Kitap Nasıl Kullanılmalıdır?	11
Kaynaklar	12
Bölüm 1: İşlem sırası ve temel programlama	14
001 İki sayının toplamı	14
002 Dikdörtgenin alanı ve çevresi	17
003 Celsius ve Fahrenheit dönüşümü	20
004 İki değişkenin değerini takas etme	23
005 Sayının pozitif negatif veya sıfır olduğunu belirleme	27
006 Bir sayının çift veya tek olduğunu belirleme	30
007 Üç sayıdan en büyüğünü bulma	33
008 1 ile N arasındaki sayıların toplamı	36
009 Bir sayının faktöriyelini hesaplama	39
010 Bir dizinin toplamını ve ortalamasını bulma	42
Bölüm 2: Birleşik koşullar ve karar yapıları	46
011 Sayının kapalı bir aralıkta olup olmadığını belirleme	46
012 Puanı harf notuna dönüştürme	50
013 Atölyeye katılım uygunluğunu belirleme	53
014 Artık yılı belirleme.....	57
015 Dört işlem hesap makinesi	60
016 Üçgenin geçerliliğini ve türünü belirleme	64
017 Kademeli tüketim bedelini hesaplama	67
018 Saate saniye ekleme	70
019 Bir ayın gün sayısını bulma	74
020 Bir tarihin ertesi gününü bulma.....	77
Bölüm 3: Döngüler ve sayı algoritmaları	82
021 Bitiş değeriyle sayıların toplamını ve ortalamasını bulma	82
022 Basamak sayısını ve basamak toplamını bulma.....	87
023 Sayının basamaklarını ters çevirme	90
024 Palindrom sayı olup olmadığını belirleme.....	93
025 Bir tamsayının kuvvetini hesaplama	96
026 Fibonacci dizisinin N numaralı terimini hesaplama	99
027 Bir sayının asal olup olmadığını belirleme	102
028 Öklid algoritmasıyla EBOB hesaplama.....	105
029 EBOB yardımıyla EKOK hesaplama	109

030 Bir sayıyı asal çarpanlarına ayırma	112
Bölüm 4: İç içe döngüler ve kısıtlı arama	116
031 $N \times N$ çarpım tablosu oluşturma	116
032 Yıldızlarla dik üçgen oluşturma.....	120
033 Floyd sayı üçgeni oluşturma.....	123
034 Ortalanmış yıldız piramidi oluşturma	126
035 İç boş dikdörtgen oluşturma	129
036 Pascal üçgeni oluşturma	133
037 Bir aralıktaki asal sayıları bulma.....	137
038 Bir üst sınıra kadar mükemmel sayıları bulma	141
039 Pisagor üçlülerini üretme.....	145
040 Bir tutarı 1, 2 ve 5 birimlik paralarla oluşturma.....	149
Bölüm 5: Diziler ve veri işleme.....	154
041 Dizinin en küçük ve en büyük değerlerini bulma	154
042 Dizide doğrusal arama yapma.....	158
043 Diziyi yerinde ters çevirme	162
044 Diziyi sağa döndürme.....	165
045 Ön ek toplamıyla aralık toplamı hesaplama.....	169
046 Dizinin azalmayan sırada olduğunu denetleme	173
047 Dizide ikili arama ile ilk eşleşmeyi bulma	177
048 Dizide değerlerin frekanslarını hesaplama	181
049 İki dizinin ortak değerlerini bulma	185
050 Dizide tekrarları yerinde kaldırma.....	190
Bölüm 6: Sıralama ve algoritma karşılaştırmaları.....	194
051 Seçmeli sıralama ile diziyi sıralama.....	194
052 Erken bitişli kabarcık sıralama.....	198
053 Eklemeli sıralama ile diziyi sıralama	202
054 Sayarak sıralama ile diziyi sıralama	206
055 İki sıralı diziyi birleştirme	210
056 Sıralı diziyi eleman ekleme	215
057 Dizinin medyanını bulma	219
058 K numaralı farklı küçük değeri bulma	223
059 Dizinin modunu sıralama ile bulma.....	227
060 İki sıralama algoritmasını karşılaştırma	231
Bölüm 7: Metin işleme ve ayrıştırma	238
061 Karakter sınıflandırma ve harf frekansı.....	238
062 Boşlukları normalleştirme ve sözcük sayma	242
063 En uzun sözcüğü ve başlangıç dizinini bulma	246
064 Metinde iki uçtan palindrom denetimi	250
065 Harf frekanslarıyla anagram denetimi.....	254
066 Örtüşen alt metin eşleşmelerini bulma.....	257
067 Ardışık harfleri adetleriyle kodlama	261

068 Metni taşıma denetimiyle elle tamsayıya dönüştürme	265
069 Harf ve adet kodunu doğrulayarak açma	269
070 Yığınla farklı parantez türlerini eşleştirme.....	273
Bölüm 8: Metotlar ve özyineleme	278
071 Metotlarla aralıktaki asal sayıları özetleme	278
072 Metot zinciriyle kesri sadeleştirme	282
073 Tekrar kullanılabilir giriş metoduyla not raporu	285
074 ref ve dizi parametreleriyle yerinde döndürme	289
075 out parametrelerle tek geçişte dizi özeti	293
076 Metotlarla sözcük frekansı raporu oluşturma	297
077 Özyinelemeli ikili aramayla ilk eşleşmeyi bulma	302
078 Özyinelemeli birleştirmeli sıralama	306
079 Geri izlemeyle hedef toplamı alt kümeleri sayma.....	310
080 Metotlarla postfix ifade değerlendirme	314
Bölüm 9: Dosyalar ve ilişkili veri kümeleri	320
081 Dosyanın satır envanteri	320
082 Sensör dosyasında artış serisi	324
083 Stok dosyasında kayıt doğrulama	328
084 Tarihli ölçümlerde eksik günler	333
085 Dosyada ardışık veri paketleri	338
086 Tırnaklı CSV kayıtlarını çözümleme	342
087 Hesap hareketlerini sırayla denetleme	347
088 İki sayım dosyasını uzlaştırma	352
089 Katalog ve satışlardan rapor	357
090 İş ağacında birikimli süre raporu.....	363
Bölüm 10: Birleşik uygulamalar ve optimizasyon	370
091 Ders denemelerinden ağırlıklı başarı.....	370
092 Salon rezervasyonlarında çakışma taraması.....	377
093 Ödünç ve iadelerden gecikme raporu.....	384
094 Depolar arası transfer ve stok korunumu.....	391
095 Reçete ve siparişlerden üretim planı	398
096 Vardiyaları ücret dilimlerine bölerek maliyet hesaplama	405
097 Bütçe kısıtlı proje seçimi.....	413
098 Yönlü ulaşım ağında en düşük maliyetli rota	419
099 Bağımlı işlerde kritik yol ve zaman payı.....	426
100 Uygunluk ve maliyetle en iyi görev ataması.....	433

Şekiller

Şekil 1 Geçici değişkenle takasın işlem sırası.....	24
Şekil 2 Sayının işaretine göre karar verme	28
Şekil 3 Sayaç ve biriktirici ile toplam döngüsü.....	37
Şekil 4 Artık yıl kuralının sıralı bölünebilme kontrolleri	58
Şekil 5 Geçerli bir tarihte gün, ay ve yıl taşması	78
Şekil 6 Bitiş değeri ve en fazla 1000 veri ile giriş döngüsü	83
Şekil 7 Öklid algoritmasında kalan ile küçülen durum	106
Şekil 8 Üst iş ve alt işlerin süreleri	364

Giriş

Bir programı anlamamanın başlangıç noktası, ekranda görünen sonucu o sonuca ulaşan işlemlerle ilişkilendirmektir. Doğru sonuç hangi girdiler için geçerlidir? Değişkenler neyi temsil eder? İşlem sırası değişirse ne olur? Bu kitap, bu soruları C# ile çözülen 100 bağımsız problem üzerinden ele alır. Giriş bölümü, kullanılan yaklaşımın kavramsal çerçevesini kurar ve problemleri etkin biçimde çalışmak için bir yöntem önerir. Bütün teknikleri önceden bilmeniz beklenmez; yeni kavramları ilgili problemle birlikte öğrenip önceki kazanımlara bağlamanız beklenir.

Prosedürel Programlama Nedir?

Prosedürel programlama, bir problemi belirli görevleri yerine getiren işlem birimlerine ayırarak çözme yaklaşımıdır. Bu birimler, gerektiğinde çağrılan prosedürler veya metotlar olarak düzenlenir. Örneğin bir ölçüm raporu hazırlamak; veriyi okuma, doğrulama, hesaplama ve sonucu sunma görevlerine ayrılabilir. Her görev açık bir giriş, işlem ve çıkış ilişkisine sahip olduğunda ana programın akışı daha kolay izlenir. Ayırıştırmanın ölçütü yalnız kod uzunluğu değildir; bir parçanın neyi yaptığı ve hangi kuralları koruduğu da belirleyicidir.

Prosedür sözcüğü genel olarak çağrılabilir bir işlem birimini anlatır. Bazı dil ve kaynaklarda sonuç döndüren birimler fonksiyon, sonuç döndürmeyenler prosedür diye ayrılır. C# bağlamında metot, bir tür içinde tanımlanan çağrılabilir kod birimidir. Kitapta üst düzey ifadelerle yazılmış konsol programları ve static yerel fonksiyonlar kullanılır; bu yerel fonksiyonlar da prosedürel ayırıştırma işlevi görür. Öğretim anlatımında bunların görevlerinden metot olarak söz edilir. Sınıf tasarlamadan alt görevler kurabilmek, C# dilinin nesneye yönelik olanaklarını reddetmek anlamına gelmez.

Değişken, belirli bir türdeki değeri adıyla kullanmamızı sağlar. Tür, hangi değerlerin ve işlemlerin anlamlı olduğunu sınırlar. Bir adet için tamsayı, bir karar için bool, bir metin için string seçilebilir. Değişken adını yalnız kısa olduğu için seçmek yerine anlamına göre seçin: gecerliAdet, saklanan veri sayısını kapasiteden ayırır. Programın belirli bir andaki ilgili değerleri birlikte onun durumunu oluşturur. Durumu okumak, yalnız sonuç değişkenine bakmak değildir; sayaçları, dizileri ve işlem konumunu da birlikte değerlendirmektir.

Sıralı yürütmede bir işlem tamamlanır, ardından sonraki işleme geçilir. Atama, eşitlik bildiren matematiksel bir önerme değil, bir değeri güncelleyen işlemdir. Aşağıdaki bağımsız örnekte ikinci atama, ilk atamanın değiştirdiği durumu kullanır:

```
using System;

int kalan = 12;
int kullanılan = 5;
kalan = kalan - kullanılan;
kullanılan = kalan + 1;
Console.WriteLine($"Kalan: {kalan}, kullanılan: {kullanılan}");
```

Çıktı Kalan: 7, kullanılan: 8 olur. Başlangıç değerleri ile son değerler farklıdır. Satırları yer değiştirip yeniden çalıştırdığınızda aynı sonucu bekleyemezsiniz. İzleme sırasında her atamadan sonraki değerleri ayrı satıra yazmak bu bağımlılığı görünür kılar. Büyük bir uygulamada da aynı düşünce geçerlidir: henüz doğrulanmamış veriyi sonuç üretiminde kullanmak, yanlış bir yürütme sırası oluşturabilir.

Karar yapısı, koşulun doğru veya yanlış olmasına göre yürütülecek işlemi seçer. if, alternatif yolları; switch, belirli seçenekleri düzenlemeye yardım eder. Yineleme ise bir işlem grubunun koşula veya veri kümesine bağlı olarak tekrar yürütülmesidir. for, while ve do / while farklı denetim düzenleri sunar. Döngünün başlangıcını, devam koşulunu ve ilerleme adımını birlikte düşünmek gerekir. Sıfır elemanlı veri, bir kez yürütme ve hiç yürütmeme arasındaki farkı ortaya çıkarır. Sonlanmanın gerekçesi, yalnız kodun bugün çalışmış olmasıyla açıklanamaz.

Parametre, işlem biriminin dışarıdan beklediği bilgiyi tanımlar; çağrıda verilen somut değer argümandır. Dönüş değeri, hesaplanan sonucu çağırana taşır. return çağırıp bitirip denetimi çağırana verir; void normal bir sonuç değeri döndürülmediğini belirtir. Ekran yazdırma ile değer döndürme aynı işlem değildir. Hesabı konsoldan ayırmak, aynı işlemi başka bir programda kullanmayı ve doğrudan sınamayı kolaylaştırır. Örnekte yalnız miktar ve eşik arasındaki ilişki hesaplanır:

```
using System;

Console.WriteLine(EsigiGectiMi(18, 15));
Console.WriteLine(EsigiGectiMi(15, 15));

static bool EsigiGectiMi(int miktar, int esik)
{
    return miktar > esik;
}
```

Çıktılar sırayla True ve False olur; eşitlik, kesin aşma değildir. Bu işlem dış değişken kullanmaz ve veriyi değiştirmez. Her metot için böyle bir sözleşme düşünün: parametrelerin anlamı, kabul edilen değerler, sonuç ve varsa yan etkiler. Bir dizi parametresinde hücre değişikliği çağırının verisini etkileyebilir; parametrenin değer olarak aktarılması, dizinin kopyalandığı anlamına gelmez. Kitabın metot bölümünde bu ayırım, ref ve out kullanımıyla birlikte ayrıntılandırılır.

Neden Öğrenilmelidir?

Prosedürel yaklaşım, problem ayrıştırmayı gözlemlenebilir adımlarla öğretir. Bir rapor isteğinde önce geçerli kaydın ne olduğunu tanımlamak, sonra hesaplamak, en son yazmak gerekir. Bu görevleri ayırdığınızda bir dosya biçimi değişikliği ile bir hesaplama değişikliğinin farklı sorumluluklar olduğunu görebilirsiniz. Her parçanın bağımsız açıklanabilmesi, parçaların doğru bağlandığını kendiliğinden garanti etmez; bu yüzden ana akışın sırası ve veri aktarımı da incelenmelidir. Ayrıştırma, bütün çözümün ilişkilerini görünür tutmalıdır.

Algoritmik düşünme, belirsiz bir amacı uygulanabilir ve sonlanan adımlara dönüştürmektir. “En uygun seçimi bul” ifadesinde uygunluk ve iyilik ölçüsü tanımlanmadan algoritma kurulamaz. Son gruptaki üretim planı belirli bir öncelik politikasını uygular; görev ataması ise tanımlanmış kısıtlar altında toplam maliyeti en aza indirir. Bu iki hedefi ayırabilmek, kullanılan döngüden daha temel bir beceridir. Yerel olarak ucuz görünen bir seçim, diğer seçenekleri kapatarak bütün çözümü pahalı hale getirebilir.

Doğrulama, programın kabul ettiği girdilerin sözleşmeye uyup uymadığını denetlemektir. Bir sayının ayrıştırılabilmesi onun problem için geçerli olduğunu göstermez; eksi bir adet, sayısal açıdan okunabilirken iş kuralına aykırı olabilir. Benzer biçimde bir ürün kodu biçimce geçerli, fakat katalogda bulunmuyor olabilir. Kitabın dosya problemleri biçim, aralık, benzersizlik ve ilişki denetimlerini ayırır. Sonuçların doğrulama tamamlanınca üretilmesi, eksik veriyle doğruymuş gibi görünen rapor yayımlanmasını önlemeye yardımcı olur.

Hata ayıklama, gözlenen davranış ile beklenen davranış arasındaki ilk ayrılığı bulmayı gerektirir. Sonucun yanlış olduğunu bilmek başlangıçtır; hangi adımda hangi varsayımın bozulduğunu göstermek çözümün yolunu açar. Bir toplamın hatalı çıkması, toplama işleminden önce yanlış hücrenin okunmasından kaynaklanabilir. Küçük girdiler, ara değer tabloları ve hata konumları bu nedenle değerlidir. Rastgele satır değiştirmek yerine hatayı tekrarlayan girdiyi saklayın, beklenen sonucu bağımsız hesaplayın ve düzeltmeden sonra aynı girdiyi yeniden deneyin.

Maliyet analizi, veri büyüdüğünde gereken işlemlerin ve belleğin nasıl değiştiğini inceler. n elemanı bir kez taramak ile her elemanı diğer bütün elemanlarla karşılaştırmak aynı büyüme davranışını göstermez. Sonuçları karşılaştırırken hangi işlemin sayıldığını belirtin; değer karşılaştırması, veri hareketi ve süre farklı ölçülerdir. Sınır analizi ayrıca tür kapasitesini, dizi boyutunu, boş veriyi ve

dosya büyüklüğünü kapsar. Sayısal taşma, doğru bir matematiksel formülün bilgisayardaki sınırlı türle yanlış uygulanmasına yol açabilir.

Bu temel, nesneye yönelik programlamaya geçişte de kullanılır. Bir rezervasyonun başlangıcı, bitişi ve çakışma denetimi sürekli birlikte kullanılıyorsa veri ile davranışı aynı sorumlulukta toplama gereksinimi doğar. Sınıf tasarımı bu ilişkinin korunmasına yardımcı olabilir; ancak metotların sözleşmesi ve algoritmanın doğruluğu yine gereklidir. Prosedürel yaklaşım küçük hesaplar ve belirgin işlem hatları için elverişlidir. Uzun ömürlü durum ve çok sayıda ilişki başka düzenlemeler gerektirebilir. Hiçbir paradigmanın her problemde üstün olduğu varsayılmamalıdır.

Farklı Programlama Paradigmaları Nelerdir?

Programlama paradigması, çözümü hangi kavramlarla kurduğumuzu ve kodu nasıl düzenlediğimizi anlatan yaklaşım ailesidir. Emirsel programlama, durumu değiştiren komutlarla işlemlerin nasıl yapılacağını belirtir. Prosedürel programlama, bu emirsel işlemleri çağrılabilir görevler etrafında düzenler. Dolayısıyla bu iki kavram eş düzeyde ve birbirinden kopuk seçenekler değildir. Bir prosedürel program emirsel özellikler taşıırken, bir nesnenin metodunun içi de sıralı atamalar ve döngülerle yazılabilir.

Nesneye yönelik yaklaşım, veriyi ve o veri üzerindeki davranışları nesneler çevresinde düzenler. Sorumluluk, kapsülleme ve nesneler arasındaki ilişkiler tasarımın merkezindedir. Öğrenci numarası, notlar ve başarı hesabı farklı dizilerde tutulduğunda aynı dizinin doğru hücrelerini eşlemek gerekir. Bu verileri uygun bir öğrenci modeli içinde bir araya getirmek tutarlılığın korunmasını kolaylaştırabilir. Bunun bedeli yeni türler ve ilişkiler tasarlamaktır; yalnız her fonksiyonu bir sınıfa taşımak iyi bir model oluşturmaz.

İşlevsel yaklaşım, hesaplamayı işlevlerin uygulanması ve birleştirilmesi üzerinden düşünür; değişmeyen veriyi ve yan etkilerin sınırlandırılmasını öne çıkarır. Saf işlev, aynı girdiler için aynı sonucu verir ve gözlenebilir yan etki oluşturmaz. Önceki eşik örneği bu düşünceye yakındır; dosyaya yazan bir işlem ise yan etkilidir. C# içinde işlev değerleri ve lambda ifadeleri kullanılabilir; F# bu yaklaşımı daha belirgin biçimde destekler. Her lambda ifadesi saf değildir: dışarıdaki değişkeni artıran bir lambda yine durum değiştirir.

Bildirimsel yaklaşım, istenen sonucu veya ilişkiyi ifade etmeye ağırlık verir; bütün yürütme adımlarını tek tek kullanıcı yazmaz. SQL sorgusu, hangi kayıtların seçileceğini belirtir; veritabanı motoru yürütme planını belirler. C# içindeki LINQ, veriler üzerinde sorgu ve dönüşüm ifadeleri sağlar. “Pozitif değerleri say” isteği bir sorguyla ifade edilebilir. Bu açıklık, yürütme maliyetini düşünme gereğini ortadan kaldırmaz; kaynağın türü ve sorgunun ne zaman yürütüldüğü davranışı etkileyebilir.

Mantıksal yaklaşımda olgular ve kurallar tanımlanır, bir sorgunun bu ilişkilerden çıkarılıp çıkarılmadığı araştırılır. Prolog, bu yaklaşımın bilinen örneğidir. Örneğin üst iş ilişkileri ve bir atalık kuralı tanımlanarak iki iş arasında dolaylı bağ olup olmadığı sorulabilir. C# bu çıkarım düzenini yerleşik Prolog sözdizimiyle sunmaz; benzer gereksinim bir kural motoruyla veya açık algoritmayla karşılanabilir. Mantıksal yaklaşım bildirimsel düşünceyle ilişkilidir, fakat onu genel sorgu ifadeleriyle aynı mekanizma olarak değerlendirmemek gerekir.

Olay güdümlü yaklaşım, çalışmanın ne zaman başlatıldığını ve etkileşimlerin nasıl düzenlendiğini ele alır. Düğmeye basılması, zamanlayıcının tetiklenmesi veya bir mesajın gelmesi ilgili işleyiciyi çalıştırabilir. C# arayüzlerinde olaylar ve temsilciler bu düzeni destekler. İşleyicinin içi prosedürel adımlardan oluşabilir; olay kaynağı ise bir nesne olabilir. Olay güdümlü düzen, hesaplama yönteminden farklı bir boyut taşır. İşlem sırası ve paylaşılan durumun yönetimi burada da önemlidir.

Yaklaşım	Temel odak	Örnek dil veya araç	C# ile ilişki
Emirsel	Durum değiştiren adımlar	C, C#	Atama, koşul ve döngü
Prosedürel	Görevlere ayrılmış işlemler	C, Pascal	Metot ve yerel fonksiyon
Nesneye yönelik	Veri ve davranışın sorumluluğu	C#, Java	Sınıflar ve nesneler
İşlevsel	İşlevler ve veri dönüşümü	F#, Haskell	Saf hesaplar, lambda
Bildirimsel	İstenen sonuç ve ilişkiler	SQL	LINQ sorguları
Mantıksal	Olgulardan ve kurallardan çıkarım	Prolog	Kural motoru ile uyarılma
Olay güdümlü	Olaylara verilen yanıt	C# arayüzleri	Olaylar ve işleyiciler

Tablo bir dil sınıflandırma sınavı değildir; odakları karşılaştırmak içindir. Kategorilerin tümü birbirini dışlamaz ve birçok dil birden fazla yaklaşımı destekler. C# ile nesneye yönelik bir uygulamada prosedürel bir hesaplama, işlevsel bir dönüşüm ve olay işleyicisi birlikte bulunabilir. Bir programın paradigmasını yalnız kullandığı dilin adına veya tek bir sözdizimi ögesine bakarak belirlemek, tasarımın önemli özelliklerini gözden kaçırır.

Paradigmalara Hakim Olmak Neden Önemlidir?

Farklı paradigmaları tanımak, problemi uygun sorularla incelemeyi sağlar. İşlemler belirgin bir sırada mı ilerliyor? Verinin geçerliliği bir nesnenin yaşamı boyunca mı korunmalı? Amaç bir koleksiyondan sonuç seçmek mi, ilişkilerden çıkarım yapmak mı? Bu sorular, yaklaşım seçimini alışkanlıktan tasarım kararına dönüştürür. Küçük bir konsol hesabını büyük bir nesne hiyerarşisine dönüştürmek gereksiz olabilir; çok sayıda ilişkiyi tek uzun prosedürde yönetmek de bakım yükünü artırabilir.

Farklı kodları okuyabilmek, aynı isteğin başka biçimlerde ifade edildiğini görmeyi gerektirir. Aşağıdaki bağımsız örnek, pozitif eleman sayısını hem açık döngüyle hem LINQ ile hesaplar:

```
using System;
using System.Linq;

int[] degerler = { -2, 0, 4, 7 };
int adet = 0;
foreach (int deger in degerler)
{
    if (deger > 0) adet++;
}
int sorguAdedi = degerler.Count(deger => deger > 0);
Console.WriteLine($"Döngü: {adet}, sorgu: {sorguAdedi}");
```

İki sonuç da 2'dir. İlk biçim sayaç güncellemesini gösterir; ikinci biçim seçme koşulunu ve sayma isteğini öne çıkarır. Count bir yöntem çağrısıdır ve bu dizide koşulu değerlendirerek sonuç üretir. Kısa yazılması, işlem yapılmadığı veya her kaynakta daha hızlı olduğu anlamına gelmez. Kitapta açık döngüler kullanılması, algoritmanın adımlarını öğrenme amacına hizmet eder. Hazır araçlarla çalışırken de sonuç, yan etki ve maliyet soruları sorulmalıdır.

Bakım ve test açısından amaç, değişikliğin etkisini sınırlayabilmektir. Saf bir hesaplama kolayca farklı girdilerle çağrılabilir; durum taşıyan bir nesne başlangıç ve geçiş kurallarıyla sınanır. Olay işleyicisini veri okuma ve hesaplama ayırmak, arayüz olmadan test yapmayı kolaylaştırır. Ekipte “önkoşul”, “yan etki”, “sorumluluk” ve “durum geçişi” sözcüklerinin ortak anlamda kullanılması anlaşmazlıkları azaltır. Ortak dil, herkesin aynı paradigmayı seçmesini değil, kararının nedenini açıklayabilmesini gerektirir.

Veri ile davranışın ilişkisini kavramak, sonraki öğrenmenin merkezindedir. Paralel dizileri birlikte sıralarken aynı kayda ait hücrelerin ayrılması gerekir. Nesneye yönelik model bu bütünlüğü bir türün sorumluluğu olarak ele alabilir; işlevsel dönüşüm özgün veriyi değiştirmeden yeni sonuç oluşturabilir. Her düzenin sağladığı güvenceler ve taşıdığı maliyetler vardır. Hakimiyet, bütün yaklaşımları aynı anda kullanmak değil, sorunu tanıyıp seçilen yaklaşımın sınırlarını açıklamaktır.

Bu Kitap Nasıl Kullanılmalıdır?

Kitap, onar soruluk on bölümde kolaydan zora ilerler. İlk bölüm işlem sırası, değişken, temel koşul, döngü ve diziyle başlar. İkinci bölüm birleşik karar kurallarını, üçüncü bölüm döngüleri ve sayı algoritmalarını, dördüncü bölüm iç içe döngülerle kısıtlı aramayı geliştirir. Beşinci bölüm dizilerde veri işleme ve aramaya, altıncı bölüm sıralama ve algoritma karşılaştırmalarına ayrılır. Yedinci bölüm metin işleme ve ayrıştırma, sekizinci bölüm metotlar ve özyineleme, dokuzuncu bölüm dosyalar ve ilişkili veri kümeleri üzerinde çalışır. Onuncu bölüm önceki becerileri birleşik uygulamalar ve optimizasyonla tamamlar.

Her problem kendi sözleşmesi ve tam çözümüyle bağımsızdır; önceki problemin programını çalıştırmayı gerektirmez. Bununla birlikte öğrenme sırası ve önkoşullar vardır. Dosyanın başındaki önkoşul kimlikleri, kullanılan kazanımlara geri dönmeniz için yol gösterir. Bir örnekte özyineleme veya dizi sınırı anlaşılmıyorsa ilgili önceki kazanımı yeniden çalışın. Son gruptaki dinamik programlama, çizge ve geri izleme kavramları problem içinde açıklanır; kitabın başında bunların tümüne hakim olmanız beklenmez. Zorluk yalnız satır sayısı ile ölçülmez.

Çözümü açmadan önce problem tanımını okuyun, girdi aralıklarını ve beklenen çıktı biçimini yazın. Küçük bir örneği elle çözün; hangi ara bilgilerin gerektiğini belirleyin. Ardından algoritmayı kendi cümlelerinizle kurun. Kendi çözümünüz kitapla farklı olabilir; karşılaştırmayı metnin aynı olması üzerinden değil, sözleşmeyi karşılaması üzerinden yapın. Örneğin eşit değerlerde ilk konumu koruma şartı varsa herhangi bir doğru değeri bulmak yeterli değildir. Hata davranışı da çözümün parçasıdır.

Adım izleme için girdiyi küçük tutun. Bir tabloda yürütülen satırı, koşul sonucunu, sayaçları ve değişen hücreleri kaydedin. Döngüye girmeden önceki durum ile ilk turun sonunu ayrı gösterin. Özyinelemede çağrı parametrelerini ve dönüş sonuçlarını izleyin. Bir adımın neden doğru olduğunu açıklayamıyorsanız yalnız son çıktıyı ezberlemeyin. Geçerli veri sayısı ile dizi kapasitesi, kimlik ile dizin, boş kayıt ile dosya sonu gibi ayrımları özellikle işaretleyin; son problemlerde bu ayrımlar birlikte kullanılır.

C# kodunu bağımsız bir .NET 10 konsol projesinin Program.cs dosyasında çalıştırın. Her problem için tek çözümü kopyalayın; farklı programların üst düzey ifadelerini aynı dosyada birleştirmeyin. Konsol girdilerini örnekteki sırayla verin. Dosyalı sorularda verilen içerikleri kendi örnek klasörünüzde oluşturun, yolları ve argüman sırasını kontrol edin. `dotnet run --` sonrasındaki argümanlar programa aktarılır. Ondalık nokta, tarih biçimi ve çıktı satırları için ilgili sözleşmeyi izleyin; görünen küçük biçim farkları veri yorumunu değiştirebilir.

Normal örneklerden sonra sınır durumlarını deneyin: boş veri, tek eleman, eşit değerler, alt ve üst sınırlar, geçersiz son kayıt ve bulunamayan ilişki. Beklenen sonucu çalıştırmadan önce belirleyin. Rapor yazan örneklerde girişlerin korunmasını ve mevcut raporun ezilmemesini inceleyin; yeni dosya adları kullanın. Derlenen kodun çalışması, bütün girdiler için doğruluk kanıtı değildir. Testler

belirli hataları ortaya çıkarır; algoritma açıklaması, korunan kurallar ve sonlanma gerekçesi daha genel güven oluşturur.

Son olarak kazanımları kendi cümlelerinizle açıklayın ve alıştırmaları çözün. Bir alıştırma yalnız yeni çıktı istemez; değişikliğin veri düzeni, sözleşme ve maliyet üzerindeki etkisini de düşündürür. Son grupta tekrar birlikte kullanılan veri ve davranışları listeleyin: rezervasyonun aralığı, stokun ilişkileri veya atamanın uygunluk kuralı gibi. Bunları hangi sorumlulukların koruyabileceğini tartışmak nesneye yönelik programlamaya hazırlıktır. Bu kitabın başarısı, 100 çözümü kopyalamakla değil, yeni bir problemi açık adımlara ayırıp çözümünüzü gerekçelendirebilmekle ölçülür.

Kaynaklar

Kavramları derinleştirmek ve dil ayrıntılarını incelemek için aşağıdaki birincil öğretim kaynaklarına başvurabilirsiniz:

- Microsoft Learn: C# metotları - parametreler, argümanlar, dönüş ve değer/referans aktarımı.
- Microsoft Learn: C# ile LINQ - sorgu ifadeleri ve koleksiyon işlemleri.
- Microsoft Learn: F# nedir? - işlevsel yaklaşım ve diğer yaklaşımlarla birliktelik.
- Learn Prolog Now!: Olgular, kurallar ve sorgular - mantıksal yaklaşımın temel yapıları.

Bölüm 1: İşlem sırası ve temel programlama

001 İki sayının toplamı

Düzy: Başlangıç Önkoşullar: Yok

Kavramlar: Girdi ve çıktı, Değişken, Tamsayı, Aritmetik işlem, Tür dönüşümü

Problem tanımı

Kullanıcının girdiği iki tamsayının toplamını hesaplayıp ekrana yazdırın. Örneğin 12 ve 8 için sonuç 20 olmalıdır. Amaç, bir programın **girdi** → **işlem** → **çıktı** akışını görmektir. Kullanıcıdan gelen metin önce sayıya dönüştürülür; ardından toplama işlemi yapılır.

Her girdi, int türünün -2147483648 ile 2147483647 arasındaki değerlerinden biri olmalıdır. İki geçerli int değerinin toplamı bu aralığı aşabileceği için sonuç long değişkende tutulur.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	birinci	Birinci tamsayı
Girdi	ikinci	İkinci tamsayı
Çıktı	toplam	İki sayının toplamı

Girdiler ayrı satırlarda verilir. Sayılar kültürden bağımsız tamsayı biçimiyle okunur ve yazılır; basamak ayırıcı kullanmayın. Geçersiz bir girdi için program Hata: Tamsayı giriniz. yazar ve sona erer.

Algoritma

1. Birinci sayıyı isteyin ve okuyun.
2. Girdi geçerli bir int değilse hata yazıp bitirin.
3. İkinci sayıyı isteyin ve okuyun.
4. Girdi geçerli bir int değilse hata yazıp bitirin.
5. Birinci sayıyı long türüne dönüştürüp ikinci sayıyla toplayın.
6. Toplamı ekrana yazdırın.

Tür dönüşümü işleminden **önce** yapılır. Yalnızca sonucu long değişkene atamak, toplamının int olarak yapılmasını engellemez.

C# çözümü

```
using System;
using System.Globalization;
```

```
Console.Write("Birinci tamsayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int birinci))
{
    Console.WriteLine("Hata: Tamsayı giriniz.");
    return;
}
```

```
Console.Write("İkinci tamsayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ikinci))
{
    Console.WriteLine("Hata: Tamsayı giriniz.");
    return;
}
```

```
long toplam = (long)birinci + ikinci;
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
```

Kod, bir C# konsol projesinin Program.cs dosyasına tek başına yerleştirilebilir. TryParse, dönüşümün başarılı olup olmadığını bildirir; out ile dönüştürülen değeri verir. return, bu örnekte programı sonlandırır. Kendi sınıfınızı veya nesnenizi tanımlamanız gerekmez.

Örnek çalıştırmalar

Tablo, giriş istemlerinden sonra yazılan sonuç satırını gösterir. Girdi sütunundaki virgöl, iki ayrı satırı ayırır.

Girdi (sırasıyla)	Sonuç satırı
12, 8	Toplam: 20
-5, 3	Toplam: -2
0, 0	Toplam: 0
2147483647, 2147483647	Toplam: 4294967294
abc	Hata: Tamsayı giriniz.

Sınır durumları

- Negatif değerler ve sıfır geçerli girdilerdir.
- 2147483648, int aralığının dışında olduğu için reddedilir.
- 2.5 bir tamsayı olmadığı için reddedilir.
- Boş satır veya giriş akışının bitmesi hata mesajıyla sonlanır.
- İki int değerinin olası bütün toplamaları long aralığında kalır.

Kazanımlar

- Metin girdisiyle sayısal değer arasındaki farkı açıklamak.
- Değişken tanımlamak ve bir aritmetik ifadeyi değerlendirmek.
- Girdiyi doğrulamak ve hata durumunda işlemi durdurmak.
- İşlem türünün taşma riskiyle ilişkisini görmek.

Alıştırmalar

1. Aynı iki sayının farkını da yazdırın; çıkarma işleminden önce tür dönüşümünü uygulayın.
2. Üç tamsayının toplamını hesaplayacak biçimde programı genişletin.
3. `long toplam = birinci + ikinci;` yazıldığında büyük girdiler için oluşabilecek sorunu açıklayın.

002 Dikdörtgenin alanı ve çevresi

Düzey: Başlangıç Önkoşullar: 001

Kavramlar: Ondalıklı sayı, Çarpma, İşlem önceliği, Girdi doğrulama, Çıktı biçimlendirme

Problem tanımı

Kısa ve uzun kenarı verilen bir dikdörtgenin alanını ve çevresini hesaplayın. İki kenar aynı ölçü biriminde girilmelidir. Kenarlar metre ise alan metrekare, çevre metre olarak yorumlanır.

Alan iki kenarın çarpımıdır. Çevre ise kenarların toplamının iki katıdır. Bu alıştırmada kenarların her biri sıfırdan büyük ve en fazla 1000000000 olabilir. Ondalıklı kenarlar decimal türünde saklanır. Sonuçlar iki ondalık basamakla gösterilir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	birinciKenar	$0 < \text{kenar} \leq 1000000000$
Girdi	ikinciKenar	$0 < \text{kenar} \leq 1000000000$
Çıktı	alan	$\text{birinciKenar} * \text{ikinciKenar}$
Çıktı	cevre	$2 * (\text{birinciKenar} + \text{ikinciKenar})$

Ondalık ayırıcı olarak nokta kullanın: 3.5. Program virgüllü girdiyi kabul etmez. Geçersiz biçim veya aralık dışındaki kenar için hata yazılır ve program biter.

Algoritma

1. Birinci kenarı okuyun ve ondalıklı sayıya dönüştürün.
2. Dönüşüm başarısızsa veya kenar izin verilen aralıkta değilse hata yazıp bitirin.
3. İkinci kenarı okuyun ve aynı denetimi uygulayın.
4. İki kenarı çarparak alanı hesaplayın.
5. Kenarları toplayıp sonucu ikiyle çarparak çevreyi hesaplayın.
6. Alanı ve çevreyi iki ondalık basamakla yazdırın.

Çevre formülündeki parantez, önce kenarların toplanacağını belirtir. $2 * \text{birinciKenar} + \text{ikinciKenar}$ farklı bir sonuç verir.

C# çözümü

```
using System;
using System.Globalization;

NumberStyles bicim = NumberStyles.AllowLeadingSign
    | NumberStyles.AllowDecimalPoint;
CultureInfo kultur = CultureInfo.InvariantCulture;

Console.Write("Birinci kenar (ondalık için nokta): ");
if (!decimal.TryParse(Console.ReadLine(), bicim, kultur,
    out decimal birinciKenar)
    || birinciKenar <= 0 || birinciKenar > 1000000000m)
{
    Console.WriteLine("Hata: Kenar 0'dan büyük ve en fazla 1000000000 olmalıdır.");
    return;
}

Console.Write("İkinci kenar (ondalık için nokta): ");
if (!decimal.TryParse(Console.ReadLine(), bicim, kultur,
    out decimal ikinciKenar)
    || ikinciKenar <= 0 || ikinciKenar > 1000000000m)
{
    Console.WriteLine("Hata: Kenar 0'dan büyük ve en fazla 1000000000 olmalıdır.");
    return;
}

decimal alan = birinciKenar * ikinciKenar;
decimal cevre = 2m * (birinciKenar + ikinciKenar);

Console.WriteLine($"Alan: {alan.ToString("F2", kultur)}");
Console.WriteLine($"Çevre: {cevre.ToString("F2", kultur)}");
```

Kodu bir konsol projesinin Program.cs dosyasında çalıştırın. m eki, sabitin decimal olduğunu belirtir. InvariantCulture, bilgisayarın bölge ayarından bağımsız olarak noktayı kullanır. F2, sonucu iki ondalık basamakla gösterir; hesaplanan değişkenleri değiştirmez.

Örnek çalıştırmalar

Tablo giriş istemlerini içermez; sonuç satırları sırayla gösterilir.

Girdi (ayrı satırlarda)	Sonuç satırları
5, 3	Alan: 15.00 Çevre: 16.00
2.5, 4	Alan: 10.00 Çevre: 13.00
6, 6	Alan: 36.00 Çevre: 24.00
0	Hata: Kenar 0'dan büyük ve en fazla 1000000000 olmalıdır.

003 Celsius ve Fahrenheit dönüşümü

Düzey: Başlangıç Önkoşullar: 001, 002

Kavramlar: Formül uygulama, Ondalık bölme, Sabit, Negatif değer, Çıktı biçimlendirme

Problem tanımı

Celsius ölçeğinde girilen sıcaklığı Fahrenheit ölçeğine dönüştürün. Dönüşüm formülü $F = C * 9 / 5 + 32$ şeklindedir. Örneğin 0 Celsius, 32 Fahrenheit; 100 Celsius, 212 Fahrenheit olur.

Bu alıştırma tek yönlü dönüşüm yapar: kullanıcı Celsius değerini girer. Hesaplama kapsamı -1000000 ile 1000000 arasındaki değerlerdir. Değer decimal türünde saklanır. Fahrenheit sonucu iki ondalık basamakla gösterilir. Buradaki aralık, uygulamanın kabul ettiği sayısal aralıktır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	celsius	-1000000 ile 1000000 arasında ondalıklı sayı
Çıktı	fahrenheit	$celsius * 9 / 5 + 32$

Ondalık ayırıcı nokta olmalıdır. Örneğin 36.5 geçerlidir. Hatalı biçimde veya aralık dışında girdi verilirse program hata mesajıyla sona erer.

Algoritma

1. Celsius sıcaklığını isteyin ve okuyun.
2. Girdiyi ondalıklı sayıya dönüştürmeyi deneyin.
3. Dönüşüm başarısızsa veya değer kabul edilen aralıkta değilse hata yazıp bitirin.
4. Celsius değerini 9 ile çarpın ve 5'e bölün.
5. Elde edilen değere 32 ekleyin.
6. Fahrenheit sonucunu iki ondalık basamakla yazdırın.

Çarpma ve bölme, toplamadan önce yapılır. C# içinde $9 / 5$ ifadesi iki tamsayının bölünmesidir ve sonuç 1 olur. Formülde $9m / 5m$ kullanmak, ondalıklı hesaplama niyetini açıkça gösterir.

C# çözümü

```
using System;
using System.Globalization;

NumberStyles bicim = NumberStyles.AllowLeadingSign
    | NumberStyles.AllowDecimalPoint;
CultureInfo kultur = CultureInfo.InvariantCulture;

Console.WriteLine("Celsius sıcaklığı (ondalık için nokta): ");
if (!decimal.TryParse(Console.ReadLine(), bicim, kultur,
    out decimal celsius)
    || celsius < -1000000m || celsius > 1000000m)
{
    Console.WriteLine("Hata: -1000000 ile 1000000 arasında sayı giriniz.");
    return;
}

decimal fahrenheit = celsius * 9m / 5m + 32m;
Console.WriteLine($"Fahrenheit: {fahrenheit.ToString("F2", kultur)}");
```

Bu kod tek başına bir konsol projesinin Program.cs dosyasında çalışır. TryParse, sayı biçimini denetler; karşılaştırmalar değer aralığını denetler. ||, bu hata koşullarından herhangi birinin gerçekleşmesini ifade eder. InvariantCulture, girdi ve çıktıdaki ondalık ayracını bilgisayarın bölge ayarlarından bağımsız kılar.

Örnek çalıştırmalar

Tablo, giriş isteminden sonra yazılan sonuç satırını gösterir.

Celsius girdisi	Sonuç satırı
0	Fahrenheit: 32.00
100	Fahrenheit: 212.00
-40	Fahrenheit: -40.00
36.5	Fahrenheit: 97.70
1000001	Hata: -1000000 ile 1000000 arasında sayı giriniz.
36,5	Hata: -1000000 ile 1000000 arasında sayı giriniz.

Sınır durumları

- Sıfır ve negatif değerler geçerli girdilerdir.
- Alt ve üst sınırlar aralığa dahildir.
- Boş girdi ve sayı olmayan metinler reddedilir.
- Kabul edilen aralıktaki hesaplamalar decimal kapasitesini aşmaz.
- Çok fazla ondalık basamak içeren girdiler decimal hassasiyetine göre yuvarlanabilir; F2 ayrıca yalnızca iki basamak gösterir.

Kazanımlar

- Sözel bir dönüşüm kuralını aritmetik ifadeye çevirmek.
- Tamsayı bölmesiyle ondalıklı bölme arasındaki farkı görmek.
- Negatif girdileri ve bilinen referans değerlerini test etmek.
- Hesaplama hassasiyetiyle ekrandaki basamak sayısını ayırt etmek.

Alıştırmalar

1. Ters dönüşümü uygulayın: $C = (F - 32) * 5 / 9$.
2. -40 değerinin iki ölçekte de aynı olmasını formülden gösterin.
3. decimal oran = $9 / 5$; ile decimal oran = $9m / 5m$; ifadelerinin sonuçlarını karşılaştırın.

004 İki değişkenin değerini takas etme

Düzey: Başlangıç Önkoşullar: 001

Kavramlar: Atama, Geçici değişken, İşlem sırası, Değişken izleme

Problem tanımı

İki değişkende saklanan tamsayıların yerini değiştirin. Başlangıçta `birinci = 5` ve `ikinci = 9` ise işlem sonunda `birinci = 9`, `ikinci = 5` olmalıdır. Program, değerleri takastan önce ve sonra gösterir.

Bu problem, atama işlemlerinin sırayla yürütüldüğünü öğretir. Bir değişkene yeni değer atanınca eski değeri kaybolur. Eski değeri ikinci değişkene aktarabilmek için geçici bir değişken kullanılır. Her girdi `int` türünün -2147483648 ile 2147483647 aralığında olmalıdır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	<code>birinci</code>	Birinci tamsayı
Girdi	<code>ikinci</code>	İkinci tamsayı
Ara değer	<code>gecici</code>	Birinci değişkenin eski değeri
Çıktı	Önce / sonra	İki değişkenin takas öncesi ve sonrası değerleri

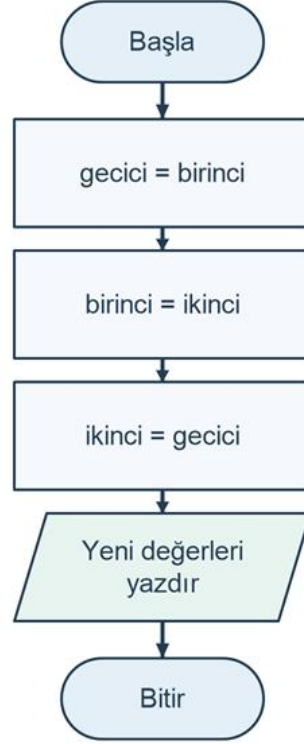
Girdiler iki ayrı satırda verilir. Sayılar kültürden bağımsız tamsayı biçimiyle okunur ve yazılır; basamak ayırıcı kullanmayın. Sayıya dönüştürülemeyen bir girdi için hata yazılır ve program sona erer.

Algoritma

1. Birinci tamsayıyı okuyun; geçersizse hata yazıp bitirin.
2. İkinci tamsayıyı okuyun; geçersizse hata yazıp bitirin.
3. İki değişkenin başlangıç değerlerini yazdırın.
4. Birinci değişkenin değerini `gecici` değişkenine kopyalayın.
5. İkinci değişkenin değerini birinci değişkene atayın.
6. Geçici değişkendeki eski birinci değeri ikinci değişkene atayın.
7. İki değişkenin yeni değerlerini yazdırın.

`birinci = ikinci`; ifadesi, sağdaki değeri soldaki değişkene kopyalar. Matematikteki bir eşitlik iddiasından farklı olarak programın durumunu değiştirir.

Şema, geçerli girdiler okunduktan sonraki atama sırasını gösterir.



Şekil 1 Geçici değişkenle takasın işlem sırası

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Birinci tamsayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int birinci))
{
    Console.WriteLine("Hata: Tamsayı giriniz.");
    return;
}

Console.Write("İkinci tamsayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ikinci))
{
    Console.WriteLine("Hata: Tamsayı giriniz.");
    return;
}

Console.WriteLine("Önce: birinci = " + birinci.ToString(CultureInfo.InvariantCulture) +
    ", ikinci = " + ikinci.ToString(CultureInfo.InvariantCulture));

int gecici = birinci;
birinci = ikinci;
ikinci = gecici;

Console.WriteLine("Sonra: birinci = " + birinci.ToString(CultureInfo.InvariantCulture) +
    ", ikinci = " + ikinci.ToString(CultureInfo.InvariantCulture));
```

Kodu bir konsol projesinin Program.cs dosyasında çalıştırın. Takas yalnızca değer kopyalar; toplama veya çıkarma yapmaz. Bu nedenle geçerli int girdileriyle aritmetik taşma oluşmaz. gecici, birinci değerini takas boyunca korunmasını sağlar.

Örnek çalıştırmalar

Tablo giriş istemlerini içermez; sonuç satırları sırayla gösterilir.

Girdi (ayrı satırlarda)	Sonuç satırları
5, 9	Önce: birinci = 5, ikinci = 9 Sonra: birinci = 9, ikinci = 5
-3, 0	Önce: birinci = -3, ikinci = 0 Sonra: birinci = 0, ikinci = -3
7, 7	Önce: birinci = 7, ikinci = 7 Sonra: birinci = 7, ikinci = 7
abc	Hata: Tamsayı giriniz.

Sınır durumları

- İki değer aynıysa ekrandaki değerler değişmez; algoritma yine geçerlidir.
- Negatif değerler, sıfır ve int sınırları takas edilebilir.
- Aralık dışı sayı, ondalıklı sayı veya boş satır reddedilir.
- Geçici değer alınmadan `birinci = ikinci; ikinci = birinci;` yazılırsa iki değişken de eski ikinci değeri taşır.

Kazanımlar

- Atama işleminin yönünü ve etkisini açıklamak.
- İşlem sırasının programın sonucunu belirlediğini görmek.
- Kaybolacak bir değeri geçici değişkende korumak.
- Programı adım adım izleyerek değişken değerlerini takip etmek.

Alıştırmalar

1. Girdiler 4 ve 11 iken her atamadan sonra üç değişkenin değerini bir tabloya yazın.
2. Takas bloğunu art arda iki kez çalıştırın; sonucu açıklayın.
3. Üç değişkenin değerlerini döndürün: eski birinci ikinciye, eski ikinci üçüncüye, eski üçüncü birinciye geçsin.

005 Sayının pozitif negatif veya sıfır olduğunu belirleme

Düzy: Bařlangıç Önkořullar: 001, 004

Kavramlar: Karřılařtırma, Kořul, if / else if / else, Birbirini dıřlayan durumlar

Problem tanımı

Kullanıcının girdiđi tamsayının pozitif, negatif veya sıfır olduđunu belirleyin. Sıfırdan büyük sayılar pozitif, sıfırdan küçük sayılar negatif olarak sınıflandırılır. Sıfır için ayrı bir sonuç yazdırılır.

Önceki örneklerde doğrulama amacıyla görülen kořullar, burada problemin asıl çözümlünü oluřturur. Geçerli bir tamsayı için üç durumdan yalnızca biri gerçekteřir. Girdi, int türünün -2147483648 ile 2147483647 aralıđında olmalıdır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	İřareti belirlenecek tamsayı
Çıktı	Sınıflandırma	Sonuç: Pozitif, Sonuç: Negatif veya Sonuç: Sıfır

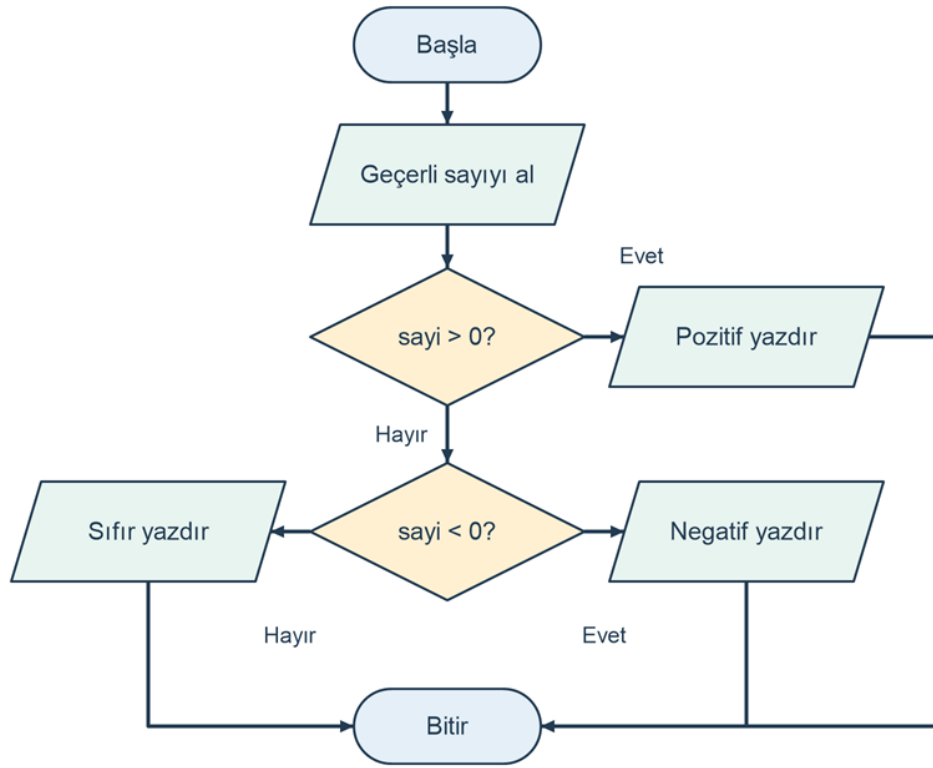
Sayı költürden bağımsız tamsayı biçimiyle okunur; basamak ayırıcı kullanmayın. Geçersiz girdi için Hata: Tamsayı giriniz. yazdırılır. Böyle bir durumda sayı sınıflandırılmaz; program sona erer.

Algoritma

1. Tamsayıyı isteyin ve okuyun.
2. Girdi geçerli bir int deđilse hata yazıp bitirin.
3. Sayı sıfırdan büyükse Sonuç: Pozitif yazdırın.
4. Deđilse, sayı sıfırdan küçükse Sonuç: Negatif yazdırın.
5. Bu kořulların ikisi de sađlanmıyorsa Sonuç: Sıfır yazdırın.
6. Programı bitirin.

Bir tamsayı ne sıfırdan büyük ne de sıfırdan küçükse sıfırdır. Bu nedenle son dalda ayrıca sayi == 0 karřılařtırması yapmaya gerek yoktur.

řema, geçerli bir tamsayı için birbirini dıřlayan üç sonucu gösterir.



Şekil 2 Sayının işaretine göre karar verme

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Tamsayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi))
{
    Console.WriteLine("Hata: Tamsayı giriniz.");
    return;
}

if (sayi > 0)
{
    Console.WriteLine("Sonuç: Pozitif");
}
else if (sayi < 0)
{
    Console.WriteLine("Sonuç: Negatif");
}
else
{
    Console.WriteLine("Sonuç: Sıfır");
}
```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. > ve < işlemleri karşılaştırma yapar. Sonuçları doğru veya yanlış değeridir. if / else if / else zinciri, ilk doğru koşulun dalını çalıştırır; sonraki dalları atlar. Böylece geçerli girdi için yalnızca bir sonuç satırı yazılır.

Örnek çalıştırmalar

Tablo, giriş isteminden sonra yazılan sonuç satırını gösterir.

Girdi	Sonuç satırı
8	Sonuç: Pozitif
-8	Sonuç: Negatif
0	Sonuç: Sıfır
2147483647	Sonuç: Pozitif
-2147483648	Sonuç: Negatif
2.5	Hata: Tamsayı giriniz.

Sınır durumları

- Sıfır, pozitif veya negatif kategorisine eklenmez; ayrı dalda işlenir.
- -0 tamsayı olarak sıfırdır ve Sonuç: Sıfır üretir.
- int alt ve üst sınırları doğrudan karşılaştırılabilir; aritmetik işlem yapılmadığı için taşma oluşmaz.
- Boş satır, metin, ondalıklı sayı ve aralık dışı tamsayı reddedilir.
- İlk koşul $sayi \geq 0$ olsaydı sıfır yanlışlıkla pozitif olarak sınıflandırılırdı.

Kazanımlar

- Karşılaştırma işlemleriyle koşul oluşturmak.
- Birbirini dışlayan durumları tek bir koşul zincirinde düzenlemek.
- Sınırdaki değeri ayrıca test etmek.
- Girdi doğrulama ile problemin karar adımını birbirinden ayırt etmek.

Alıştırmalar

1. Negatif koşulu önce denetleyecek biçimde dalları yeniden düzenleyin.
2. Pozitif sayılarda Sıfırdan büyük., negatif sayılarda Sıfırdan küçük. açıklamalarını da yazdırın.
3. Test kümesine neden hem 0 hem de 1 ve -1 eklenmesi gerektiğini açıklayın.

006 Bir sayının çift veya tek olduğunu belirleme

Düzey: Başlangıç Önkoşullar: 005

Kavramlar: Tamsayı, Kalan operatörü, Koşul, Girdi doğrulama

Problem tanımı

Klavyeden girilen bir tamsayının çift mi, tek mi olduğunu belirleyin. İkiye tam bölünen tamsayılar çifttir; diğer tamsayılar tektir. Bu tanım sıfır ve negatif sayılar için de geçerlidir. Örneğin -3 tektir; 0 ve -8 çifttir.

Bu problemde sonuç, sayının büyüklüğüne değil, ikiye bölümünden kalan değere bağlıdır. C# dilindeki % operatörü bölümün kalanını verir. Sayı için `sayi % 2 == 0` ifadesi doğruysa çift, yanlışsa tek sonucu üretilir. Negatif tek sayılarda kalan negatif olabilir; bu nedenle tekliği yalnızca `kalan == 1` koşuluyla sınamayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	int aralığında bir tamsayı: -2147483648 ile 2147483647.
Çıktı	Sonuç	Sayı çifttir. veya Sayı tektir.

Program Bir tamsayı girin: istemini gösterir. Bir sayı yazıp Enter tuşuna basın. Ondalık sayı, boş giriş veya aralık dışındaki değer geçersizdir. Sayı yazımında kültürden bağımsız tamsayı biçimi kullanılır; basamak ayırıcı eklemeyin.

Algoritma

1. Kullanıcıdan bir tamsayı isteyin.
2. Girdiyi tamsayıya dönüştürmeyi deneyin.
3. Dönüştürme başarısızsa hata mesajını gösterip programı bitirin.
4. Sayının ikiye bölümünden kalanı hesaplayın.
5. Kalan sıfırsa çift, değilse tek olduğunu yazdırın.

Karar yapısının yalnızca iki sonucu vardır. Koşulun doğru olduğu durumda `if` gövdesi, yanlış olduğu durumda `else` gövdesi çalışır. Her geçerli giriş için tam bir sonuç üretilir.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Bir tamsayı girin: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi))
{
    Console.WriteLine("Hata: int aralığında bir tamsayı girin.");
    return;
}

if (sayi % 2 == 0)
{
    Console.WriteLine("Sayı çifttir.");
}
else
{
    Console.WriteLine("Sayı tektir.");
}
```

TryParse, geçersiz bir girişte programın dönüşüm hatasıyla kesilmesini önler. Başarısız dönüşümden sonra kullanılan return, karar adımına geçilmeden programı sonlandırır.

Örnek çalıştırmalar

Aşağıdaki tabloda istem metni yerine yalnızca giriş ve sonuç satırı gösterilir.

Girdi	Sonuç
12	Sayı çifttir.
7	Sayı tektir.
0	Sayı çifttir.
-3	Sayı tektir.
-2147483648	Sayı çifttir.

Sınır durumları

- Sıfırın ikiye bölümünden kalan sıfırdır; sıfır çifttir.
- Negatif sayılar da aynı koşulla değerlendirilir.
- 2147483648 ve abc için hata mesajı yazılır; çiftlik testi yapılmaz.
- Kalan işlemi geçerli int girdileri için ek bir aralık kısıtı gerektirmez.

Kazanımlar

- Bölme işlemi ile kalan işleminin farklı amaçlarını açıklama.
- Bir matematiksel tanımlı mantıksal koşula dönüştürme.
- if ve else ile birbirini dışlayan sonuçları üretme.
- Negatif sayılar ve sıfırla sınır durumlarını denetleme.

Alıştırmalar

1. Aynı yaklaşımı kullanarak sayının üçe tam bölünüp bölünmediğini gösterin.
2. Sayının hem ikiye hem üçe tam bölünüp bölünmediğini belirleyin.
3. `kalan == 1` koşulunun neden -3 için yanlış sonuç verdiğini açıklayın.

007 Üç sayıdan en büyüğünü bulma

Düzey: Başlangıç Önkoşullar: 005

Kavramlar: Karşılaştırma, Koşul, Değişken güncelleme, Girdi doğrulama

Problem tanımı

Kullanıcıdan üç tamsayı alın ve bunların en büyüğünü ekrana yazdırın. Sayılar pozitif, negatif veya sıfır olabilir. İki ya da üç sayının eşit olduğu durumlarda da tek bir en büyük değer üretilmelidir.

Çözümde önce ilk sayı geçici en büyük değer kabul edilir. İkinci sayı bu değerden büyükse geçici değer güncellenir. Aynı işlem üçüncü sayı için tekrar edilir. Böylece her karşılaştırmadan sonra değişken, o ana kadar incelenmiş sayıların en büyüğünü tutar. Başlangıç değerini ilk sayıdan almak, tüm girdilerin negatif olduğu durumlarda da doğru sonuç verir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	birinci, ikinci, üçüncü	Her biri int aralığında üç tamsayı.
Çıktı	enBuyuk	Girilen üç değerden en büyüğü.

Program sırasıyla Birinci sayı: , İkinci sayı: ve Üçüncü sayı: istemlerini gösterir. Her değeri ayrı satırda girin. Sayıların izin verilen aralığı -2147483648 ile 2147483647'dir. Basamak ayırıcı kullanmayın; ilk geçersiz girişte hata mesajı gösterilir ve program biter.

Algoritma

1. Birinci sayıyı okuyun ve doğrulayın.
2. İkinci sayıyı okuyun ve doğrulayın.
3. Üçüncü sayıyı okuyun ve doğrulayın.
4. enBuyuk değişkenine birinci sayıyı atayın.
5. İkinci sayı enBuyuk değerinden büyükse değişkeni ikinci sayıyla güncelleyin.
6. Üçüncü sayı enBuyuk değerinden büyükse değişkeni üçüncü sayıyla güncelleyin.
7. enBuyuk değerini yazdırın.

İkinci karşılaştırma, ilk karşılaştırmada güncellenmiş olabilen değerle yapılır. Burada iki bağımsız if kullanılır; üçüncü sayının değerlendirilmesi ikinci sayının sonucuna bağlı olarak atlanmaz.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Birinci sayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int birinci))
{
    Console.WriteLine("Hata: int aralığında bir tamsayı girin.");
    return;
}

Console.Write("İkinci sayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ikinci))
{
    Console.WriteLine("Hata: int aralığında bir tamsayı girin.");
    return;
}

Console.Write("Üçüncü sayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ucuncu))
{
    Console.WriteLine("Hata: int aralığında bir tamsayı girin.");
    return;
}

int enBuyuk = birinci;
if (ikinci > enBuyuk)
{
    enBuyuk = ikinci;
}
if (ucuncu > enBuyuk)
{
    enBuyuk = ucuncu;
}

Console.WriteLine("En büyük sayı: " +
    enBuyuk.ToString(CultureInfo.InvariantCulture));
```

Örnek çalıştırmalar

Girdi sütunu, üç ayrı satırda girilen sayıları sırasıyla gösterir. Sonuç sütununda istemler yer almaz.

Girdiler	Sonuç
4, 9, 2	En büyük sayı: 9
12, 3, 8	En büyük sayı: 12
-8, -2, -5	En büyük sayı: -2
7, 7, 1	En büyük sayı: 7
0, 0, 0	En büyük sayı: 0

Sınır durumları

- Eşit değerler için güncelleme gerekmesi de sonuç doğrudur.
- Tüm sayılar negatifse en büyük sayı sıfır kabul edilmez.
- Toplama veya çıkarma yapılmadığından karşılaştırmalar int sınırlarında taşma üretmez.
- Geçersiz girişte eksik verilerle karşılaştırma yapılmaz.

Kazanımlar

- Bir sonucun önceki adımlarda nasıl korunduğunu izleme.
- Karşılaştırma operatörleriyle değişken güncelleme.
- Başlangıç değerinin algoritmanın doğruluğuna etkisini açıklama.
- Eşitlik ve negatif değerler için örnekler hazırlama.

Alıştırmalar

1. Üç sayıdan en küçüğünü bulan çözümü yazın.
2. Dördüncü bir sayı ekleyerek aynı yaklaşımı genişletin.
3. enBuyuk başlangıçta sıfır olsaydı hangi girdilerde hata oluşurdu?

008 1 ile N arasındaki sayıların toplamı

Düzey: Temel Önkoşullar: 005, 007

Kavramlar: For döngüsü, Sayaç, Toplam biriktirme, Girdi doğrulama

Problem tanımı

Kullanıcıdan sıfır veya pozitif bir N tamsayısı alın. 1 ile N arasındaki bütün tamsayıları, iki uç dahil olacak şekilde toplayın. Bu çalışmada amaç, toplamı hazır bir formülle bulmak yerine sayıları bir döngü içinde tek tek işlemektir.

Örneğin $N = 4$ için sonuç $1 + 2 + 3 + 4 = 10$ olur. $N = 0$ için toplanacak sayı yoktur ve sonuç sıfırdır. Biriktirici değişken, döngü boyunca işlenmiş sayıların toplamını saklar; sayaç ise sıradaki sayıyı temsil eder.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	0 ile 1000000 arasında bir tamsayı.
Çıktı	toplam	1'den n değerine kadar tamsayıların toplamı; long türünde.

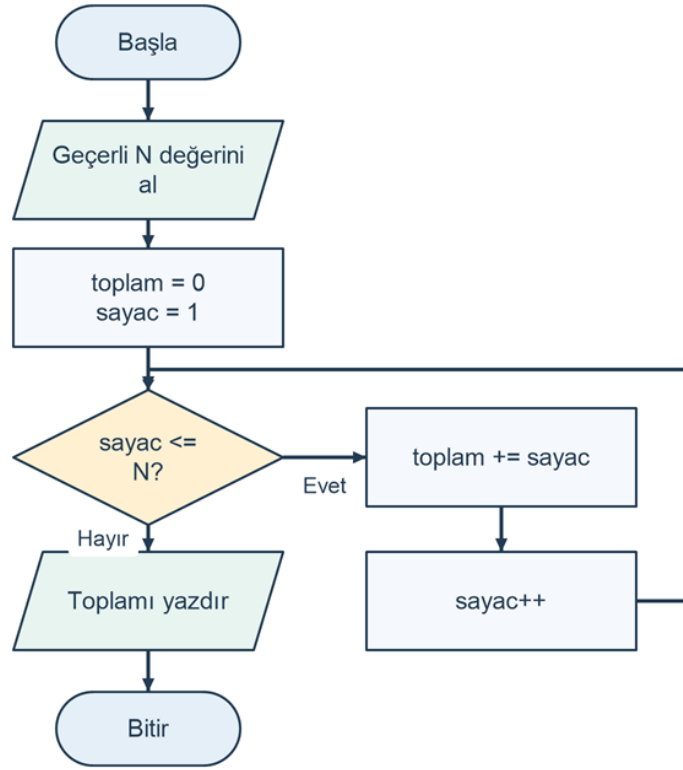
Program N (0..1000000): istemini gösterir. Tek bir tamsayı yazıp Enter tuşuna basın. Basamak ayırıcı kullanmayın. Bu üst sınır, döngünün başlangıç düzeyi bir uygulamada makul sürede tamamlanmasını sağlar. Sonuçlar kültürden bağımsız biçimde yazdırılır.

Algoritma

1. N değerini okuyun.
2. Girdinin tamsayı olduğunu ve 0 ile 1000000 aralığında bulunduğunu doğrulayın.
3. Geçersiz girişte hata mesajı gösterip programı bitirin.
4. toplam değişkenini sıfırla başlatın.
5. Sayacı 1'den başlatın; sayaç N değerini aşmadığı sürece döngüyü sürdürün.
6. Her adımda sayaç değerini toplama ekleyip sayacı bir artırın.
7. Döngü bitince toplamı yazdırın.

Sayaç 1 iken toplam 1, sayaç 2 işlendiğinde toplam 3 olur. Her adımda önceki sonuç yeni sayıyla genişletilir. Döngü koşulundaki $\leq$, N değerinin de toplama katılmasını sağlar.

Şema, doğrulanmış N girdisi için döngünün koşul, gövde ve artış adımlarını gösterir. $N = 0$ durumunda ilk koşul yanlış olur.



Şekil 3 Sayaç ve biriktirici ile toplam döngüsü

C# çözümü

```
using System;
using System.Globalization;
```

```
Console.Write("N (0..1000000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 0 || n > 1000000)
{
    Console.WriteLine("Hata: 0 ile 1000000 arasında bir tamsayı girin.");
    return;
}
```

```
long toplam = 0;
for (int sayac = 1; sayac <= n; sayac++)
{
    toplam += sayac;
}
```

```
Console.WriteLine("Toplam: " +
    toplam.ToString(CultureInfo.InvariantCulture));
```

`toplam += sayac`, `toplam = toplam + sayac` ifadesinin kısa yazımıdır. Toplamın long olması, izin verilen büyük girdilerde sonucun int sınırını aşmasına olanak tanır. Sayaç ise verilen üst sınır nedeniyle int türünde tutulabilir.

Örnek çalıştırmalar

Tabloda istem metni gösterilmez.

Girdi	Sonuç
0	Toplam: 0
1	Toplam: 1
4	Toplam: 10
10	Toplam: 55
1000000	Toplam: 500000500000

Sınır durumları

- $N = 0$ olduğunda döngü gövdesi hiç çalışmaz; başlangıç toplamı korunur.
- $N = 1$ olduğunda döngü tam bir kez çalışır.
- Negatif sayı, 1000001, boş giriş ve ondalık sayı reddedilir.
- En büyük izin verilen toplam 500000500000, long aralığına sığar.

Kazanımlar

- Tekrarlı işlemleri for döngüsüyle ifade etme.
- Sayaç ve biriktirici değişkenlerinin görevlerini ayırma.
- Döngünün başlangıç, koşul ve artış bölümlerini açıklama.
- Girdi sınırına göre sonuç türünü seçme.

Alıştırmalar

1. Yalnızca 1 ile N arasındaki çift sayıları toplayın.
2. Döngüyü while kullanarak yeniden yazın.
3. $N = 5$ için her adımın sayaç ve toplam değerlerini bir tabloda gösterin.

009 Bir sayının faktöriyelini hesaplama

Düzey: Temel Önkoşullar: 008

Kavramlar: For döngüsü, Çarpım biriktirme, Başlangıç değeri, Sayısal sınırlar

Problem tanımı

Sıfır veya pozitif bir tamsayının faktöriyelini hesaplayın. Pozitif N için $N!$, 1'den N değerine kadar tamsayıların çarpımıdır. Örneğin $4! = 1 \times 2 \times 3 \times 4 = 24$ olur. Sıfırın faktöriyeli tanım gereği $0! = 1$ kabul edilir.

Çözümde çarpım, bir döngü ile biriktirilir. Toplama biriktiricisini sıfırla başlatırken, çarpma biriktiricisini birle başlatmak gerekir. Başlangıç değeri sıfır olsaydı sonraki bütün çarpımlar sıfır kalırdı. Bu seçim, sıfır girdisini ek bir hesaplama gerektirmeden doğru sonuçlandırır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	0 ile 20 arasında bir tamsayı.
Çıktı	faktoriyel	$n!$ değeri; long türünde.

Program N (0..20): istemini gösterir. Bir tamsayı yazıp Enter tuşuna basın. Basamak ayırıcı kullanmayın. İzin verilen üst sınır, sonucu long türünde güvenle saklayabilmek için seçilmiştir. Sayılar kültürden bağımsız biçimde okunur ve yazdırılır.

Algoritma

1. N değerini okuyun.
2. Girdinin tamsayı olduğunu ve 0 ile 20 aralığında bulunduğunu doğrulayın.
3. Geçersiz girişte hata mesajı gösterip programı bitirin.
4. faktoriyel değişkenini 1 ile başlatın.
5. Sayacı 1'den N değerine kadar artırın.
6. Her adımda mevcut faktöriyel değerini sayaçla çarpın.
7. Sonucu yazdırın.

Döngüde sayaç k işlendiğinde biriktiricinin değeri $k!$ olur. Böylece her adımın sonucunu önceki adım üzerinden açıklamak mümkündür. $N = 0$ olduğunda ilk döngü koşulu yanlış olur ve başlangıç değeri 1 korunur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("N (0..20): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 0 || n > 20)
{
    Console.WriteLine("Hata: 0 ile 20 arasında bir tamsayı girin.");
    return;
}

long faktoriyel = 1;
for (int sayac = 1; sayac <= n; sayac++)
{
    faktoriyel *= sayac;
}

Console.WriteLine("Faktöriyel: " +
    faktoriyel.ToString(CultureInfo.InvariantCulture));
```

faktoriyel *= sayac, faktoriyel = faktoriyel * sayac ifadesine karşılık gelir. Döngü koşulunda <= kullanmak son çarpanı da dahil eder. Girdi doğrulaması çarpma başlamadan yapılır; bu nedenle izin verilmeyen bir sayı kısmi sonuç üretmez.

Örnek çalıştırmalar

Tabloda istem metni yerine sonuç satırı gösterilir.

Girdi	Sonuç
0	Faktöriyel: 1
1	Faktöriyel: 1
4	Faktöriyel: 24
10	Faktöriyel: 3628800
20	Faktöriyel: 2432902008176640000

Sınır durumları

- 0 ve 1 farklı sayıda döngü adımıyla aynı sonucu üretir.
- Negatif sayıların faktöriyeli bu problem kapsamında hesaplanmaz.
- 20!, long aralığına sığar; 21! sığmadığı için 21 reddedilir.
- 12! değeri int aralığına sığsa da 13! sığmaz; biriktirici bu yüzden long seçilmiştir.
- Boş veya sayısal olmayan girişte hata mesajı yazdırılır.

Kazanımlar

- Çarpım biriktiricisinin başlangıç değerini gerekçelendirme.
- Matematiksel bir işlemi sayaçlı döngüye dönüştürme.
- Veri türünün sonuç büyüklüğünü sınırladığını fark etme.
- Döngünün hiç çalışmaması durumunu açıklama.

Alıştırmalar

1. $N = 5$ için her adımın sayaç ve çarpım değerlerini yazın.
2. Döngüyü `while` ile yeniden kurun.
3. Başlangıç değerini sıfır yapmanın sonuçlara etkisini açıklayın.

010 Bir dizinin toplamını ve ortalamasını bulma

Düzey: Temel Önkoşullar: 008, 009

Kavramlar: Dizi, Dizin, For döngüsü, Toplam biriktirme, Tür dönüşümü

Problem tanımı

Kullanıcının belirlediği sayıda tamsayıyı bir dizide saklayın. Ardından dizi elemanlarını dolaşarak toplamalarını ve aritmetik ortalamalarını hesaplayın. Aritmetik ortalama, bütün elemanların toplamının eleman sayısına bölünmesiyle bulunur.

Örneğin 2, 5, 8 değerlerinin toplamı 15, ortalaması 5'tir. Dizi, aynı türde birden fazla değeri tek bir değişken adı altında saklar. Her elemana sıfırdan başlayan bir dizinle erişilir. Üç elemanlı bir dizinin geçerli dizinleri 0, 1 ve 2 olur.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	adet	1 ile 100 arasında eleman sayısı.
Girdi	sayılar	Her biri -1000000 ile 1000000 arasında adet tamsayı.
Çıktı	toplam	Elemanların toplamı; long türünde.
Çıktı	ortalama	double türündeki ortalama; iki ondalık basamakla gösterilir.

Önce Eleman sayısı (1..100): istemine cevap verin. Sonra 1. eleman: ile başlayan istemlerde her tamsayıyı ayrı satıra yazın. Basamak ayırıcı kullanmayın. Ortalama çıktısında ondalık ayırıcı noktadır; örneğin 1.50.

Algoritma

1. Eleman sayısını okuyun ve 1 ile 100 aralığında olduğunu doğrulayın.
2. Bu uzunlukta bir tamsayı dizisi oluşturun.
3. Dizinleri sıfırdan başlayarak dolaşın; her girdiyi doğrulayıp ilgili elemana kaydedin.
4. Geçersiz girişte hata mesajı gösterip programı bitirin.
5. toplam değişkenini sıfırla başlatın.
6. Diziyi ikinci bir döngüyle dolaşarak her elemanı toplama ekleyin.
7. Toplamı double türüne dönüştürüp eleman sayısına bölün.
8. Toplamı ve iki ondalık basamakla ortalamayı yazdırın.

İlk döngü verileri saklar, ikinci döngü saklanmış veriler üzerinde hesaplama yapar. Bu ayırım, aynı diziyi daha sonra farklı işlemler için kullanmayı kolaylaştırır.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..100): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int adet) || adet < 1 || adet > 100)
{
    Console.WriteLine("Hata: 1 ile 100 arasında bir tamsayı girin.");
    return;
}

int[] sayilar = new int[adet];
for (int i = 0; i < sayilar.Length; i++)
{
    Console.WriteLine((i + 1).ToString(CultureInfo.InvariantCulture) + ". eleman: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int sayi) ||
        sayi < -1000000 || sayi > 1000000)
    {
        Console.WriteLine("Hata: -1000000 ile 1000000 arasında bir tamsayı girin.");
        return;
    }
    sayilar[i] = sayi;
}

long toplam = 0;
for (int i = 0; i < sayilar.Length; i++)
{
    toplam += sayilar[i];
}

double ortalama = (double)toplam / sayilar.Length;
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Ortalama: " + ortalama.ToString("F2", CultureInfo.InvariantCulture));
```

Tür dönüşümü bölme işleminden önce yapılır. Böylece 3 / 2 tamsayı bölmesi yerine 3.0 / 2 işlemi gerçekleşir ve kesirli sonuç korunur. F2 yalnızca gösterimi iki ondalık basamağa yuvarlar.

Örnek çalıştırmalar

Elemanlar giriş sırasındadır; sonuç sütunu iki çıktı satırını gösterir.

Eleman sayısı	Elemanlar	Sonuç
3	2, 5, 8	Toplam: 15 Ortalama: 5.00
2	1, 2	Toplam: 3 Ortalama: 1.50
3	-4, 0, 4	Toplam: 0 Ortalama: 0.00
1	-7	Toplam: -7 Ortalama: -7.00
100	Her eleman 1000000	Toplam: 100000000 Ortalama: 1000000.00

Sınır durumları

- Eleman sayısı sıfır olamaz; sıfıra bölme engellenir.
- $i < \text{sayılar.Length}$, son geçerli dizinin aşılmasını önler.
- Negatif değerler ve sıfır normal biçimde toplama katılır.
- Toplam -100000000 ile 100000000 arasındadır ve long türüne sığar.
- Geçersiz eleman girişinde kısmi dizi için sonuç yazılmaz.

Kazanımlar

- Dizi oluşturma, doldurma ve dizinle okuma.
- Dizi uzunluğuna bağlı bir döngü kurma.
- Bölme öncesi tür dönüşümünün etkisini açıklama.
- Hesaplanan değerle ekranda gösterilen biçimi ayırma.

Alıştırmalar

1. Pozitif ve negatif elemanların sayılarını bulun.
2. Dizideki en büyük değeri 007 numaralı problemdeki yaklaşımla belirleyin.
3. Ortalamanın üzerinde olan elemanları yazdırın.

Bölüm 2: Birleşik koşullar ve karar yapıları

011 Sayının kapalı bir aralıkta olup olmadığını belirleme

Düzey: Temel Önkoşullar: 005, 006

Kavramlar: Kapalı aralık, Mantıksal VE, Birleşik koşul, Girdiler arası doğrulama

Problem tanımı

Kullanıcıdan bir tamsayı, bir alt sınır ve bir üst sınır alın. Sayının bu iki sınırla tanımlanan kapalı aralıkta olup olmadığını belirleyin. Kapalı aralık, sınırların kendisini de içerir: alt sınır 3 ve üst sınır 8 ise 3 ve 8 de aralıktadır.

Her girdi -1000000 ile 1000000 arasında olmalıdır. Alt sınır üst sınırdan büyükse aralık geçersizdir; program sınırları kendiliğinden değiştirmek yerine hata verir. Alt ve üst sınırın eşit olması geçerlidir ve yalnızca tek sayıyı içeren bir aralık oluşturur.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	Aralıkta olup olmadığı araştırılan tamsayı.
Girdi	altSınir	Aralığın dahil edilen alt sınırı.
Girdi	ustSınir	Aralığın dahil edilen üst sınırı.
Çıktı	Karar	Sonuç: Aralıkta veya Sonuç: Aralık dışında.

Sayı: , Alt sınır: ve Üst sınır: istemlerine sırasıyla birer tamsayıyı ayrı satırda yazın. Basamak ayırıcı kullanmayın. Her tamsayı -1000000 ile 1000000 arasında olmalıdır. Bir girdinin biçimi veya aralığı geçersizse sonraki girdiler istenmeden program sona erer.

Algoritma

1. Sayıyı okuyun; belirtilen aralıkta bir tamsayı değilse hata yazıp bitirin.
2. Alt sınırı okuyun ve aynı biçimde doğrulayın.
3. Üst sınırı okuyun ve aynı biçimde doğrulayın.
4. Alt sınır üst sınırdan büyükse hata yazıp bitirin.
5. `sayi >= altSınir` ve `sayi <= ustSınir` koşullarını `&&` ile birleştirin.
6. İki koşul da doğruysa Sonuç: Aralıkta, aksi durumda Sonuç: Aralık dışında yazdırın.

`&&` mantıksal VE işlecidir; birleşik koşulun doğru olması için iki karşılaştırmanın da doğru olması gerekir. Örneğin 2, üst sınır 8'den küçük olsa bile alt sınır 3'ten küçük olduğu için aralıkta değildir. `>=` ve `<=` kullanılması sınırların dahil edilmesini sağlar.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Sayı: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi) ||
    sayi < -1000000 || sayi > 1000000)
{
    Console.WriteLine("Hata: Sayı -1000000 ile 1000000 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Alt sınır: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int altSınir) ||
    altSınir < -1000000 || altSınir > 1000000)
{
    Console.WriteLine("Hata: Alt sınır -1000000 ile 1000000 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

Console.Write("Üst sınır: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ustSınir) ||
    ustSınir < -1000000 || ustSınir > 1000000)
{
    Console.WriteLine("Hata: Üst sınır -1000000 ile 1000000 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

if (altSınir > ustSınir)
{
    Console.WriteLine("Hata: Alt sınır üst sınırdan büyük olamaz.");
    return;
}

bool aralikta = sayi >= altSınir && sayi <= ustSınir;
if (aralikta)
{
    Console.WriteLine("Sonuç: Aralıkta");
}
else
{
    Console.WriteLine("Sonuç: Aralık dışında");
}
```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. bool değişkeni doğru veya yanlış değerini saklar. Girdi doğrulaması önce tek tek değerleri, sonra sınırların birbiriyle ilişkisini denetler. return, hata durumunda sonraki adımların çalışmasını engeller.

Örnek çalıştırmalar

Tablo, istemleri hariç tutarak yazdırılan tam sonuç veya hata satırını gösterir. –, hata nedeniyle o girdinin henüz istenmediğini belirtir.

Sayı	Alt sınır	Üst sınır	Sonuç satırı
5	3	8	Sonuç: Aralıkta
3	3	8	Sonuç: Aralıkta
8	3	8	Sonuç: Aralıkta
2	3	8	Sonuç: Aralık dışında
9	3	8	Sonuç: Aralık dışında
-4	-4	-4	Sonuç: Aralıkta
0	-1000000	1000000	Sonuç: Aralıkta
5	8	3	Hata: Alt sınır üst sınırdan büyük olamaz.
2.5	–	–	Hata: Sayı -1000000 ile 1000000 arasında bir tamsayı olmalıdır.
0	-1000001	–	Hata: Alt sınır -1000000 ile 1000000 arasında bir tamsayı olmalıdır.
0	0	1000001	Hata: Üst sınır -1000000 ile 1000000 arasında bir tamsayı olmalıdır.

Sınır durumları

- Alt veya üst sınıra eşit sayı aralıktadır; > ve < kullanılsaydı bu değerler dışarıda kalırdı.
- Eşit sınırlar geçerlidir. Sayı bu ortak değerden farklıysa aralık dışındadır.
- Negatif sınırlarda da aynı karşılaştırmalar kullanılır; ek bir işaret kontrolü gerekmez.
- Sınırların her biri geçerli bir sayı olsa bile alt sınırın üst sınırdan büyük olması ayrıca reddedilir.
- Boş satır, metin, ondalıklı sayı veya izin verilen aralık dışında bir değer için karar yazılmaz.
- Yalnızca karşılaştırma yapıldığı için toplama veya çarpma taşması oluşmaz.

Kazanımlar

- Kapalı bir aralığı >=, <= ve && ile doğru biçimde ifade etme.
- İki ayrı karşılaştırmamanın sonucunu bir bool değişkeninde saklama.
- Tek bir girdinin doğrulanması ile iki girdi arasındaki ilişkinin doğrulanmasını ayırt etme.
- Alt sınır, üst sınır ve sınırların hemen dışındaki değerler için test sonuçlarını açıklama.

Alıştırmalar

1. Sınırları içermeyen açık aralık için koşulu değiştirin; 3 ile 8 aralığında 3, 4 ve 8'in sonuçlarını karşılaştırın.
2. Aralık dışındaki sayılar için Alt sınırın altında ve Üst sınırın üstünde biçiminde iki farklı sonuç yazdırın.
3. Koşulda && yerine || kullanılırsa alt sınır 3 ve üst sınır 8 iken 2 için neden yanlış sonuç çıkacağını açıklayın.

012 Puanı harf notuna dönüştürme

Düzy: Temel Önkoşullar: 005, 011

Kavramlar: Aralık sınıflandırması, Sıralı koşul zinciri, Eşik değeri, İlk doğru dal

Problem tanımı

Kullanıcının girdiği 0 ile 100 arasındaki tamsayı puanı bir harf notuna dönüştürün. Bu problem için kullanılan ölçek yalnızca programlama alıştırmalarına ait örnek bir sınıflandırmadır; herhangi bir kurumun not yönetmeliği değildir.

90–100 aralığı AA, 80–89 BA, 70–79 BB, 60–69 CB, 50–59 CC ve 0–49 FF olarak sınıflandırılır. Aralıklar bütün geçerli puanları kapsar ve bir puan yalnızca tek harf notuna karşılık gelir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	puan	0 ile 100 arasında tamsayı.
Çıktı	harfNotu	AA, BA, BB, CB, CC veya FF.

Puan (0..100): istemine bir tamsayı girin. Basamak ayırıcı veya ondalık bölüm kullanmayın. Geçerli girişte Harf notu: ile başlayan tek sonuç satırı yazılır. Geçersiz girişte Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır. yazılır ve program sona erer.

Algoritma

1. Puanı okuyun.
2. Girdi tamsayı değilse veya 0 ile 100 arasında değilse hata yazıp bitirin.
3. Puan en az 90 ise harf notunu AA belirleyin.
4. Aksi durumda puan en az 80 ise BA belirleyin.
5. Aksi durumda puan en az 70 ise BB belirleyin.
6. Aksi durumda puan en az 60 ise CB belirleyin.
7. Aksi durumda puan en az 50 ise CC belirleyin.
8. Hiçbir eşik sağlanmadıysa FF belirleyin.
9. Harf notunu yazdırın.

Koşullar yüksek eşikten düşük eşığe doğru denenir. Örneğin 85 puanı 90 kontrolünü geçemez, 80 kontrolünü geçer ve BA olur. Bu dalda ayrıca puan ≤ 89 yazılması gerekmez; önceki koşulun yanlış olması zaten puanın 90'dan küçük olduğunu gösterir.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Puan (0..100): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int puan) || puan < 0 || puan > 100)
{
    Console.WriteLine("Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.");
    return;
}

string harfNotu;
if (puan >= 90)
{
    harfNotu = "AA";
}
else if (puan >= 80)
{
    harfNotu = "BA";
}
else if (puan >= 70)
{
    harfNotu = "BB";
}
else if (puan >= 60)
{
    harfNotu = "CB";
}
else if (puan >= 50)
{
    harfNotu = "CC";
}
else
{
    harfNotu = "FF";
}

Console.WriteLine("Harf notu: " + harfNotu);
```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. if / else if / else zinciri ilk doğru dalı seçer. Her dal harfNotu değişkenine bir metin atar; tek ortak çıktı satırı sınıflandırma tamamlandıktan sonra çalışır. Ayrı if ifadeleri kullanılması aynı puan için birden fazla koşulun doğru olmasına ve önceki atamanın değiştirilmesine yol açabilir.

Örnek çalıştırmalar

Tablo, giriş isteminden sonra yazılan tam satırı gösterir.

Puan	Sonuç satırı	Denenen durum
100	Harf notu: AA	Geçerli en büyük puan.
90	Harf notu: AA	AA alt sınırı.
89	Harf notu: BA	AA sınırının hemen altı.
80	Harf notu: BA	BA alt sınırı.
79	Harf notu: BB	BA sınırının hemen altı.

Puan	Sonuç satırı	Denenen durum
70	Harf notu: BB	BB alt sınırı.
60	Harf notu: CB	CB alt sınırı.
50	Harf notu: CC	CC alt sınırı.
49	Harf notu: FF	CC sınırının hemen altı.
0	Harf notu: FF	Geçerli en küçük puan.
101	Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.	Aralık dışı.
-1	Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.	Aralık dışı.
89.5	Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.	Tamsayı değil.

Sınır durumları

- 0 ve 100 kabul edilir; bu değerlerin dışındaki puanlar sınıflandırılmaz.
- Eşiklerde eşitlik dahildir. 90, BA yerine AA olur.
- puan ≥ 50 koşulu zincirin başına taşınırsa 90 puanı yanlışlıkla CC olarak sınıflandırılır.
- FF dalına yalnızca doğrulanmış 0–49 puanları ulaşır; negatif puanlar bu dala bırakılmaz.
- Boş giriş ve sayı olmayan metin hata üretir. Ondalıklı puanlar yuvarlanmaz; reddedilir.

Kazanımlar

- Birbirini dışlayan sayı aralıklarını yüksekten düşüğe bir koşul zincirinde sınıflandırma.
- Önceki dalın yanlış olmasının sonraki dalın aralığını nasıl daralttığını açıklama.
- Bir metin sonucunu dallarda belirleyip ortak bir satırda yazdırma.
- Her eşik için eşik değerini ve hemen altındaki değeri içeren testler tasarlama.

Alıştırmalar

1. Aynı sınıflandırmayı puan < 50 koşulundan başlayarak düşükten yükseğe kurun ve sonuçları karşılaştırın.
2. 59, 60, 69 ve 70 için hangi koşulların hangi sırayla denendiğini yazın.
3. Örnek ölçeğe 95–100 aralığında AA+ ekleyin; yeni koşulun zincirde nereye gelmesi gerektiğini açıklayın.

013 Atölyeye katılım uygunluğunu belirleme

Düzey: Temel Önkoşullar: 011, 012

Kavramlar: Mantıksal VE, Mantıksal VEYA, Parantez, Karar tablosu, Birleşik koşul

Problem tanımı

Örnek bir programlama atölyesi için bir adayın katılım koşulunu sağlayıp sağlamadığını belirleyin. Bu alıştırmada adayın yaşı en az 18 olmalıdır. Ayrıca aday ya ön hazırlık çalışmasını tamamlamış olmalı ya da hazırlık değerlendirmesinden en az 70 puan almış olmalıdır.

Ön hazırlığın tamamlanması ve değerlendirme puanı iki alternatif koşuldur; ikisinin birden sağlanması gerekmez. Yaş koşulu ise her durumda zorunludur. Kurallar yalnızca bu alıştırmadaki kurgusal atölyeye aittir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	yas	0 ile 120 arasında tamsayı.
Girdi	onHazirlik	Tamamlandıysa 1, tamamlanmadıysa 0.
Girdi	puan	0 ile 100 arasında tamsayı değerlendirme puanı.
Çıktı	Uygunluk	Sonuç: Katılabilir veya Sonuç: Katılamaz.

Yaş (0..120): , Ön hazırlık tamamlandı mı (0/1): ve Değerlendirme puanı (0..100): istemlerine sırayla ayrı satırlarda yanıt verin. Bütün girdiler doğrulanır; ön hazırlık tamamlanmış olsa da değerlendirme puanı istenir. Geçersiz girişte hata yazılır ve sonraki girdiler istenmeden program sona erer.

Algoritma

1. Yaşı okuyun ve 0 ile 120 arasında tamsayı olduğunu doğrulayın.
2. Ön hazırlık durumunu okuyun ve yalnızca 0 veya 1 olduğunu doğrulayın.
3. Değerlendirme puanını okuyun ve 0 ile 100 arasında tamsayı olduğunu doğrulayın.
4. $yas \geq 18$ ile yaş koşulunu kurun.
5. $onHazirlik == 1 \ || \ puan \geq 70$ ile iki alternatif hazırlık koşulunu kurun.
6. Yaş koşulunu hazırlık koşuluyla && kullanarak birleştirin.
7. Birleşik koşul doğruysa katılabilir, yanlışsa katılamaz sonucunu yazdırın.

&& iki koşulun da doğru olmasını ister. || en az bir koşul doğru olduğunda doğrudur. Parantez, iki hazırlık seçeneğinin birlikte değerlendirilmesini açıkça gösterir.

Yaş en az 18 mi?	Ön hazırlık tamam mı?	Puan en az 70 mi?	Karar
Hayır	Hayır	Hayır	Katılamaz
Hayır	Hayır	Evet	Katılamaz
Hayır	Evet	Hayır	Katılamaz
Hayır	Evet	Evet	Katılamaz

Yaş en az 18 mi?	Ön hazırlık tamam mı?	Puan en az 70 mi?	Karar
Evet	Hayır	Hayır	Katılamaz
Evet	Hayır	Evet	Katılabilir
Evet	Evet	Hayır	Katılabilir
Evet	Evet	Evet	Katılabilir

C# çözümü

```

using System;
using System.Globalization;

Console.Write("Yaş (0..120): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int yas) || yas < 0 || yas > 120)
{
    Console.WriteLine("Hata: Yaş 0 ile 120 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Ön hazırlık tamamlandı mı (0/1): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int onHazirlik) ||
    (onHazirlik != 0 && onHazirlik != 1))
{
    Console.WriteLine("Hata: Ön hazırlık için 0 veya 1 girin.");
    return;
}

Console.Write("Değerlendirme puanı (0..100): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int puan) || puan < 0 || puan > 100)
{
    Console.WriteLine("Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.");
    return;
}

bool uygun = yas >= 18 && (onHazirlik == 1 || puan >= 70);
if (uygun)
{
    Console.WriteLine("Sonuç: Katılabilir");
}
else
{
    Console.WriteLine("Sonuç: Katılamaz");
}

```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. == eşitliği, != eşit olmamayı denetler. C# dilinde &&, || işlecinden önce değerlendirilir. Parantez kaldırılıp yas >= 18 && onHazirlik == 1 || puan >= 70 yazılıysa yüksek puanlı 17 yaşındaki bir aday da kabul edilirdi. Parantez bu hatayı engeller.

Mantıksal işlemler kısa devre yapar: && sol tarafı yanlışsa sağ tarafı değerlendirmez; || sol tarafı doğruysa sağ tarafı değerlendirmez. Burada bütün girdilerin önce alınması ve doğrulanması, kısa devre nedeniyle bir girdinin kontrol dışı kalmasını önler.

Örnek çalıştırmalar

Tablo, istemlerden sonra yazılan tam satırı gösterir. –, hata nedeniyle henüz istenmeyen girdidir.

Yaş	Ön hazırlık	Puan	Sonuç satırı
18	0	70	Sonuç: Katılabilir
18	0	69	Sonuç: Katılamaz
18	1	0	Sonuç: Katılabilir
17	1	100	Sonuç: Katılamaz
17	0	70	Sonuç: Katılamaz
25	1	85	Sonuç: Katılabilir
0	1	100	Sonuç: Katılamaz
120	0	100	Sonuç: Katılabilir
121	–	–	Hata: Yaş 0 ile 120 arasında bir tamsayı olmalıdır.
20	2	–	Hata: Ön hazırlık için 0 veya 1 girin.
20	1	101	Hata: Puan 0 ile 100 arasında bir tamsayı olmalıdır.
yirmi	–	–	Hata: Yaş 0 ile 120 arasında bir tamsayı olmalıdır.

Sınır durumları

- 18 yaş dahil edilir, 17 yaş dahil edilmez; yüksek puan yaş şartını kaldırmaz.
- 70 puan yeterlidir; 69 puan ancak ön hazırlık tamamlanmışsa yeterlidir.
- Ön hazırlık tamamlandıysa sıfır puanla katılım mümkündür, ancak puanın geçerli aralıkta olması yine zorunludur.
- Ön hazırlık için 0 ve 1 dışında değerler, evet gibi metinler ve boş giriş reddedilir.
- Yaş ile puan için sınırlar dahil edilir; ondalıklı girdiler tamsayıya dönüştürülmez.
- Adayın koşulu sağlamaması normal bir karar sonucudur; biçimi geçersiz girdiden doğan hata ile aynı şey değildir.

Kazanımlar

- Zorunlu koşulu alternatif koşullarla && ve || kullanarak birleştirme.
- Parantezlerin bir uygunluk kuralının anlamını nasıl değiştirdiğini örnekle gösterme.
- Üç mantıksal değişken için sekiz satırlık karar tablosunu yorumlama.
- Geçerli fakat uygun olmayan bir girdiyi geçersiz girdiden ayırt etme.

Alıřtırmalar

1. Atölye için iki hazırlık koşulunun da sağlanmasını isteyin; deęişen karar tablosu satırlarını bulun.
2. Yaş üst sınırını katılım kuralında 65 yapın; 65 ve 66 yaşları için test ekleyin. Girdi doğrulama aralığının neden aynı kalabileceğini açıklayın.
3. Parantezsiz hatalı koşulu deneyin ve 17 yaş, ön hazırlık 0, puan 100 girdisinin iki sürümdeki sonuçlarını karşılaştırın.

014 Artık yılı belirleme

Düzey: Orta Önkoşullar: 006, 013

Kavramlar: Bölünebilme, Kalan, İstisna kuralı, Birleşik mantıksal koşul, Karar sırası

Problem tanımı

Kullanıcının girdiği yılın artık yıl olup olmadığını belirleyin. Bu alıştırmada Gregoryen takviminin artık yıl hesabı kullanılır: 400'e tam bölünen yıl artık yıldır. 400'e bölünmeyen ama 100'e tam bölünen yıl artık yıl değildir. Diğer yıllarda 4'e tam bölünenler artık yıldır.

Bu hesap modeli 1 ile 9999 arasındaki bütün girdilere uygulanır. Geçmişte farklı toplumların takvim kullanım tarihleri veya takvim geçişleri araştırılmaz. Artık yılda Şubat 29 gün, diğer yıllarda 28 gün olarak kabul edilir.

Örneğin 1900, 4'e bölünmesine rağmen 100'e bölünüp 400'e bölünmediği için artık yıl değildir. 2000 ise 400'e bölündüğü için artık yıldır. Yalnızca 4'e bölünme kontrolü bu iki durumu ayıramaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	yıl	1 ile 9999 arasında tamsayı yıl.
Çıktı	Karar	Sonuç: Artık yıl veya Sonuç: Artık yıl değil.

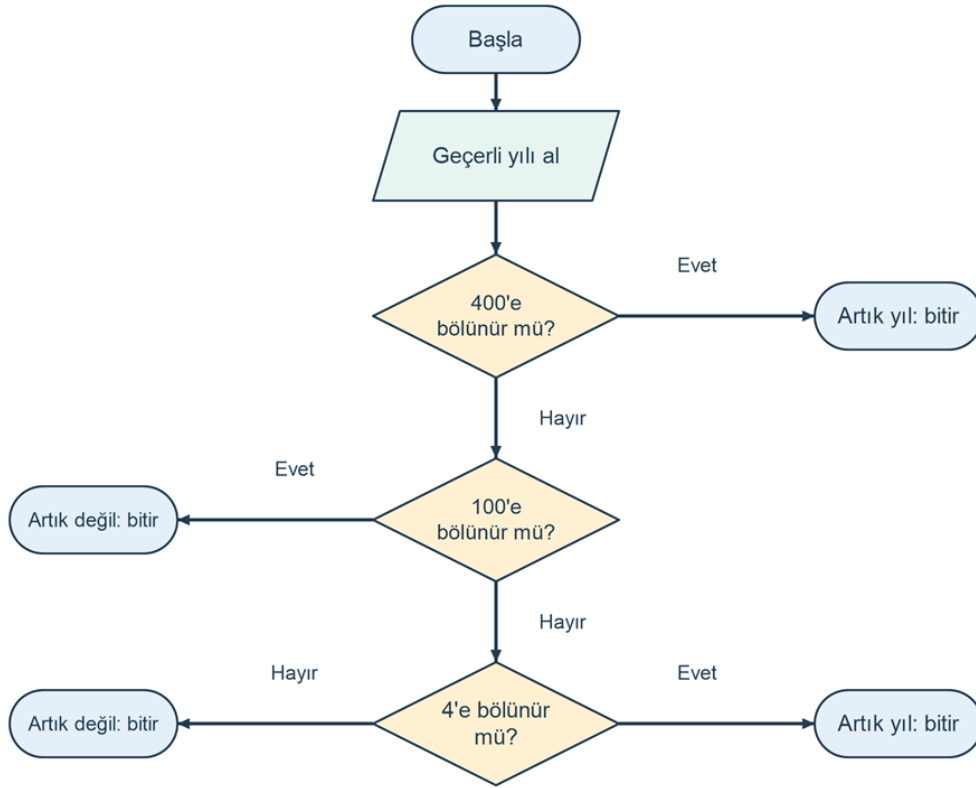
Yıl (1..9999): istemine bir tamsayı yazın. Basamak ayırıcı veya ondalık bölüm kullanmayın. Geçersiz girişte Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır. yazılır ve program sona erer.

Algoritma

1. Yılı okuyun; 1 ile 9999 arasında tamsayı değilse hata yazıp bitirin.
2. Yıl 400'e tam bölünüyorsa artık yıl sonucunu seçin.
3. Değilse, yıl 100'e tam bölünüyorsa artık yıl değil sonucunu seçin.
4. Değilse, yıl 4'e tam bölünüyorsa artık yıl sonucunu seçin.
5. Bu koşullar sağlanmıyorsa artık yıl değil sonucunu seçin.
6. Sonucu yazdırıp bitirin.

% bölme işleminden kalanı verir. $yıl \% 4 == 0$, yılın 4'e tam bölündüğünü belirtir. Kural tek bir koşul olarak $yıl \% 400 == 0 \ || \ (yıl \% 4 == 0 \ \&\& \ yıl \% 100 != 0)$ biçiminde de yazılabilir. İlk parça 400 kuralını, parantezli parça ise 100 istisnası dışındaki 4 kuralını ifade eder.

Şema doğrulanmış girdi üzerinde kararların sırasını gösterir. Sonuç yazılan uçlar programın bitişini de temsil eder.



Şekil 4 Artık yıl kuralının sıralı bölünebilme kontrolleri

C# çözümü

```

using System;
using System.Globalization;

Console.Write("Yıl (1..9999): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int yıl) || yıl < 1 || yıl > 9999)
{
    Console.WriteLine("Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.");
    return;
}

bool artıkYıl = yıl % 400 == 0 || (yıl % 4 == 0 && yıl % 100 != 0);
if (artıkYıl)
{
    Console.WriteLine("Sonuç: Artık yıl");
}
else
{
    Console.WriteLine("Sonuç: Artık yıl değil");
}
  
```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. Şemadaki sıralı kararlar kodda tek bir bool ifadesinde birleştirilmiştir; iki anlatım aynı sonucu üretir. != 0, tam bölünmeme

durumunu ifade eder. DateTime gibi takvim hesabını hazır yapan bir bileşen kullanılmadan kuralın kendisi uygulanır.

Örnek çalıştırmalar

Tablo, giriş isteminden sonra yazılan tam satırı gösterir.

Yıl	Sonuç satırı	Gerekçe
2024	Sonuç: Artık yıl	4'e bölünür, 100'e bölünmez.
2023	Sonuç: Artık yıl değil	4'e bölünmez.
1900	Sonuç: Artık yıl değil	100'e bölünür, 400'e bölünmez.
2000	Sonuç: Artık yıl	400'e bölünür.
2100	Sonuç: Artık yıl değil	100'e bölünür, 400'e bölünmez.
1	Sonuç: Artık yıl değil	Geçerli en küçük yıl.
4	Sonuç: Artık yıl	Hesap modeli bu girdiye de uygulanır.
9999	Sonuç: Artık yıl değil	Geçerli en büyük yıl.
0	Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.	Alt sınırın dışında.
10000	Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.	Üst sınırın dışında.
2024.5	Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.	Tamsayı değil.

Sınır durumları

- Sıfır veya negatif yıl kabul edilmez; hesap modeli 1 ile 9999 arasındaki pozitif yıllarla sınırlıdır.
- 100'e bölünen yıllarda 4 koşulu tek başına yeterli değildir; 400 istisnası ayrıca uygulanır.
- 400'e bölünen her yıl 100'e ve 4'e de bölünür. Bu nedenle şemada 400 kontrolü önce yapılır.
- 9996 kabul edilen aralıktaki son artık yıldır; 9999 geçerli ama artık olmayan bir yıldır.
- Boş giriş, sayı olmayan metin ve ondalıklı sayı için karar üretilmez.
- Bölme bölenleri sabit ve sıfırdan farklı olduğundan sıfıra bölme oluşmaz.

Kazanımlar

- Tam bölünmeyi % işleci ve sıfır karşılaştırmasıyla denetleme.
- Genel bir kuralın istisnasını ve istisnanın istisnasını mantıksal koşullara dönüştürme.
- Sıralı karar şeması ile birleşik bool ifadesinin eşdeğerliğini örneklerle gösterme.
- 1900 ve 2000 gibi kuralları birbirinden ayıran test girdileri seçme.

Alıştırmalar

1. Birleşik koşulu şemadaki sıraya uygun if / else if / else zinciriyle yeniden yazın.
2. Karara ek olarak Şubat ayındaki gün sayısını 28 veya 29 olarak yazdırın.
3. Yalnızca `yil % 4 == 0` kullanan sürümün 1900, 2000, 2024 ve 2100 girdilerinde hangi sonuçları yanlış ürettiğini belirleyin.

015 Dört işlem hesap makinesi

Düzey: Orta Önkoşullar: 001, 003, 012, 014

Kavramlar: Switch, İşlem seçimi, Decimal, Sıfıra bölme, Taşma, Çıktı biçimlendirme

Problem tanımı

Kullanıcının girdiği iki ondalıklı sayı üzerinde seçilen tek bir işlemi uygulayan hesap makinesi yazın. Desteklenen işlemler toplama +, çıkarma -, çarpma * ve bölme / işlemleridir. Çıkarma ve bölmede sıra önemlidir: ilk sayıdan ikinci sayı çıkarılır veya ilk sayı ikinci sayıya bölünür.

Her sayı -1000000 ile 1000000 arasında olmalıdır. Bölmede ikinci sayı sıfır olamaz. Desteklenmeyen bir işlem işareti girilirse hesaplama yapılmaz. Bu sürüm tek işlem tamamlandıktan sonra sona erer; yeniden işlem istemez.

Ondalık değerleri saklamak için decimal kullanılır. Sonuç ekranda iki ondalık basamakla gösterilir. Örneğin 1'in 3'e bölümü 0.33 olarak yazılır; hesaplanan değer gösterimden önce iki basamağa yuvarlanmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	ilkSayi	-1000000 ile 1000000 arasında decimal değer.
Girdi	ikinciSayi	Aynı aralıkta decimal değer; bölmede sıfır olamaz.
Girdi	islem	Tam olarak +, -, * veya / metni.
Çıktı	sonuc	Sonuç: ile başlayan, iki ondalık basamaklı değer.

İlk sayı: , İkinci sayı: ve İşlem (+, -, *, /): istemlerine sırayla ayrı satırlarda yanıt verin. Ondalık ayırıcı olarak nokta kullanın: 2.5 geçerlidir, 2,5 geçersizdir. Sayıların başında isteğe bağlı + veya - olabilir. Basamak ayırıcı, bilimsel gösterim ve girişin başında veya sonunda boşluk kabul edilmez. İşlem işaretinin yanında da boşluk yazmayın.

Algoritma

1. İlk sayıyı okuyun, biçimini ve aralığını doğrulayın. Geçersizse hata yazıp bitirin.
2. İkinci sayıyı okuyun, aynı biçimde doğrulayın.
3. İşlem işaretini okuyun.
4. switch ile işlem işaretine uygun dalı seçin.
5. + için toplayın, - için ilk sayıdan ikinciyi çıkarın, * için çarpın.
6. / dalında ikinci sayı sıfırsa hata yazıp bitirin; değilse bölün. Bölüm decimal türüne sığmazsa hata yazıp bitirin.
7. İşaret desteklenmiyorsa hata yazıp bitirin.
8. Sonucu nokta ondalık ayırıcıyla ve iki ondalık basamakla yazdırın.

switch, tek bir değer için belirli seçeneklerden hangisine eşit olduğunu denetler. case ilgili seçeneği, default seçeneklerin hiçbirine uymayan girdiyi temsil eder. Her başarılı dalın sonundaki break, switch işleminden çıkılmasını sağlar; program ortak çıktı satırına devam eder.

C# çözümü

```
using System;
using System.Globalization;
NumberStyles sayiBicimi = NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint;

Console.Write("İlk sayı: ");
if (!decimal.TryParse(Console.ReadLine(), sayiBicimi,
    CultureInfo.InvariantCulture, out decimal ilkSayi) ||
    ilkSayi < -1000000m || ilkSayi > 1000000m)
{
    Console.WriteLine("Hata: İlk sayı -1000000 ile 1000000 arasında olmalıdır.");
    return;
}

Console.Write("İkinci sayı: ");
if (!decimal.TryParse(Console.ReadLine(), sayiBicimi,
    CultureInfo.InvariantCulture, out decimal ikinciSayi) ||
    ikinciSayi < -1000000m || ikinciSayi > 1000000m)
{
    Console.WriteLine("Hata: İkinci sayı -1000000 ile 1000000 arasında olmalıdır.");
    return;
}

Console.Write("İşlem (+, -, *, /): ");
string? islem = Console.ReadLine();
decimal sonuc;

switch (islem)
{
    case "+":
        sonuc = ilkSayi + ikinciSayi;
        break;
    case "-":
        sonuc = ilkSayi - ikinciSayi;
        break;
    case "*":
        sonuc = ilkSayi * ikinciSayi;
        break;
    case "/":
        if (ikinciSayi == 0m)
        {
            Console.WriteLine("Hata: Sıfıra bölme yapılamaz.");
            return;
        }
        try
        {
            sonuc = ilkSayi / ikinciSayi;
        }
        catch (OverflowException)
        {
            Console.WriteLine("Hata: Bölme sonucu decimal türüne sığmıyor.");
            return;
        }
        break;
    default:
        Console.WriteLine("Hata: İşlem +, -, * veya / olmalıdır.");
        return;
}

Console.WriteLine("Sonuç: " + sonuc.ToString("F2", CultureInfo.InvariantCulture));
```

Kod, bir konsol projesinin Program.cs dosyasına tek başına konabilir. Sayısal sabitlerdeki m eki decimal türünü belirtir. NumberStyles seçeneklerini birleştiren |, burada kabul edilen sayı biçimlerini seçer; koşullarda kullanılan || ile aynı işleç değildir. string?, giriş akışı sona erdiğinde işlem metninin null olabileceğini belirtir; bu değer de default halinde reddedilir.

F2, yalnızca çıktı metnini iki ondalık basamağa yuvarlar. decimal sonlu hassasiyete sahiptir; 1/3 gibi sonsuz ondalık açılımlar bu türde de tam olarak saklanamaz. Çözüm dört işlemi doğrudan uygular, hesaplamaları iki ondalık basamakla sınırlayan ek bir yuvarlama yapmaz.

Çok küçük bir bölen, girdi aralığı geçerli olsa bile saklanamayacak kadar büyük bir bölüm oluşturabilir. try içinde bölme denir; böyle bir taşmada catch (OverflowException) hata mesajı yazıp programı bitirir. Toplama, çıkarma ve çarpma ise belirtilen girdi sınırlarıyla türün kapasitesi içinde kalır.

Örnek çalıştırmalar

Tablo, istemleri hariç tutarak yazdırılan tam satırı gösterir. –, hata nedeniyle henüz istenmeyen girdidir.

İlk sayı	İkinci sayı	İşlem	Sonuç satırı
2.5	1.25	+	Sonuç: 3.75
2	5	-	Sonuç: -3.00
-4	2.5	*	Sonuç: -10.00
7	2	/	Sonuç: 3.50
1	3	/	Sonuç: 0.33
0	5	/	Sonuç: 0.00
1000000	1000000	*	Sonuç: 1000000000000.00
5	0	/	Hata: Sıfıra bölme yapılamaz.
5	-0.0	/	Hata: Sıfıra bölme yapılamaz.
5	2	%	Hata: İşlem +, -, * veya / olmalıdır.
2,5	–	–	Hata: İlk sayı -1000000 ile 1000000 arasında olmalıdır.
5	1000000.01	–	Hata: İkinci sayı -1000000 ile 1000000 arasında olmalıdır.
1e3	–	–	Hata: İlk sayı -1000000 ile 1000000 arasında olmalıdır.
1000000	0.000000000000000000000000000001	/	Hata: Bölme sonucu decimal türüne sığmıyor.

Sınır durumları

- -1000000 ve 1000000 dahil edilir. Toplama, çıkarma ve çarpma sonuçları bu girdi aralığının dışında olabilir; çıktı aynı aralıkla sınırlanmaz.
- En büyük mutlak çarpım 1000000000000'dır ve decimal türüne sığar.
- Bölmede çok küçük ama sıfır olmayan bir bölen çok büyük bir bölüm üretebilir. Türün kapasitesini aşan bölüm için taşma yakalanır ve sonuç yazılmaz.
- 0, 0.0 ve -0.0 aynı sıfır değerini ifade eder ve bölen olarak reddedilir. Sıfırla çarpma veya sıfırı sıfır olmayan sayıya bölme geçerlidir.
- Büyük/küçük harf dönüşümü gerekmez; işlem girdisi tek bir işarettir. Boş satır veya boşluk eklenmiş işaret reddedilir.
- Gösterim yuvarlaması nedeniyle farklı hesaplanan değerler aynı iki basamaklı metni üretebilir.

Kazanımlar

- Bir işlem seçimini switch, case, default ve break ile gerçekleştirme.
- İşleme özel bir ön koşulu, sıfıra bölme örneğinde doğru dalda doğrulama.
- decimal girdiyi belirtilen ondalık ayırıcı ve sayı biçimiyle okuma.
- Geçerli girdilerin bile taşma oluşturabileceğini bölme örneğiyle açıklama ve hata sonucunu denetleme.
- Hesaplanan değeri iki basamaklı çıktı gösteriminden ayırt etme.
- Çıkarma ve bölmede operand sırasının sonucunu örneklerle gösterme.

Alıştırmalar

1. İşlem seçimini if / else if / else zinciriyle yeniden yazın ve dört işlem sonuçlarını karşılaştırın.
2. Sonucu önce F2, ardından F4 biçiminde yazdırın; 1/3 girdisinde hesap değişmeden gösterimin nasıl değiştiğini açıklayın.
3. Bölmede sıfır olmayan bölenin mutlak değerinin en az 0.01 olmasını isteyen bir sürüm yazın. Bu yeni kuralın en büyük mutlak bölümü nasıl sınırladığını açıklayın.

016 Üçgenin geçerliliğini ve türünü belirleme

Düzey: Orta Önkoşullar: 007, 011, 013

Kavramlar: Bileşik koşul, Mantıksal VE, Mantıksal VEYA, Üçgen eşitsizliği, Önce doğrulama sonra sınıflandırma

Problem tanımı

Üç kenar uzunluğunun bir üçgen oluşturup oluşturmadığını belirleyin. Üçgen oluşuyorsa kenarların eşitliğine göre eşkenar, ikizkenar veya çeşitkenar sonucunu yazdırın. Kenarlar aynı birimle ölçülmüş pozitif tamsayılardır; açıları hesaplamayın.

Bir üçgende herhangi iki kenarın toplamı üçüncü kenardan kesinlikle büyük olmalıdır. Örneğin 2, 3, 5 uzunlukları üçgen oluşturmaz: iki kenarın toplamı üçüncü kenara eşittir. Üç kenarı eşit üçgen eşkenar, tam iki kenarı eşit üçgen ikizkenar, bütün kenarları farklı üçgen çeşitkenardır. Bu çalışmada eşkenar sonucu ayrıca verildiği için ikizkenar dalı yalnızca tam iki kenarın eşit olduğu durumu kapsar.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	a, b, c	Her biri 1 ile 1000000 arasında, aynı birimde üç int kenar uzunluğu.
Çıktı	Geçerlilik veya tür	Sonuç: Üçgen oluşmaz., Sonuç: Eşkenar, Sonuç: İkizkenar veya Sonuç: Çeşitkenar.

a kenarı (1..1000000): , b kenarı (1..1000000): ve c kenarı (1..1000000): istemlerinde her kenarı ayrı satıra girin. Ondalık veya basamak ayırıcı kullanmayın. Biçimi veya aralığı yanlış olan ilk girdide Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır. yazılır ve program sona erer.

Algoritma

1. a, b ve c uzunluklarını sırayla okuyun; her birinin kabul edilen aralıkta bir tamsayı olduğunu doğrulayın.
2. $a + b > c$, $a + c > b$ ve $b + c > a$ koşullarının üçünün de sağlanıp sağlanmadığını hesaplayın.
3. Koşullardan en az biri sağlanmıyorsa üçgen oluşmadığını yazıp bitirin.
4. $a == b$ ve $b == c$ birlikte doğruysa eşkenar sonucunu yazdırın.
5. Değilse $a == b$, $a == c$ veya $b == c$ karşılaştırmalarından biri doğruysa ikizkenar sonucunu yazdırın.
6. Hiçbir eşitlik yoksa çeşitkenar sonucunu yazdırın.

&& işleci bütün koşulların birlikte doğru olmasını, || işleci en az bir koşulun doğru olmasını ister. Üçgen geçerliliği &&, iki kenarın eşitliğini bulma || gerektirir. Eşkenar kontrolünü önce yapmak, üç kenarı eşit bir girdinin ikizkenar dalına girmesini engeller.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("a kenarı (1..1000000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int a) || a < 1 || a > 1000000)
{
    Console.WriteLine("Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("b kenarı (1..1000000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int b) || b < 1 || b > 1000000)
{
    Console.WriteLine("Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("c kenarı (1..1000000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int c) || c < 1 || c > 1000000)
{
    Console.WriteLine("Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır.");
    return;
}

bool ucgenOlur = a + b > c && a + c > b && b + c > a;
if (!ucgenOlur)
{
    Console.WriteLine("Sonuç: Üçgen oluşmaz.");
}
else if (a == b && b == c)
{
    Console.WriteLine("Sonuç: Eşkenar");
}
else if (a == b || a == c || b == c)
{
    Console.WriteLine("Sonuç: İkizkenar");
}
else
{
    Console.WriteLine("Sonuç: Çeşitkenar");
}
```

Girdi doğrulaması, üçgenin geometrik geçerliliğinden ayrıdır. 1, 2, 3 geçerli üç tamsayı girdisidir; üçgen oluşturmaması hata mesajı gerektirmez ve normal bir sonuç olarak bildirilir.

Örnek çalıştırmalar

Sonuç sütunu, giriş istemlerinden sonra yazılan satırı gösterir. Girdiler a, b, c sırasındadır.

Girdi	Sonuç	Açıklama
3, 3, 3	Sonuç: Eşkenar	Üç kenar eşittir.
5, 3, 5	Sonuç: İkizkenar	Birinci ve üçüncü kenar eşittir.
3, 4, 5	Sonuç: Çeşitkenar	Eşitsizlikler sağlanır, kenarlar farklıdır.

Girdi	Sonuç	Açıklama
2, 3, 5	Sonuç: Üçgen oluşmaz.	Toplamın eşit olması yeterli değildir.
8, 2, 3	Sonuç: Üçgen oluşmaz.	En uzun kenar ilk sırada olsa da kontrol edilir.
1, 1, 1	Sonuç: Eşkenar	Alt sınırdaki geçerli üçgen.
1000000, 1000000, 1000000	Sonuç: Eşkenar	Üst sınırdaki geçerli üçgen.
İlk kenar 0	Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır.	Diğer kenarlar istenmez.
İlk kenar 2.5	Hata: Kenar 1 ile 1000000 arasında bir tamsayı olmalıdır.	Ondalıklı girdi reddedilir.

Sınır durumları

- Sıfır veya negatif bir kenar uzunluğu geometrik sınıflandırmadan önce reddedilir.
- Toplam üçüncü kenara eşitse üçgen oluşmaz; > yerine >= kullanılmamalıdır.
- En uzun kenarın hangi sırada girildiği önemli değildir; üç eşitsizlik de denetlenir.
- İki kenarın toplamı en fazla 2000000 olur ve int aralığında kalır.
- Boş satır, metin, ondalıklı sayı veya aralık dışı değer ilk görüldüğü anda programı bitirir.

Kazanımlar

- Üç ayrı gerekliliği && ile tek bir mantıksal ifadede birleştirme.
- Birden çok eşitlik olasılığını || ile sınama.
- Karar dallarının sırasının sınıflandırmayı nasıl etkilediğini örneklerle gösterme.
- Geçersiz girdi ile geçerli girdinin olumsuz sonucunu ayırt etme.

Alıştırmalar

1. Kenarları farklı sıralarda girip sonucun neden değişmediğini üç örneklerle gösterin.
2. İlk tür kontrolünü ikizkenar koşulu yaparsanız 3, 3, 3 girdisinin neden yanlış sınıflanacağını açıklayın.
3. Geçerli bir üçgen için çevreyi de hesaplayıp sonuç satırının ardından yazdırın.

017 Kademeli tüketim bedelini hesaplama

Düzey: Orta Önkoşullar: 012, 015, 016

Kavramlar: Aralık sınırları, Kademeli hesaplama, Decimal, Koşul zinciri, Karar tablosu

Problem tanımı

Kurgusal bir hizmetin tüketim bedelini üç kademeli bir tarifeye göre hesaplayın. İlk 100 birimin her biri 2.00 TL, sonraki 100 birimin her biri 3.00 TL, 200 birimin üzerindeki her birim 5.00 TL olarak ücretlendirilir. Bu öğretim örneği herhangi bir kurumun gerçek tarifesini temsil etmez; vergi, sabit ücret ve indirim içermez.

Kademeli tarifiede her aralıktaki miktar kendi birim fiyatıyla çarpılır. Örneğin 201 birim tüketimde ilk 100 birim için 200.00 TL, ikinci 100 birim için 300.00 TL ve kalan bir birim için 5.00 TL ödenir. Toplam 505.00 TL olur; 201 birimin tamamına 5.00 TL uygulanmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	tuketim	0 ile 10000 arasında tüketim miktarı; int ve birimi bu örneğe özgü soyut tüketim birimidir.
Çıktı	bedel	decimal olarak hesaplanan toplam; Bedel: 505.00 TL biçiminde, iki ondalık basamakla.

Tüketim (0..10000 birim): isteminde tamsayıyı tek satıra girin. Çıktının ondalık ayırıcısı noktadır. Geçersiz biçim veya aralık için Hata: Tüketim 0 ile 10000 arasında bir tamsayı olmalıdır. yazılır ve bedel hesaplanmaz.

Algoritma

1. Tüketimi okuyun; 0 ile 10000 arasında bir tamsayı olduğunu doğrulayın.
2. Tüketim en fazla 100 ise bedeli $tuketim * 2.00$ olarak hesaplayın.
3. Tüketim 100'den büyük ve en fazla 200 ise ilk kademenin bedeline, $tuketim - 100$ birimin 3.00 TL üzerinden bedelini ekleyin.
4. Tüketim 200'den büyükse ilk iki kademenin toplam bedeline, $tuketim - 200$ birimin 5.00 TL üzerinden bedelini ekleyin.
5. Bedeli iki ondalık basamakla ve TL birimiyle yazdırın.

Karar tablosu sınırların hangi dala ait olduğunu açıkça gösterir:

Tüketim aralığı	İlk kademe	İkinci kademe	Üçüncü kademe
$0 \leq tuketim \leq 100$	$tuketim * 2.00$	Yok	Yok
$100 < tuketim \leq 200$	$100 * 2.00$	$(tuketim - 100) * 3.00$	Yok
$200 < tuketim \leq 10000$	$100 * 2.00$	$100 * 3.00$	$(tuketim - 200) * 5.00$

Her satırdaki bedeller toplanır. İlk dal başarısız olduğunda tüketimin 100'den büyük olduğu zaten bilinir; ikinci dalda yalnızca $tuketim \leq 200$ kontrolü yeterlidir.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Tüketim (0..10000 birim): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int tüketim) || tüketim < 0 || tüketim > 10000)
{
    Console.WriteLine("Hata: Tüketim 0 ile 10000 arasında bir tamsayı olmalıdır.");
    return;
}

decimal bedel;
if (tüketim <= 100)
{
    bedel = tüketim * 2.00m;
}
else if (tüketim <= 200)
{
    bedel = 100 * 2.00m + (tüketim - 100) * 3.00m;
}
else
{
    bedel = 100 * 2.00m + 100 * 3.00m + (tüketim - 200) * 5.00m;
}

Console.WriteLine("Bedel: " + bedel.ToString("F2", CultureInfo.InvariantCulture) + " TL");
```

Ondalık sabitlerin sonundaki m, değerin decimal türünde olduğunu belirtir. Bu tür ondalık parasal hesaplamalar için uygundur. F2, sonuçtaki iki ondalık basamağın her çalıştırmada görünmesini sağlar.

Örnek çalıştırmalar

Tablo giriş isteminden sonra yazılan sonuç satırını gösterir.

Girdi	Sonuç	Açıklama
0	Bedel: 0.00 TL	Hiç tüketim yoktur.
100	Bedel: 200.00 TL	İlk kademenin son birimi.
101	Bedel: 203.00 TL	Yalnızca bir birim 3.00 TL ile ücretlenir.
150	Bedel: 350.00 TL	İlk kademe 200.00 TL, kalan 50 birim 150.00 TL.
200	Bedel: 500.00 TL	İkinci kademenin son birimi.
201	Bedel: 505.00 TL	Yalnızca bir birim 5.00 TL ile ücretlenir.
10000	Bedel: 49500.00 TL	İlk iki kademe 500.00 TL, kalan 9800 birim 49000.00 TL.
-1	Hata: Tüketim 0 ile 10000 arasında bir tamsayı olmalıdır.	Negatif tüketim reddedilir.
10001	Hata: Tüketim 0 ile 10000 arasında bir tamsayı olmalıdır.	Üst sınır aşılr.

Girdi	Sonuç	Açıklama
1.5	Hata: Tüketim 0 ile 10000 arasında bir tamsayı olmalıdır.	Bu model yalnızca tamsayı tüketim kabul eder.

Sınır durumları

- 100 ve 200 değerleri önceki kademelerin son birimleridir; sonraki fiyatlar 101 ve 201 ile başlar.
- Sıfır tüketimde sabit ücret bulunmadığından bedel sıfırdır.
- Tüketim arttığında önceki kademelerde hesaplanmış ücretler korunur.
- En yüksek bedel 49500.00 TL'dir ve decimal türüne rahatça sığar.
- Boş satır, metin ve tamsayı olmayan girdiler için hesaplama yapılmaz.

Kazanımlar

- Kademeli bir kuralı birbirini dışlayan aralıklara ayırma.
- Eşik değeri ve hemen sonraki değeri ayrı örneklerle sınıma.
- Her kademedede yalnızca o aralığa düşen miktarı ücretlendirme.
- decimal sabitleri ve F2 biçimini kullanarak parasal sonuç yazdırma.

Alıştırmalar

1. 99, 100 ve 101 birim için bedelleri hesaplayıp ardışık artışların neden değiştiğini açıklayın.
2. Üç kademenin bedelini ayrı değişkenlerde tutup toplamın yanında ayrı satırlarda gösterin.
3. Üst sınırı koruyarak 500 birimden sonraki miktara 7.00 TL uygulayan dördüncü kademe ekleyin.

018 Saate saniye ekleme

Düzey: Orta Önkoşullar: 006, 008

Kavramlar: Tamsayı bölmesi, Kalan işleci, Birim dönüşümü, Gün taşması, Sabit genişlikte biçimleme

Problem tanımı

Saat, dakika ve saniye olarak verilen bir saate belirli sayıda saniye ekleyin. Yeni saati ve başlangıç gününden kaç gün sonrasına geçildiğini bulun. Bir gün 24 saat, bir saat 60 dakika, bir dakika 60 saniye kabul edilir. Tarih, saat dilimi, yaz saati ve artık saniye kuralları bu modelin dışında tutulur.

Örneğin 23:59:50 saatine 20 saniye eklenirse bir sonraki günün 00:00:10 saatine ulaşılır. Gündeki saat bir noktayı, eklenen saniye ise bir süreyi belirtir. Bütün değerleri önce saniyeye çevirmek, dakika ve saat taşmalarını tek hesapta yönetmeyi sağlar.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	saat	0 ile 23 arasında int.
Girdi	dakika	0 ile 59 arasında int.
Girdi	saniye	0 ile 59 arasında int.
Girdi	eklenen	0 ile 172800 arasında eklenecek saniye sayısı; int.
Çıktı	gun	Başlangıç gününün sonrasına geçilen gün sayısı; Geçen gün: N.
Çıktı	Yeni saat	Yeni saat: HH:mm:ss biçiminde iki basamaklı saat, dakika ve saniye.

Sırasıyla Saat (0..23): , Dakika (0..59): , Saniye (0..59): ve Eklenecek saniye (0..172800): istemlerine her tamsayıyı ayrı satırda girin. Başlangıç günü 0 kabul edilir. Örneğin 1 gün geçmesi, başlangıç gününün ertesi gününe ulaşılmasıdır; mutlaka 24 saatlik bir sürenin eklenmiş olması değildir.

Algoritma

1. Saat, dakika, saniye ve eklenecek süreyi sırayla okuyup aralıklarını doğrulayın.
2. Başlangıç saatini $\text{saat} * 3600 + \text{dakika} * 60 + \text{saniye}$ ile saniyeye çevirin.
3. Eklenecek saniyeyi bu değere ekleyin.
4. Toplamı 86400'e tamsayı bölerek geçilen gün sayısını bulun.
5. Toplamın 86400'e bölümünden kalanı alarak yeni günün içindeki saniye sayısını bulun.
6. Gün içindeki saniyeyi 3600'e bölerek yeni saati bulun.
7. Gün içindeki saniyenin 3600'e bölümünden kalanını 60'a bölerek yeni dakikayı bulun.
8. Gün içindeki saniyenin 60'a bölümünden kalanını alarak yeni saniyeyi bulun.
9. Gün sayısını ve her alanı iki basamaklı yeni saati yazdırın.

Pozitif tamsayılar için / bölümün tamsayı kısmını, % kalanını verir. $\text{toplam} = \text{gun} * 86400 + \text{gunIci}$ eşitliği, tam günlerle yeni gün içindeki saati birbirinden ayırır. gunIci her zaman 0 ile 86399 arasındadır.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Saat (0..23): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int saat) || saat < 0 || saat > 23)
{
    Console.WriteLine("Hata: Saat 0 ile 23 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Dakika (0..59): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int dakika) || dakika < 0 || dakika > 59)
{
    Console.WriteLine("Hata: Dakika 0 ile 59 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Saniye (0..59): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int saniye) || saniye < 0 || saniye > 59)
{
    Console.WriteLine("Hata: Saniye 0 ile 59 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Eklenecek saniye (0..172800): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int eklenen) || eklenen < 0 || eklenen > 172800)
{
    Console.WriteLine("Hata: Eklenecek saniye 0 ile 172800 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int toplam = saat * 3600 + dakika * 60 + saniye + eklenen;
int gun = toplam / 86400;
int gunIci = toplam % 86400;
int yeniSaat = gunIci / 3600;
int yeniDakika = (gunIci % 3600) / 60;
int yeniSaniye = gunIci % 60;

Console.WriteLine("Geçen gün: " + gun.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Yeni saat: " +
    yeniSaat.ToString("D2", CultureInfo.InvariantCulture) + ":" +
    yeniDakika.ToString("D2", CultureInfo.InvariantCulture) + ":" +
    yeniSaniye.ToString("D2", CultureInfo.InvariantCulture));
```

D2, tamsayının en az iki basamakla yazılmasını sağlar; örneğin 7 değeri 07 olur. Hesaplama tamsayı aritmetiğiyle yapılır; saat parçalarını bulmak için yuvarlama veya ondalık tür gerekmez.

Örnek çalıştırmalar

Girdi sütununda saat, dakika, saniye ve eklenecek saniye sırasıyla verilmiştir. Sonuç sütunu iki çıktı satırını gösterir.

Girdi	Sonuç	Açıklama
12, 30, 0, 90	Geçen gün: 0 Yeni saat: 12:31:30	Aynı gün içinde dakika taşması.
23, 59, 50, 20	Geçen gün: 1 Yeni saat: 00:00:10	Gece yarısı aşılar.
0, 0, 0, 0	Geçen gün: 0 Yeni saat: 00:00:00	Alt sınırlar ve sıfır süre.
23, 59, 59, 1	Geçen gün: 1 Yeni saat: 00:00:00	Tam gece yarısına ulaşılır.
7, 8, 9, 86400	Geçen gün: 1 Yeni saat: 07:08:09	Tam bir gün eklenir, saat aynı kalır.
23, 59, 59, 172800	Geçen gün: 2 Yeni saat: 23:59:59	En büyük geçerli toplam.
İlk girdi 24	Hata: Saat 0 ile 23 arasında bir tamsayı olmalıdır.	Sonraki girdiler istenmez.
0, 60	Hata: Dakika 0 ile 59 arasında bir tamsayı olmalıdır.	Doğrulama dakika adımında durur.
0, 0, 0, -1	Hata: Eklenecek saniye 0 ile 172800 arasında bir tamsayı olmalıdır.	Negatif süre reddedilir.
İlk girdi sabah	Hata: Saat 0 ile 23 arasında bir tamsayı olmalıdır.	Metin girişi reddedilir.

Sınır durumları

- Saat 24 olamaz; tam gece yarısı 00:00:00 ve bir sonraki gün olarak gösterilir.
- Sıfır saniye eklenmesi başlangıç saatini ve günü değiştirmez.
- En büyük toplam $86399 + 172800 = 259199$ saniyedir; int aralığında kalır.
- Gün sayısı bu aralıkta 0, 1 veya 2 olur; örneğin gece yarısına yakın başlangıç, kısa ek sürede bile günü değiştirebilir.
- Negatif süre kabul edilmediği için negatif kalanların yorumlanması gerekmez.
- Geçersiz alan görüldüğünde sonraki alanlar okunmaz ve sonuç hesaplanmaz.

Kazanımlar

- Farklı zaman birimlerini ortak bir tamsayı birimine dönüştürme.
- Tamsayı bölmesi ve kalan işleciyle gün, saat, dakika ve saniyeyi ayırma.
- Bir değer tam katlarını ve kalan kısmını birlikte yorumlama.
- D2 biçimini kullanarak sabit genişlikli saat yazdırma.

Alıřtırmalar

1. 01:59:59 saatine bir saniye eklenirken yeni saatin 02:00:00 oluřmasını ara deęerlerle gsterin.
2. Bařlangı saatini deęiřtirmeden 86399, 86400 ve 86401 saniye iin  sonucu karřılařtıřın.
3. Yeni gn iindeki saniyeyi ıktı olarak ekleyip toplam = gun * 86400 + gunIci eřitlięini doęrulayın.

019 Bir ayın gün sayısını bulma

Düzey: Orta Önkoşullar: 014, 015

Kavramlar: Switch deyimi, Gruplanmış case dalları, Artık yıl, Mantıksal ifade, Takvim kuralları

Problem tanımı

Yıl ve ay numarasını kullanarak o ayın kaç gün sürdüğünü bulun. Ocak, mart, mayıs, temmuz, ağustos, ekim ve aralık 31 gün; nisan, haziran, eylül ve kasım 30 gündür. Şubat normal yılda 28, artık yılda 29 gün sürer.

Bu çalışmada Gregoryen takvimin hesap kuralları 1 ile 9999 arasındaki bütün yıllara aynı biçimde uygulanır; 1582 öncesindeki yıllar da aynı hesap modeline dahildir. Tarihsel takvim geçişleri ve bölgesel uygulamalar dikkate alınmaz. Bir yıl 400'e tam bölünüyorsa artık yıldır. 400'e bölünmüyorsa, 4'e bölünüp 100'e bölünmeyen yıllar artık yıldır. Bu nedenle 2000 artık yıl, 1900 normal yıldır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	yıl	1 ile 9999 arasında int.
Girdi	ay	1 ile 12 arasında int; 1 ocak, 2 şubat, 12 aralık.
Çıktı	gunSayisi	Gün sayısı: N biçiminde 28, 29, 30 veya 31.

Önce Yıl (1..9999): , sonra Ay (1..12): istemine birer tamsayı girin. Geçersiz biçimde veya aralıkta bir değer görülürse ilgili alanın hata mesajı yazılır ve program sona erer. Ondalık veya basamak ayırıcı kullanmayın.

Algoritma

1. Yılı okuyun ve 1 ile 9999 arasında bir tamsayı olduğunu doğrulayın.
2. Ayı okuyun ve 1 ile 12 arasında bir tamsayı olduğunu doğrulayın.
3. $yil \% 400 == 0 \ || \ (yil \% 4 == 0 \ \&\& \ yil \% 100 != 0)$ ifadesiyle artık yıl durumunu hesaplayın.
4. switch deyiminde ayı seçin; 31 gün süren ayları aynı dalda, 30 gün süren ayları başka bir dalda gruplayın.
5. Şubat dalında artık yıl doğruysa 29, değilse 28 gün atayın.
6. Gün sayısını yazdırın.

switch, bir değer farklı sabitlerle eşleşmesini düzenler. Arka arkaya yazılan case etiketleri aynı işlem grubunu kullanabilir. Böylece yedi ayrı 31 gün ataması yerine tek atama yeterli olur. Artık yıl kuralı yalnızca şubatın gün sayısını değiştirir.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Yıl (1..9999): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int yil) || yil < 1 || yil > 9999)
{
    Console.WriteLine("Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Ay (1..12): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ay) || ay < 1 || ay > 12)
{
    Console.WriteLine("Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.");
    return;
}

bool artikYil = yil % 400 == 0 || (yil % 4 == 0 && yil % 100 != 0);
int gunSayisi;
switch (ay)
{
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        gunSayisi = 31;
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        gunSayisi = 30;
        break;
    case 2:
        if (artikYil)
        {
            gunSayisi = 29;
        }
        else
        {
            gunSayisi = 28;
        }
        break;
    default:
        Console.WriteLine("Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.");
        return;
}

Console.WriteLine("Gün sayısı: " + gunSayisi.ToString(CultureInfo.InvariantCulture));
```

Her dalın sonundaki break, switch deyiminden çıkışı sağlar. default geçerli aralıktaki girdiler için çalışmaz; ancak hiçbir ay etiketiyle eşleşmeyen bir durumda programın gün sayısı yazdırmasını engeller. Şubat dalı, seçilmiş bir durumun içinde ek koşul kurulabileceğini gösterir.

Örnek çalıştırmalar

Girdiler yıl ve ay sırasındadır; sonuç sütunu giriş istemlerinden sonraki satırı gösterir.

Girdi	Sonuç	Açıklama
2025, 1	Gün sayısı: 31	Ocak yılın artık olmasından etkilenmez.
2025, 4	Gün sayısı: 30	Nisan 30 gün sürer.
2024, 2	Gün sayısı: 29	Dörde bölünen, yüze bölünmeyen yıl.
2025, 2	Gün sayısı: 28	Normal yılın şubatı.
1900, 2	Gün sayısı: 28	Yüze bölünür, dört yüze bölünmez.
2000, 2	Gün sayısı: 29	Dört yüze bölünür.
1, 2	Gün sayısı: 28	Modelin alt yıl sınırı.
9999, 12	Gün sayısı: 31	Üst yıl ve ay sınırı.
İlk girdi 0	Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.	Ay istenmez.
2024, 13	Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.	Geçersiz ay.
2024, şubat	Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.	Ayın adı yerine numarası beklenir.

Sınır durumları

- Yıl sıfır kabul edilmez; kabul edilen yıl aralığı 1 ile başlar.
- Ay numarası 0 veya 13 olamaz.
- Yalnızca 4'e bölünebilirlik kontrolü, 1900 gibi yüzyıl yıllarında yanlış sonuç üretir.
- Yalnızca 100'e bölünebilirlik kontrolü de 2000 gibi 400'e bölünen artık yılları kaçıır.
- Her geçerli ay için gün sayısı mutlaka atanır; şubat dışındaki aylarda yıl değeri sonucu değiştirmez.
- Boş satır, metin veya ondalıklı girdi için gün sayısı hesaplanmaz.

Kazanımlar

- Bir tamsayı değerini switch ile sabit durumlara ayırma.
- Aynı sonucu üreten ayları tek işlem dalında toplama.
- &&, || ve parantezlerle artık yıl kuralını tam olarak ifade etme.
- 1900 ve 2000 örnekleriyle genel kuralın istisnalarını sınama.

Alıştırmalar

1. Aynı işlemi yalnızca if / else if / else kullanarak yazıp iki çözümün okunabilirliğini karşılaştırın.
2. 2100 ve 2400 yıllarının şubat gün sayılarını bulun ve artık yıl ifadesindeki her karşılaştırmayı açıklayın.
3. İkinci bir switch ekleyip gün sayısının yanında ayın Türkçe adını da yazdırın.

020 Bir tarihin ertesi gününü bulma

Düzey: Orta Önkoşullar: 018, 019

Kavramlar: İç içe doğrulama, Ay ve yıl taşması, Durum güncelleme, Takvim kuralları, Karar ağacı

Problem tanımı

Yıl, ay ve gün olarak girilen geçerli bir tarihin ertesi gününü hesaplayın. Ayın ortasında yalnızca gün artar; ayın son gününde gün 1'e döner ve ay artar; aralık sonundaysa gün ve ay 1'e döner, yıl artar. Şubatın son gününü belirlerken artık yıl kuralını kullanın.

Gregoryen takvimin hesap kuralları 1 ile 9999 arasındaki bütün yıllara uygulanır. 1582 öncesi yıllar için de aynı model kullanılır; tarihsel takvim geçişleri ele alınmaz. Artık yıl, 400'e bölünen ya da 4'e bölünüp 100'e bölünmeyen yıldır. DateTime gibi hazır tarih araçları kullanmadan kuralları koşullar ve tamsayı işlemleriyle uygulayın.

Hem girdi hem çıktı 1 ile 9999 arasındaki yılları kullanmalıdır. Bu nedenle 9999-12-31 geçerli bir tarih olsa da ertesi günü bu çalışmanın çıktı aralığına sığmaz; program açık bir aralık mesajı verir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	yıl	1 ile 9999 arasında int.
Girdi	ay	1 ile 12 arasında int.
Girdi	gun	Önce 1 ile 31 arasında kontrol edilen, sonra ilgili ayın gün sayısıyla doğrulanmış int.
Çıktı	Ertesi gün	Ertesi gün: YYYY-MM-DD biçiminde dört basamaklı yıl, iki basamaklı ay ve gün.

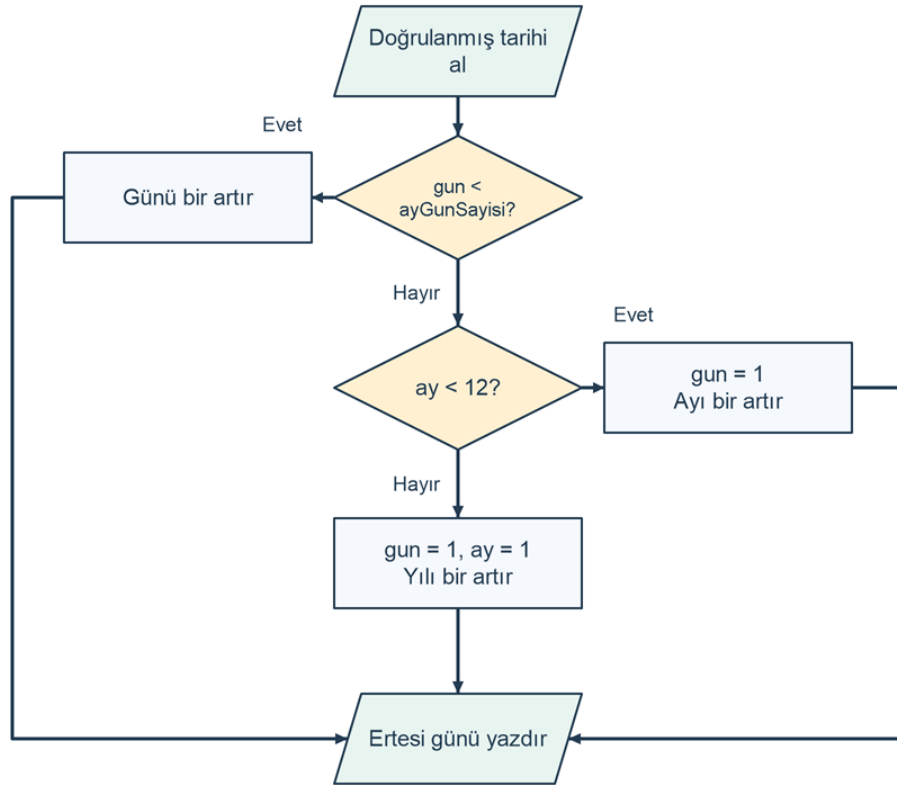
Yıl (1..9999): , Ay (1..12): ve Gün (1..31): istemlerine sırasıyla ayrı satırlarda tamsayı girin. Alan biçimi veya aralığı yanlışsa ilgili hata satırı yazılır. Ayda bulunmayan bir gün için Hata: Girilen ayda bu gün yoktur.; üst çıktı sınırı için Hata: Ertesi gün 1..9999 yıl aralığının dışındadır. yazılır. Bu durumlarda tarih çıktısı verilmez.

Algoritma

1. Yılı, ayı ve günü sırayla okuyun; temel aralıklarını doğrulayın.
2. Artık yıl durumunu hesaplayın.
3. Ayı switch ile seçerek 31, 30 veya şubat için 28/29 değerini ayGunSayisi değişkenine atayın.
4. Gün bu ayın gün sayısından büyükse geçersiz tarih mesajı yazıp bitirin.
5. Tarih 9999-12-31 ise ertesi gün çıktı aralığının dışına çıkacağı için hata yazıp bitirin.
6. Gün ayın son gününden küçükse yalnızca günü bir artırın.
7. Ayın son günündeyseniz ve ay 12'den küçükse günü 1 yapıp ayı bir artırın.
8. Ayın son günündeyseniz ve ay 12 ise günü ve ayı 1 yapıp yılı bir artırın.
9. Güncellenmiş tarihi dört ve iki basamaklı alanlarla yazdırın.

Bir tarihin alanlarını ayrı ayrı doğrulamak yeterli değildir: 2025, 2, 31 değerlerinin her biri temel aralığındadır, ancak bir arada geçerli bir tarih oluşturmaz. Önce ilgili ayın uzunluğu bulunmalı, sonra gün bu uzunlukla karşılaştırılmalıdır.

Şema, girdi doğrulaması ve üst sınır kontrolü tamamlandıktan sonraki üç güncelleme yolunu gösterir.



Şekil 5 Geçerli bir tarihte gün, ay ve yıl taşması

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Yıl (1..9999): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int yıl) || yıl < 1 || yıl > 9999)
{
    Console.WriteLine("Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Ay (1..12): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ay) || ay < 1 || ay > 12)
{
    Console.WriteLine("Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.");
    return;
}
```

```

Console.Write("Gün (1..31): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int gun) || gun < 1 || gun > 31)
{
    Console.WriteLine("Hata: Gün 1 ile 31 arasında bir tamsayı olmalıdır.");
    return;
}

bool artikYil = yil % 400 == 0 || (yil % 4 == 0 && yil % 100 != 0);
int ayGunSayisi;
switch (ay)
{
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        ayGunSayisi = 31;
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        ayGunSayisi = 30;
        break;
    case 2:
        if (artikYil)
        {
            ayGunSayisi = 29;
        }
        else
        {
            ayGunSayisi = 28;
        }
        break;
    default:
        Console.WriteLine("Hata: Ay 1 ile 12 arasında bir tamsayı olmalıdır.");
        return;
}

if (gun > ayGunSayisi)
{
    Console.WriteLine("Hata: Girilen ayda bu gün yoktur.");
    return;
}

if (yil == 9999 && ay == 12 && gun == 31)
{
    Console.WriteLine("Hata: Ertesi gün 1..9999 yıl aralığının dışındadır.");
    return;
}

```

```

if (gun < ayGunSayisi)
{
    gun++;
}
else if (ay < 12)
{
    gun = 1;
    ay++;
}
else
{
    gun = 1;
    ay = 1;
    yil++;
}

Console.WriteLine("Ertesi gün: " + yil.ToString("D4", CultureInfo.InvariantCulture) + "-" +
    ay.ToString("D2", CultureInfo.InvariantCulture) + "-" +
    gun.ToString("D2", CultureInfo.InvariantCulture));

```

gun++, ay++ ve yil++ ilgili tamsayıyı bir artırır. Koşul zinciri güncelleme yollarından yalnızca birini çalıştırır. Bir ayın son gününden sonra önce günün 1'e dönmesi gerekir; yalnızca gun++ kullanmak 32. gün gibi geçersiz bir sonuç üretebilir. Tarih araçlarına başvurmadan tüm kurallar görünür biçimde uygulanır.

Örnek çalıştırmalar

Girdiler yıl, ay ve gün sırasındadır; sonuç sütunu giriş istemlerinden sonraki satırı gösterir.

Girdi	Sonuç	Açıklama
2025, 3, 14	Ertesi gün: 2025-03-15	Ayın içinde normal gün artışı.
2025, 4, 30	Ertesi gün: 2025-05-01	30 günlük ayın sonu.
2025, 1, 31	Ertesi gün: 2025-02-01	31 günlük ayın sonu.
2024, 2, 28	Ertesi gün: 2024-02-29	Artık yılda şubatın içinde kalınır.
2024, 2, 29	Ertesi gün: 2024-03-01	Artık yılın şubat sonu.
1900, 2, 28	Ertesi gün: 1900-03-01	1900 normal yıldır.
2000, 2, 28	Ertesi gün: 2000-02-29	2000 artık yıldır.
2025, 12, 31	Ertesi gün: 2026-01-01	Yıl değişir.
1, 1, 1	Ertesi gün: 0001-01-02	En küçük tarih, dört basamaklı yıl.
9999, 12, 30	Ertesi gün: 9999-12-31	En büyük üretilebilir sonuç.
9999, 12, 31	Hata: Ertesi gün 1..9999 yıl aralığının dışındadır.	Girdi geçerli, çıktı aralık dışıdır.
2025, 2, 29	Hata: Girilen ayda bu gün yoktur.	Normal yılda 29 şubat yoktur.
2025, 4, 31	Hata: Girilen ayda bu gün yoktur.	Nisanın 31. günü yoktur.
2025, 1, 0	Hata: Gün 1 ile 31 arasında bir tamsayı olmalıdır.	Temel gün aralığı geçersizdir.

Girdi	Sonuç	Açıklama
İlk girdi yıl	Hata: Yıl 1 ile 9999 arasında bir tamsayı olmalıdır.	Metin girişi reddedilir.

Sınır durumları

- 28, 29, 30 ve 31 günlük ayların sonları farklı olsa da aynı güncelleme koşulları kullanılabilir.
- Gün, ay uzunluğuyla karşılaştırılmadan tarih geçerli kabul edilmez.
- 1900 ve 2000 yılları, şubatın sonunu belirleyen yüzyıl istisnalarını sınar.
- Ayın son gününde ay < 12 doğruysa yıl değişmez; aralık sonunda yıl da artar.
- 9999-12-31 için aralık kontrolü yıl artırılmadan önce yapılır; 10000 yılı yazdırılmaz.
- Her geçersiz girişte veya aralık dışı sonuçta tek hata satırı yazılır ve tarih çıktısı verilmez.

Kazanımlar

- Birden çok alanın tek bir geçerli durum oluşturduğunu birlikte doğrulama.
- Önce ay uzunluğunu hesaplayıp daha sonra gün doğrulamasında kullanma.
- Normal gün, ay sonu ve yıl sonu yollarını birbirini dışlayan dallarla yönetme.
- Geçerli girdi ile kabul edilen aralığa sığmayan çıktı durumunu ayırt etme.
- Güncellenen tarih parçalarını D4 ve D2 biçimleriyle tutarlı yazdırma.

Alıştırmalar

1. 2100-02-28 ve 2400-02-28 için ertesi günü hesaplayıp artık yıl koşullarını adım adım açıklayın.
2. 2025-11-30, 2025-12-31 ve 2026-01-01 girdilerinin üç farklı geçişte hangi dalları çalıştırdığını gösterin.
3. Geçerli bir tarihin bir önceki gününü bulan çözümü yazın; 0001-01-01 için çıktı sınırı davranışını ayrıca belirleyin.

Bölüm 3: Döngüler ve sayı algoritmaları

021 Bitiş değeriyle sayıların toplamını ve ortalamasını bulma

Düzy: Orta Önkoşullar: 008, 010, 011

Kavramlar: while döngüsü, Bitiş değeri, Sayaç, Biriktirme, Boş veri, Döngü sınırı

Problem tanımı

Kaç sayı girileceği önceden belli olmayan bir veri dizisinin toplamını ve aritmetik ortalamasını bulun. Her sayı -1000000 ile 1000000 arasında bir tamsayıdır. Kullanıcı 0 girdiğinde veri girişi biter. Bu problemde 0 yalnızca bitiş işaretidir; toplama ve sayaca eklenmez. Program ayrıca en fazla 1000 sıfır dışı sayı kabul eder. Bu sayıya ulaşınca başka girdi istemeden sonucu yazdırır.

Örneğin 12, -3, 6 ve 0 girildiğinde üç sayı işlenir, toplam 15 ve ortalama 5.00 olur. İlk girdi 0 ise veri yoktur; ortalama hesaplanmaz. Geçersiz bir giriş veya bitiş değerinden önce girişin sona ermesi halinde hata yazılır ve önceki sayıların kısmi sonucu verilmez. Böylece eksik bir giriş dizisi tamamlanmış veri gibi gösterilmez.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayı	-1000000 ile 1000000 arasında int; her satırda bir sayı.
Kontrol	0	Veri girişi için bitiş değeri; veri sayılmaz.
Çıktı	adet	Kabul edilen sıfır dışı sayı sayısı; Adet: N.
Çıktı	toplam	long türünde toplam; Toplam: N.
Çıktı	ortalama	double türünde ortalama; Ortalama: F2.

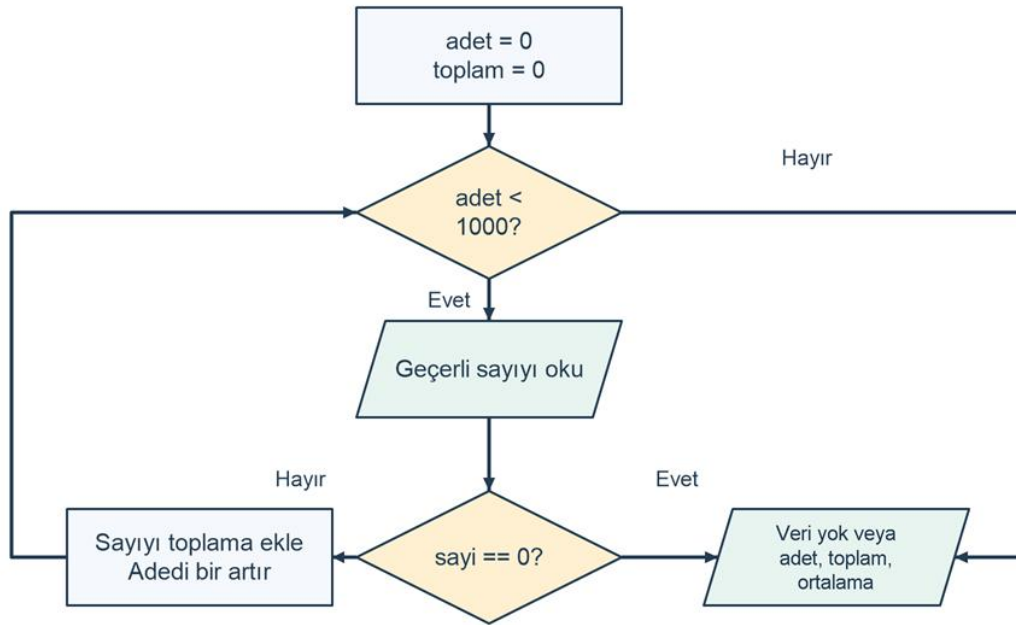
Her okuma için Sayı (-1000000..1000000, bitirmek için 0): istemi yazılır. En fazla 1000 sıfır dışı sayı okunur. Hiç veri girilmezse yalnızca Sonuç: Veri girilmedi. yazılır. Hatalı biçim, aralık dışı sayı, boş satır veya erken giriş sonu için Hata: -1000000 ile 1000000 arasında bir tamsayı giriniz. yazılır. Sonuçlarda ondalık ayırıcı nokta, ortalama iki ondalık basamak kullanılır.

Algoritma

1. adet değerini 0, toplam değerini 0 olarak başlatın.
2. adet < 1000 olduğu sürece döngüye girin.
3. Bir sayı okuyun; biçimi veya aralığı geçersizse hata yazıp programı bitirin.
4. Sayı 0 ise break ile veri giriş döngüsünden çıkın.
5. Sıfır dışı sayıyı toplama ekleyin ve adet değerini bir artırın.
6. Döngü bitince adet == 0 ise veri yok mesajı yazıp bitirin.
7. Toplamı double türüne çevirerek adet değerine bölün.
8. Adedi, toplamı ve ortalamayı yazdırın.

Döngü başında adet, o ana kadar kabul edilen sıfır dışı sayıların sayısını; toplam, aynı sayıların toplamını temsil eder. Yeni sayı eklendiğinde iki değişken birlikte güncellenir. Bu ilişki her yinelemeye korunur. Bitiş değeri kontrolü güncellemeden önce yapıldığı için 0 hiçbir zaman veri sayılmaz. while koşulu ise 1001. sayının istenmesini önler.

Şema yalnızca geçerli girişlerin akışını gösterir. Her okumadan sonra kodda uygulanan biçim ve aralık doğrulaması, şemayı sade tutmak için gösterilmemiştir. Son çıktı adımı, adet sıfırsa veri yok mesajını; aksi halde üç sonuç satırını seçer.



Şekil 6 Bitiş değeri ve en fazla 1000 veri ile giriş döngüsü

C# çözümü

```
using System;
using System.Globalization;

int adet = 0;
long toplam = 0;

while (adet < 1000)
{
    Console.WriteLine("Sayı (-1000000..1000000, bitirmek için 0): ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int sayi) ||
        sayi < -1000000 || sayi > 1000000)
    {
        Console.WriteLine("Hata: -1000000 ile 1000000 arasında bir tamsayı giriniz.");
        return;
    }

    if (sayi == 0)
    {
        break;
    }

    toplam += sayi;
    adet++;
}

if (adet == 0)
{
    Console.WriteLine("Sonuç: Veri girilmedi.");
    return;
}

double ortalama = (double)toplam / adet;
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Ortalama: " + ortalama.ToString("F2", CultureInfo.InvariantCulture));
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. break yalnızca döngüyü bitirir; ardından sonuç hesaplanır. return ise tüm programı bitirdiği için geçersiz girişte kısmi sonuç yazılmaz. (double)toplam dönüşümü tamsayı bölmesini önler. Örneğin toplam 1 ve adet 2 olduğunda ortalama 0.50 olur.

Örnek çalıştırmalar

Girdi sütunundaki değerler ayrı satırlarda girilir. Sonuç sütunu giriş istemleri dışındaki tüm çıktı satırlarını gösterir. "1000 kez" ifadesi, ilgili değer 1000 ayrı satırda verilmesini belirtir.

Girdi	Sonuç	Açıklama
12, -3, 6, 0	Adet: 3 Toplam: 15 Ortalama: 5.00	Bitiş değeri adede eklenmez.
0	Sonuç: Veri girilmedi.	İlk girişte boş veri.
1, 0	Adet: 1 Toplam: 1 Ortalama: 1.00	Tek veri.

Girdi	Sonuç	Açıklama
1, -1, 0	Adet: 2 Toplam: 0 Ortalama: 0.00	Toplamın sıfır olması veri yok anlamına gelmez.
2, -1, 0	Adet: 2 Toplam: 1 Ortalama: 0.50	Ondalıklı ortalama.
-1000000, 1000000, 0	Adet: 2 Toplam: 0 Ortalama: 0.00	İki geçerli uç değer.
1000 kez 1000000	Adet: 1000 Toplam: 1000000000 Ortalama: 1000000.00	Üst adet sınırında kendiliğinden biter.
1000 kez -1000000	Adet: 1000 Toplam: -1000000000 Ortalama: -1000000.00	En küçük toplam.
5, 1000001	Hata: -1000000 ile 1000000 arasında bir tamsayı giriniz.	Daha önceki 5 için sonuç verilmez.
İlk girdi iki	Hata: -1000000 ile 1000000 arasında bir tamsayı giriniz.	Metin reddedilir.
5, sonra girişin sonu	Hata: -1000000 ile 1000000 arasında bir tamsayı giriniz.	0 gelmeden eksik kalan giriş reddedilir.

Sınır durumları

- İlk sayı 0 ise adet sıfır kalır; sıfıra bölme yapılmaz.
- Pozitif ve negatif sayıların toplamı sıfır olabilir; boş veri kontrolünde toplam yerine adet kullanılır.
- Bitiş değeri nedeniyle 0 bu çalışmada ölçüm değeri olarak girilemez. Sıfırın veri olması gereken bir tasarım başka bitiş yöntemi kullanmalıdır.
1000. sıfır dışı sayı eklendiğinde döngü koşulu yanlış olur. 1001. satır ve ayrıca bitiş değeri okunmaz.
- Toplam -1000000000 ile 1000000000 arasında kalır. Bu sınır int içine de sığsa da toplam için long kullanılması, biriktirme değişkeninin aralığını açıkça ayırır.
- Hatalı giriş, boş satır veya erken giriş sonu, sonuç hesaplamasından önce programı bitirir.

Kazanımlar

- Tekrar sayısı önceden bilinmeyen girişleri while döngüsüyle işleme.
- Bitiş değerini gerçek veriden ayırıp güncellemeden önce denetleme.
- Sayaç ve toplamın her yinelemede aynı veri kümesini temsil etmesini sağlama.
- Boş veri için bölme öncesinde kontrol yapma.
- Döngüden çıkış ile programdan çıkışın sonuç üzerindeki farkını açıklama.

Alıřtırmalar

1. Toplam ve ortalamaya ek olarak pozitif ve negatif sayı adetlerini iki ayrı sayaçla yazdırın. Bitiş deęerini hiçbir sayaca katmayın.
2. Giriş sınırını 1000'den 2000'e çıkarın. Toplamın olası alt ve üst sınırlarını hesaplayıp tür seçimini yeniden deęerlendirin.
3. Geçersiz satırda programı bitirmek yerine aynı sıradaki veriyi yeniden isteyen bir sürüm yazın. Giriş sonu için döngünün nasıl biteceğini ayrıca belirleyin.

022 Basamak sayısını ve basamak toplamını bulma

Düzey: Orta Önkoşullar: 006, 018, 021

Kavramlar: do / while döngüsü, Tamsayı bölmesi, Kalan işleci, Basamak ayırma, Döngü değişmezi

Problem tanımı

0 ile 2147483647 arasında girilen bir tamsayının onluk yazımındaki basamak sayısını ve basamaklarının toplamını bulun. Sayıyı metne çevirerek karakterlerini dolaşmayın; her adımda % 10 ile son basamağı alın ve / 10 ile bu basamağı sayıdan çıkarın.

Örneğin 407 üç basamaklıdır ve basamak toplamı $4 + 0 + 7 = 11$ 'dir. Sıfırın onluk yazımı tek basamaklıdır; basamak toplamı 0'dır. Giriş sayısal olarak yorumlanır: 00407 yazılırsa 407 değeri işlenir ve sonuç üç basamaktır. Negatif sayılar kabul edilmez; böylece eksi işareti ve negatif kalanların yorumu bu problemde yer almaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	0 ile 2147483647 arasında int.
Ara değer	kalanSayi	Basamakları çıkarıldıkça küçülen kopya.
Çıktı	basamakSayisi	Onluk yazımın basamak adedi; Basamak sayısı: N.
Çıktı	basamakToplami	Basamak değerlerinin toplamı; Basamak toplamı: N.

Sayı (0..2147483647): istemine bir tamsayı girin. Biçimi veya aralığı yanlışsa Hata: 0 ile 2147483647 arasında bir tamsayı giriniz. yazılır ve sonuç verilmez. Sıfır için basamak sayısı 1 olarak kabul edilir.

Algoritma

1. Sayıyı okuyup biçimini ve aralığını doğrulayın.
2. Sayıyı kalanSayi değişkenine kopyalayın; sayaç ve toplamı 0 yapın.
3. do gövdesinde kalanSayi % 10 ile son basamağı bulun.
4. Bu basamağı toplama ekleyin ve basamak sayısını bir artırın.
5. kalanSayi /= 10 ile işlenen son basamağı çıkarın.
6. kalanSayi > 0 olduğu sürece 3. adımdan devam edin.
7. Basamak sayısını ve toplamını yazdırın.

do / while, koşulu gövdeden sonra denetler; gövde en az bir kez çalışır. Bu nedenle 0 için de bir basamak işlenir. Her yinleme sonunda sayaç, çıkarılmış basamakların adedidir; toplam ise aynı basamakların toplamıdır. Kalan sayı, henüz işlenmemiş soldaki basamakları temsil eder. Pozitif bir kalan sayı 10'a tamsayı bölününce küçülür; en fazla 10 yinelemede sifıra ulaşılır.

407 için ara değerler aşağıdaki gibidir. Tablo her yinelemenin sonundaki sayaç ve toplamı gösterir.

Yineleme	İşlem öncesi kalan sayı	Alınan basamak	Yeni kalan sayı	Basamak sayısı	Basamak toplamı
1	407	7	40	1	7
2	40	0	4	2	7

Yineleme	İşlem öncesi kalan sayı	Alınan basamak	Yeni kalan sayı	Basamak sayısı	Basamak toplamı
3	4	4	0	3	11

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi) || sayi < 0)
{
    Console.WriteLine("Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.");
    return;
}

int kalanSayi = sayi;
int basamakSayisi = 0;
int basamakToplami = 0;

do
{
    int basamak = kalanSayi % 10;
    basamakToplami += basamak;
    basamakSayisi++;
    kalanSayi /= 10;
}
while (kalanSayi > 0);

Console.WriteLine("Basamak sayısı: " +
    basamakSayisi.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Basamak toplamı: " +
    basamakToplami.ToString(CultureInfo.InvariantCulture));
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. `int.TryParse` üst tür sınırını aşan girdiyi reddeder. `/=` değişkeni bölümün tamsayı sonucuyla günceller. Örneğin $407 / 10$ işlemi 40 olur; $\% 10$ ise 7 verir. Asıl sayı değiştirilmediği için daha sonra başlangıç değeri gerektiğinde kullanılabilir.

Örnek çalıştırmalar

Sonuç sütunu giriş isteminden sonraki iki satırı veya hata satırını tam olarak gösterir.

Girdi	Sonuç	Açıklama
407	Basamak sayısı: 3 Basamak toplamı: 11	Aradaki sıfır da bir basamaktır.
0	Basamak sayısı: 1 Basamak toplamı: 0	Gövde en az bir kez çalışır.
7	Basamak sayısı: 1 Basamak toplamı: 7	Tek basamak.
10	Basamak sayısı: 2 Basamak toplamı: 1	Sondaki sıfır sayaca katılır.

Girdi	Sonuç	Açıklama
1000	Basamak sayısı: 4 Basamak toplamı: 1	Birden çok sıfır basamak.
999	Basamak sayısı: 3 Basamak toplamı: 27	Tekrarlanan basamaklar.
00407	Basamak sayısı: 3 Basamak toplamı: 11	Başta yazılan sıfırlar sayısal değeri değiştirmez.
2147483647	Basamak sayısı: 10 Basamak toplamı: 46	En büyük geçerli girdi.
-1	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Negatif sayı reddedilir.
2147483648	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	int aralığı aşılr.
4.5	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Ondalıklı yazım kabul edilmez.

Sınır durumları

- Başlangıç değeri 0 olsa da do / while bir yineleme yapar. Yalnızca başta koşul denetleyen bir while, ek bir sıfır kuralı olmadan yanlışlıkla sıfır basamak bulurdu.
- Sayının içindeki veya sonundaki sıfırlar toplamı değiştirmez; basamak sayısını artırır.
- Girdi en fazla 10 basamaklıdır. Her basamak en fazla 9 olduğu için basamak toplamı en fazla 90 olabilecek bir üst sınırla güvence altındadır; her iki sonuç int içinde kalır.
- kalanSayı sıfıra ulaştığında işlem biter; başlangıç sayısı korunur.
- Negatif, aralık dışı, metin, boş satır ve giriş sonu için hata yazılır; basamak sonuçları verilmez.

Kazanımlar

- % 10 ve / 10 ile onluk basamakları aritmetik yoldan ayırma.
- En az bir işlem gerektiren bir durumda do / while seçme.
- Bir yineleme tablosuyla sayaç, biriktirici ve kalan sayıyı birlikte izleme.
- Asıl girdiyi korumak için çalışma kopyası kullanma.
- Döngünün sonlanmasını küçülen bir ara değerle açıklama.

Alıştırmalar

1. Tek ve çift basamak adetlerini ayrı sayaçlarda bulun. Sıfır basamağının çift sayılması gerektiğini örneklerle kontrol edin.
2. Basamaklar arasındaki en büyük değeri de hesaplayın. 0, 407 ve 999 girdileri için başlangıç değerinizin neden doğru olduğunu açıklayın.
3. Aynı çözümü while döngüsüyle yazın; 0 için tek basamak kuralını ayrıca uygulayın ve iki sürümün sonuçlarını karşılaştırın.

023 Sayının basamaklarını ters çevirme

Düzey: Orta Önkoşullar: 022

Kavramlar: Basamak ayırma, Sayısal birleştirme, Çalışma kopyası, long türü, Döngü değişmezi

Problem tanımı

0 ile 2147483647 arasında girilen bir tamsayının basamaklarını ters sırayla birleştirerek yeni bir sayı oluşturun. Hazır ters çevirme işlemleri veya metin karakterlerini kullanmayın. Son basamağı % 10 ile çıkarın; yeni sayıyı 10 ile çarpıp çıkarılan basamağı ekleyin.

Örneğin 407 değeri 704'e dönüşür. 1200 için ters basamak sırası 0021 olsa da çıktı bir sayıdır ve baştaki sıfırlar gösterilmez; sonuç 21 olur. Sıfırın tersi 0'dır. Girdi int aralığındadır, ancak ters çevrilen sayı aynı aralığa sığmayabilir. Bu nedenle sonuç long türünde tutulur; 2147483647'nin tersi 7463847412'dir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	0 ile 2147483647 arasında int; değiştirilmeden korunur.
Ara değer	kalanSayi	İşlenmemiş basamakları taşıyan kopya.
Çıktı	tersSayi	long türünde sayısal ters; Ters sayı: N.

Sayı (0..2147483647): istemine tamsayı girin. Geçersiz biçim veya aralık için Hata: 0 ile 2147483647 arasında bir tamsayı giriniz. yazılır. Çıktı metin basamak dizisi değil, sayısal değerdir; başta sıfır doldurulmaz.

Algoritma

1. Sayıyı okuyup biçimini ve aralığını doğrulayın.
2. Başlangıç değerini koruyup kalanSayi içine kopyalayın.
3. long türündeki tersSayi değerini 0 yapın.
4. do gövdesinde son basamağı kalanSayi % 10 ile alın.
5. tersSayi = tersSayi * 10 + basamak işlemini yapın.
6. kalanSayi /= 10 ile işlenen basamağı çıkarın.
7. Kalan sayı pozitif olduğu sürece 4. adımdan devam edin.
8. Ters sayıyı yazdırın.

Her yineleme sonunda tersSayi, o ana kadar sağdan çıkarılan basamakların çıkarılma sırasıyla oluşturduğu sayıdır. kalanSayi ise işlenmemiş sol kısmı taşır. 407 için ters sayı sırasıyla 7, 70 ve 704 olur. 10 ile çarpmak mevcut basamakları bir basamak sola kaydırır; yeni basamağı eklemek sağdaki yeri doldurur. Sıfır basamakları da aynı güncelleme kuralından geçer.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi) || sayi < 0)
{
    Console.WriteLine("Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.");
    return;
}

int kalanSayi = sayi;
long tersSayi = 0;

do
{
    int basamak = kalanSayi % 10;
    tersSayi = tersSayi * 10 + basamak;
    kalanSayi /= 10;
}
while (kalanSayi > 0);

Console.WriteLine("Ters sayı: " + tersSayi.ToString(CultureInfo.InvariantCulture));
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. tersSayi daha ilk işlemten itibaren long olduğu için çarpma ve ekleme de bu türde yapılır. Sonucu yalnızca çıktıdan hemen önce long türüne çevirmek, daha önce gerçekleşmiş bir int taşmasını engellemez. Başlangıç sayısı bu çözümde yazdırılmasa da değişmeden tutulur; sonraki problemde karşılaştırma için kullanılabilir.

Örnek çalıştırmalar

Tablo giriş isteminden sonra yazılan sonuç veya hata satırını gösterir.

Girdi	Sonuç	Açıklama
407	Ters sayı: 704	Aradaki sıfır korunur.
0	Ters sayı: 0	Bir yineleme sonucu sıfır.
7	Ters sayı: 7	Tek basamak değişmez.
10	Ters sayı: 1	Başta oluşan sıfır gösterilmez.
1200	Ters sayı: 21	İki son sıfır sayısal tersin başında kaybolur.
1001	Ters sayı: 1001	Simetrik basamaklar.
00120	Ters sayı: 21	Girdi sayısal olarak 120'dir.
2147483647	Ters sayı: 7463847412	Sonuç int sınırını aşar, long içine sığar.
-12	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Negatif sayı reddedilir.
2147483648	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Girdi tür sınırını aşar.

Girdi	Sonuç	Açıklama
sayı	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Metin reddedilir.

Sınır durumları

- Sıfır için döngü bir kez çalışır ve ters sayı 0 olur.
- Son sıfırlar sayısal tersin başına geldiğinde görünmez. Bu yüzden ters çevirme işlemini iki kez uygulamak her sayıyı geri vermez: $1200 \rightarrow 21 \rightarrow 12$.
- Başta yazılmış sıfırlar `int`.TryParse sonrasında sayısal değerin parçası değildir.
- Girdi en fazla 10 basamaklıdır; ters sayı 9999999999 değerinden büyük olamaz. Bu üst sınır `long` aralığına rahatça sığar.
- `kalanSayi` her pozitif yinelemede küçülür; son basamak alındığında 0 olur ve döngü biter.
- Negatif, aralık dışı, boş veya metin girişinde ters sayı yazdırılmaz.

Kazanımlar

- Çıkarılan basamaklarla yeni bir sayı kurma.
- Çarpma ve toplamının basamak konumunu nasıl değiştirdiğini açıklama.
- Girdi türü ile sonuç türünün farklı aralıklar gerektirebileceğini görme.
- Çalışma kopyası kullanarak ilk değeri koruma.
- Sayısal ters çevirme ile metin ters çevirmenin sıfırlardaki farkını açıklama.

Alıştırmalar

1. 5080 için her yinelemenin basamak, kalan sayı ve ters sayı değerlerini tabloya yazın.
2. Başlangıç sayısını da Başlangıç sayısı: N satırıyla yazdırın. Sonuca ulaştıktan sonra sayı ve `kalanSayi` değişkenlerinin farkını gösterin.
3. Ters çevirme sırasında kaç sıfır basamağı işlendiğini ayrı sayaçta tutun. 1000 ve 1001 için sonuçları karşılaştırın.

024 Palindrom sayı olup olmadığını belirleme

Düzey: Orta Önkoşullar: 005, 023

Kavramlar: Palindrom, Aritmetik ters çevirme, Asıl değeri koruma, Karşılaştırma, Döngü ve karar

Problem tanımı

0 ile 2147483647 arasında girilen bir tamsayının onluk basamaklarının soldan sağa ve sağdan sola aynı olup olmadığını belirleyin. Böyle sayılara palindrom sayı denir. Örneğin 1221 ve 707 palindromdur; 123 ve 10 değildir. Tek basamaklı sayılar, 0 dahil, palindrom kabul edilir.

Sayıyı metne dönüştürmeden aritmetik işlemlerle tersini oluşturun ve başlangıç sayısı ile karşılaştırın. Karşılaştırmada ilk sayı gerektiği için basamakları çıkarırken bir kopyayı değiştirin. Girdi sayısal olarak yorumlanır: 00100 girişi 100 değeridir, baştaki sıfırlar karşılaştırmaya katılmaz. Negatif sayılar çalışma kapsamına alınmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	sayi	0 ile 2147483647 arasında int; karşılaştırma için korunur.
Ara değer	kalanSayi	Basamakları çıkarılan kopya.
Ara değer	tersSayi	Aritmetik ters, long türünde.
Çıktı	Sınıflandırma	Sonuç: Palindrom veya Sonuç: Palindrom değil.

Sayı (0..2147483647): istemine tamsayı girin. Hatalı biçim veya aralıkta Hata: 0 ile 2147483647 arasında bir tamsayı giriniz. yazılır. Başta yazılan sıfırlar değil, dönüştürülen tamsayının basamakları incelenir.

Algoritma

1. Sayıyı okuyup biçimini ve aralığını doğrulayın.
2. Sayıyı kalanSayi içine kopyalayın; tersSayi değerini 0 yapın.
3. Son basamağı kalanSayi % 10 ile alın.
4. $tersSayi = tersSayi * 10 + basamak$ ile ters sayıya ekleyin.
5. $kalanSayi /= 10$ ile işlenen basamağı çıkarın.
6. do / while ile kalan sayı pozitif olduğu sürece 3. adımdan devam edin.
7. Başlangıç sayısı ters sayıya eşitse palindrom, değilse palindrom değil sonucunu yazdırın.

Döngü boyunca sayi sabit kalır. Her yinelemede tersSayi, sağdan çıkarılan basamakları çıkarılma sırasıyla taşır; kalanSayi işlenmemiş sol kısmı tutar. Döngü sonunda bütün basamaklar çıkarılmıştır. 1221 için ters sayı sırasıyla 1, 12, 122 ve 1221 olur; son karşılaştırma doğrudur. Başlangıç değerini de bölerek değiştirmek, doğru karşılaştırma için gerekli bilgiyi kaybettirirdi.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sayi) || sayi < 0)
{
    Console.WriteLine("Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.");
    return;
}

int kalanSayi = sayi;
long tersSayi = 0;

do
{
    int basamak = kalanSayi % 10;
    tersSayi = tersSayi * 10 + basamak;
    kalanSayi /= 10;
}
while (kalanSayi > 0);

if (tersSayi == sayi)
{
    Console.WriteLine("Sonuç: Palindrom");
}
else
{
    Console.WriteLine("Sonuç: Palindrom değil");
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. long ters sayı ile int girdi karşılaştırılırken int değeri sayısal olarak long türüne genişletilir; bilgi kaybı olmaz. Girdi int içine sığsa da tersi sığmayabileceği için ters sayı yine long türündedir. Koşul zinciri yalnızca bir sınıflandırma satırı üretir.

Örnek çalıştırmalar

Tablo giriş isteminden sonraki tam sonuç veya hata satırını gösterir.

Girdi	Sonuç	Açıklama
1221	Sonuç: Palindrom	Çift sayıda simetrik basamak.
707	Sonuç: Palindrom	Ortada sıfır bulunabilir.
123	Sonuç: Palindrom değil	Ters sayı 321'dir.
0	Sonuç: Palindrom	Sıfırın tersi sıfırdır.
9	Sonuç: Palindrom	Tek basamaklı sayı.
10	Sonuç: Palindrom değil	Sayısal ters 1'dir.
1001	Sonuç: Palindrom	İki iç sıfır simetriyi bozmaz.
00100	Sonuç: Palindrom değil	İncelenen değer 100, tersi 1'dir.
2147447412	Sonuç: Palindrom	int üst sınırına yakın palindrom.

Girdi	Sonuç	Açıklama
2147483647	Sonuç: Palindrom değil	Tersi 7463847412 olur.
-121	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Negatif sayılar kapsam dışıdır.
2147483648	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Girdi aralık dışındadır.
palindrom	Hata: 0 ile 2147483647 arasında bir tamsayı giriniz.	Metin reddedilir.

Sınır durumları

- 0 ile 9 arasındaki bütün sayılar palindromdur; tek basamak sağdan ve soldan aynıdır.
- Sıfırla biten pozitif sayının sayısal tersi daha az basamaklı olur; bu sayı palindrom olamaz.
- Sayının içindeki sıfırlar korunur. 1001 palindrom olduğu halde 1000 değildir.
- Başta yazılan sıfırlar girdinin metinsel görünümünü değiştirir; tamsayı değerini değiştirmez.
- Ters sayı en fazla 10 basamaklı olduğundan long aralığında kalır. Sonucun int türüne zorla daraltılması gerekmez.
- Geçersiz sayı, boş satır veya giriş sonu halinde karşılaştırma yapılmaz ve sınıflandırma yazılmaz.

Kazanımlar

- Bir önceki basamak algoritmasını karar problemi içinde yeniden uygulama.
- Döngüyle dönüştürülen kopya ile korunan asıl değer görevlerini ayırma.
- Sayısal eşitlik üzerinden palindrom tanımını gerçekleştirme.
- Daha geniş türde üretilen ara sonucu güvenle karşılaştırma.
- Metinsel görünüm ile sayısal değer arasındaki farkı örneklerle açıklama.

Alıştırmalar

1. Palindrom kararından önce Ters sayı: N satırını ekleyin. 120, 1001 ve 0 için iki çıktı arasındaki ilişkiyi açıklayın.
2. Palindrom olmayan sayılarda başlangıç ile ters sayı arasındaki farkı long türünde hesaplayın. 2147483647 için neden long gerektiğini gösterin.
3. 100 ile 150 arasındaki sayıları dolaşarak palindrom olanları yazdırın. Her yeni adayda kalan sayı ve ters sayı değişkenlerini yeniden başlatın.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Taban (-10..10): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int taban) || taban < -10 || taban > 10)
{
    Console.WriteLine("Hata: Taban -10 ile 10 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Üs (0..18): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int us) || us < 0 || us > 18)
{
    Console.WriteLine("Hata: Üs 0 ile 18 arasında bir tamsayı olmalıdır.");
    return;
}

long sonuc = 1;
for (int i = 0; i < us; i++)
{
    sonuc *= taban;
}

Console.WriteLine("Sonuç: " + sonuc.ToString(CultureInfo.InvariantCulture));
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. sonuc değişkeni long olduğu için her çarpma geniş tamsayı türünde yapılır. $i < us$ yerine $i \leq us$ yazılıysaydı bir fazla çarpma yapılırdı. Tam tamsayı çarpımları kullanıldığı için ondalıklı yuvarlama yapılmaz.

Örnek çalıştırmalar

Girdiler taban ve üs sırasındadır. Tablo giriş istemlerinden sonraki tam sonuç veya hata satırını gösterir.

Girdi	Sonuç	Açıklama
3, 4	Sonuç: 81	Dört çarpma.
5, 0	Sonuç: 1	Hiç çarpma yapılmaz.
0, 0	Sonuç: 1	Bu programın boş çarpım kabulü.
0, 7	Sonuç: 0	İlk çarpmadan itibaren sonuç sıfır.
-2, 3	Sonuç: -8	Negatif taban, tek üs.
-2, 4	Sonuç: 16	Negatif taban, çift üs.
1, 18	Sonuç: 1	Birin tüm kabul edilen kuvvetleri 1'dir.
-10, 17	Sonuç: -100000000000000000	Büyük negatif sonuç.
10, 18	Sonuç: 100000000000000000	En büyük sonuç büyüklüğü.
-10, 18	Sonuç: 100000000000000000	Çift üs işareti pozitif yapar.
İlk girdi 11	Hata: Taban -10 ile 10 arasında bir tamsayı olmalıdır.	Üs istenmeden durur.

Girdi	Sonuç	Açıklama
2, -1	Hata: Üs 0 ile 18 arasında bir tamsayı olmalıdır.	Negatif üs reddedilir.
2, 19	Hata: Üs 0 ile 18 arasında bir tamsayı olmalıdır.	Üst üs sınırı aşılr.
İlk girdi iki	Hata: Taban -10 ile 10 arasında bir tamsayı olmalıdır.	Metin reddedilir.

Sınır durumları

- Üs 0 ise taban ne olursa olsun bu algoritma 1 üretir; 0 üzeri 0 kuralı da açıkça bu kabulden gelir.
- Taban 0 ve üs pozitifse sonuç 0 olur; bölme yapılmadığı için sıfıra bölme sorunu yoktur.
- Negatif tabanın tek üssü negatif, çift üssü pozitifdir; bu davranış çarpmanın işaret kurallarıyla kendiliğinden oluşur.
- Sonuç büyüklüğü en fazla 1000000000000000000'dır. long üst sınırı 9223372036854775807 olduğundan tüm ara çarpımlar da tür aralığında kalır.
- Girdi aralıkları büyütülürse taşma sınırı yeniden incelenmelidir; mevcut long seçimi sınırsız kuvvet hesabı sağlamaz.
- Her hatada sonuç yazılmadan program biter. Boş satır veya giriş sonu da ilgili alandaki geçersiz tamsayı olarak ele alınır.

Kazanımlar

- Kuvveti hazır fonksiyon yerine tekrarlı tamsayı çarpmasıyla hesaplama.
- Çarpma biriktiricisinde başlangıç değeri 1'in nedenini açıklama.
- Sayaç koşulundan gerçek yineleme adedini çıkarma.
- Girdi sınırlarını kullanarak sonuç ve ara işlem taşmasını değerlendirme.
- Sıfır ve negatif tabanların davranışını algoritmanın güncellemeleriyle izleme.

Alıştırmalar

1. Her yinelemeden sonra çarpma adedini ve ara sonucu yazdırın. -3 tabanı, 4 üssü için işaret değişimini izleyin.
2. Taban 0 ve üs 0 olduğunda sonuç yerine özel bir hata mesajı veren sürüm hazırlayın. Bu kontrolün döngüden önce yapılmasını açıklayın.
3. Aynı tabanın 0'dan girilen üssüne kadar tüm kuvvetlerini tek döngüyle yazdırın. Biriktiriciyi her kuvvet için baştan hesaplamadan güncelleyin.

026 Fibonacci dizisinin N numaralı terimini hesaplama

Düzey: Orta Önkoşullar: 004, 008, 025

Kavramlar: Ardışık durum saklama, For döngüsü, Güncelleme sırası, Sayısal taşma sınırı

Problem tanımı

Fibonacci dizisinin kullanıcı tarafından belirtilen numaradaki terimini hesaplayın. Bu çalışmada numaralandırma sıfırdan başlar: $F(0) = 0$, $F(1) = 1$. Sonraki her terim önceki iki terimin toplamıdır: $F(n) = F(n - 1) + F(n - 2)$. İlk terimler 0, 1, 1, 2, 3, 5, 8 olur; örneğin 6 numaralı terim 8'dir.

Bütün diziyi bir dizide saklamak yerine yalnızca önceki ve güncel terimi tutun. Yeni terim hesaplandıktan sonra bu iki değişkenin görevini ilerletin. Amaç, birbirine bağlı iki durumun döngü içinde hangi sırada güncellenmesi gerektiğini öğrenmektir. Sonuç long aralığında kalmalıdır; kabul edilen en büyük terim numarası 92'dir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	0 ile 92 arasında int; istenen terimin sıfırdan başlayan numarası.
Çıktı	sonuc	long türünde Fibonacci terimi; $F(n)$: değer biçiminde.

Terim numarası (0..92): istemine tek tamsayı girin. Sayının içinde basamak ayırıcı kullanmayın. Örneğin 10 girildiğinde sonuç satırı $F(10)$: 55 olur. Geçersiz girişte Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır. yazılır.

Algoritma

1. Terim numarasını okuyup 0 ile 92 arasında bir tamsayı olduğunu doğrulayın.
2. Önceki terimi 0, güncel terimi 1 yapın.
3. $n = 0$ ise sonucu 0 kabul edin; döngü çalıştırmayın.
4. Diğer durumlarda sayacı 2'den başlatın ve n dahil olacak şekilde artırın.
5. Her adımda yeni terimi önceki terim ile güncel terimin toplamı olarak hesaplayın.
6. Önceki terime güncel terimi, sonra güncel terime yeni terimi atayın.
7. Döngü bitince güncel terimi sonuç olarak seçip terim numarasıyla birlikte yazdırın.

i numaralı terimi hesaplamadan hemen önce $onceki = F(i - 2)$ ve $guncel = F(i - 1)$ olur. Toplam ayrı bir değişkende tutulduğu için eski değerler kaybolmaz. Gövde sonunda bu iki değişken sırasıyla $F(i - 1)$ ve $F(i)$ olur. Sayaç her adımda artar; n sabit ve en fazla 92 olduğu için döngü sonlanır. $n = 1$ durumunda döngü çalışmaz ve başlangıç güncel terimi olan 1 korunur.

İşlenen terim	Adım başında önceki	Adım başında güncel	Yeni terim	Adım sonunda önceki / güncel
2	0	1	1	1 / 1
3	1	1	2	1 / 2
4	1	2	3	2 / 3

İşlenen terim	Adım başında önceki	Adım başında güncel	Yeni terim	Adım sonunda önceki / güncel
5	2	3	5	3 / 5

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Terim numarası (0..92): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 0 || n > 92)
{
    Console.WriteLine("Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır.");
    return;
}

long onceki = 0;
long guncel = 1;
long sonuc;
if (n == 0)
{
    sonuc = 0;
}
else
{
    for (int i = 2; i <= n; i++)
    {
        long yeni = onceki + guncel;
        onceki = guncel;
        guncel = yeni;
    }
    sonuc = guncel;
}

Console.WriteLine("F(" + n.ToString(CultureInfo.InvariantCulture) + "): " +
    sonuc.ToString(CultureInfo.InvariantCulture));
```

yeni hesaplanmadan önceki = guncel yapılırsa toplama katılacak eski terim kaybolur. Döngü yalnızca istenen terime kadar ilerler. 92 numaralı terim hesaplandıktan sonra kullanılmayacak bir 93 numaralı terim hesaplanmaz; bu ek toplama long sınırını aşardı.

Örnek çalıştırmalar

İstem metni gösterilmez; sonuç sütunu programın ürettiği tam sonuç veya hata satırıdır.

Girdi	Sonuç	Açıklama
0	F(0): 0	Ayrı ele alınan ilk terim.
1	F(1): 1	Döngü çalışmadan başlangıç terimi alınır.
2	F(2): 1	Tek toplama yapılır.
6	F(6): 8	Birkaç ardışık durum güncellemesi.
10	F(10): 55	Normal örnek.

Girdi	Sonuç	Açıklama
92	$F(92)$: 7540113804746346429	En büyük kabul edilen terim.
93	Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır.	Taşabilecek terim kabul edilmez.
-1	Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır.	Negatif numara reddedilir.
iki	Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır.	Tamsayı olmayan giriş.
2.5	Hata: Terim numarası 0 ile 92 arasında bir tamsayı olmalıdır.	Kesirli numara reddedilir.

Sınır durumları

- Sıfır numaralı terim, güncel değişkeninin başlangıç değeri olan 1'den ayrı ele alınır.
- 1 numaralı terimde sayaç 2'den başladığı için gövde hiç çalışmaz.
- $F(92) = 7540113804746346429$, long üst sınırı olan 9223372036854775807 değerinden küçüktür.
- $F(93) = 12200160415121876738$ aynı aralığa sığmaz. Girdi sınırı hesaplama başlamadan denetlenir.
- Boş giriş, ondalık sayı, metin ve 0..92 dışındaki tamsayılar tek hata satırıyla reddedilir.
- Dizi oluşturulmaz; saklanan ara değerlerin sayısı terim numarasıyla artmaz.

Kazanımlar

- $F(0)$ ve $F(1)$ taban durumlarını döngü başlangıcıyla birlikte açıklama.
- Ardışık iki terimi saklayarak bir sonraki terimi hesaplama.
- $n = 5$ için her adımdaki önceki, güncel ve yeni değerleri tabloya yazma.
- Yanlış atama sırasının hangi eski değeri kaybettirdiğini gösterme.
- Kullanılmayacak bir sonraki terimin neden hesaplanmaması gerektiğini sayısal sınırla açıklama.

Alıştırmalar

- 0 ile n arasındaki bütün Fibonacci terimlerini sırayla yazdırın; $n = 0$ için yalnızca 0 yazıldığını kontrol edin.
- Başlangıcı $F(0) = 2$, $F(1) = 1$ olan benzer bir dizi için çözümü uyarlayın; girdi üst sınırını yeniden hesaplayın.
- Döngüyü while ile yazın ve 1, 2, 6 girdilerinde iki sürümün aynı sonuç verdiğini ara değerlerle doğrulayın.

027 Bir sayının asal olup olmadığını belirleme

Düzey: İleri Önkoşullar: 006, 021, 026

Kavramlar: Bölen arama, Erken döngü çıkışı, Boolean durum, Taşmasız döngü sınırı

Problem tanımı

Sıfır veya pozitif bir tamsayının asal olup olmadığını belirleyin. Asal sayı, 1'den büyük ve yalnızca 1 ile kendisine tam bölünen tamsayıdır. 0 ve 1 asal değildir; bunlar geçerli girdilerdir. Örneğin 13 asal, 15 ise 3'e bölünebildiği için asal değildir.

Sayıyı 2'den başlayarak olası bölenlere bölün. Bir bölen bulunca aramayı sonlandırın. Her bileşik sayı, bir çarpan çiftine sahiptir; çiftin en az bir üyesi sayının karekökünü aşmaz. Bu nedenle sayıya kadar bütün adayları denemek gerekmez. Karekökü hazır bir işlevle hesaplamak yerine tamsayı bölmesiyle güvenli bir döngü sınırı kurun.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	0 ile 2147483647 arasında int.
Çıktı	Asallık durumu	Sonuç: Asal veya Sonuç: Asal değil.

Sayı (0..2147483647): istemine tek tamsayı girin. Geçersiz girişte Hata: Sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır. yazılır; asallık sonucu verilmez.

Algoritma

1. Sayıyı okuyup 0 ile 2147483647 arasında bir tamsayı olduğunu doğrulayın.
2. asal değişkenini $n \geq 2$ ifadesinin sonucuyla başlatın.
3. Asal olabilecek sayılarda bölen adayını 2'den başlatın.
4. $bolen \leq n / bolen$ olduğu sürece adayları deneyin.
5. $n \% bolen == 0$ ise sayının asal olmadığını kaydedip döngüden break ile çıkın.
6. Bölen bulunmadıysa adayı bir artırıp koşulu yeniden denetleyin.
7. asal durumuna göre sonuç satırını yazdırın.

Her adımın başında 2 ile bolen - 1 arasındaki adayların hiçbirinin sayıyı tam bölmediği bilinir. $bolen \leq n / bolen$ koşulu, pozitif adaylar için $bolen * bolen \leq n$ sınırını tamsayı çarpımı yapmadan ifade eder. Örneğin 49 için 7 denenir; eşitlik korunmalıdır. Her başarısız denemede aday artar, üst sınır sınırlı kalır; bir bölen bulunması da aramayı hemen bitirir.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 0)
{
    Console.WriteLine("Hata: Sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

bool asal = n >= 2;
if (asal)
{
    for (int bolen = 2; bolen <= n / bolen; bolen++)
    {
        if (n % bolen == 0)
        {
            asal = false;
            break;
        }
    }
}

if (asal)
{
    Console.WriteLine("Sonuç: Asal");
}
else
{
    Console.WriteLine("Sonuç: Asal değil");
}
```

int.TryParse, int üst sınırını aşan girdileri reddeder; bu nedenle ayrıca 2147483647 ile karşılaştırmak gerekmez. bolen 2'den başladığı için bölmede sıfır kullanılmaz. break, döngüden çıkar; programın sonuç yazdıran bölümü yine çalışır.

Örnek çalıştırmalar

İstem metni sonuç sütununa dahil değildir.

Girdi	Sonuç	Açıklama
0	Sonuç: Asal değil	Geçerli girdi; asal tanımının dışında.
1	Sonuç: Asal değil	1 asal kabul edilmez.
2	Sonuç: Asal	En küçük asal, gövde çalışmaz.
13	Sonuç: Asal	2 ve 3 bölen değildir.
15	Sonuç: Asal değil	3 bulunduğu arama biter.
49	Sonuç: Asal değil	Karekök sınırındaki 7 mutlaka denenir.
2147483647	Sonuç: Asal	En büyük int değeri asal sayıdır.
2147483646	Sonuç: Asal değil	İlk aday olan 2'ye bölünür.

Girdi	Sonuç	Açıklama
-1	Hata: Sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.	Negatif sayı reddedilir.
2147483648	Hata: Sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.	int sınırı aşılr.
asal	Hata: Sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.	Metin girişı reddedilir.

Sınır durumları

- 0 ve 1 için döngüye girilmez; sonuç doğrudan asal değildir.
- 2 ve 3 için ilk sınır koşulu yanlıştır; bir bölen aranmadan doğru asal sonucu elde edilir.
- Bir asalın karesinde karekök adayı denenmelidir; < kullanılması 49'u yanlış sınıflandırabilir.
- Büyük int girdisinde bolen * bolen ifadesi taşıabilir. Bölme kullanan koşul bu çarpımı yapmaz.
- Bir bölen bulunması yeterlidir; bütün bölenlerin çıkarılması bu problemde gerekli değildir.
- Boş giriş, kesirli sayı, negatif sayı ve int aralığını aşan metin tek hata satırıyla reddedilir.

Kazanımlar

- Asal tanımını 0, 1 ve 2 örnekleri üzerinden doğru uygulama.
- Kalan işleciyle bir bölenin sayıyı tam bölüp bölmediğini belirleme.
- Bir çarpan çiftinden en az birinin karekök sınırında veya altında olduğunu açıklama.
- break kullanarak sonucu değiştirmeyecek sonraki denemeleri durdurma.
- Çarpma kullanan döngü sınırını taşmasız bölme koşuluna dönüştürme.

Alıştırmalar

1. Asal olmayan, 1'den büyük sayılarda ilk bulunan böleni de yazdırın; 0 ve 1 için bölen mesajı üretmeyin.
2. Yapılan bölünebilirlik denemelerini sayın; 2, 49 ve 97 için sayaç değerlerini adım adım çıkarın.
3. 2'yi ayrı denedikten sonra yalnızca tek adayları deneyecek biçimde çözümü değiştirin; 2 ve çift sayılardaki davranışı koruyun.

028 Öklid algoritmasıyla EBOB hesaplama

Düzey: İleri Önkoşullar: 004, 006, 021, 027

Kavramlar: Öklid algoritması, Kalanla durum küçültme, While döngüsü, Geçici değişken

Problem tanımı

Sıfır veya pozitif iki tamsayının en büyük ortak bölenini (EBOB) hesaplayın. EBOB, iki sayıyı da kalansız bölen pozitif tamsayıların en büyüğüdür. Örneğin 48 ve 18'in ortak bölenleri 1, 2, 3 ve 6'dır; sonuç 6 olur.

Öklid algoritması, bütün bölenleri listelemek yerine $EBOB(a, b) = EBOB(b, a \% b)$ özelliğini kullanır. Kalan sıfır olduğunda son sıfır olmayan değer sonuçtur. Bir girdi sıfırsa diğer pozitif sayı EBOB olur. Her iki girdi sıfır olduğunda en büyük pozitif ortak bölen yoktur; bu çalışma bu durumu hata olarak ele alır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	a	0 ile 2147483647 arasında ilk int.
Girdi	b	0 ile 2147483647 arasında ikinci int.
Çıktı	EBOB	EBOB: değer biçiminde pozitif tamsayı.

İlk sayı (0..2147483647): ve İkinci sayı (0..2147483647): istemlerine sırayla tamsayı girin. Aralık veya biçim yanlışsa ilgili sayı için hata yazılır ve sonraki işlem yapılmaz. İki sıfır için Hata: İki sayı aynı anda sıfır olamaz; EBOB tanımsızdır. yazılır.

Algoritma

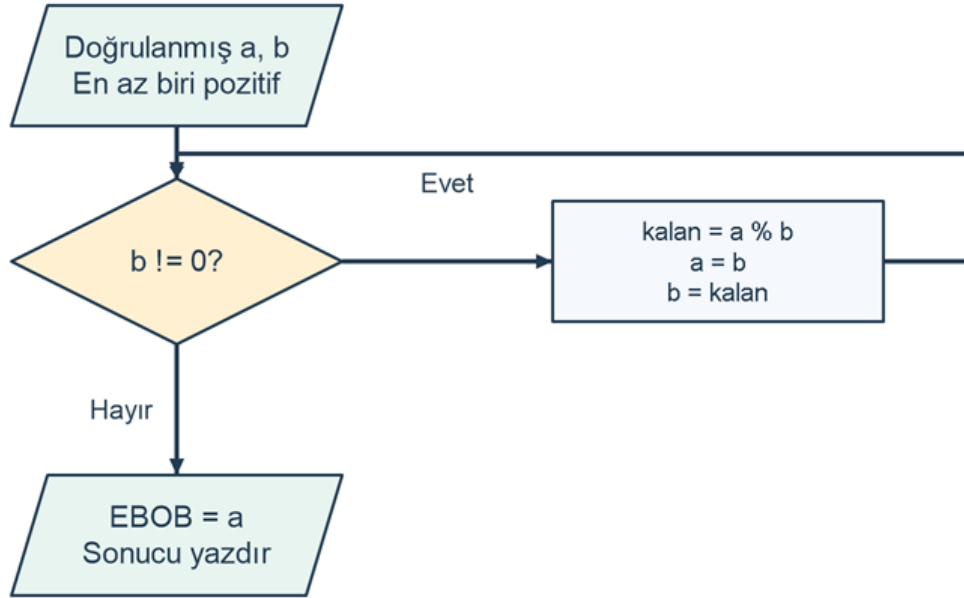
1. İlk ve ikinci sayıyı okuyup ayrı ayrı doğrulayın.
2. Her iki sayı sıfırsa hata yazıp bitirin.
3. İkinci sayı b sıfır olmadığı sürece döngüyü sürdürün.
4. $a \% b$ değerini kalan geçici değişkenine alın.
5. a değişkenine eski b değerini, ardından b değişkenine kalanı atayın.
6. Döngü bittiğinde a değişkenini EBOB olarak yazdırın.

Bir ortak bölen a ve b sayılarını bölüyorsa $a = q * b$ farkını da böler; bu fark uygun q için kalandır. Tersine, b ile kalanı bölen sayı $a = q * b + kalan$ değerini de böler. Bu nedenle her güncellemede ortak bölenler ve EBOB korunur. $b > 0$ iken $0 \leq kalan < b$ olur. Yeni ikinci sayı eskisinden küçüldüğü için sıfıra ulaşılır ve döngü biter. % işlemi yalnızca $b \neq 0$ koşulu sağlandığında yapılır.

48 ve 18 için her satır bir döngü adımını gösterir.

Adım	Adım başında a	Adım başında b	Kalan	Adım sonunda a / b
1	48	18	12	18 / 12
2	18	12	6	12 / 6
3	12	6	0	6 / 0

Şema, iki girdi doğrulandıktan ve ikisinin birden sıfır olmadığı belirlendikten sonraki hesaplamayı gösterir.



Şekil 7 Öklid algoritmasında kalan ile küçülen durum

C# çözümü

```
using System;
using System.Globalization;

Console.Write("İlk sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int a) || a < 0)
{
    Console.WriteLine("Hata: İlk sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("İkinci sayı (0..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int b) || b < 0)
{
    Console.WriteLine("Hata: İkinci sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

if (a == 0 && b == 0)
{
    Console.WriteLine("Hata: İki sayı aynı anda sıfır olamaz; EBOB tanımsızdır.");
    return;
}

while (b != 0)
{
    int kalan = a % b;
    a = b;
    b = kalan;
}

Console.WriteLine("EBOB: " + a.ToString(CultureInfo.InvariantCulture));
```

Girdiler hesaplama sırasında değiştirilir. kalan, iki atamadan önce eski çiftin kalanını saklar. Önce $a = b$ yapıp sonra $a \% b$ hesaplamak eski ilk sayıyı kaybettirir ve yanlış bir sıfır kalanı üretir. Başlangıçta $a < b$ olabilir; ilk adım çiftin sırasını değiştirerek algoritmayı devam ettirir.

Örnek çalıştırmalar

Girdiler ilk ve ikinci sayı sırasındadır. Sonuç sütunu istemlerden sonraki tam sonuç veya hata satırını gösterir.

Girdi	Sonuç	Açıklama
48, 18	EBOB: 6	Üç kalan adımı.
18, 48	EBOB: 6	Girdi sırası sonucu değiştirmez.
17, 13	EBOB: 1	Sayılar aralarında asaldır.
24, 24	EBOB: 24	İlk kalan sıfırdır.
0, 12	EBOB: 12	İlk sayı sıfırdır.
12, 0	EBOB: 12	Döngü çalışmadan sonuç yazılır.
2147483647, 0	EBOB: 2147483647	En büyük girdi ve sıfır.

Girdi	Sonuç	Açıklama
2147483647, 2147483646	EBOB: 1	Ardışık tamsayılar aralarında asaldır.
0, 0	Hata: İki sayı aynı anda sıfır olamaz; EBOB tanımsızdır.	Özel geçersiz çift.
İlk girdi -1	Hata: İlk sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.	İkinci sayı istenmez.
12, sayı	Hata: İkinci sayı 0 ile 2147483647 arasında bir tamsayı olmalıdır.	İkinci alan metindir.

Sınır durumları

- Bir sayı sıfırsa diğer sayı pozitif olmalıdır; EBOB pozitif olan sayıdır.
- Her iki sıfır için hesaplama başlamadan hata verilir.
- Girdilerin büyükten küçüğe sıralanması gerekmez.
- $b = 0$ olduğunda % gövdesi çalışmaz; sıfıra bölme oluşmaz.
- Kalan önceki ikinci sayıdan küçük veya sıfırdır. Çarpma yapılmadığından büyük girdilerde ek bir taşma riski oluşmaz.
- Boş giriş, kesirli sayı, negatif sayı ve int sınırı dışındaki değerler ilgili alanın hata mesajıyla reddedilir.

Kazanımlar

- EBOB tanımını tek sıfır ve iki sıfır durumlarını ayırarak uygulama.
- Öklid eşitliğinin ortak bölenleri neden koruduğunu bölme-kalan ilişkisiyle açıklama.
- 48 ve 18 için tüm durum çiftlerini sırasıyla yazma.
- Geçici değişkenle eski iki değeri kullanarak güvenli durum güncellemesi yapma.
- Azalan ikinci değişkeni kullanarak while döngüsünün sonlanmasını açıklama.

Alıştırmalar

1. Her döngü adımında a, b ve kalan değerlerini yazdırın; 18 ve 48 girdisindeki ilk adımı yorumlayın.
2. Döngüye eklenen bir sayaçla kalan hesaplama sayısını çıktı olarak verin; 12 ve 0 için sayacın sıfır kaldığını doğrulayın.
3. Üç sayının EBOB'unu iki aşamada hesaplayın. İlk iki sayı aynı anda sıfırsa ara EBOB hesabına girmeyin; üçüncü sayı pozitifse sonuç doğrudan bu sayıdır. Diğer durumlarda önce ilk ikinin EBOB'unu, sonra bu sonuçla üçüncü sayının EBOB'unu bulun. Üç sıfır için hata verin; (0, 0, 12) örneğinin sonucunu ayrıca doğrulayın.

029 EBOB yardımıyla EKOK hesaplama

Düzey: İleri Önkoşullar: 025, 028

Kavramlar: Algoritmaları birleştirme, EKOK ve EBOB ilişkisi, Girdi kopyası, İşlem sırası, Tür dönüşümü

Problem tanımı

İki pozitif tamsayının en küçük ortak katını (EKOK) bulun. EKOK, iki sayıya da kalansız bölünen pozitif tamsayıların en küçüğüdür. Örneğin 12 ve 18 için 36, her iki sayının da katıdır ve en küçük ortak kattır.

Ortak katları tek tek aramak yerine $EKOK(a, b) = (a / EBOB(a, b)) * b$ ilişkisini kullanın. EBOB'u önceki problemdeki Öklid döngüsüyle hesaplayın. Döngü çalışma değerlerini değiştirdiği için asıl girdileri ayrıca koruyun. Önce bölme yapıp çarpmayı long türünde gerçekleştirmek, büyük girdilerde yanlış taşmış sonuç üretmeyi önler. Bu çalışmada sıfır ve negatif sayılar kabul edilmez.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	ilk	1 ile 2147483647 arasında ilk int.
Girdi	ikinci	1 ile 2147483647 arasında ikinci int.
Çıktı	ekok	long türünde sonuç; EKOK: değer biçiminde.

İlk sayı (1..2147483647): ve İkinci sayı (1..2147483647): istemlerine ayrı satırlarda tamsayı girin. Geçersiz bir alan için o alanın hata satırı yazılır; EKOK hesaplanmaz.

Algoritma

- İki girdiyi okuyup pozitif int olduklarını doğrulayın.
- Girdileri değiştirmeden $a = \text{ilk}$, $b = \text{ikinci}$ çalışma kopyalarını oluşturun.
- $b \neq 0$ olduğu sürece $\text{kalan} = a \% b$, $a = b$, $b = \text{kalan}$ güncellemelerini sırayla uygulayın.
- Döngü bitince EBOB'u a olarak alın.
- İlk girdiyi EBOB'a bölün; bu bölüm tam sayıdır.
- Bölümü çarpmadan önce long türüne dönüştürüp ikinci girdiyi bununla çarpın.
- EKOK'u yazdırın.

EBOB ortak çarpanların bir kopyasını temsil eder; iki sayıyı doğrudan çarpmak bu ortak kısmı iki kez içerir. EBOB'a bölmek fazladan kopyayı çıkarır. ilk / ebob bölümünde kalan yoktur. Öklid döngüsü her adımda ikinci çalışma değerini küçülttüğünden sonlanır; sonrasında yalnızca bir bölme ve bir çarpma yapılır.

Örneğin 12 ve 18 için EBOB 6'dır. Önce $12 / 6 = 2$, sonra $2 * 18 = 36$ bulunur. İlk sayının döngü sonunda 6'ya dönüşmüş çalışma kopyası, 12 değerinin yerine çarpıma konulmamalıdır.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("İlk sayı (1..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ilk) || ilk < 1)
{
    Console.WriteLine("Hata: İlk sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("İkinci sayı (1..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ikinci) || ikinci < 1)
{
    Console.WriteLine("Hata: İkinci sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

int a = ilk;
int b = ikinci;
while (b != 0)
{
    int kalan = a % b;
    a = b;
    b = kalan;
}

int ebob = a;
long ekok = (long)(ilk / ebob) * ikinci;
Console.WriteLine("EKOK: " + ekok.ToString(CultureInfo.InvariantCulture));
```

(long)(ilk / ebob) dönüşümü çarpımdan önce yapılır; diğer çarpan da çarpma sırasında long olarak değerlendirilir. Yalnızca sonucun saklandığı değişkeni long yapmak yeterli değildir: long ekok = ilk * ikinci; ifadesinin sağındaki iki int önce int olarak çarpılır ve sonuç atanmadan taşabilir.

Örnek çalıştırmalar

Girdiler ilk ve ikinci sayı sırasındadır. İstem metinleri sonuç sütununa dahil değildir.

Girdi	Sonuç	Açıklama
12, 18	EKOK: 36	EBOB 6 kullanılır.
18, 12	EKOK: 36	Girdi sırası sonucu değiştirmez.
7, 9	EKOK: 63	Aralarında asal girdiler doğrudan çarpılır.
24, 24	EKOK: 24	Eşit sayılarda sonuç sayının kendisidir.
1, 1	EKOK: 1	Her iki alt sınır.
1, 2147483647	EKOK: 2147483647	Bir girdinin 1 olması.
2147483647, 2147483647	EKOK: 2147483647	Büyük ortak bölen çarpımdan önce çıkarılır.

Girdi	Sonuç	Açıklama
2147483647, 2147483646	EKOK: 4611686011984936962	EBOB 1; sonuç int sınırını aşar.
İlk girdi 0	Hata: İlk sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.	Pozitif girdi şartı.
12, -3	Hata: İkinci sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.	Negatif ikinci sayı reddedilir.
İlk girdi 2147483648	Hata: İlk sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.	int sınırı aşılır.
12, kat	Hata: İkinci sayı 1 ile 2147483647 arasında bir tamsayı olmalıdır.	Metin girişi reddedilir.

Sınır durumları

- Sıfır veya negatif girdi reddedildiğinden bulunan EBOB en az 1'dir; EBOB'a bölmede sıfır kullanılmaz.
- Girdiler eşitse EKOK aynı değerdir; 1 ile başka sayının EKOK'u diğer sayıdır.
- ilk / ebob tamsayı bölmesi tamdır; EBOB, ilk sayının bölenidir.
- Öklid için kullanılan a ve b kopyaları değiştirilir; ilk ve ikinci korunur.
- En büyük olası çarpım $(2147483647)^2 = 4611686014132420609$ olur. EKOK bu çarpımdan büyük olamaz; long üst sınırı 9223372036854775807 olduğundan kabul edilen girdilerde sonucun taşması mümkün değildir.
- Boş giriş, ondalık sayı, metin ve int dışındaki değerler alan doğrulamasında reddedilir.

Kazanımlar

- Öklid döngüsünün sonucunu ikinci bir hesaplama adımında kullanma.
- Çalışma kopyalarıyla başlangıç girdilerini hesaplama boyunca koruma.
- EBOB ve EKOK ilişkisini 12 ile 18 üzerinden ara değerlerle açıklama.
- Çarpma öncesi tür dönüşümünün neden gerekli olduğunu büyük girdilerle gösterme.
- Girdi üst sınırından hareketle long sonuç aralığının yeterliliğini hesaplama.

Alıştırmalar

- EBOB'u ve EKOK'u iki ayrı sonuç satırında gösterin; 12 ve 18 için aradaki ilişkiyi doğrulayın.
- 6, 8 ve 12 sayılarına ait üçlü EKOK'u iki aşamada hesaplayın. Ara EKOK'un int sınırını aşabileceği durumda çalışma türlerinin nasıl değiştirilmesi gerektiğini açıklayın.
- Döngü kopyaları yerine doğrudan girdileri değiştiren hatalı bir sürümü 12 ve 18 ile izleyin; EKOK hesabında hangi bilginin kaybolduğunu gösterin.

030 Bir sayıyı asal çarpanlarına ayırma

Düzey: İleri Önkoşullar: 021, 027, 028, 029

Kavramlar: Asal çarpanlara ayırma, Değişen döngü sınırı, Tekrarlı bölme, Aday ilerletme, Çarpan çokluğu

Problem tanımı

En az 2 olan bir tamsayıyı asal çarpanlarına ayırın. Çarpanları küçükten büyüğe ve tekrarlarını koruyarak tek satırda yazdırın. Örneğin $12 = 2 * 2 * 3$ olduğundan sonuç Asal çarpanlar: 2 2 3 olmalıdır. Çıktıdaki çarpanların çarpımı başlangıç girdisini vermelidir.

Bir aday kalan sayıyı bölüyorsa bu çarpanı yazdırıp kalanı küçültün; aynı çarpan yeniden bulunabileceksa adayı değiştirmeyin. Bölünmüyorsa sonraki adaya geçin. Kalan küçüldüğü için arama sınırı da değişir. Her adayı ayrıca asallık testinden geçirmek gerekli değildir: daha küçük çarpanlar çıkarıldığı için kalan sayıyı bölen sıradaki aday zaten asaldır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	2 ile 2147483647 arasında int.
Çıktı	Asal çarpanlar	Asal çarpanlar: önekiyle, aralarında bir boşluk bulunan artan çarpanlar.

Sayı (2..2147483647): istemine tek tamsayı girin. Örneğin 36 için çıktı Asal çarpanlar: 2 2 3 3 olur. Geçersiz girişte Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır. yazılır. 1, asal çarpanı bulunmadığı için bu çalışmanın giriş aralığına dahil değildir.

Algoritma

1. Girdiyi okuyup 2 ile 2147483647 aralığını doğrulayın.
2. kalan değerini girdiye, aday değerini 2'ye eşitleyin.
3. Çıktı öneki olan Asal çarpanlar: metnini satırı bitirmeden yazın.
4. $ad\ ay \leq kalan / ad\ ay$ olduğu sürece döngüyü sürdürün.
5. Kalan adayın katıysa bir boşlukla beraber adayı yazın ve $kalan /= ad\ ay$ uygulayın. Aynı adayla tekrar deneyin.
6. Kalan adayın katı değilse adayı bir artırın.
7. Döngü bittiğinde kalan 1'den büyükse bu son asal çarpanı da yazın.
8. Sonuç satırını bitirin.

Her adımda başlangıç sayısı, şimdiye kadar yazılmış çarpanların çarpımı ile kalan değerinin çarpımına eşittir. Bölme yolunda kalan en az 2 kat küçülür; diğer yolda aday bir artar. Pozitif kalan büyümmez ve aday gerilemez; bu nedenle döngü sonlanır. Koşul yanlış olduğunda kalan bileşik olsaydı kareköküne kadar bir böleni bulunması gerekirdi. Bu aralıktaki tüm adaylar denendiği için 1'den büyük son kalan asaldır.

84 için döngüdeki değişim şöyle izlenir. Son satır döngü sonrası işlemdir.

Kalan	Aday	İşlem	Yazılan çarpan	Yeni kalan / aday
84	2	Bölünür	2	42 / 2

Kalan	Aday	İşlem	Yazılan çarpan	Yeni kalan / aday
42	2	Bölünür	2	21 / 2
21	2	Bölünmez	Yok	21 / 3
21	3	Bölünür	3	7 / 3
7	3	3 <= 7 / 3 yanlış; son kalanı yaz	7	Döngü bitti

C# çözümü

```

using System;
using System.Globalization;

Console.WriteLine("Sayı (2..2147483647): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 2)
{
    Console.WriteLine("Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır.");
    return;
}

int kalan = n;
int aday = 2;
Console.WriteLine("Asal çarpanlar:");
while (aday <= kalan / aday)
{
    if (kalan % aday == 0)
    {
        Console.WriteLine(" " + aday.ToString(CultureInfo.InvariantCulture));
        kalan /= aday;
    }
    else
    {
        aday++;
    }
}

if (kalan > 1)
{
    Console.WriteLine(" " + kalan.ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();

```

Çarpandan önce boşluk yazılması ilk ve sonraki çarpanların aynı biçimde eklenmesini sağlar. Satır sonunda fazladan boşluk bırakılmaz. `kalan /= aday`, `kalan = kalan / aday` işleminin kısa yazımıdır. Adayı her turda artırmak, 8 gibi sayılarda tekrar eden 2 çarpanlarını kaybettirir.

Örnek çalıştırmalar

İstem metni sonuç sütununa dahil değildir. Çarpanlar tekrarlarıyla gösterilir.

Girdi	Sonuç	Açıklama
2	Asal çarpanlar: 2	En küçük geçerli sayı; gövde çalışmaz.
13	Asal çarpanlar: 13	Asal sayının tek çarpanı kendisidir.

Girdi	Sonuç	Açıklama
8	Asal çarpanlar: 2 2 2	Aynı adayla tekrar bölme.
36	Asal çarpanlar: 2 2 3 3	İki farklı asalın tekrarları.
84	Asal çarpanlar: 2 2 3 7	Değişen kalan sınırı ve son çarpan.
2147117569	Asal çarpanlar: 46337 46337	Büyük bir asalın karesi; eşitlik sınırı.
2147483646	Asal çarpanlar: 2 3 3 7 11 31 151 331	Üst sınıra yakın bileşik sayı.
2147483647	Asal çarpanlar: 2147483647	En büyük int değeri asaldır.
1	Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır.	Asal çarpanı olmayan sayı reddedilir.
0	Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır.	Alt sınır dışıdır.
2147483648	Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır.	int sınırı aşılar.
çarpan	Hata: Sayı 2 ile 2147483647 arasında bir tamsayı olmalıdır.	Metin girişi reddedilir.

Sınır durumları

- 1 için boş bir çarpan satırı üretilmez; sayı doğrulamada reddedilir.
- Başlangıç girdisi asal ise döngü hiçbir çarpan yazmaz; son kalan başlangıç sayısı olarak yazılır.
- Tam karelerde sınırdaki aday denenmelidir. $\leq$ yerine $<$ kullanılması 4 veya büyük asal karelerinde ayrıştırmayı eksik bırakır.
- Bölünme halinde aday sabit kalır; aynı asalın çokluğu böyle korunur.
- $\text{aday} \leq \text{kalan} / \text{aday}$ koşulu değişen kalanla yeniden hesaplanır ve taşabilecek aday $\text{aday} * \text{aday}$ çarpımından kaçınır.
- Bu giriş aralığında son kalan 1'den büyüktür: döngüye giriş koşulu $\text{kalan} \geq \text{aday} * \text{aday}$ anlamına geldiği için her başarılı bölme sonrasında kalan en az aday kadardır. Son kontrol bu asal kalanın yazdırılmasını sağlar.
- Boş giriş, ondalık sayı, negatif sayı ve int dışındaki değerler tek hata satırıyla reddedilir.

Kazanımlar

- Bir sayının asal çarpanlarını artan sırada ve tekrarlarıyla üretme.
- Bölünebilirliğe göre kalan küçültme veya aday artırma yollarını ayırma.
- 84 için aday, kalan ve yazılan çarpanları adım adım izleme.
- Daha küçük çarpanlar çıkarıldıktan sonra bölünen adayın neden asal olduğunu açıklama.
- Döngü sonunda kalan büyük çarpanın neden asal olduğunu arama sınırıyla gerekçelendirme.
- Tek satırdaki çarpanlar arasında birer boşluk kullanıp sondaki fazladan boşluğu önleme.

Alıştırımlar

1. Çarpanların toplam sayısını bir sayaçla hesaplayın; 13, 8 ve 36 için sırasıyla 1, 3 ve 4 bulunduğunu doğrulayın.
2. Her çarpanı yazdırırken bir long biriktiricide çarpın; işlem sonunda bu çarpımın başlangıç sayısına eşit olduğunu gösterin.
3. Ardışık aynı çarpanları sayarak sonucu $36 = 2^2 * 3^2$ biçiminde yazdırın; tek çarpan ve üs 1 durumları için çıktı biçimini tanımlayın.

Bölüm 4: İç içe döngüler ve kısıtlı arama

031 N × N çarpım tablosu oluşturma

Düzye: İleri Önkoşullar: 008, 025

Kavramlar: İç içe döngü, Satır ve sütun, İç sayacı yeniden başlatma, Sabit genişlik, Çıktı düzeni

Problem tanımı

1 ile 10 arasında verilen N değeri için 1'den N'ye kadar sayıların çarpım tablosunu oluşturun. Tablonun satır ve sütun numaraları 1'den başlar. Her hücrede satır numarası ile sütun numarasının çarpımı bulunur. Örneğin üçüncü satırın ikinci hücresinde $3 \times 2 = 6$ yazılır.

Çıktı tam N satırdan oluşmalıdır; her satırda N ürün bulunmalıdır. Sayıları sağa yaslanan dört karakterlik alanlarda yazdırın. Tablonun ayrıca başlığı, ayraçları veya Sonuç : öneki yoktur. Değerleri bir dizide biriktirmeden, her hücreyi hesaplandığı anda yazdırın. Bu problemde daha önce tek boyutta kullanılan döngü, satır ve sütun için iki ayrı sayaca genişletilir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 10 arasında int; satır ve sütun sayısı.
Ara değer	satır, sütun	1 ile N arasında dolaşan iki bağımsız sayaç.
Çıktı	Çarpım tablosu	N satır; her satır dört karakterlik N sayı alanı içerir.

N (1..10): istemine tek tamsayı girin. Geçersiz girişte Hata: N 1 ile 10 arasında bir tamsayı olmalıdır. yazılır; tablo üretilmez. Sayının solundaki boşluklar alanın parçasıdır. Satır sonuna ayrıca boşluk eklenmez; son sayı yazıldıktan sonra yalnızca yeni satıra geçilir.

Algoritma

1. N değerini okuyup 1 ile 10 aralığını doğrulayın.
2. Dış döngüde satır sayacını 1'den N'ye kadar ilerletin.
3. Her dış döngü yinelemesinde iç döngünün sütun sayacını yeniden 1 yapın.
4. İç döngüde sütunu 1'den N'ye kadar ilerletin; her hücre için satır * sütun hesaplayın.
5. Ürünü tamsayı metnine çevirip PadLeft(4) ile dört karakterlik alana sağa yaslayarak yazdırın.
6. İç döngü tamamlandığında satırı bitirin; sonraki dış döngü yinelemesine geçin.
7. N. satır tamamlanınca programı bitirin.

Dış döngünün gövdesine her girişte iç for ifadesinin başlangıç adımı yeniden çalışır. Önceki satır bittiğinde sütun N + 1'e ulaşmış olsa da yeni satırda tekrar 1 olur. Böylece her satırın ilk ürünü satır * 1'dir. İç döngü başında, o satırın 1'den sütun - 1'e kadar hücreleri yazılmıştır. Dış döngü başında ise önceki satırlar tamamlanmıştır. N² ürün ve N satır sonu üretilir.

N = 3 için satır geçişi aşağıdaki gibi izlenir. Tablo yalnızca sayaçların görevlerini gösterir; tam çıktı örnekler bölümündedir.

Adım	Satır	Sütun	İşlem
İç döngü başlar	1	1	1 yazılır.

Adım	Satır	Sütun	İşlem
Aynı satır ilerler	1	2, sonra 3	2 ve 3 yazılır.
İç koşul yanlış olur	1	4	Yeni satıra geçilir.
Dış döngü ilerler	2	Yeniden 1	2 yazılır; iç sayaç sıfırlanır.
Aynı satır ilerler	2	2, sonra 3	4 ve 6 yazılır.
Son satır başlar	3	Yeniden 1	Sırasıyla 3, 6 ve 9 yazılır.

Buradaki "sıfırlama", sayacı başlangıç değeri olan 1'e döndürmek anlamındadır; sütunun değeri 0 yapılmaz.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("N (1..10): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 10)
{
    Console.WriteLine("Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.");
    return;
}

for (int satir = 1; satir <= n; satir++)
{
    for (int sutun = 1; sutun <= n; sutun++)
    {
        int urun = satir * sutun;
        Console.Write(urun.ToString(CultureInfo.InvariantCulture).PadLeft(4));
    }
    Console.WriteLine();
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Console.WriteLine satırı bitirmediği için aynı satırın hücreleri yan yana yazılır. Parametresiz Console.WriteLine yalnızca iç döngünün dışında olduğundan her tamamlanmış satırda bir kez çalışır. PadLeft(4), hesaplamayı değiştirmez; yalnızca sayı metninin soluna gerekli boşlukları ekler.

Örnek çalıştırmalar

Aşağıdaki metin blokları, istem ve kullanıcının yanıtı dışındaki tam tablo çıktılarıdır. Soldaki boşluklar korunmalıdır.

N = 1 için tek hücre dört karakterlik alanda yazılır:

1

N = 3 için:

```
1  2  3
2  4  6
3  6  9
```

N = 4 için iki basamaklı ürünler de sütunlarını korur:

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

Üst sınırın büyük çıktısını aşağıdaki özelliklerle denetleyin. Bu satır tam çıktı gösterimi değildir.

Girdi	Beklenen özellik
10	10 satır; her satır 40 karakter; son hücre 100, bu hücrenin alanı 100; toplam 100 ürün.

Geçersiz örneklerde tablo yerine aşağıdaki tek hata satırı yazılır.

Girdi	Hata çıktısı
0	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
11	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
-2	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
3.5	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
üç, boş satır veya giriş sonu	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.

Sınır durumları

- N = 1 için iki döngü de bir kez çalışır; tek ürün ve bir satır sonu oluşur.
- N = 10 için en büyük ürün 100'dür. Üç basamaklı bu sayı dört karakterlik alana sığar; bütün ürünler int aralığındadır.
- Sütun her yeni satırda 1'den başlamalıdır. İç sayacı döngülerin dışında bir kez başlatmak sonraki satırların boş kalmasına yol açabilir.
- Yeni satır komutu iç döngünün gövdesinde olursa ürünler tek tek alt alta yazılır; istenen tablo oluşmaz.
- Alan genişliğinin çarpımından her satırın $N \times 4$ karakter olduğu hesaplanabilir. Satır sonu karakterleri bu genişliğe dahil değildir.
- Negatif, sıfır, üst sınır dışı, ondalıklı, metin veya boş giriş ve giriş sonu tek hata satırıyla reddedilir.

Kazanımlar

- İki boyutlu bir çıktıyı dış satır ve iç sütun döngüleriyle kurma.
- İç döngü sayacının her dış yinelemede neden yeniden başlatıldığını açıklama.
- Hücre hesabını döngü sayaçlarından üretme.
- Write ve WriteLine konumlarıyla satır düzenini yönetme.
- Sabit genişlikli sayı alanlarıyla farklı basamak sayısındaki değerleri hizalama.

Alıřtırmalar

1. Satır ve sřtřn sayılarını ayrı iki girdi olarak alın. 2×4 ve 4×2 tablolarını karşılařtırıp dıř ve iç dōngř sınırlarını açıklayın.
2. Çarpım yerine satır + sřtřn yazdıran toplama tablosunu oluřturun. $N = 3$ için břtřn hřcrelerin beklenen deęerlerini önceden hesaplayın.
3. Her satır sonunda o satırın řrřnler toplamını yazdırın. Toplam biriktiricisini dıř dōngřnřn gōvdesinde bařlatın ve neden orada olması gerektięini açıklayın.

032 Yıldızlarla dik üçgen oluşturma

Düzey: İleri Önkoşullar: 031

Kavramlar: İç içe döngü, Değişen iç sınır, Satır numarası, Karakter çıktısı, Üçgensel toplam

Problem tanımı

1 ile 20 arasında verilen N değeri için yıldız karakterlerinden N satırlı bir dik üçgen oluşturun. İlk satırda bir, ikinci satırda iki yıldız bulunur; her sonraki satır bir yıldız daha içerir. Son satırda N yıldız vardır. Bütün satırlar aynı sol kenardan başlar.

Bir satırın uzunluğu, satır numarasına eşittir. Önceki çarpım tablosunda iç döngünün sınırı her satırda sabitti. Bu problemde iç sınır dış döngünün satır sayacına bağlıdır. Yıldızları bir dizide veya uzun bir metinde biriktirmeden, iç döngüde birer birer yazdırın. Yıldızların arasına veya satır sonuna boşluk koymayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 20 arasında int; üçgenin satır sayısı.
Ara değer	satır	1'den N'ye kadar satır numarası.
Çıktı	Dik üçgen	Satır R'de R adet *; toplam N satır.

N (1..20): istemine tamsayı girin. Geçersiz girişte Hata: N 1 ile 20 arasında bir tamsayı olmalıdır. yazılır. Geçerli çıktıda başlık, sayı, ayrıç veya Sonuç: önceki bulunmaz; yalnızca yıldız satırları yazılır.

Algoritma

1. N değerini okuyup 1 ile 20 aralığını doğrulayın.
2. Dış döngüde satır değerini 1'den N'ye kadar ilerletin.
3. Her satırın başında iç döngünün sütun sayacını 1 yapın.
4. $sutun \leq satır$ olduğu sürece bir yıldız yazdırıp sütunu artırın.
5. İç döngü tamamlandığında satırı bitirin.
6. Son satır tamamlanınca programı bitirin.

Dış döngü başında önceki satırlar tam olarak yazılmıştır. İç döngü başında mevcut satırda $sutun - 1$ yıldız vardır. Bir yıldız yazıp sütunu artırınca bu ilişki korunur. Koşul yanlış olduğunda $sutun = satır + 1$ değerindedir; tam satır yıldız yazılmıştır. N = 4 için iç gövde sırasıyla 1, 2, 3 ve 4 kez çalışır; toplam 10 yıldız üretir. Genel toplam $N \times (N + 1) / 2$ 'dir.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("N (1..20): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 20)
{
    Console.WriteLine("Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.");
    return;
}

for (int satir = 1; satir <= n; satir++)
{
    for (int sutun = 1; sutun <= satir; sutun++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. İç döngünün üst sınırı n olsaydı her satır N yıldız içeren bir kare oluşurdu. Üçgen için bu sınırın satır numarası olması gerekir. Her satırın başında sutun = 1 yeniden çalışır; satır içindeki tekrarlar önceki satırdan devam etmez.

Örnek çalıştırmalar

Metin blokları, istem ve kullanıcının yanıtı dışındaki tam desen çıktılarıdır. Satırların başında veya sonunda boşluk bulunmaz.

N = 1 için:

*

N = 4 için:

*
**

N = 5 için:

*
**

Aşağıdaki üst sınır satırı tam çıktı yerine denetlenecek özellikleri belirtir.

Girdi	Beklenen özellik
20	20 satır; ilk satır 1, son satır 20 yıldız; toplam 210 yıldız; en geniş satır 20 karakter.

Geçersiz girişlerde yalnızca ilgili hata satırı yazılır.

Girdi	Hata çıktısı
0	Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.

Girdi	Hata çıktısı
21	Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.
-4	Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.
4.5	Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.
dört, boş satır veya giriş sonu	Hata: N 1 ile 20 arasında bir tamsayı olmalıdır.

Sınır durumları

- $N = 1$ için tek yıldız ve bir satır sonu yazılır; özel bir dal gerekmez.
- İç koşuldaki eşitlik korunmalıdır. $\text{sutun} < \text{satır}$ kullanılması her satırdan bir yıldız eksiltir ve ilk satırı boş bırakır.
- İç sayaç her satır için 1'den başlar; önceki satırın son sütun değeri yeni satırda kullanılmaz.
- En büyük çıktıda yalnızca 210 yıldız yazılır. Üst sınır hem sayacı hem ekran çıktısının uzunluğunu küçük tutar.
- Yıldızların arasına boşluk eklemek geometrik genişliği değiştirir; bu sürümde her yıldız tam bir sütun kaplar.
- Boş, metin, ondalıklı, aralık dışı giriş ve giriş sonu reddedilir; kısmi desen oluşmaz.

Kazanımlar

- İç döngü sınırını dış sayaca bağlayarak satır uzunluğunu değiştirme.
- Bir satırın yıldız adedini döngü koşulundan çıkarma.
- Üçgen çıktıda toplam işlem adedini üçgensel toplamla hesaplama.
- Sabit ve değişken iç döngü sınırlarının ürettiği şekilleri karşılaştırma.
- Satır sonu işlemini doğru döngü düzeyinde tutma.

Alıştırmalar

1. İlk satır N , son satır 1 yıldız içeren ters dik üçgeni oluşturun. İç sınırı dış sayaca bağlı bir ifadeyle kurun.
2. Yıldız yerine satır numarasını yazdırın; satırdaki değerleri birer boşlukla ayırın. $N = 4$ için beklenen dört satırı önce elle yazın.
3. Satır R 'de R yerine $2 \times R$ yıldız yazdırın. $N = 5$ için toplam karakter adedini hesaplayıp çıktıyla doğrulayın.

033 Floyd sayı üçgeni oluşturma

Düzey: İleri Önkoşullar: 031, 032

Kavramlar: İç içe döngü, Satırlar arası durum, Artan sayı, Değişen iç sınır, Sabit genişlik

Problem tanımı

1 ile 10 arasında verilen N değeri için Floyd sayı üçgeni oluşturun. İlk satırda 1, ikinci satırda 2 ve 3, üçüncü satırda 4, 5 ve 6 yer alır. Her satır, satır numarası kadar sayı içerir. Sayılar 1'den başlayıp her hücrede bir artar; yeni satıra geçerken numaralandırma yeniden başlamaz.

Sayıları sağa yaslanan üç karakterlik alanlarda yazdırın. Çıktı N satırdan oluşur; başlık veya Sonuç: öneki içermez. Hücre değerini satır ve sütun çarpımından hesaplamak yerine, satırlar arasında korunan ayrı bir numara değişkeni kullanın. Böylece döngü sayaçları yerleşimi, bu değişken ise sıradaki yazılacak değeri yönetir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 10 arasında int; satır sayısı.
Ara değer	numara	1'den başlayan, her yazdırmadan sonra artan int.
Çıktı	Sayı üçgeni	Satır R'de R sayı; her sayı üç karakterlik alanda.

N (1..10): istemine tamsayı girin. Geçersiz girişte Hata: N 1 ile 10 arasında bir tamsayı olmalıdır. yazılır. Alanların solundaki boşluklar korunur; son sayıdan sonra ek bir boşluk yazılmaz. N = 10 için son yazılan değer 55'tir.

Algoritma

1. N değerini okuyup 1 ile 10 aralığında doğrulayın.
2. Döngülerin dışında numara değerini 1 yapın.
3. Dış döngüde satır sayacını 1'den N'ye ilerletin.
4. Her satır için sütunu 1'den o satırın numarasına kadar ilerletin.
5. Her iç yinelemede numara değerini PadLeft(3) ile yazdırın; sonra numarayı bir artırın.
6. İç döngü tamamlandığında satırı bitirin.
7. N. satır tamamlanınca programı bitirin.

Her hücreden önce numara, şimdiye kadar yazılan hücrelerin toplam adedinden bir fazladır. Değer yazılıp artırılınca bir sonraki hücre için hazır olur. Sütun sayacı her satırda 1'e döner; numara ise dış döngünün dışında başlatıldığı için satırlar arasında korunur. N = 4 için satırlardaki son değerler 1, 3, 6 ve 10'dur. Genel olarak R. satırın son değeri $R \times (R + 1) / 2$ olur.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("N (1..10): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 10)
{
    Console.WriteLine("Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.");
    return;
}

int numara = 1;
for (int satir = 1; satir <= n; satir++)
{
    for (int sutun = 1; sutun <= satir; sutun++)
    {
        Console.Write(numara.ToString(CultureInfo.InvariantCulture).PadLeft(3));
        numara++;
    }
    Console.WriteLine();
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Yazdırma artırmadan önce yapılır; aksi sırada ilk değer 2 olurdu. numara = 1 ifadesini dış döngünün gövdesine taşımak her satırda numaralandırmayı yeniden başlatır ve Floyd üçgeni kuralını bozar. Sayılar bir dizide saklanmaz; çıktı ilerledikçe sıradaki sayı üretilir.

Örnek çalıştırmalar

Metin blokları istem ve kullanıcının yanıtı dışında tam çıktıyı gösterir. Sabit genişlikli alanların baştaki boşlukları korunmalıdır.

N = 1 için:

1

N = 3 için:

1
2 3
4 5 6

N = 4 için tek basamaklı ve iki basamaklı değerler aynı genişlikli alandadır:

1
2 3
4 5 6
7 8 9 10

Üst sınırın tam çıktısını göstermeden aşağıdaki özellikleri denetleyin.

Girdi	Beklenen özellik
10	10 satır; toplam 55 sayı; son satır 46'dan 55'e kadar 10 sayı içerir ve 30 karakterdir.

Hatalı girişlerde aşağıdaki tek satır yazılır ve üçgen üretilmez.

Girdi	Hata çıktısı
0	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.

Girdi	Hata çıktısı
11	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
-1	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
3.2	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.
üç, boş satır veya giriş sonu	Hata: N 1 ile 10 arasında bir tamsayı olmalıdır.

Sınır durumları

- N = 1 için 1 yazılır; numara daha sonra 2 olur ancak bu ara değer çıktıya eklenmez.
- N = 10 için toplam hücre sayısı 55'tir. Son yazılan sayı 55, döngü sonundaki numara 56 olur; bunlar farklı değerlerdir.
- En büyük sayı iki basamaklı olduğundan üç karakterlik alan yeterlidir; tüm sayaçlar int aralığında kalır.
- Sütunun satırlar arasında yeniden başlaması gerekirken numaranın yeniden başlamaması gerekir. İki değişkenin yaşam alanları farklı görevleri destekler.
- Satır R'nin genişliği R × 3 karakterdir; satır sonu karakterleri bu hesaba katılmaz.
- Aralık dışı veya sayı olmayan giriş, ondalıklı yazım, boş satır ve giriş sonu üçgen başlamadan reddedilir.

Kazanımlar

- Yerleşim sayaçları ile hücreye yazılan değerın görevlerini ayırma.
- Bir değişkeni döngü dışında başlatarak satırlar arasında durumu koruma.
- Yazdırma ve artırma sırasının ilk ve son değerlere etkisini açıklama.
- Son yazılan sayı ile döngü sonundaki sıradaki sayıyı ayırt etme.
- Hücre adedini üçgensel toplamla doğrulama.

Alıştırmalar

1. Başlangıç numarasını ayrı girdi olarak alın. 5'ten başlayan üç satırlı çıktının son değerini önceden hesaplayın.
2. Sayıları bir yerine iki artırarak yalnızca pozitif tek sayılardan bir üçgen oluşturun. N = 4 için son değeri bulun.
3. Her satır sonunda o satırda yazılmış sayıların toplamını ayrı bir çıktı alanına ekleyin. Satır toplamını sıfırlayın fakat numarayı sıfırlamayın.

034 Ortalanmış yıldız piramidi oluşturma

Düzey: İleri Önkoşullar: 032, 033

Kavramlar: İç içe döngü, Ardışık iç döngüler, Başlangıç boşlukları, Tek sayılı genişlik, Merkez hizalama

Problem tanımı

1 ile 15 arasında verilen N değeri için N satırlı ortalanmış bir yıldız piramidi oluşturun. İlk satır bir, ikinci satır üç, üçüncü satır beş yıldız içerir. Her sonraki satır iki yıldız genişler. Satır R için yıldız adedi $2 \times R - 1$ 'dir; son satır $2 \times N - 1$ yıldız içerir.

Üst satırları aynı merkez sütuna hizalamak için satır R'nin başına $N - R$ boşluk yazın. Bu boşluklardan sonra yıldızlar kesintisiz gelir. Satırın yıldızlarından sonra boşluk eklemeyin. Piramit için sol girinti yeterlidir; satır sonuna sağ girintiyi doldurmak gerekmez. Aynı dış yineleme içinde önce boşluklar, sonra yıldızlar için iki ayrı iç döngü çalıştırın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 15 arasında int; piramidin yüksekliği.
Ara değer	satır	1 ile N arasında satır numarası.
Çıktı	Piramit	Satır R'de $N - R$ başlangıç boşluğu ve $2 \times R - 1$ yıldız.

N (1..15): istemine tamsayı girin. Geçersiz girişte Hata: N 1 ile 15 arasında bir tamsayı olmalıdır. yazılır. Başlık veya Sonuç: önceki bulunmaz. Baştaki boşluklar şeklin hizasını belirler; satır sonunda fazladan boşluk yoktur.

Algoritma

1. N değerini okuyup 1 ile 15 aralığında doğrulayın.
2. Dış döngüde satırı 1'den N'ye kadar ilerletin.
3. İlk iç döngüyle $N -$ satır kadar boşluk yazdırın.
4. İlk iç döngü bittikten sonra ikinci iç döngüyle $2 \times \text{satır} - 1$ yıldız yazdırın.
5. İki iç döngü de tamamlandığında satırı bitirin.
6. Son satır tamamlandığında programı bitirin.

Satır başında önceki satırlar tamamlanmıştır. Boşluk döngüsü bittiğinde yıldızların başlayacağı sütun hazırdır. Yıldız döngüsü başladığında boşluk üretimi bitmiş olur; bu iki döngü birbirinin içinde değil, dış döngünün içinde art arda yer alır. Yıldız sayısı tek olduğundan bir orta yıldız vardır. Bu yıldızdan önce $N - R$ boşluk ve $R - 1$ yıldız bulunduğu için orta yıldız her satırda N. sütundadır.

$N = 4$ için boşluk adetleri 3, 2, 1, 0; yıldız adetleri 1, 3, 5, 7 olur. Son satırda boşluk döngüsü hiç çalışmaz. Toplam yıldız sayısı ilk N tek sayının toplamı olan N^2 'dir.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("N (1..15): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 15)
{
    Console.WriteLine("Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.");
    return;
}

for (int satir = 1; satir <= n; satir++)
{
    for (int bosluk = 0; bosluk < n - satir; bosluk++)
    {
        Console.Write(" ");
    }
    for (int yildiz = 0; yildiz < 2 * satir - 1; yildiz++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. İki iç sayaç 0'dan başlayıp istenen adet kadar ilerler; bosluk < adet ve yildiz < adet koşulları tam bu kadar karakter üretir. Yıldız döngüsü boşluk döngüsünün gövdesine konulursa her boşluk için yıldız grubu tekrar edilir ve piramit bozulur. Satır sonu yalnızca iki iç döngü tamamlandıktan sonra yazılır.

Örnek çalıştırmalar

Metin blokları istem ve kullanıcının yanıtı dışındaki tam desen çıktılarıdır. Baştaki boşluklar şeklin parçasıdır; satır sonunda boşluk bulunmaz.

N = 1 için boşluk döngüsü çalışmaz:

*

N = 3 için merkez sütun 3'tür:

*

N = 4 için:

*

Üst sınırı tam desen yerine aşağıdaki çıktı özellikleriyle denetleyin.

Girdi	Beklenen özellik
15	15 satır; ilk satırda 14 boşluk ve 1 yıldız; son satırda 29 yıldız ve hiç boşluk yok; toplam 225 yıldız.

Geçersiz girişlerde desen yerine aşağıdaki hata satırı yazılır.

Girdi	Hata çıktısı
0	Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.
16	Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.
-3	Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.
3.5	Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.
üç, boş satır veya giriş sonu	Hata: N 1 ile 15 arasında bir tamsayı olmalıdır.

Sınır durumları

- $N = 1$ için boşluk sayısı 0, yıldız sayısı 1'dir; tek yıldız özel dal olmadan oluşur.
- Son satırda $N - \text{satır} = 0$ olur. Başlangıç koşulu yanlış olduğu için boşluk döngüsü gövdesi hiç çalışmaz.
- Yıldız sayısı her satırda en az 1 ve en fazla 29'dur; sayaç ifadeleri int aralığında kalır.
- Yıldızların bittiği sütun $N + R - 1$ 'dir. Satırlar sağ tarafta boşlukla doldurulmadığından satır uzunlukları eşit değildir; merkezleri yine aynıdır.
- Üst sınırdaki toplam başlangıç boşluğu 105, yıldız adedi 225'tir. Satır sonları bunların dışındadır.
- Aralık dışı, metin, ondalıklı veya boş giriş ve giriş sonu doğrulamada reddedilir; desen yazılmaya başlanmaz.

Kazanımlar

- Aynı dış döngü içinde ardışık iki iç döngüyle farklı karakter grupları üretme.
- Satır numarasından başlangıç boşluğu ve yıldız sayısı formüllerini türetme.
- Tek sayıda yıldızın merkezini ortak sütuna hizalama.
- Bir iç döngünün sıfır kez çalışmasının geçerli olabileceğini gösterme.
- Başlangıç boşluğu ile gereksiz satır sonu boşluğunu ayırt etme.

Alıştırmalar

1. Satırları N 'den 1'e doğru dolaşarak ters piramit üretin. $N = 4$ için boşluk ve yıldız sayılarını tabloya yazın.
2. Piramidin son satırından sonra iki yıldız genişliğinde bir gövde ekleyin. Gövdenin hangi sütundan başlayacağına açık bir hizalama kuralı belirleyin.
3. Her satırın orta yıldızını `|` karakteriyle değiştirin. Ortayı belirlemek için yıldız sayacını kullanın ve $N = 1$ durumunu ayrıca deneyin.

035 İçi boş dikdörtgen oluşturma

Düzey: İleri Önkoşullar: 013, 031, 034

Kavramlar: İç içe döngü, Sınır hücreleri, Mantıksal VEYA, Satır ve sütun konumu, Tek satır ve sütun

Problem tanımı

Satır ve sütun sayıları verilen bir dikdörtgenin yalnızca kenarlarını yıldızla, iç kısmını boşlukla çizin. Satır sayısı 1 ile 15, sütun sayısı 1 ile 30 arasında olmalıdır. Her çıktı satırı tam sütun sayısı kadar karakter içerir. İlk ve son satır bütünüyle yıldızdır; aradaki satırların yalnızca ilk ve son sütununda yıldız bulunur.

Bir hücrenin kenarda olması için şu dört koşuldan en az biri doğru olmalıdır: ilk satır, son satır, ilk sütun veya son sütun. Bu koşulları mantıksal VEYA ile birleştirin. Satır sayısı 1 ya da sütun sayısı 1 olduğunda bütün hücreler kenardadır; iç boşluk oluşmaz. Özel durumları ayrı çizim algoritmalarıyla değil, aynı sınır koşuluyla ele alın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	satirSayisi	1 ile 15 arasında int; dikdörtgenin yüksekliği.
Girdi	sutunSayisi	1 ile 30 arasında int; her satırın genişliği.
Çıktı	İçi boş dikdörtgen	Kenar hücreleri *, iç hücreleri boşluk; istenen sayıda satır.

Satır sayısı (1..15): ve Sütun sayısı (1..30): istemlerine sırasıyla iki ayrı satırda tamsayı girin. Satır geçersizse sütun istenmez ve Hata: Satır sayısı 1 ile 15 arasında bir tamsayı olmalıdır. yazılır. Sütun geçersizse Hata: Sütun sayısı 1 ile 30 arasında bir tamsayı olmalıdır. yazılır. Geçerli çıktıda başlık ve Sonuç: öneki bulunmaz.

Algoritma

1. Satır sayısını okuyup 1 ile 15 aralığında doğrulayın.
2. Sütun sayısını okuyup 1 ile 30 aralığında doğrulayın.
3. Dış döngüde satırı 1'den satır sayısına kadar dolaşın.
4. Her satırda iç döngüyle sütunu 1'den sütun sayısına kadar dolaşın.
5. Hücre ilk veya son satırdaysa ya da ilk veya son sütundaysa bir yıldız yazdırın.
6. Bu dört koşulun hepsi yanlışsa bir boşluk yazdırın.
7. İç döngü bittiğinde satırı bitirin; bütün satırlar bitince programı sonlandırın.

Her hücre tam bir kez değerlendirilir ve tam bir karakter üretir. İç döngü başında mevcut satırın önceki sütunları yazılmıştır; dış döngü başında önceki satırlar tamamlanmıştır. Kenar koşulu doğruysa hücrenin diğer koşullarını ayrıca sınıflandırmaya gerek yoktur. Örneğin bir köşe hem bir satır sınırında hem bir sütun sınırındadır, ancak yine yalnızca bir yıldız yazılır.

3 × 6 dikdörtgende ikinci satırın birinci ve altıncı sütunları kenardır. İkinci ile beşinci sütunlar ise dört koşulu da sağlamaz; dört boşlukla iç kısmı oluşturur. Toplam karakter adedi satır sayısı × sütun sayısıdır; satır sonları bu sayıya katılmaz.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Satır sayısı (1..15): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int satirSayisi) ||
    satirSayisi < 1 || satirSayisi > 15)
{
    Console.WriteLine("Hata: Satır sayısı 1 ile 15 arasında bir tamsayı olmalıdır.");
    return;
}

Console.Write("Sütun sayısı (1..30): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int sutunSayisi) ||
    sutunSayisi < 1 || sutunSayisi > 30)
{
    Console.WriteLine("Hata: Sütun sayısı 1 ile 30 arasında bir tamsayı olmalıdır.");
    return;
}

for (int satir = 1; satir <= satirSayisi; satir++)
{
    for (int sutun = 1; sutun <= sutunSayisi; sutun++)
    {
        bool kenar = satir == 1 || satir == satirSayisi ||
            sutun == 1 || sutun == sutunSayisi;
        if (kenar)
        {
            Console.Write("*");
        }
        else
        {
            Console.Write(" ");
        }
    }
    Console.WriteLine();
}
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. ||, kenar koşullarından en az biri doğru olduğunda kenar değerini doğru yapar. if / else her hücrede yalnızca bir karakter yazdırır. Dört ayrı if ile yazdırmak köşelerde birden fazla yıldız üretebilirdi. Doğrulamalar çizimden önce tamamlandığı için ikinci girdi geçersiz olduğunda bile kısmi dikdörtgen oluşmaz.

Örnek çalıştırmalar

Girdiler satır ve sütun sırasındadır. Metin blokları istemler ve kullanıcının yanıtları dışında tam çıktıları gösterir. İç boşluklar korunmalıdır.

1 ve 1 için tek hücre hem satır hem sütun sınırıdır:

*

3 ve 6 için:

```
*****
*      *
*****
```

4 ve 7 için:

```
*****
*       *
*       *
*****
```

1 ve 5 için iç satır bulunmaz:

```
*****
```

4 ve 1 için her hücre ilk sütundadır:

```
*
*
*
*
```

Üst sınırın tam çıktısı yerine aşağıdaki özellikler denetlenir.

Girdi	Beklenen özellik
15, 30	15 satır ve her satır 30 karakter; 86 yıldız, 364 iç boşluk; ilk ve son satırda 30 yıldız.
2, 5	2 satırın her biri 5 yıldız; iç boşluk yok.

Geçersiz örneklerde yalnızca ilgili hata satırı yazılır.

Girdi	Hata çıktısı
İlk girdi 0	Hata: Satır sayısı 1 ile 15 arasında bir tamsayı olmalıdır.
İlk girdi 16	Hata: Satır sayısı 1 ile 15 arasında bir tamsayı olmalıdır.
3, 0	Hata: Sütun sayısı 1 ile 30 arasında bir tamsayı olmalıdır.
3, 31	Hata: Sütun sayısı 1 ile 30 arasında bir tamsayı olmalıdır.
İlk girdi 2.5, üç, boş satır veya giriş sonu	Hata: Satır sayısı 1 ile 15 arasında bir tamsayı olmalıdır.
İlk girdi 3; ikinci girdi 2.5, altı, boş satır veya giriş sonu	Hata: Sütun sayısı 1 ile 30 arasında bir tamsayı olmalıdır.

Sınır durumları

- Tek satırda bütün hücreler ilk ve son satır koşulunu sağlar; yıldızlar kesintisizdir.
- Tek sütunda bütün hücreler ilk ve son sütun koşulunu sağlar; her satır tek yıldızdır.
- Satır veya sütun sayısı 2 ise iç bölge yoktur; bütün hücreler kenardadır.
- Her iki boyut en az 2 olduğunda yıldız sayısı $2 \times \text{satır sayısı} + 2 \times \text{sütun sayısı} - 4$ 'tür. Köşeler bu formülde iki kez sayılmamak için çıkarılır; tek boyutlu durumlarda bu formül doğrudan kullanılmaz.
- Her iki boyut en az 3 olduğunda iç boşluk sayısı $(\text{satır sayısı} - 2) \times (\text{sütun sayısı} - 2)$ 'dir. 15×30 için $13 \times 28 = 364$ boşluk ve $450 - 364 = 86$ yıldız bulunur.
- Satır sonu her zaman son kenar yıldızından sonra gelir; satır sonunda fazladan boşluk yoktur.
- Negatif, aralık dışı, ondalıklı, metin veya boş giriş ve giriş sonu ilgili alanın doğrulamasında reddedilir.

Kazanımlar

- Satır ve sütun konumlarından bir hücrenin sınırdaki olup olmadığını belirleme.
- Dört alternatif kenar koşulunu mantıksal VEYA ile birleştirme.
- Köşelerde birden çok koşul doğru olsa da tek karakter üretme.
- Tek satır ve tek sütun durumlarını genel koşulla doğru çizme.
- Çıktının genişliğini ve kenar karakter adedini bağımsız özelliklerle doğrulama.

Alıştırmalar

1. Kenarda *, iç bölgede boşluk yerine . yazdırın. 3×6 ve 1×6 için yıldız sayısının değişmediğini gösterin.
2. Kenar koşulunu tersine çevirip yalnızca iç bölgeye yıldız, kenarlara boşluk yazdırın. Tek satır ve tek sütun için çıktının nasıl tanımlanacağını açıklayın.
3. Çizim sırasında yıldız ve boşluk adetlerini iki sayaçla bulun. 1×1 , 4×1 ve 15×30 için sonuçları formüllerle karşılaştırın.

036 Pascal üçgeni oluşturma

Düzey: İleri Önkoşullar: 025, 031, 033

Kavramlar: İç içe döngü, Satır içi yineleme, Pascal katsayıları, Tam bölme, Sabit genişlikli alan

Problem tanımı

İstenen sayıda satır içeren Pascal üçgenini yazdırın. Satırları sıfırdan numaralandırın: 0 numaralı satır yalnızca 1, 1 numaralı satır iki tane 1 içerir. Her satırın ilk ve son değeri 1'dir. İç değerler önceki satırda üstündeki iki değer toplamına eşittir. Örneğin 4 numaralı satır 1 4 6 4 1 olur.

Önceki satırı bir dizide saklamadan, aynı satırdaki ardışık katsayıları hesaplayın. r satırında k konumundaki katsayı biliniyorsa sonraki katsayı $katsayi * (r - k) / (k + 1)$ ile bulunur. Bu problem iç içe döngülerin yanı sıra, önce çarpıp sonra tam bölmenin önemini gösterir. Çıktıda her katsayı dört karakter genişliğindeki bir alana sağa hizalanır; satırların başlangıcı sola hizalıdır ve bütün üçgen ortalanmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 10 arasında int; yazdırılacak satır sayısı.
Çıktı	Pascal satırları	$r = 0$ ile $r = n - 1$ arasında, satır başına $r + 1$ katsayı.
Biçim	Katsayı alanı	Dört karakter genişliğinde, soldan boşlukla doldurulmuş tamsayı.

Satır sayısı (1..10): istemine tek tamsayı girin. Sonuçların başında ek bir başlık veya sonuç öneki yazılmaz. İlk satırda 1'den önce üç boşluk bulunur. Geçersiz girişte Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır. yazılır.

Algoritma

1. Satır sayısını okuyup 1 ile 10 arasında bir tamsayı olduğunu doğrulayın.
2. Dış döngüde r değerini 0'dan $n - 1$ değerine kadar artırın.
3. Her yeni satırda katsayıyı 1 yapın.
4. İç döngüde k değerini 0'dan r dahil olacak şekilde artırın.
5. Katsayıyı dört karakter genişliğindeki alanda yazdırın.
6. Satırın son konumunda değilseniz sonraki katsayıyı $katsayi * (r - k) / (k + 1)$ ile hesaplayın.
7. İç döngü bitince satırı bitirin ve sonraki dış döngü adımına geçin.

İç döngü başında $katsayi$, r satırındaki k konumunun değeridir. İlk konum için bu değer 1'dir; ardışık katsayı ilişkisi her güncellemede korunur. Bölme çarpımdan sonra yapılır ve sonuç tamdır. Örneğin $4 * 3 / 2 = 6$ olur; önce $3 / 2$ tamsayı bölmesi yapılırsa 1 elde edilir ve yanlış katsayı bulunur. Her iki sayaç da sabit üst sınıra kadar arttığı için döngüler sonlanır.

4 numaralı satırın katsayıları şöyle hesaplanır. Son konumdan sonra yeni katsayı hesaplanmaz.

Konum k	Yazılan katsayı	Sonraki katsayının hesabı
0	1	$1 * 4 / 1 = 4$
1	4	$4 * 3 / 2 = 6$

Konum k	Yazılan katsayı	Sonraki katsayının hesabı
2	6	$6 * 2 / 3 = 4$
3	4	$4 * 1 / 4 = 1$
4	1	Son konum; hesaplama yok.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Satır sayısı (1..10): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 10)
{
    Console.WriteLine("Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.");
    return;
}

for (int r = 0; r < n; r++)
{
    long katsayi = 1;
    for (int k = 0; k <= r; k++)
    {
        Console.Write(katsayi.ToString(CultureInfo.InvariantCulture).PadLeft(4));
        if (k < r)
        {
            katsayi = katsayi * (r - k) / (k + 1);
        }
    }
    Console.WriteLine();
}
```

PadLeft(4), kısa sayının başına boşluk ekler; sayının değerini değiştirmez. katsayi her dış döngü adımında yeniden 1 olur. Böylece bir satırın son değeri sonraki satırın iç hesabına taşınmaz. Katsayıların tamamını saklayan bir dizi kullanılmaz.

Örnek çalıştırmalar

Aşağıdaki metin blokları giriş istemi dışındaki bütün çıktı satırlarını gösterir. Başlangıç boşlukları çıktı biçiminin parçasıdır.

Girdi 1 için tam çıktı:

```
1
```

Girdi 2 için tam çıktı:

```
1
1 1
```

Girdi 5 için tam çıktı:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Üst sınır için aşağıdaki tablo tam çıktıyı göstermez; üretilen çıktıda denetlenecek özellikleri özetler.

Girdi	Beklenen özellik veya denetim
10	10 satır; son satırın değerleri 1 9 36 84 126 126 84 36 9 1; son satır genişliği 40 karakter.

Aşağıdaki sonuçlar tek hata satırıdır; Pascal satırları üretilmez.

Girdi	Tam sonuç	Açıklama
0	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	Alt sınır dışı.
11	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	Üst sınır dışı.
satır	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	Metin.
2.5	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	Ondalık sayı.
Boş satır	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	Sayı verilmemiştir.
Hiç satır vermeden girişin sonu	Hata: Satır sayısı 1 ile 10 arasında bir tamsayı olmalıdır.	ReadLine() sonuç vermez.

Sınır durumları

- $n = 1$ için dış döngü bir kez, iç döngü bir kez çalışır; yalnızca ilk 1 yazılır.
- $r = 0$ ve her satırın son konumunda $k < r$ yanlış olduğu için sonraki katsayı hesabı yapılmaz.
- En büyük satır numarası 9'dur; en büyük katsayı 126 ve katsayı hesabındaki en büyük ara çarpım 630'dur. Bunlar long aralığındadır.
- En uzun satır 10 alandan oluşur: $10 * 4 = 40$ karakter. Dört karakterlik alan katsayıları ayırmaya yeterlidir.
- Önce bölme yapmak tamsayı kırpması nedeniyle yanlış sonuç verebilir; çarpma-bölme sırası korunmalıdır.
- Metin, ondalık sayı, boş satır, giriş sonu ve aralık dışı değerler hesaplamadan önce reddedilir.

Kazanımlar

- Dış döngüyü satır, iç döngüyü satırdaki konum olarak yorumlama.
- Her satır başında yeniden başlatılması gereken değişkeni belirleme.
- 4 numaralı satırın katsayılarını yalnızca önceki katsayıdan hesaplama.
- Tam bölme için işlem sırasını koruyup yanlış sıralamanın sonucunu gösterme.
- Dört karakterlik alanların toplam çıktı genişliğini hesaplama.

Alıştırımlar

1. Her satırın katsayılarını toplamak için bir biriktirici ekleyin; ilk beş satırın toplamlarının 1, 2, 4, 8 ve 16 olduğunu doğrulayın.
2. Dört karakterlik alanı altı karaktere çıkarın; 10 satırlık çıktının son satır genişliğini hesaplayın.
3. Satır başına uygun sayıda boşluk ekleyerek üçgeni ortalayın; katsayı hesabını koruyup yalnızca çıktı yerleşimini değiştirin.

037 Bir aralıktaki asal sayıları bulma

Düzey: İleri Önkoşullar: 027, 031, 036

Kavramlar: İç içe döngü, Aralık tarama, Her adayda yeniden başlatma, Erken çıkış, Sonuç sayacı

Problem tanımı

Kullanıcının verdiği iki sınır arasındaki bütün asal sayıları küçükten büyüğe yazdırın ve kaç tane olduklarını bulun. İki uç da aralığa dahildir. Asal sayı, 1'den büyük olup yalnızca 1 ile kendisine bölünen tamsayıdır; 0 ve 1 aralıkta bulunabilir, ancak sonuç listesine girmez.

027 numaralı problemde tek sayıya uygulanan bölen aramasını her aralık adayı için yeniden çalıştırın. Dış döngü hangi sayının incelendiğini, iç döngü o sayının olası bölenlerini belirler. Her yeni sayıda asallık durumu yeniden başlatılmalıdır; önceki sayının sonucu sonraki sayıya taşınmaz. Aralık en fazla 0..10000 olabilir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	alt	0 ile 10000 arasında int; dahil edilen alt sınır.
Girdi	ust	alt ile 10000 arasında int; dahil edilen üst sınır.
Çıktı	Asal listesi	Asal sayılar: öneki ve birer boşlukla ayrılmış asal sayılar; bulunamazsa Asal sayılar: Yok.
Çıktı	adet	Adet: N biçiminde bulunan asal sayısı.

Önce Alt sınır (0..10000): , sonra Üst sınır (alt..10000): istemine tamsayı girin. İkinci istemdeki alt, daha önce girilmiş değeri temsil eder. Üst sınır alt sınırdan küçükse hata verilir; sınırlar kendiliğinden yer değiştirmez. İlk alan hatalıysa ikinci alan istenmez.

Algoritma

1. Alt sınırı okuyup 0..10000 aralığını doğrulayın.
2. Üst sınırı okuyup alt sınırdan küçük olmadığını ve 10000'i aşmadığını doğrulayın.
3. Adet sayacını 0 yapın; liste önekini satırı bitirmeden yazın.
4. Dış döngüde sayıyı alt sınırdan üst sınır dahil olacak şekilde artırın.
5. Her sayı için asallık durumunu $sayi \geq 2$ ile yeniden başlatın.
6. Asal olabilecek sayı için 2'den başlayıp bölen $\leq sayi / bölen$ koşuluyla bölen arayın.
7. Tam bölen bulunursa durumu yanlış yapıp yalnızca iç döngüden çıkın.
8. Durum hâlâ doğruysa sayıyı listeye ekleyip adedi bir artırın.
9. Dış döngü bittiğinde adet 0 ise listeye Yok ekleyin; ardından satırı bitirip adedi yazdırın.

İç döngünün her adımında önceki bölen adaylarının sayıyı bölmediği bilinir. Bir tam bölen sonucu kesinleştirdiği için break kalan bölen adaylarını atlar; dış döngü sıradaki sayıyla sürer. Dış döngü adımının sonunda liste ve adet, o ana kadar işlenmiş aralık bölümünün asal sayılarını temsil eder. Sayı ve bölen sayaçları sınırlı aralıklarda arttığından bütün işlem sonlanır.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Alt sınır (0..10000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int alt) || alt < 0 || alt > 10000)
{
    Console.WriteLine("Hata: Alt sınır 0 ile 10000 arasında bir tamsayı olmalıdır.");
    return;
}
Console.Write("Üst sınır (alt..10000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ust) || ust < alt || ust > 10000)
{
    Console.WriteLine("Hata: Üst sınır alt sınır ile 10000 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}
int adet = 0;
Console.Write("Asal sayılar:");
for (int sayi = alt; sayi <= ust; sayi++)
{
    bool asal = sayi >= 2;
    if (asal)
    {
        for (int bolen = 2; bolen <= sayi / bolen; bolen++)
        {
            if (sayi % bolen == 0)
            {
                asal = false;
                break;
            }
        }
    }
    if (asal)
    {
        Console.Write(" " + sayi.ToString(CultureInfo.InvariantCulture));
        adet++;
    }
}
if (adet == 0)
{
    Console.Write(" Yok");
}
Console.WriteLine();
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
```

asal değişkeni dış döngünün gövdesinde oluşturulur. Örneğin 4 için yanlış olan durum, 5 için yeniden doğru başlayabilir. İçteki break, yalnızca en yakın bölen döngüsünü bitirir; aralık taramasını sona erdirmez. Çıktı listesi bir dizide biriktirilmeden doğrudan yazdırılır.

Örnek çalıştırmalar

Girdiler alt, üst sırasındadır. Metin blokları istemlerden sonraki bütün çıktı satırlarını gösterir.

0, 5 için tam çıktı:

```
Asal sayılar: 2 3 5
Adet: 3
```

2, 2 için tam çıktı:

Asal sayılar: 2

Adet: 1

4, 4 için tam çıktı:

Asal sayılar: Yok

Adet: 0

10, 20 için tam çıktı:

Asal sayılar: 11 13 17 19

Adet: 4

Uzun aralığın aşağıdaki denetimi tam liste değildir; sonucun özetidir.

Girdi	Beklenen özellik veya denetim
0, 10000	1229 asal; ilk sayı 2, son sayı 9973; adet satırı Adet : 1229.

Aşağıdaki örneklerde tek hata satırı yazılır; liste ve adet satırları üretilmez.

Girdi	Tam sonuç
İlk girdi -1	Hata: Alt sınır 0 ile 10000 arasında bir tamsayı olmalıdır.
5, 4	Hata: Üst sınır alt sınır ile 10000 arasında bir tamsayı olmalıdır.
0, 10001	Hata: Üst sınır alt sınır ile 10000 arasında bir tamsayı olmalıdır.
İlk girdi alt	Hata: Alt sınır 0 ile 10000 arasında bir tamsayı olmalıdır.
0, 2.5	Hata: Üst sınır alt sınır ile 10000 arasında bir tamsayı olmalıdır.
0, sonra boş satır	Hata: Üst sınır alt sınır ile 10000 arasında bir tamsayı olmalıdır.
0, sonra girişin sonu	Hata: Üst sınır alt sınır ile 10000 arasında bir tamsayı olmalıdır.

Sınır durumları

- Eşit sınırlar tek aday oluşturur; o aday asal değilse liste boştur.
- 0 ve 1 için bölen döngüsü çalışmaz; ikisi de listeden dışlanır.
- Bir asalın karesinde eşitlik sınırı denenmelidir; 49 için bölen 7 kontrol edilir.
- Aralıkta asal bulunmaması geçerli bir sonuçtur; hata değildir ve adet 0 yazılır.
- Her adayda asallık yeniden başlatılır; bir adayın yanlış durumu sonraki adayı etkilemez.
- En fazla 10001 sayı incelenir; bölen 2'den başladığı için sıfıra bölme yapılmaz.
- Metin, ondalık sayı, boş satır, giriş sonu ve alan aralığının dışındaki girdiler sonuç başlıkları yazılmadan reddedilir.

Kazanımlar

- Tek sayı algoritmasını dış döngüyle bir aralıktaki bütün adaylara uygulama.
- İç ve dış döngülerin farklı sayaç görevlerini açıklama.
- Asallık durumunu her aday için yeniden başlatma.
- İç döngüden erken çıkışın dış döngüyü neden durdurmadığını örnekle gösterme.
- Boş sonuç listesini adet 0 ile tutarlı biçimde yazdırma.

Alıştırmalar

1. Her aday için kaç bölen denendiğini ayrıca sayın; 2, 4, 5 ve 49 için deneme sayısını karşılaştırın.
2. Bulunan asal sayıların toplamını bir long biriktiricide hesaplayıp adet satırının ardından yazdırın.
3. Aralığı büyükten küçüğe tarayan bir sürüm yazın; listedeki sayı sırasının değiştiğini, adedin aynı kaldığını doğrulayın.

038 Bir üst sınıra kadar mükemmel sayıları bulma

Düzey: İleri Önkoşullar: 027, 031, 037

Kavramlar: İç içe döngü, Bölen çiftleri, Çift saymayı önleme, Biriktirici, Mükemmel sayı

Problem tanımı

2'den başlayıp kullanıcı tarafından verilen üst sınıra kadar bütün mükemmel sayıları bulun. İki uç da aramaya dahildir. Bir pozitif tamsayı, kendisi dışındaki pozitif bölenlerinin toplamına eşitse mükemmeldir. Örneğin 6'nın kendisi dışındaki bölenleri 1, 2 ve 3'tür; $1 + 2 + 3 = 6$ olduğu için 6 mükemmel sayıdır. 8 için toplam $1 + 2 + 4 = 7$ olur; 8 mükemmel değildir.

Bölenleri sayının sonuna kadar tek tek aramak yerine bölen çiftlerini kullanın. 28'in 2 böleni, aynı zamanda 14 bölenini de verir. Bir tam karede eşleşen iki bölen aynı olabilir; örneğin 36 için 6 ile 6 bulunur ve yalnızca bir 6 toplama eklenmelidir. Dış döngü sayıları tarar, iç döngü bir sayının bölen çiftlerini bulur. Her yeni sayı için toplam yeniden başlatılır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	ust	2 ile 10000 arasında int; dahil edilen üst sınır.
Çıktı	Mükemmel sayı listesi	Mükemmel sayılar: öneki ve birer boşlukla ayrılmış sonuçlar; boşsa Mükemmel sayılar: Yok.
Çıktı	adet	Adet: N biçiminde bulunan mükemmel sayı sayısı.

Üst sınır (2..10000): istemine tek tamsayı girin. Geçersiz girişte Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır. yazılır; sonuç listesi ve adet üretilmez. 1 bu aramada aday değildir.

Algoritma

1. Üst sınırı okuyup 2..10000 aralığını doğrulayın.
2. Adedi 0 yapıp liste önekini satırı bitirmeden yazın.
3. Dış döngüde sayıyı 2'den üst sınır dahil olacak şekilde artırın.
4. Her sayı için bölen toplamını 1 ile başlatın; 1 bütün adayların kendileri dışındaki bir bölenidir.
5. İç döngüde böleni 2'den başlatıp $bolen \leq sayi / bolen$ koşuluyla artırın.
6. Tam bölünme varsa böleni toplama ekleyin; eşini $sayi / bolen$ ile hesaplayın.
7. Eş bölen mevcut bölenle aynı değilse onu da ekleyin.
8. İç döngü bittiğinde toplam sayıya eşitse sayıyı yazıp adedi artırın.
9. Dış döngü bitince hiç sonuç yoksa Yok ekleyin; satırı bitirip adedi yazdırın.

Kareköke kadar bulunan her bölenin diğer eşi karekök sınırında veya üstündedir. Bu sayede uygun bölenlerin tamamı bulunur. Adayın kendisi toplama girmez: 1 başta eklenmiştir; 2 ve daha büyük bölenlerin eşi en fazla sayının yarısıdır. Tam kare sınırında iki eş aynı olduğundan ikinci ekleme atlanır. İki döngünün sayaçları sınırlı üst değerlere kadar arttığı için arama sonlanır.

28 için toplam 1 ile başlar. Bölen 2 bulunduğunda toplam $1 + 2 + 14 = 17$, bölen 4 bulunduğunda $17 + 4 + 7 = 28$ olur. Diğer denemeler toplamı değiştirmez; sayı sonuç listesine girer.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Üst sınır (2..10000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ust) || ust < 2 || ust > 10000)
{
    Console.WriteLine("Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.");
    return;
}
int adet = 0;
Console.Write("Mükemmel sayılar:");
for (int sayi = 2; sayi <= ust; sayi++)
{
    int toplam = 1;
    for (int bolen = 2; bolen <= sayi / bolen; bolen++)
    {
        if (sayi % bolen == 0)
        {
            toplam += bolen;
            int es = sayi / bolen;
            if (es != bolen)
            {
                toplam += es;
            }
        }
    }
    if (toplam == sayi)
    {
        Console.Write(" " + sayi.ToString(CultureInfo.InvariantCulture));
        adet++;
    }
}
if (adet == 0)
{
    Console.Write(" Yok");
}
Console.WriteLine();
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
```

toplam dış döngü gövdesinde başlatılır. Eş bölen denetimi bir tam karede aynı değerin iki kez eklenmesini önler. Asal sayıların kendileri dışındaki tek pozitif böleni 1'dir; toplamları 1 kaldığı için hiçbir asal sayı bu aramada mükemmel sayılmaz.

Örnek çalıştırmalar

Metin blokları giriş isteminden sonraki bütün çıktı satırlarını gösterir.

Girdi 2 için tam çıktı:

Mükemmel sayılar: Yok
Adet: 0

Girdi 6 için tam çıktı:

Mükemmel sayılar: 6
Adet: 1

Girdi 28 için tam çıktı:

Mükemmel sayılar: 6 28
Adet: 2

Girdi 10000 için tam çıktı:

Mükemmel sayılar: 6 28 496 8128

Adet: 4

Aşağıdaki örneklerde tek hata satırı yazılır.

Girdi	Tam sonuç	Açıklama
1	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Alt sınır dışı.
10001	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Üst sınır dışı.
mükemmel	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Metin girişi.
6.5	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Ondalık sayı.
Boş satır	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Değer verilmemiştir.
Hiç satır vermeden girişin sonu	Hata: Üst sınır 2 ile 10000 arasında bir tamsayı olmalıdır.	Okunacak satır yoktur.

Sınır durumları

- Üst sınır 2 ile 5 arasında olduğunda sonuç listesi boştur; bu geçerli bir sonuçtur.
- Üst sınır 6 olduğunda 6 listeye katılır; üst sınırın dahil edilmesi için $\leq$ kullanılır.
- Asal adaylarda toplam 1 kalır; bölen yokluğu için özel bir hata verilmez.
- Tam karede eş bölen aynıysa yalnızca bir kez eklenir. 36 için kendisi dışındaki bölenlerin toplamı 55'tir; 6'nın ikinci kez eklenmesi bu hesabı bozar.
- Her aday için toplamın yeniden 1 yapılması, önceki sayının bölenlerinin sonraki sayıya taşınmasını önler.
- 10000 sınırında bütün olası pozitif uygun bölenler 1..9999 aralığındadır. Bu aralıktaki bütün tamsayıların toplamı 49995000 olduğundan bölen toplamı `int` içine sığar.
- Hatalı biçim, boş satır, giriş sonu veya aralık dışı değer tek hata satırıyla reddedilir.

Kazanımlar

- Bir sayının kendisi dışındaki pozitif bölenlerini doğru ayırma.
- Bölen çiftleriyle kareköke kadar aramadan bütün uygun bölenleri toplama.
- Tam karede aynı bölenin iki kez sayılmasını önleme.
- Her aralık adayı için toplamı yeniden başlatma.
- Bulunan sayıların listesiyle adet sayacının aynı sonuç kümesini temsil etmesini sağlama.

Alıřtırmalar

1. 28 ve 36 iin bulunan blen iftlerini ve her eklemekten sonraki toplamı yazdırın; karedeki eřit iftin davranıřını gsterin.
2. Uygun blen toplamı sayısından byk olan sayıları aynı yntemle listeleyin; 12 ve 18 iin beklenen durumu hesaplayın.
3. nce blenleri 1'den $n - 1$ deėerine kadar deneyen bir srm yazın, sonra iki srmdeki blnebilirlik denemelerini sayarak karřılařtırın.

039 Pisagor üçlülerini üretme

Düzey: İleri Önkoşullar: 031, 037, 038

Kavramlar: Üç iç içe döngü, Aday uzayı, Sıralama kısıtı, Tekrarsız üretim, Pisagor eşitliği

Problem tanımı

Hipotenüsü verilen üst sınırı aşmayan bütün pozitif tamsayı Pisagor üçlülerini üretin. Bir üçlü $a^2 + b^2 = c^2$ koşulunu sağlamalıdır; a ve b dik kenarlar, c hipotenüstür. Örneğin $3^2 + 4^2 = 5^2$ olduğu için bir sonuçtur.

Her üçlüyü yalnızca bir kez yazmak için $a < b < c$ düzenini kullanın. $3\ 4\ 5$ yazılırken $4\ 3\ 5$ ayrıca yazılmaz. Ortak çarpanı olan üçlüler de kabul edilir; $6\ 8\ 10$, $3\ 4\ 5$ üçlüsünün iki katı olsa da ayrı bir geçerli sonuçtur. Dış döngü önce a , orta döngü b , iç döngü c değerini ilerletir. Çıktı bu döngü sırasındadır; hipotenüse göre sıralanmış olması gerekmez.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 100 arasında int; hipotenüs için dahil edilen üst sınır.
Çıktı	Üçlüler	Pisagor üçlülere: başlığının ardından her satırda $a\ b\ c$; sonuç yoksa bir satır Yok.
Çıktı	adet	Son satırda Adet: N ; bulunan farklı sıralı üçlü sayısı.

Hipotenüs üst sınırı (1..100): istemine tek tamsayı girin. Üçlülerde sayılar birer boşlukla ayrılır. Sınır 5'ten küçük olduğunda geçerli girişe rağmen sonuç bulunmaz. Hatalı girişte Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır. yazılır; liste başlığı üretilmez.

Algoritma

- Üst sınırı okuyup 1..100 aralığını doğrulayın.
- Adedi sıfır yapıp liste başlığını yazdırın.
- a değerini 1'den $n - 2$ dahil olacak şekilde artırın.
- Her a için b değerini $a + 1$ ile $n - 1$ arasında artırın.
- Her a, b çifti için c değerini $b + 1$ ile n arasında artırın.
- Pisagor eşitliği sağlanıyorsa üçlüyü yazıp adedi artırın.
- Bütün döngüler bitince adet sıfırsa Yok yazın.
- Adedi sonuç satırında yazdırın.

Başlangıç sınırları her adayda $a < b < c \leq n$ ilişkisini kurar. Uygun aralıktaki her artan üçlü bu döngülerde tam bir kez denenir. Bu nedenle kenarları yer değiştirerek aynı çözüm tekrar üretilmez. Bir a değerine ait bütün b, c adayları tamamlandıktan sonra a artar. Sayaçların her biri sonlu aralıkta arttığı için üç döngü de sonlanır.

Üst sınır 13 olduğunda $3\ 4\ 5$, ardından $5\ 12\ 13$, ardından $6\ 8\ 10$ yazılır. Sonucun hipotenüsleri 5, 13, 10 sırasındadır; sıralamayı hipotenüs değil dıştaki a belirler.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Hipotenüs üst sınırı (1..100): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 100)
{
    Console.WriteLine("Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.");
    return;
}
int adet = 0;
Console.WriteLine("Pisagor üçlüleri:");
for (int a = 1; a <= n - 2; a++)
{
    for (int b = a + 1; b <= n - 1; b++)
    {
        for (int c = b + 1; c <= n; c++)
        {
            if (a * a + b * b == c * c)
            {
                Console.WriteLine(a.ToString(CultureInfo.InvariantCulture) + " " +
                    b.ToString(CultureInfo.InvariantCulture) + " " +
                    c.ToString(CultureInfo.InvariantCulture));
                adet++;
            }
        }
    }
}
if (adet == 0)
{
    Console.WriteLine("Yok");
}
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
```

Karekök hesaplamak yerine her hipotenüs adayının karesi karşılaştırılır. $b = a + 1$ ve $c = b + 1$ başlangıçları, bir son koşulla elenecek tekrarları en baştan aday uzayının dışına çıkarır. Doğrulama sonucunda sınır en fazla 100 olduğu için bütün kareler ve toplamaları `int` içinde kalır.

Örnek çalıştırmalar

Metin blokları istemden sonraki bütün çıktı satırlarını gösterir.

Girdi 1 için tam çıktı:

```
Pisagor üçlüleri:
Yok
Adet: 0
```

Girdi 5 için tam çıktı:

```
Pisagor üçlüleri:
3 4 5
Adet: 1
```

Girdi 13 için tam çıktı:

```
Pisagor üçlüleri:
3 4 5
5 12 13
6 8 10
Adet: 3
```

Hacimli çıktılar için aşağıdaki tablo tam liste değildir; doğrulanacak özellikleri özetler.

Girdi	Beklenen özellik veya denetim
4	Hiç üçlü bulunmaz; adet 0'dır.
20	Altı üçlü bulunur; katlar dahildir; son satır Adet: 6.
100	52 üçlü bulunur; bütün adaylarda $a < b < c \leq 100$; son satır Adet: 52.

Aşağıdaki örneklerin sonucu tek hata satırıdır.

Girdi	Tam sonuç
0	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.
101	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.
üçlü	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.
5.5	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.
Boş satır	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.
Hiç satır vermeden girişin sonu	Hata: Üst sınır 1 ile 100 arasında bir tamsayı olmalıdır.

Sınır durumları

- $n = 1$ veya $n = 2$ olduğunda dış döngü bile çalışmaz; başlık, Yok ve adet 0 yazılır.
- 3 ve 4 sınırlarında aday denenebilir, ancak en küçük Pisagor üçlüsünün hipotenüsü 5'tir; sonuç bulunmaz.
- Üst sınır dahil olduğundan $n = 5$ için 3 4 5 yazılır.
- Kat üçlüler dışlanmaz; yalnızca aynı kenarların farklı sıralamaları dışlanır.
- a, b, c en fazla 100 olabilir. Bir kare en fazla 10000, iki kare toplamı en fazla 20000 olur; int taşmaz.
- Üst sınırdaki en fazla $100 * 99 * 98 / 6 = 161700$ artan üçlü aday vardır; adet int aralığındadır.
- Boş satır, giriş sonu, metin, ondalık sayı ve aralık dışı girişlerde tek hata yazılır.

Kazanımlar

- Üç bağımsız aday değişkenini üç iç içe döngüyle üretme.
- Başlangıç ve bitiş sınırlarıyla $a < b < c$ ilişkisini kurma.
- Tekrarlı çözümleri sonradan silmek yerine aday üretiminden dışlama.
- Çıktı sırasını dış, orta ve iç döngülerin sırasından çıkarma.
- Girdi üst sınırından karelerin ve toplamın güvenli sayı aralığını hesaplama.

Alıřtırmalar

1. Çözüm sayacına ek olarak denenen aday üçlü sayısını yazdırın; 5 için 10 adayın denendiğini doğrulayın.
2. Döngü sırasını önce c, sonra a, sonra b olacak biçimde deęiřtirin; aynı üçlülerin hipotenüse göre sıralandığını kontrol edin.
3. 028 numaralı problemdeki Öklid algoritmasını her bulunan üçlüye uygulayın; üç kenarı aralarında asal olan üçlülerini filtreleyerek 6, 8, 10 gibi katları dışlayın.

040 Bir tutarı 1, 2 ve 5 birimlik paralarla oluşturma

Düzey: İleri Önkoşullar: 018, 031, 039

Kavramlar: İç içe döngü, Kısıtlı kombinasyon üretimi, Kalan tutar, Eşitlikten değişken bulma, Sıfır çözümü

Problem tanımı

Bir tamsayı tutarı, 1, 2 ve 5 birimlik paralardan kaç adet kullanarak oluşturabileceğinizi bulun. Her para türünden sınırsız sayıda bulunduğunu kabul edin. Bütün farklı adet kombinasyonlarını üretin ve kaç tane olduklarını bulun.

Paraların diziliş sırası önemli değildir. Örneğin iki tane 1 ile bir tane 2 kullanılması, bu paraların hangi sırayla dizildiğinden bağımsız olarak tek kombinasyondur. Her çözüm birler + 2 * ikiler + 5 * besler = tutar eşitliğini sağlamalıdır. Beşlik ve ikilik adetlerini iki döngüyle seçin; birlik adedini kalan tutardan bulun. Üçüncü bir arama döngüsüne gerek yoktur. Tutar 0 ise hiçbir para kullanılmayan tek çözüm vardır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	tutar	0 ile 100 arasında int; oluşturulacak soyut birim sayısı.
Çıktı	Kombinasyonlar	Çözümler: başlığından sonra 1: X, 2: Y, 5: Z biçiminde her satırda para adetleri.
Çıktı	adet	Son satırda Adet: N; farklı kombinasyon sayısı.

Tutar (0..100): istemine tek tamsayı girin. Çözümler önce beşlik adedi, aynı beşlik adedinde ikilik adedi artacak şekilde yazılır. Geçersiz girişte Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır. yazılır; çözüm başlığı üretilmez.

Algoritma

1. Tutarı okuyup 0..100 aralığını doğrulayın.
2. Adedi 0 yapıp çözüm başlığını yazdırın.
3. Dış döngüde beşlik adedini 0'dan tutar / 5 dahil olacak şekilde artırın.
4. Seçilen beşliklerden sonra kalan tutarı $tutar - 5 * besler$ ile hesaplayın.
5. İç döngüde ikilik adedini 0'dan kalan / 2 dahil olacak şekilde artırın.
6. Birlik adedini $kalan - 2 * ikiler$ ile hesaplayın.
7. Üç adedi belirtilen biçimde yazdırıp çözüm sayacını bir artırın.
8. Bütün döngüler bitince adedi yazdırın.

İç döngünün üst sınırı ikiliklerin kalan tutarı aşmasını önler; birlik adedi bu nedenle hiçbir zaman negatif olmaz. Her beşlik-ikilik çifti için birlik adedi eşitlikle tek bir değere belirlenir. Bütün geçerli çiftler birer kez denenir; farklı para dizilişleri ayrıca üretilmez. İki sayaç da sınırlı aralıkta arttığından işlem sonlanır. Birimlik para bulunduğu için her geçerli tutar en az bir çözüme sahiptir.

Tutar 5 için aday üretimi şöyledir.

Beşlik adedi	Beşliklerden kalan	İkilik adedi	Birlik adedi	Eşitlik
0	5	0	5	$5 + 2 * 0 + 5 *$ $0 = 5$
0	5	1	3	$3 + 2 * 1 + 5 *$ $0 = 5$
0	5	2	1	$1 + 2 * 2 + 5 *$ $0 = 5$
1	0	0	0	$0 + 2 * 0 + 5 *$ $1 = 5$

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Tutar (0..100): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int tutar) || tutar < 0 || tutar > 100)
{
    Console.WriteLine("Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.");
    return;
}
int adet = 0;
Console.WriteLine("Çözümler:");
for (int besler = 0; besler <= tutar / 5; besler++)
{
    int kalan = tutar - 5 * besler;
    for (int ikiler = 0; ikiler <= kalan / 2; ikiler++)
    {
        int birler = kalan - 2 * ikiler;
        Console.WriteLine("1: " + birler.ToString(CultureInfo.InvariantCulture) +
            ", 2: " + ikiler.ToString(CultureInfo.InvariantCulture) +
            ", 5: " + besler.ToString(CultureInfo.InvariantCulture));
        adet++;
    }
}
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
```

tutar / 5 ve kalan / 2 tamsayı bölmeleridir; sığabilecek en büyük para adedini verir. Örneğin kalan 5 iken en fazla iki tane ikilik kullanılabilir. Birlik adedi aranmadan doğrudan hesaplandığı için her yazdırılan aday baştan geçerli bir çözümdür.

Örnek çalıştırmalar

Metin blokları giriş isteminden sonraki bütün çıktı satırlarını gösterir.

Girdi 0 için tam çıktı:

```
Çözümler:
1: 0, 2: 0, 5: 0
Adet: 1
```

Girdi 2 için tam çıktı:

Çözümler:

1: 2, 2: 0, 5: 0

1: 0, 2: 1, 5: 0

Adet: 2

Girdi 5 için tam çıktı:

Çözümler:

1: 5, 2: 0, 5: 0

1: 3, 2: 1, 5: 0

1: 1, 2: 2, 5: 0

1: 0, 2: 0, 5: 1

Adet: 4

Hacimli çıktılar için aşağıdaki tablo tam çözüm listesi değildir; denetlenecek özellikleri özetler.

Girdi	Beklenen özellik veya denetim
10	10 farklı kombinasyon; son satır Adet: 10.
100	541 farklı kombinasyon; ilk çözüm 1: 100, 2: 0, 5: 0, son çözüm 1: 0, 2: 0, 5: 20; son satır Adet: 541.

Aşağıdaki örneklerde tek hata satırı yazılır.

Girdi	Tam sonuç
-1	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.
101	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.
para	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.
2.5	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.
Boş satır	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.
Hiç satır vermeden girişin sonu	Hata: Tutar 0 ile 100 arasında bir tamsayı olmalıdır.

Sınır durumları

- Tutar 0 olduğunda dış ve iç döngüler 0 adedi için birer kez çalışır; hiçbir para kullanılmayan tek çözüm sayılır.
- Tutar 1 olduğunda yalnızca bir birlik para içeren çözüm vardır.
- Tutarın 5'e veya kalan tutarın 2'ye tam bölünmesi gerekmez; tamsayı bölümünün kalan kısmı birliklerle tamamlanır.
- Bütün adetler sıfır veya pozitifdir. Birlikler en fazla 100, ikilikler en fazla 50, beşlikler en fazla 20 olabilir; hesaplamalar int içine sığar.
- Her sabit beşlik adedi için iç döngü $kalan / 2 + 1$ çözüm üretir. Tutar 100 için bu sayıların toplamı 541'dir.
- Aynı adetler farklı para sıralamasıyla yeniden sayılmaz. Çıktı sırası önce beşlik, sonra ikilik sayacına bağlıdır.
- Metin, ondalık sayı, boş satır, giriş sonu ve aralık dışı değerler sonuç başlığı yazılmadan reddedilir.

Kazanımlar

- Üç değişkenli bir eşitlikte iki değişkeni seçip üçüncüyü hesaplama.
- İç döngü sınırını dış döngüde kalan tutara bağlı kurma.
- Para adetlerinin negatif olmamasını aday sınırlarıyla sağlama.
- Para sıralamasıyla adet kombinasyonunu birbirinden ayırma.
- Sıfır tutarın boş seçim içeren bir geçerli çözüm olduğunu açıklama.
- 5 birimlik tutar için dört çözümü döngü sırasıyla üretme.

Alıştırmalar

1. Her çözümde kullanılan toplam para adedini de yazdırın; tutar 5 için dört satırdaki adetleri karşılaştırın.
2. Yalnızca toplam para sayısı çift olan kombinasyonları yazdırın ve sayın; sıfır tutarın bu koşuldaki durumunu açıklayın.
3. Para değerlerini 1, 3 ve 7 olarak değiştirin; döngü sınırlarını ve kalan tutar eşitliğini güncelleyip tutar 7'nin bütün kombinasyonlarını çıkarın.

Bölüm 5: Diziler ve veri işleme

041 Dizinin en küçük ve en büyük değerlerini bulma

Düzey: İleri Önkoşullar: 007, 010, 031

Kavramlar: Dizi tarama, Sıfır tabanlı dizin, İlk elemanla başlatma, En küçük ve en büyük, İlk geçişi koruma

Problem tanımı

Kullanıcının girdiği 1 ile 50 arasında tamsayıyı bir dizide saklayın. Dizinin en küçük ve en büyük değerlerini, bu değerlerin ilk görüldüğü dizinlerle birlikte bulun. Elemanlar `int` türünün bütün aralığında, yani -2147483648 ile 2147483647 arasında olabilir.

Dizi dizinleri sıfırdan başlar: beş elemanın dizinleri 0, 1, 2, 3 ve 4'tür. Aynı en küçük veya en büyük değer birden çok kez bulunabilir. Örneğin 7, -2, 7, 4, -2 dizisinde en küçük değer -2'nin ilk dizini 1; en büyük değer 7'nin ilk dizini 0'dır. Eşit değerde kayıtlı dizini değiştirmeyerek ilk geçişi koruyun. Hazır en küçük/en büyük araçları yerine diziyi kendiniz tarayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında <code>int</code> ; eleman sayısı.
Girdi	sayılar	<code>int[n]</code> ; her eleman tam <code>int</code> aralığında.
Çıktı	En küçük ve dizini	En küçük: X, ardından En küçük dizini: I.
Çıktı	En büyük ve dizini	En büyük: Y, ardından En büyük dizini: J.

Önce Eleman sayısı (1..50): istemini yanıtlayın; sonra `Eleman[0]:` ile başlayan istemlere her elemanı ayrı satırda girin. İstemdeki sayı elemanın sıfır tabanlı dizinidir. Basamak ayırıcı veya ondalık bölüm kullanmayın. Bütün elemanlar doğrulanmadan sonuç yazılmaz. Geçersiz eleman sayısında Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., geçersiz elemanda Hata: Eleman geçerli bir `int` tamsayı olmalıdır. yazılır.

Algoritma

1. Eleman sayısını okuyup 1..50 aralığını doğrulayın.
2. N uzunluklu diziyi oluşturun; bütün elemanları doğrulayarak doldurun.
3. En küçük ve en büyük değerleri dizinin ilk elemanı ile başlatın; iki dizini de 0 yapın.
4. Diziyi 1. dizinden son dizine kadar dolaşın.
5. Eleman kayıtlı en küçükten kesinlikle küçükse en küçük değeri ve dizinini güncelleyin.
6. Eleman kayıtlı en büyükten kesinlikle büyükse en büyük değeri ve dizinini güncelleyin.
7. Değerleri ve dizinleri belirtilen dört satırla yazdırın.

Tarama başında kayıtlı değerler, daha önce incelenmiş elemanların en küçüğü ve en büyüğüdür; kayıtlı dizinler bunların ilk görüldüğü yerlerdir. Yeni değer daha küçük veya daha büyükse ilgili kayıt değişir. Eşitlikte güncelleme yapılmadığı için ilk dizin korunur. Başlangıçta gerçek bir dizi elemanını kullanmak, bütün değerler negatif veya pozitif olduğunda da geçerli bir aday sağlar. En küçüğü 0 yapmak, yalnızca pozitif elemanlarda dizide bulunmayan yanlış bir sonuç oluşturabilirdi.

7, -2, 7, 4, -2 dizisi için tarama:

İncelenen dizin	Değer	En küçük / ilk dizin	En büyük / ilk dizin
0, başlangıç	7	7 / 0	7 / 0
1	-2	-2 / 1	7 / 0
2	7	-2 / 1	7 / 0
3	4	-2 / 1	7 / 0
4	-2	-2 / 1	7 / 0

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}
int[] sayilar = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("Eleman[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    sayilar[i] = deger;
}

int enKucuk = sayilar[0], enBuyuk = sayilar[0];
int enKucukDizini = 0, enBuyukDizini = 0;
for (int i = 1; i < n; i++)
{
    if (sayilar[i] < enKucuk)
    {
        enKucuk = sayilar[i];
        enKucukDizini = i;
    }
    if (sayilar[i] > enBuyuk)
    {
        enBuyuk = sayilar[i];
        enBuyukDizini = i;
    }
}
Console.WriteLine("En küçük: " + enKucuk.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("En küçük dizini: " +
    enKucukDizini.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("En büyük: " + enBuyuk.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("En büyük dizini: " +
    enBuyukDizini.ToString(CultureInfo.InvariantCulture));
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. sayılar[0] güvenlidir çünkü N en az 1'dir. Veri okuma ve tarama ayrı döngülerdir; geçersiz bir son elemanda bile sonuç üretilmez. int.TryParse tür sınırının dışını reddeder; eleman değerlerine ayrıca dar bir aralık konulmaz.

Örnek çalıştırmalar

Her girdiyi ayrı satırda verin: önce N, sonra listelenen elemanlar. Metin blokları istemler dışındaki bütün çıktı satırlarını gösterir.

N = 5; elemanlar 7, -2, 7, 4, -2:

En küçük: -2
En küçük dizini: 1
En büyük: 7
En büyük dizini: 0

N = 1; tek eleman -8:

En küçük: -8
En küçük dizini: 0
En büyük: -8
En büyük dizini: 0

N = 4; elemanlar -5, -2, -9, -2:

En küçük: -9
En küçük dizini: 2
En büyük: -2
En büyük dizini: 1

N = 3; elemanlar -2147483648, 0, 2147483647:

En küçük: -2147483648
En küçük dizini: 0
En büyük: 2147483647
En büyük dizini: 2

Uzun dizi için tam eleman listesi yerine aşağıdaki özelliği denetleyin.

Girdi	Beklenen özellik
N = 50; bütün elemanlar 2147483647	En küçük ve en büyük aynı değerdir; ikisinin ilk dizini 0'dır.

N için 0, 51, -1, 1.5, metin, boş satır veya giriş sonu verilirse tek çıktı:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 3 sonrasında ilk eleman 7 ve ikinci eleman iki ise aşağıdaki hata yazılır. Aynı hata herhangi bir elemanda 2147483648, -2147483649, 2.5, boş satır veya giriş sonu için de geçerlidir; sonuç satırları yazılmaz.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- Tek eleman hem en küçük hem en büyüktür; tarama döngüsü çalışmadan iki dizin de 0 kalır.
- Bütün değerler eşitse ilk elemanın dizini korunur. < yerine <= veya > yerine >= kullanılması son eşit elemanın dizinini seçerdi.
- Bütün değerler negatif olduğunda en büyük değer yine dizideki bir elemandır; başlangıçta 0 kullanılmaz.
- Son geçerli dizin $N - 1$ 'dir; $i < n$ koşulu dizinin dışına erişimi engeller.
- Değerlere aritmetik işlem yapılmadığı için iki `int` uç değeri arasında karşılaştırma taşma oluşturmaz.
- Bütün girişler doğru olmadan taramaya başlanmaz; eksik veya hatalı dizinin kısmi sonucu yoktur.

Kazanımlar

- Diziyi sıfır tabanlı dizinlerle doldurup ikinci döngüde tarama.
- Aday değerleri gerçek bir dizi elemanı ile başlatma.
- Bir değerle birlikte onun bulunduğu dizini tutarlı güncelleme.
- Kesin karşılaştırma kullanarak tekrarlanan değerlerin ilk geçişini koruma.
- Tek eleman, eşit elemanlar ve bütün değerlerin negatif olduğu durumları açıklama.

Alıştırmalar

1. İlk yerine son geçişin dizinini seçen sürümü yazın; 7, -2, 7, 4, -2 dizisindeki iki dizin sonucunu karşılaştırın.
2. En küçük ve en büyük değerlerin kaç kez geçtiğini ikinci bir taramayla bulun. Bütün değerler eşitken iki adedin de N olmasını sağlayın.
3. En büyük ile en küçük arasındaki farkı hesaplayın. İki uç `int` değeri için sonucu `long` türünde güvenle üretmenin neden gerekli olduğunu açıklayın.

042 Dizide doğrusal arama yapma

Düzey: İleri Önkoşullar: 041

Kavramlar: Doğrusal arama, Sıfır tabanlı dizin, İlk eşleşme, Erken döngü çıkışı, Bulunamadı işareti

Problem tanımı

1 ile 50 tamsayı içeren bir dizide, ayrıca girilen bir tamsayıyı arayın. Elemanlar ve aranan değer `int` türünün bütün aralığında olabilir. Dizinin sıralı olması gerekmez. Elemanları baştan sona birer birer karşılaştırarak aranan değerın ilk bulunduğu dizini yazdırın. Hiç eşleşme yoksa bulunamadı sonucu verin.

Dizinler sıfırdan başlar. Örneğin 4, 9, 4, -1, 9 dizisinde 9'un ilk dizini 1'dir. Sonraki 9 da eşleşse bile sonuç 4 olmamalıdır. İlk eşleşmede döngüden çıkın. Arama sonucunu tutan değişkende -1 kullanın; bu değer geçerli bir dizi dizini olmadığı için bulunamadı durumunu temsil edebilir. Dizideki bir elemanın -1 olması ise normal veridir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında <code>int</code> ; dizi uzunluğu.
Girdi	sayılar	N ayrı <code>int</code> eleman.
Girdi	aranan	Elemanlardan sonra girilen, tam <code>int</code> aralığında arama değeri.
Çıktı	İlk dizin veya bulunamadı	İlk dizin: I ya da Sonuç: Bulunamadı.

Önce Eleman sayısı (1..50): , sonra Eleman[0]: ile başlayan N eleman istemi, en son Aranan: yanıtlanır. Her tamsayı ayrı satırda girilir. Eleman sayısı ve bütün elemanlar doğrulandıktan sonra aranan değer istenir; aranan da doğrulanmadan arama sonucu verilmez. Hata satırları alanına göre Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., Hata: Eleman geçerli bir `int` tamsayı olmalıdır. veya Hata: Aranan geçerli bir `int` tamsayı olmalıdır. olur.

Algoritma

1. N değerini doğrulayın; N uzunluklu diziyi oluşturup bütün elemanları doğrulayarak okuyun.
2. Aranan değeri okuyup geçerli bir `int` olduğunu doğrulayın.
3. Sonuç dizinini -1 yapın.
4. Dizinleri 0'dan N - 1'e kadar sırayla dolaşın.
5. Eleman aranan değere eşitse sonuç dizinini kaydedin ve `break` ile arama döngüsünü bitirin.
6. Sonuç dizini -1 ise bulunamadı mesajını, aksi halde ilk dizini yazdırın.

Her yinelemenin başında daha önce denenmiş dizinlerin hiçbirinde eşleşme bulunmamıştır. Bu nedenle ilk doğru karşılaştırma, en küçük eşleşen dizini verir. Döngü bu noktada biter; eşit sonraki değerler sonucu değiştiremez. Başta sonuç -1'dir; eşleşme bulunmadığında değiştirilmeden kalır. Arama en fazla N karşılaştırma yapar ve dizideki değerleri değiştirmez.

4, 9, 4, -1, 9 dizisinde 9 arandığında:

Dizin	Eleman	Eşit mi?	İşlem
0	4	Hayır	Sonraki dizine geç.
1	9	Evet	Sonucu 1 yap; döngüden çık.
2, 3, 4	Önceden okunmuş değerler	Karşılaştırılmaz	İlk eşleşme zaten bulunmuştur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] sayilar = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("Eleman[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    sayilar[i] = deger;
}

Console.Write("Aranan: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int aranan))
{
    Console.WriteLine("Hata: Aranan geçerli bir int tamsayı olmalıdır.");
    return;
}
```

```

int dizin = -1;
for (int i = 0; i < n; i++)
{
    if (sayilar[i] == aranan)
    {
        dizin = i;
        break;
    }
}
if (dizin == -1)
{
    Console.WriteLine("Sonuç: Bulunamadı");
}
else
{
    Console.WriteLine("İlk dizin: " + dizin.ToString(CultureInfo.InvariantCulture));
}

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. break yalnızca arama döngüsünden çıkar; program daha sonra sonuç satırını yazdırır. Veri doğrulamasındaki return tüm programı bitirir. Dizinin tamamı okunmadan ilk elemanda arama yapıp sonuç yazmak, sonraki geçersiz girdiyi gizleyebilirdi; iki aşama bu yüzden ayrıdır.

Örnek çalıştırmalar

Girdi sırası N, N eleman ve aranan değerdir; her değer ayrı satıra yazılır. Metin blokları istemler dışındaki tam çıktıları gösterir.

N = 5; elemanlar 4, 9, 4, -1, 9; aranan 9:

İlk dizin: 1

N = 3; elemanlar 8, 0, -8; aranan 5:

Sonuç: Bulunamadı

N = 1; eleman -2147483648; aranan -2147483648:

İlk dizin: 0

N = 4; elemanlar 2, 7, 0, -1; aranan -1:

İlk dizin: 3

Uzun diziler için tam çıktı yerine aşağıdaki özellikleri denetleyin.

Girdi	Beklenen özellik
N = 50; elemanlar sırasıyla 0..49; aranan 49	Son dizin 49 bulunur; 50 karşılaştırma gerekir.
N = 50; bütün elemanlar 2147483647; aranan 2147483647	İlk dizin 0 bulunur; yalnızca bir karşılaştırma yapılır.

N için 0, 51, -1, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 2; ilk eleman 5, ikinci eleman 2147483648 ise aşağıdaki hata yazılır. Herhangi bir elemanda -2147483649, 3.5, metin, boş satır veya giriş sonu aynı hatayı üretir; aranan istenmez.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

N = 1; eleman 5; aranan beş ise aşağıdaki hata yazılır. Aranan değerde 2147483648, -2147483649, 5.0, boş satır veya giriş sonu da reddedilir.

Hata: Aranan geçerli bir int tamsayı olmalıdır.

Sınır durumları

- Tek elemanda eşleşme varsa dizin 0, yoksa bulunamadı sonucu verilir.
- İlk ve son elemanlar geçerli arama konumlarıdır; döngü her iki sınırı kapsar.
- Tekrarlanan eşleşmelerde ilk dizin seçilir. Erken çıkış kaldırılıp her eşleşmede kayıt yapılırsa son dizin seçilirdi.
- Sonuç değişkenindeki -1 bir konum işaretidir; eleman değeri -1'in aranmasını engellemez.
- Sıralama şartı yoktur; arama karşılaştırma dışında elemanlara işlem uygulamaz.
- Boş veya eksik dizi, geçersiz aranan ve tür sınırını aşan tamsayılar sonuç verilmeden reddedilir.

Kazanımlar

- Sırasız dizide doğrusal tarama ile ilk eşleşmeyi bulma.
- Geçerli dizinlerin dışında bir değerle bulunamadı durumunu temsil etme.
- Veri değeri ile o verinin dizinini ayırt etme.
- break kullanarak ilk eşleşmeden sonra gereksiz karşılaştırmaları durdurma.
- En iyi ve en kötü karşılaştırma adetlerini ilk ve son konum örnekleriyle açıklama.

Alıştırmalar

1. İlk eşleşme yerine aranan değer in bütün dizinlerini yazdırın; eşleşme yoksa tek bulunamadı satırı üretin.
2. Karşılaştırma adedini bir sayaçla bulun. İlk konum, son konum ve bulunamadı durumlarının sonuçlarını karşılaştırın.
3. Diziyi son dizinden başlayarak tarayıp son eşleşmenin dizinini bulun. Tekrarlı değerlerde ileri taramadan farkını gösterin.

043 Diziyi yerinde ters çevirme

Düzey: İleri Önkoşullar: 004, 010, 041, 042

Kavramlar: Dizide yerinde değişiklik, İki uçtan ilerleme, Geçici değişken, Takas, Tek ve çift uzunluk

Problem tanımı

1 ile 50 tamsayı içeren dizinin eleman sırasını, aynı dizi üzerinde ters çevirin. İlk eleman sona, son eleman başa geçer. Örneğin 1, 2, 3, 4, 5 dizisi 5, 4, 3, 2, 1 olur. Elemanlar tam int aralığındadır; sayının kendi basamakları değil, dizideki elemanların konumları ters çevrilir.

Yeni bir sonuç dizisi oluşturmayın. Başlangıçta ilk ve son dizine yerleştirilen iki sayacı merkeze doğru ilerletin. Her adımda bu iki konumdaki değeri bir geçici değişkenle takas edin. Böylece yalnızca iki dizin ve bir geçici değer kullanılarak dizi yerinde değiştirilir. Tek uzunlukta orta eleman yerinde kalır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; dizi uzunluğu.
Girdi	sayılar	Her biri tam int aralığında N eleman.
Çıktı	Ters çevrilmiş dizi	Tek satır; Dizi: ardından her eleman için bir boşluk ve değer.

Önce Eleman sayısı (1..50): istemini, sonra Eleman[0]: ile başlayan N istemi yanıtlayın. Her tamsayı ayrı satırda girilir. Dizinler sıfırdan N - 1'e kadar gider. Sonuç satırında elemanlar arasında tam bir boşluk bulunur; satır sonunda fazladan boşluk yoktur. Bütün girdiler doğrulanmadan takas ve çıktı yapılmaz.

Algoritma

1. N değerini doğrulayın; diziyi oluşturup bütün elemanları doğrulayarak doldurun.
2. sol değerini 0, sag değerini N - 1 yapın.
3. sol < sag olduğu sürece soldaki değeri geçici değişkene alın.
4. Soldaki elemana sağdaki değeri, sağdaki elemana geçici değeri atayın.
5. Sol dizini bir artırıp sağ dizini bir azaltın; 3. adımdan devam edin.
6. Döngü bitince dizinin bütün elemanlarını sırasıyla yazdırın.

Döngü başında sol değerinin solunda ve sag değerinin sağında kalan elemanlar doğru son konumlarına yerleşmiştir. Henüz çevrilmemiş bölüm iki dizin arasında kalır. Takas bu bölümün iki ucunu tamamlar; dizinlerin ilerletilmesi kalan bölümü daraltır. Dizinler buluşunca veya birbirini geçince tüm gerekli takaslar bitmiştir. N / 2 tamsayı bölümü kadar takas yapılır.

1, 2, 3, 4, 5 dizisindeki adımlar:

Sol	Sağ	İşlem sonrası dizi
0	4	5, 2, 3, 4, 1
1	3	5, 4, 3, 2, 1
2	2	Koşul yanlış; orta değer değiştirilmez.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}
int[] sayilar = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("Eleman[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    sayilar[i] = deger;
}

int sol = 0, sag = n - 1;
while (sol < sag)
{
    int gecici = sayilar[sol];
    sayilar[sol] = sayilar[sag];
    sayilar[sag] = gecici;
    sol++;
    sag--;
}
Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + sayilar[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Geçici değişken soldaki eski değeri korur; aksi halde ilk atama sonrasında bu değer kaybolurdu. Tüm değerler yalnızca taşınır, toplanmaz; bu nedenle iki uç int değerinin takası taşma oluşturmaz. Girdi dizisine ek olarak ikinci bir dizi ayrılmaz.

Örnek çalıştırmalar

Girdiler N ve ardından N elemandır; her biri ayrı satırda verilir. Bloklar istemler dışındaki tam sonucu gösterir.

N = 5; elemanlar 1, 2, 3, 4, 5:

Dizi: 5 4 3 2 1

N = 4; elemanlar -3, 0, 8, 8:

Dizi: 8 8 0 -3

N = 1; eleman -2147483648:

Dizi: -2147483648

N = 2; elemanlar -2147483648, 2147483647:

Dizi: 2147483647 -2147483648

Uzun dizi için aşağıdaki özellikleri denetleyin; tablo tam dizi çıktısı değildir.

Girdi	Beklenen özellik
N = 50; elemanlar sırasıyla 0..49	25 takas; sonuçta 49..0 sırası; ilk eleman 49, son eleman 0.

N için 0, 51, -1, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 2; ilk eleman 7, ikinci eleman 2147483648 ise aşağıdaki hata yazılır. Herhangi bir elemanda - 2147483649, 1.0, metin, boş satır veya giriş sonu aynı hatayı üretir. Kısmi dizi ters çevrilmez ve Dizi: satırı yazılmaz.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- N = 1 için sol ve sağ aynı dizindir; döngü hiç çalışmaz, tek eleman korunur.
- Çift uzunlukta dizinler birbirini geçer; tek uzunlukta orta dizinde buluşur. İki durumda da `sol < sag` koşulu biter.
- `sol <= sag` kullanılması tek uzunlukta orta elemanı kendisiyle gereksiz yere takas ederdi.
- Aynı değerler takas edildiğinde görünüm değişmeyebilir; işlem yine doğru konum çiftini ele alır.
- Her iki dizin de yalnızca döngü koşulu doğruyken kullanılır; dizinin dışına erişilmez.
- Ters çevirme iki kez uygulanırsa başlangıç sırası geri gelir. Değerlerin tekrarlanması bu özelliği bozamaz.
- Hatalı, aralık dışı veya eksik girdide sonuç verilmeden program biter.

Kazanımlar

- Diziyi ikinci bir dizi oluşturmadan yerinde dönüştürme.
- İlk ve son dizinden merkeze doğru ilerleyen iki sayaç kullanma.
- Değer kaybını geçici değişkenli takasla önleme.
- Tek ve çift uzunlukta döngünün bitiş konumlarını açıklama.
- Takas adedini dizi uzunluğunun tamsayı yarısıyla ilişkilendirme.

Alıştırmalar

1. Yalnızca iki kullanıcı dizini arasındaki bölümü ters çevirin; dizinleri doğrulayın ve bölümün dışındaki elemanları koruyun.
2. Takas adedini sayıp N = 1, 4 ve 5 için sırasıyla 0, 2 ve 2 sonucunu doğrulayın.
3. Aynı diziye ters çevirme döngüsünü iki kez uygulayın. Tek, çift ve tekrarlı elemanlı üç örnekte ilk sıranın geri geldiğini gösterin.

044 Diziye sağa döndürme

Düzey: İleri Önkoşullar: 018, 031, 043

Kavramlar: Dizi döndürme, Kalanla adım azaltma, Geriye kopyalama, Son elemanı koruma, İç içe döngü

Problem tanımı

1 ile 50 tamsayı içeren diziye, kullanıcının belirttiği K adım sağa döndürün. K, 0 ile 1000 arasında bir tamsayıdır. Bir sağa döndürme adımında son eleman başa gelir; diğer elemanlar bir konum sağa kayar. Örneğin 10, 20, 30, 40, 50 dizisi iki adım sonra 40, 50, 10, 20, 30 olur.

Elemanları ikinci bir diziye taşımadan aynı dizide işlem yapın. Dizi uzunluğu kadar döndürme başlangıç sırasını geri verdiğinizden gerekli adım sayısı $K \% N$ 'dir. Her adımda önce son elemanı saklayın; sonra değerleri son diziden ilk dizine doğru kopyalayın. İleri yönde kopyalamak, henüz kullanılmamış eski değerleri ezebilir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; dizi uzunluğu.
Girdi	sayılar	Her biri tam int aralığında N eleman.
Girdi	k	Elemanlardan sonra girilen 0..1000 döndürme sayısı.
Çıktı	Döndürülmüş dizi	Dizi: ardından her eleman için bir boşluk ve değer; tek satır.

Sırayla Eleman sayısı (1..50): , Eleman[0]: ile başlayan N istem ve Adım sayısı (0..1000): yanıtlanır. Her tamsayı ayrı satırda yazılır. Diziler sıfır tabanlıdır. Bütün elemanlar ve K doğrulanmadan dönüşüm veya sonuç yazımı yapılmaz. Sonuçta satır sonuna fazladan boşluk eklenmez.

Algoritma

1. N değerini doğrulayın; diziye oluşturup N elemanı doğrulayarak doldurun.
2. K değerini okuyup 0..1000 aralığını doğrulayın.
3. Gerçek adım sayısını $k \% n$ ile bulun.
4. Her adımda son elemanı geçici değişkene kaydedin.
5. Dizini N - 1'den 1'e kadar geriye ilerletin; her konuma sol komşusunun değerini kopyalayın.
6. Sıfıncı konuma saklanan eski son elemanı yazın.
7. Gerekli adımlar tamamlanınca diziye yazdırın.

Her adımın başında dizi o ana kadar tamamlanmış döndürmelerin sonucudur. Geriye kopyalama sırasında kaynak olan sol komşu henüz değiştirilmemiştir; eski değer güvenle alınır. Son eleman önceden saklandığı için son dizine yapılan ilk kopyada kaybolmaz. Bir tam tur bütün elemanları başlangıç konumuna döndürür; bu yüzden K'nin N'ye bölümünden kalan kadar adım yeterlidir. N pozitif olduğundan $\% n$ işlemi sıfıra bölme oluşturmaz.

10, 20, 30, 40, 50 dizisinde K = 2 için:

Tamamlanan adım	Dizi
0	10, 20, 30, 40, 50
1	50, 10, 20, 30, 40
2	40, 50, 10, 20, 30

İlk adımda 50 saklanır; 4. konuma 40, 3. konuma 30, 2. konuma 20 ve 1. konuma 10 taşınır. En son 0. konuma 50 yazılır.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}
int[] sayilar = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("Eleman[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    sayilar[i] = deger;
}
Console.Write("Adım sayısı (0..1000): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int k) || k < 0 || k > 1000)
{
    Console.WriteLine("Hata: Adım sayısı 0 ile 1000 arasında olmalıdır.");
    return;
}

int adim = k % n;
for (int tur = 0; tur < adim; tur++)
{
    int son = sayilar[n - 1];
    for (int i = n - 1; i >= 1; i--)
    {
        sayilar[i] = sayilar[i - 1];
    }
    sayilar[0] = son;
}
Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + sayilar[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Geriye giden iç döngü 1'de sonlandığı için $i - 1$ hiçbir zaman negatif olmaz. adım 0 ise döndürme gövdesi hiç çalışmaz; dizi aynı biçimde yazdırılır. Elemanlara aritmetik uygulanmaz, yalnızca değerleri taşınır. Girdi dizisinin dışında ikinci bir dizi yoktur.

Örnek çalıştırmalar

Girdi sırası N, N eleman ve K'dir; her değer ayrı satırdadır. Bloklar istemler dışındaki tüm çıktıları gösterir.

N = 5; elemanlar 10, 20, 30, 40, 50; K = 2:

Dizi: 40 50 10 20 30

N = 3; elemanlar 1, 2, 3; K = 0:

Dizi: 1 2 3

N = 3; elemanlar 7, -2, 7; K = 6:

Dizi: 7 -2 7

N = 1; eleman 2147483647; K = 1000:

Dizi: 2147483647

N = 3; elemanlar -1, 0, 1; K = 1000, gerçek adım 1:

Dizi: 1 -1 0

N = 2; elemanlar -2147483648, 2147483647; K = 1:

Dizi: 2147483647 -2147483648

Uzun dizi için tam çıktı yerine aşağıdaki özellikleri denetleyin.

Girdi	Beklenen özellik
N = 50; elemanlar 0..49; K = 1000	Gerçek adım 0; başlangıç sırası korunur.
N = 50; elemanlar 0..49; K = 999	Gerçek adım 49; ilk değer 1, son değer 0; her başlangıç elemanı bir kez bulunur.

N için 0, 51, -1, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda 2147483648, -2147483649, 1.0, metin, boş satır veya giriş sonu verilirse K istenmeden:

Hata: Eleman geçerli bir int tamsayı olmalıdır.

N = 1 ve eleman 7 sonrasında K için -1, 1001, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Adım sayısı 0 ile 1000 arasında olmalıdır.

Sınır durumları

- $K = 0$ veya K 'nin N 'ye tam bölünmesi durumunda gerçek adım 0'dır; dizi değişmez.
- $N = 1$ için bütün geçerli K değerlerinin kalanı 0'dır; tek eleman korunur.
- Bir adım son elemanı başa getirir; ters çevirme gibi bütün sırayı tersine döndürmez.
- İleri kopyalama, örneğin 1, 2, 3 dizisinde önce 1'i ikinci konuma yazıp sonra bu değiştirilmiş değeri üçüncü konuma taşıyarak veri kaybettirir.
- Gerçek adım N 'den küçüktür. $N = 50$ için en fazla 49 tur ve tur başına 49 komşu kopyası yapılır; bu yöntem öğrenim için küçük girişlerde uygundur.
- Değerlerin tekrarlanması ve int uç değerleri güvenle işlenir; değerler yalnızca saklanıp atanır.
- Eksik veya geçersiz girişte Dizi: satırı yazılmaz ve kısmi sonuç verilmaz.

Kazanımlar

- Döndürmeyi sıralamadan ve ters çevirmeden ayırt etme.
- Tam tur tekrarlarını kalan işleciyle azaltma.
- Yerinde kaydırmada kopyalama yönünü eski veriyi koruyacak şekilde seçme.
- Eski son elemanı geçici değişkenle saklayıp başa yerleştirme.
- Sıfır gerçek adım ve tek eleman durumlarını döngü koşuluyla ele alma.

Alıştırmalar

1. Bir sola döndürme sürümü yazın; ilk elemanı saklayıp kopyalama yönünü değiştirin. Aynı üç elemanı sağa ve sola bir adım döndürün.
2. Gerçek tur ve komşu kopyası adetlerini iki sayaçla bulun. $N = 5$ ve $K = 12$ için beklenen 2 tur ve 8 kopyayı doğrulayın.
3. Diziyi K adım sağa döndürdükten sonra K adım sola döndürün; tekrarlı ve uç değerli örneklerde başlangıç sırasının geri geldiğini gösterin.

045 Ön ek toplamıyla aralık toplamı hesaplama

Düzey: İleri Önkoşullar: 008, 010, 041, 044

Kavramlar: Ön ek toplamı, Ek dizi, Dahil aralık, Dizin dönüşümü, long biriktirme, Aralık sorgusu

Problem tanımı

1 ile 50 tamsayı içeren dizide, kullanıcının belirttiği başlangıç ve son dizin arasındaki elemanların toplamını bulun. İki sınır da toplama dahildir. Dizinler sıfırdan başlar; 0 ilk elemanı, N - 1 son elemanı gösterir. Başlangıç son dizinden büyük olamaz. Elemanlar tam int aralığındadır; toplam daha geniş long türünde tutulmalıdır.

Aralık toplamı için önce bir ön ek toplamı dizisi oluşturun. $onEk[j]$, asıl dizinin ilk J elemanının toplamıdır; bu bir adet bilgisidir. Bu nedenle $onEk[0] = 0$, $onEk[1] = sayilar[0]$ ve $onEk[n]$ bütün elemanların toplamıdır. Başlangıç B, son S olduğunda dahil aralığın toplamı $onEk[S + 1] - onEk[B]$ olur. Böylece istenen bölgeden önceki elemanlar çıkarılır; son eleman ise dahil kalır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; dizi uzunluğu.
Girdi	sayilar	N adet tam int eleman.
Girdi	baslangic	0 ile N - 1 arasında dahil başlangıç dizini.
Girdi	son	Başlangıç ile N - 1 arasında dahil son dizin.
Ara değer	onEk	N + 1 uzunluklu long dizisi; ilk J elemanın toplamı.
Çıktı	toplam	Toplam: X biçiminde dahil aralık toplamı.

Sırayla Eleman sayısı (1..50): , Eleman[0]: ile başlayan N eleman istemi, Başlangıç dizini: ve Son dizin: yanıtlanır. Her tamsayı ayrı satırda girilir. Bütün elemanlar ve iki dizin doğrulanmadan sonuç yazılmaz. Örneğin beş elemanda başlangıç 1 ve son 3 ise 1, 2 ve 3. dizinler toplama katılır.

Algoritma

1. N değerini doğrulayın; asıl diziyi oluşturup bütün elemanları doğrulayarak okuyun.
2. Başlangıç dizinini 0..N - 1 aralığında doğrulayın.
3. Son dizini başlangıç..N - 1 aralığında doğrulayın.
4. N + 1 uzunluğunda bir long ön ek dizisi oluşturun; başlangıç ögesi 0'dır.
5. I dizinini 0'dan N - 1'e kadar dolaşın; $onEk[I + 1] = onEk[I] + sayilar[I]$ hesaplayın.
6. $onEk[son + 1] - onEk[baslangic]$ ile aralık toplamını bulun.
7. Toplamı yazdırın.

Ön ek kurma döngüsü başında $onEk[i]$, işlenmiş ilk I elemanın toplamıdır. Sıradaki eleman eklendiğinde bir sonraki öge ilk I + 1 elemanın toplamı olur. Ön ek dizisi bu nedenle asıl diziden bir öge uzundur. son + 1 konumu en fazla N olabilir ve ayrılan diziye sığar. Ön ek dizisi hazırlandıktan sonra aralık için yeniden eleman taraması gerekmez; iki ön ek değeri okunup çıkarılır. Bu örnek bir sorgu yapar; aynı ön ek dizisi daha sonra birden çok sorguda da kullanılabilir.

3, -2, 7, 4, -1 dizisinin ön ekleri:

Ön ek dizini	Eklenen eleman	Ön ek toplamı
0	Henüz eleman yok	0
1	sayilar[0] = 3	3
2	sayilar[1] = -2	1
3	sayilar[2] = 7	8
4	sayilar[3] = 4	12
5	sayilar[4] = -1	11

Başlangıç 1, son 3 için $onEk[4] - onEk[1] = 12 - 3 = 9$ olur. Doğrudan bölüm toplamı da $-2 + 7 + 4 = 9$ 'dur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] sayilar = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("Eleman[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    sayilar[i] = deger;
}

Console.Write("Başlangıç dizini: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int baslangic) ||
    baslangic < 0 || baslangic >= n)
{
    Console.WriteLine("Hata: Başlangıç dizini dizi sınırları içinde olmalıdır.");
    return;
}

Console.Write("Son dizin: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int son) || son < baslangic || son >= n)
{
    Console.WriteLine("Hata: Son dizin başlangıç ile n-1 arasında olmalıdır.");
    return;
}
```

```

long[] onEk = new long[n + 1];
for (int i = 0; i < n; i++)
{
    onEk[i + 1] = onEk[i] + sayilar[i];
}

```

```

long toplam = onEk[son + 1] - onEk[baslangic];
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Yeni sayısal dizinin öğeleri başlangıçta 0 olduğundan onEk[0] ayrıca atanmasa da 0'dır. Toplama işleminde onEk[i] zaten long olduğu için int eleman bu türe genişletilerek eklenir; toplamı önce int içinde hesaplayıp sonradan dönüştürmek gerekmez. Asıl dizi değiştirilmez; ek dizi hesaplanmış ara bilgiyi taşır.

Örnek çalıştırmalar

Girdi sırası N, N eleman, başlangıç ve son dizindir. Hepsi ayrı satırda verilir. Bloklar istemler dışındaki tam sonucu gösterir.

N = 5; elemanlar 3, -2, 7, 4, -1; başlangıç 1, son 3:

Toplam: 9

N = 3; elemanlar -7, 0, 8; başlangıç 2, son 2:

Toplam: 8

N = 2; elemanlar 2147483647, 2147483647; başlangıç 0, son 1:

Toplam: 4294967294

N = 2; elemanlar -2147483648, -2147483648; başlangıç 0, son 1:

Toplam: -4294967296

N = 1; eleman -2147483648; başlangıç 0, son 0:

Toplam: -2147483648

Uzun dizilerde aşağıdaki özellikleri denetleyin; tablo tam eleman listesi değildir.

Girdi	Beklenen özellik
N = 50; bütün elemanlar 2147483647; başlangıç 0, son 49	Toplam 107374182350 olur; long içinde kalır.
N = 50; bütün elemanlar -2147483648; başlangıç 0, son 49	Toplam -107374182400 olur; long içinde kalır.

N için 0, 51, -1, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda 2147483648, -2147483649, 3.5, metin, boş satır veya giriş sonu verilirse dizinler istenmeden:

Hata: Eleman geçerli bir int tamsayı olmalıdır.

N = 3 ve elemanlar 1, 2, 3 sonrasında başlangıç için -1, 3, 1.5, metin, boş satır veya giriş sonu verilirse son dizin istenmez:

Hata: Başlangıç dizini dizi sınırları içinde olmalıdır.

Aynı dizide başlangıç 1 iken son için 0, 3, 2.5, metin, boş satır veya giriş sonu verilirse:

Hata: Son dizin başlangıç ile n-1 arasında olmalıdır.

Sınır durumları

- Başlangıç ve son eşitse tek eleman toplanır; $onEk[B + 1] - onEk[B]$ o elemanı verir.
- Başlangıç 0 ise çıkarılan ön ek 0'dır; son $N - 1$ ise okunan son ön ek $onEk[n]$ olur.
- Sonun başlangıçtan küçük olması boş aralık gibi yorumlanmaz; bu çalışma hatalı aralığı reddeder.
- Negatif elemanlar ön ekleri küçültebilir; ön ek dizisinin artan olması gerekmez. Fark formülü yine geçerlidir.
- En fazla 50 tam int elemanın toplamı -107374182400 ile 107374182350 arasındadır. Ön ekler ve geçerli aralık toplamaları $long$ türüne sığar; int yeterli değildir.
- Dizi doldurulduktan sonra değişirse mevcut ön ek bilgisi güncelliğini kaybeder; yeni değerler için ön ekler yeniden hesaplanmalıdır.
- Herhangi bir yanlış biçim, tür taşması, boş veya eksik giriş ve geçersiz dizin sonuç yazılmadan reddedilir.

Kazanımlar

- İlk J elemanın toplamını $N + 1$ uzunluklu ek dizide saklama.
- Asıl eleman dizini ile ön ekteki eleman adedinin farkını açıklama.
- İki ucu dahil aralık toplamını iki ön ek farkıyla hesaplama.
- İlk elemandan başlayan ve tek elemanlı aralıkları aynı formülle işleme.
- Tam int veride biriktiriciyi baştan $long$ seçerek taşmayı önleme.
- Ön hesaplamanın birden çok aralık sorgusuna nasıl hizmet ettiğini açıklama.

Alıştırmalar

1. Ön ek dizisini Ön ek: satırında 0. öge dahil yazdırın; 3, -2, 7, 4, -1 için altı değeri izleme tablosuyla karşılaştırın.
2. Aynı dizi ve aynı ön ekler için üç farklı geçerli aralık okuyup toplamalarını yazdırın. Her sorgunun dizinlerini ayrıca doğrulayın.
3. Aynı aralığı bir döngüyle doğrudan toplayın; bulunan $long$ sonuçla ön ek farkını karşılaştırın. Tek eleman ve bütün dizi durumlarını deneyin.

046 Dizinin azalmayan sırada olduğunu denetleme

Düzey: İleri Önkoşullar: 042, 043, 045

Kavramlar: Komşu karşılaştırma, Azalmayan sıra, İlk ihlal, Erken çıkış, Dizin sınırı

Problem tanımı

Bir tamsayı dizisinin azalmayan sırada olup olmadığını belirleyin. Azalmayan sıra, her elemanın kendinden önceki elemandan küçük olmaması demektir; eşit değerler kabul edilir. Örneğin -7, 0, 0, 9 sıralıdır. 2, 2, -1, 7 dizisinde ise -1, önceki 2'den küçüktür; ilk ihlal -1'in bulunduğu 2 numaralı dizindedir.

Diziyi sıralamayın veya değiştirmeyin. Komşu çiftleri soldan sağa karşılaştırıp ilk azalmada durun. İhlal dizini, karşılaştırılan çiftin sağındaki elemanın sıfırdan başlayan dizinidir. Bu denetim, sonraki problemde ikili aramanın önkoşulunu doğrulayacaktır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; dizi uzunluğu.
Girdi	a	Her biri -2147483648..2147483647 aralığında n adet int.
Çıktı	Sıra durumu	Sonuç: Azalmayan sırada veya Sonuç: Azalmayan sırada değil.
Çıktı	ihlal	Sıra bozursa ikinci satır İlk ihlal dizini: I; çiftin sağ dizini.

Önce Eleman sayısı (1..50): istemini yanıtlayın, sonra A[0]: ile başlayan istemlerde bütün elemanları ayrı satırlarda girin. Dizinler 0'dan başlar. Geçersiz uzunluk veya eleman girişinde yalnızca ilgili hata yazılır; önceki elemanlar için kısmi sıra sonucu verilmez.

Algoritma

1. Uzunluğu doğrulayıp bu uzunlukta bir dizi oluşturun.
2. Bütün elemanları sırayla okuyup doğrulayarak diziyi kaydedin.
3. İlk ihlal dizinini -1 ile başlatın; bu değer henüz ihlal olmadığını belirtir.
4. Dizini 1'den n - 1 dahil olacak şekilde ilerletin.
5. $a[i - 1] > a[i]$ ise ihlal dizinini i yapıp döngüden çıkın.
6. İhlal -1 kaldıysa sıralı mesajı; aksi halde sırasız mesajı ve ihlal dizinini yazdırın.

2, 2, -1, 7 dizisi için izleme:

Sağ dizin i	Sol değer	Sağ değer	Sol > sağ	İşlem
1	2	2	Yanlış	Eşitlik geçerli; devam.
2	2	-1	Doğru	İhlal 2; döngü biter.

Döngü başında daha önce denetlenen bütün komşu çiftlerin azalmadığı bilinir. Soldan sağa ilerlemek, bulunan ilk ihlalin en küçük sağ dizine sahip olmasını sağlar. Bir ihlal bütün dizinin sıralı olmadığını kesinleştirdiği için sonraki çiftler gereksizdir. Döngü ihlalde veya sınırlı sayıdaki komşu

çiftlerin sonunda biter. Karşılaştırma çıkarma kullanmadığından int uç değerlerinde fark taşması oluşmaz.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

int ihlal = -1;
for (int i = 1; i < n; i++)
{
    if (a[i - 1] > a[i])
    {
        ihlal = i;
        break;
    }
}

if (ihlal == -1)
{
    Console.WriteLine("Sonuç: Azalmayan sırada");
}
else
{
    Console.WriteLine("Sonuç: Azalmayan sırada değil");
    Console.WriteLine("İlk ihlal dizini: " +
        ihlal.ToString(CultureInfo.InvariantCulture));
}
```

Sayaç 1'den başladığı için $i - 1$ hiçbir zaman negatif olmaz. $i < n$, sağ elemanın da geçerli dizinde kalmasını sağlar. $>=$ karşılaştırması kullanmak eşit komşuları yanlışlıkla ihlal sayardı.

Örnek çalıştırmalar

Uzunluk ve elemanlar belirtilen sırayla ayrı satırlarda girilir. Metin blokları istemlerden sonraki bütün sonuç satırlarını gösterir.

$n = 4$, dizi -7, 0, 0, 9 için tam çıktı:

Sonuç: Azalmayan sırada

n = 4, dizi 2, 2, -1, 7 için tam çıktı:

Sonuç: Azalmayan sırada değil

İlk ihlal dizini: 2

n = 1, dizi -2147483648 için tam çıktı:

Sonuç: Azalmayan sırada

n = 2, dizi 2147483647, -2147483648 için tam çıktı:

Sonuç: Azalmayan sırada değil

İlk ihlal dizini: 1

Aşağıdaki örneklerde yalnızca ilgili tek hata satırı yazılır.

Girdi	Tam sonuç
Uzunluk 0 veya 51	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
Uzunluk iki veya 2.5	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
n = 2, ilk eleman 7, ikinci eleman sayı	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, eleman 2147483648	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, eleman -2147483649 veya 1.5	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, sonra boş satır veya girişin sonu	Hata: Eleman int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Tek elemanın karşılaştırılacak komşusu yoktur; dizi azalmayan sıradadır.
- Bütün değerlerin eşit olması geçerlidir; azalmayan sıra, kesin artış gerektirmez.
- İlk ihlal en erken 1, en geç $n - 1$ dizinindedir; 0 sağ dizin olamaz.
- İhlalden sonra gelen bir artış, daha önceki ihlali düzeltmez.
- int alt ve üst uçları doğrudan karşılaştırılır; elemanları birbirinden çıkarma yapılmaz.
- Bütün dizi doğrulanmadan denetim ve sonuç yazdırma başlamaz. Boş satır, EOF, metin, kesirli sayı ve int taşması eleman hatasıdır.

Kazanımlar

- Azalmayan sıra ile kesin artan sıra arasındaki eşitlik farkını açıklama.
- Komşu çiftlerin sağ dizininden sol dizine güvenli erişme.
- Soldan sağa taramayla ilk ihlali bulup erken çıkma.
- Veri doğrulamasıyla sıra denetimini ayrı adımlarda yürütme.
- Karşılaştırmayı çıkarma yerine doğrudan yaparak uç değerleri güvenli işleme.

Alıřtırmalar

1. Dizinleri biriktirmeden bütn ihlal çiftlerini sırayla yazdırın ve sayın; ilk ihlalde çıkışı kaldırın.
2. Kesin artan sırayı denetleyen sürümü yazın; 2, 2, 3 dizisinin iki tanımda farklı sonuç verdiğini gösterin.
3. Azalmayan yerine artmayan sırayı denetleyin; ilk ihlal için hangi karşılařtırmanın deęiřeceğini açıklayın.

047 Dizide ikili arama ile ilk eşleşmeyi bulma

Düzey: İleri Önkoşullar: 042, 046

Kavramlar: İkili arama, Sıralılık önkoşulu, Arama aralığı, İlk eşleşme, Tekrarlanan değerler

Problem tanımı

Azalmayan sırada verilmiş bir tamsayı dizisinde aranan değerın ilk bulunduđu dizini ikili aramayla belirleyin. Dizinler sıfırdan başlar. Aynı değeri birden fazla kez bulunabilir; bu durumda herhangi bir eşleşme yerine en küçük dizin istenir. Örneğin -5, 0, 0, 0, 8 içinde 0'ın ilk dizini 1'dir.

Önce bütün diziyi okuyun ve 046 numaralı problemdeki komşu karşılaştırmasıyla sıralılığını doğrulayın. Sırasız dizi için hata verin; diziyi kendiliğinden sıralamayın. Geçerli sıralı dizi için aranan değeri okuyun. İkili aramada ortadaki elemanı karşılaştırıp aday aralığının bir yarısını eleyin. Eşleşme bulunduğunda daha küçük bir dizin olabileceği için soldaki bölümde aramaya devam edin.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; dizi uzunluğu.
Girdi	a	Tam int aralığında n eleman; azalmayan sırada olmalıdır.
Girdi	aranan	-2147483648..2147483647 aralığında int.
Çıktı	İlk dizin	Bulunursa İlk dizin: I; bulunamazsa Sonuç: Bulunamadı.

Sırasıyla Eleman sayısı (1..50): , A[0]: ile başlayan eleman istemleri ve Aranan değeri: yanıtlanır. Dizi sırasızsa Hata: Dizi azalmayan sırada olmalıdır. yazılır ve aranan değeri istenmez. Girdi biçimi hatalarında ilgili hata satırı yazılır; arama sonucu verilmez.

Algoritma

1. Uzunluğu ve bütün elemanları doğrularak diziyi oluşturun.
2. Dizinin komşularını karşılaştırın; bir azalma varsa sıra hatası yazıp bitirin.
3. Aranan tamsayıyı okuyup doğrulayın.
4. sol = 0, sag = n - 1, bulunan = -1 ile başlayın.
5. sol <= sag olduğu sürece orta = sol + (sag - sol) / 2 hesaplayın.
6. Orta değeri aranan değere eşitse dizini kaydedip sag = orta - 1 yapın.
7. Orta değeri küçükse sol = orta + 1; büyükse sag = orta - 1 yapın.
8. Aralık boşaldığında kayıtlı dizini veya bulunamadı mesajını yazdırın.

Sıralılık, orta değeri küçükken sol bölümün de küçük olduğunu; büyükken sağ bölümün de büyük olduğunu garanti eder. Eşitlikte kaydedilen dizin bir çözümdür, ancak daha küçük eşleşmeler yalnızca solda bulunabilir. Her adımda orta konum aralıktan çıkarılır; kalan aralık küçülür ve sonunda boşalır. Orta hesabındaki fark önce alındığı için sol ve sağ sınırın doğrudan toplamına gerek yoktur.

-5, 0, 0, 0, 8 dizisinde 0 araması:

Sol	Sağ	Orta	Orta değer	İşlem
0	4	2	0	Bulunan 2; sağ 1 olur.
0	1	0	-5	Sol 1 olur.
1	1	1	0	Bulunan 1; sağ 0 olur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 1; i < n; i++)
{
    if (a[i - 1] > a[i])
    {
        Console.WriteLine("Hata: Dizi azalmayan sırada olmalıdır.");
        return;
    }
}

Console.Write("Aranan değer: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int aranan))
{
    Console.WriteLine("Hata: Aranan değer int aralığında bir tamsayı olmalıdır.");
    return;
}
```

```

int sol = 0;
int sag = n - 1;
int bulunan = -1;
while (sol <= sag)
{
    int orta = sol + (sag - sol) / 2;
    if (a[orta] == aranan)
    {
        bulunan = orta;
        sag = orta - 1;
    }
    else if (a[orta] < aranan)
    {
        sol = orta + 1;
    }
    else
    {
        sag = orta - 1;
    }
}

if (bulunan == -1)
{
    Console.WriteLine("Sonuç: Bulunamadı");
}
else
{
    Console.WriteLine("İlk dizin: " +
        bulunan.ToString(CultureInfo.InvariantCulture));
}

```

Eşitlik dalında break kullanılmaz; soldaki eşleşmeler denenir. bulunan değişkeni bu arada önceki çözümü korur. Sol tarafta başka eşleşme çıkmazsa kayıtlı dizin kaybolmaz. Sıralılık denetimi bütün diziyi tarar; dolayısıyla bu tam program yalnızca ikili aramanın işlem sayısı ile değerlendirilmez.

Örnek çalıştırmalar

Girdi sırası uzunluk, bütün dizi elemanları, ardından aranan değerdir. Metin blokları istemler dışındaki bütün sonuç satırlarını gösterir.

n = 5, dizi -5, 0, 0, 0, 8, aranan 0 için tam çıktı:

İlk dizin: 1

n = 4, dizi -4, 0, 3, 8, aranan 4 için tam çıktı:

Sonuç: Bulunamadı

n = 1, dizi 7, aranan 7 için tam çıktı:

İlk dizin: 0

n = 2, dizi -2147483648, 2147483647, aranan 2147483647 için tam çıktı:

İlk dizin: 1

Hatalı örnekler yalnızca aşağıdaki tek satırı üretir. Sırasız örnekte aranan değer okunmaz.

Girdi	Tam sonuç
Uzunluk 0 veya 51	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
n = 2, dizi 3, 2	Hata: Dizi azalmayan sırada olmalıdır.

Girdi	Tam sonuç
n = 1, eleman sayı, 1.5 veya 2147483648	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, eleman için boş satır veya giriş sonu	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, dizi 7, aranan yedi veya 7.5	Hata: Aranan değer int aralığında bir tamsayı olmalıdır.
n = 1, dizi 7, aranan -2147483649	Hata: Aranan değer int aralığında bir tamsayı olmalıdır.
n = 1, dizi 7, sonra boş satır veya giriş sonu	Hata: Aranan değer int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Tek elemanlı dizide ilk orta 0'dır; karşılaştırmadan sonra aralık boşalır.
- Bütün elemanlar aranan değere eşitse arama sola ilerleyerek 0 dizinini bulur.
- Aranan değer ilk veya son elemanda olabilir; sınır konumları dahildir.
- Aranan değer dizinin en küçük değerinden küçük veya en büyük değerinden büyükse aralık boşalır ve bulunamadı yazılır.
- $sag = -1$ veya $sol = n$ oluşması aramayı bitiren geçerli durumdur; koşul yanlış olduğundan bu dizinlerde elemana erişilmez.
- Tam int uçları doğrudan karşılaştırılır; eleman farkı veya hazır arama işlevi kullanılmaz.
- Girdi veya sıra hatasında sonuç verilmez. Aranan değer, sıralılık doğrulamasından sonra istenir.

Kazanımlar

- İkili aramanın sıralılık önkoşulunu denetleme.
- Orta karşılaştırmasıyla aday aralığının hangi yarısının elenebileceğini açıklama.
- Tekrarlı değerlerde eşleşmeyi kaydedip daha küçük dizinleri arama.
- Aralık boşaldığında geçersiz dizin erişimi yapılmadığını gösterme.
- Sıralılık denetiminin işlem sayısı ile ikili arama adımlarının işlem sayısını ayırma.

Alıştırmalar

1. Aynı koşullarda son eşleşmeyi bulmak için eşitlik dalındaki sınır güncellemesini değiştirin.
2. Arama sırasında denenen orta dizinleri ve karşılaştırma sayısını yazdırın; beş aynı elemanda ilk dizinin nasıl bulunduğunu izleyin.
3. 042 numaralı doğrusal aramayla aynı sıralı girdileri deneyin; iki aramanın bulduğu ilk dizinin aynı olduğunu karşılaştırın.

048 Dizide değerlerin frekanslarını hesaplama

Düzy: İleri Önkoşullar: 010, 041, 046

Kavramlar: Frekans dizisi, Değerden dizine dönüşüm, Sıfır başlangıç, Biriktirme, Artan değer sırası

Problem tanımı

-10 ile 10 arasındaki tamsayılardan oluşan bir dizide her farklı değerin kaç kez bulunduğunu hesaplayın. Yalnızca dizide bulunan değerleri küçükten büyüğe yazdırın. Örneğin -10, 0, 10, -10, 0 için -10 iki kez, 0 iki kez, 10 bir kez bulunur.

Değer aralığı dar ve bilindiği için her olası değere ayrı bir sayaç ayırın. -10 için 0, 0 için 10, 10 için 20 dizisini kullanın; dönüşüm değeri + 10 olur. Dizi elemanlarını önce okuyup saklayın, ardından 21 elemanlı frekans dizisini doldurun. Böylece bir eleman hatalı olduğunda kısmi frekans listesi yazılmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; veri dizisinin uzunluğu.
Girdi	a	Her biri -10..10 aralığında n adet int.
Ara değer	frekans	21 tamsayı sayacı; i dizini i - 10 değerini sayar.
Çıktı	Frekans listesi	Frekanslar: başlığından sonra Değer: X, Adet: Y; sıfır olmayan sayaçlar.

Eleman sayısı (1..50): isteminden sonra A[0]: ile başlayan istemlerde her elemanı ayrı satıra girin. Geçersiz uzunlukta uzunluk hatası, geçersiz elemanda aralık hatası yazılır; liste başlığı üretilmez. Çıktı değer sırasındadır; girdide ilk görülme sırası kullanılmaz.

Algoritma

1. Uzunluğu doğrulayıp veri dizisini oluşturun.
2. Bütün elemanları okuyup -10..10 aralığını doğrulayarak diziye kaydedin.
3. 21 elemanlı frekans dizisi oluşturun; başlangıçtaki bütün sayaçlar 0'dır.
4. Veri dizisinin her elemanı için frekans[a[i] + 10] sayacını bir artırın.
5. Liste başlığını yazdırın.
6. Frekans dizilerini 0'dan 20'ye tarayın.
7. Sayaç sıfırdan büyükse değeri i - 10 ile geri dönüştürüp değer ve adedi yazdırın.

Veri değeri	Frekans dizini	Ters dönüşüm
-10	0	0 - 10 = -10
0	10	10 - 10 = 0
10	20	20 - 10 = 10

Sayma döngüsünün her adımı sonunda frekanslar, işlenmiş veri bölümündeki adetleri temsil eder. Frekansların toplamı işlenmiş eleman sayısına eşittir; döngü sonunda bu toplam n olur. Değerden

dizine dönüşüm, kabul edilen bütün değerleri 0..20 aralığına taşır. Son tarama dizin artışını değer artışına dönüştürür. Bütün döngüler sonlu dizi uzunluklarıyla sınırlıdır.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < -10 || deger > 10)
    {
        Console.WriteLine("Hata: Eleman -10 ile 10 arasında " +
            "bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

int[] frekans = new int[21];
for (int i = 0; i < n; i++)
{
    frekans[a[i] + 10]++;
}

Console.WriteLine("Frekanslar:");
for (int i = 0; i < frekans.Length; i++)
{
    if (frekans[i] > 0)
    {
        Console.WriteLine("Değer: " +
            (i - 10).ToString(CultureInfo.InvariantCulture) +
            ", Adet: " + frekans[i].ToString(CultureInfo.InvariantCulture));
    }
}
```

`new int[21]`, bütün elemanları otomatik olarak 0 ile başlatır; ayrıca sıfırlama döngüsü gerekmez. `frekans[a[i] + 10]++` hem hangi sayacın kullanılacağını seçer hem o sayacı artırır. Veri dizisini sıralamak gerekmez; çıktı frekans dizisi taranarak sıralı olur.

Örnek çalıştırmalar

Uzunluk ve elemanlar ayrı satırlarda girilir. Metin blokları istemler dışındaki bütün sonuç satırlarını gösterir.

n = 5, dizi -10, 0, 10, -10, 0 için tam çıktı:

Frekanslar:

Değer: -10, Adet: 2

Değer: 0, Adet: 2

Değer: 10, Adet: 1

n = 1, dizi -10 için tam çıktı:

Frekanslar:

Değer: -10, Adet: 1

n = 4, dizi 5, 5, 5, 5 için tam çıktı:

Frekanslar:

Değer: 5, Adet: 4

Aşağıdaki hata örneklerinde yalnızca belirtilen tek satır yazılır.

Girdi	Tam sonuç
Uzunluk 0, 51 veya iki	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
n = 1, eleman 11 veya -11	Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır.
n = 2, ilk eleman 0, ikinci eleman sayı	Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır.
n = 1, eleman 1.5 veya 2147483648	Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır.
n = 1, sonra boş satır veya girişin sonu	Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır.

Sınır durumları

- -10 ve 10, sırasıyla ilk ve son frekans konumuna denk gelir; ikisi de geçerlidir.
- 0 özel bir bitiş işareti değildir; normal bir veri değeridir ve 10 numaralı sayaçta sayılır.
- Bütün elemanlar eşitse yalnızca bir değer satırı yazılır; sayacı n olur.
- Bütün sayaçlar en fazla 50'dir ve toplamı dizi uzunluğuna eşittir; int taşması oluşmaz.
- Dizide bulunmayan değerlerin sayaçları 0 kalır ve çıktı satırı üretilmez.
- Girdi aralığı denetlenmeden deger + 10 dizini kullanılmaz. Boş, EOF, metin, ondalık ve int dışı girişler tek hata satırıyla reddedilir.

Kazanımlar

- Dar bir değer aralığının tamamı için sayaç dizisi oluşturma.
- Negatif değerleri kaydırmayla geçerli sıfır tabanlı dizine dönüştürme.
- Aynı dönüşümün tersini kullanarak dizinden veri değerini elde etme.
- Frekans toplamının veri sayısına eşit olduğunu kontrol etme.
- Veri sırası değişse de frekansların aynı kaldığını gösterme.

Alıřtırmalar

1. En yksek frekansı ve bu frekansa sahip btn deęerleri yazdırın; eřit adet durumunda deęerleri artan sırada tutun.
2. Deęer aralıęını -20..20 yapın; frekans uzunluęunu, kaydırmayı ve ters dnřm birlikte gncelleyin.
3. Her deęer iin adet kadar * yazdırarak yatay bir frekans grnm oluřturun; boř sayaları yine atlayın.

049 İki dizinin ortak değerlerini bulma

Düzey: İleri Önkoşullar: 031, 042, 048

Kavramlar: İki dizide arama, Ortak değerler, Tekrar eleme, Geçerli sonuç bölümü, İlk görülme sırası

Problem tanımı

İki tamsayı dizisinde de bulunan farklı değerleri bulun. Bir değer dizilerden birinde veya ikisinde tekrarlanırsa da sonuçta yalnızca bir kez yer almalıdır. Çıktı, ilk dizide değerlerin ilk görülme sırasını korumalıdır; sayısal sıralama yapılmaz. Örneğin A dizisi 3, 1, 3, 2, B dizisi 2, 3, 3, 8, 1 ise sonuç 3 1 2 olur.

A'nın her elemanını önce B içinde arayın. B'de bulunan değeri, sonuç dizisinin doldurulmuş bölümünde de arayın; daha önce eklenmemişse sona ekleyin. Sonuç dizisinin kapasitesi A'nın uzunluğu kadar yeterlidir, ancak geçerli sonuç sayısı ayrıca tutulmalıdır. Yeni bir tamsayı dizisindeki başlangıç sıfırları, doldurulmadıkça sonuç değeri sayılmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, m	Sırasıyla A ve B uzunlukları; her biri 1..20 aralığında int.
Girdi	a, b	Tam int aralığında sırasıyla n ve m eleman.
Ara değer	ortaklar	Kapasitesi n; yalnızca 0..adet - 1 bölümü doludur.
Çıktı	Ortak liste	Ortak değerler: ardından birer boşlukla değerler; boşsa Ortak değerler: Yok.
Çıktı	adet	Adet: N biçiminde farklı ortak değer sayısı.

Önce A uzunluğu (1..20): , sonra B uzunluğu (1..20): istemlerini yanıtlayın. Ardından bütün A elemanlarını A[0]: ile, sonra bütün B elemanlarını B[0]: ile başlayan istemlerde ayrı satırlarda girin. Bütün girdiler doğrulanmadan ortak liste yazılmaz.

Algoritma

- İki uzunluğu okuyup 1..20 aralığını doğrulayın.
- İki diziyi oluşturup A ve B elemanlarını ayrı okuma döngülerinde doğrularak kaydedin.
- A uzunluğunda sonuç dizisi oluşturun ve geçerli sonuç adedini 0 yapın.
- A elemanlarını giriş sırasıyla dolaşın.
- Mevcut A değerini B içinde arayın; ilk eşleşmede B aramasını bitirin.
- B'de bulunmuşsa aynı değeri sonuç dizisinin yalnızca geçerli bölümünde arayın.
- Henüz sonuçta yoksa ortaklar[adet] konumuna yazıp adedi artırın.
- Döngü bitince geçerli sonuç bölümünü ve adedi yazdırın; adet 0 ise listeye Yok ekleyin.

Dış döngü adımı sonunda sonuç bölümünde, işlenmiş A elemanları arasından B'de bulunan bütün farklı değerler vardır. Tekrar denetimi aynı değerın yeniden eklenmesini engeller. A sırayla tarandı ve her yeni sonuç sona eklendiği için ilk görülme sırası korunur. Her arama sonlu bir dizi bölümünü tarar veya eşleşmede biter. Sonuç adedi A uzunluğunu aşamaz.

Yukarıdaki örneğin A taraması:

A dizini	Değer	B'de var mı?	Önceden sonuçta mı?	Geçerli sonuç
0	3	Evet	Hayır	3
1	1	Evet	Hayır	3, 1
2	3	Evet	Evet	3, 1
3	2	Evet	Hayır	3, 1, 2

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("A uzunluğu (1..20): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 20)
{
    Console.WriteLine("Hata: A uzunluğu 1 ile 20 arasında bir tamsayı olmalıdır.");
    return;
}
Console.WriteLine("B uzunluğu (1..20): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int m) || m < 1 || m > 20)
{
    Console.WriteLine("Hata: B uzunluğu 1 ile 20 arasında bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
int[] b = new int[m];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: A elemanı int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 0; i < m; i++)
{
    Console.WriteLine("B[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: B elemanı int aralığında bir tamsayı olmalıdır.");
        return;
    }
    b[i] = deger;
}
```

```

int[] ortaklar = new int[n];
int adet = 0;
for (int i = 0; i < n; i++)
{
    bool bdeVar = false;
    for (int j = 0; j < m; j++)
    {
        if (a[i] == b[j])
        {
            bdeVar = true;
            break;
        }
    }
    if (bdeVar)
    {
        bool ekli = false;
        for (int j = 0; j < adet; j++)
        {
            if (ortaklar[j] == a[i])
            {
                ekli = true;
                break;
            }
        }
        if (!ekli)
        {
            ortaklar[adet] = a[i];
            adet++;
        }
    }
}

Console.Write("Ortak değerler:");
for (int i = 0; i < adet; i++)
{
    Console.Write(" " + ortaklar[i].ToString(CultureInfo.InvariantCulture));
}
if (adet == 0)
{
    Console.Write(" Yok");
}
Console.WriteLine();
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));

```

İki arama farklı soruları yanıtlar: B taraması değerin ortak olup olmadığını, sonuç taraması daha önce yazılıp yazılmadığını belirler. Her A elemanında iki durum da yeniden başlatılır. Sonuç aramasında `j < ortaklar.Length` kullanmak henüz doldurulmamış sıfırları yanlışlıkla sonuç kabul edebilir; doğru sınır `j < adet` olur.

Örnek çalıştırmalar

Girdi sırası `n, m`, bütün A elemanları, bütün B elemanlarıdır. Metin blokları istemlerden sonraki bütün sonuç satırlarını gösterir.

`n = 4, m = 5, A 3, 1, 3, 2, B 2, 3, 3, 8, 1` için tam çıktı:

Ortak değerler: 3 1 2

Adet: 3

`n = 2, m = 2, A 1, 2, B 3, 4` için tam çıktı:

Ortak değerler: Yok

Adet: 0

$n = 1, m = 1$, A -2147483648, B -2147483648 için tam çıktı:

Ortak değerler: -2147483648

Adet: 1

$n = 3, m = 2$, A 0, 0, 2147483647, B 2147483647, 0 için tam çıktı:

Ortak değerler: 0 2147483647

Adet: 2

Aşağıdaki hatalar tek satır üretir; ortak liste veya adet yazılmaz.

Girdi	Tam sonuç
A uzunluğu 0, 21 veya iki	Hata: A uzunluğu 1 ile 20 arasında bir tamsayı olmalıdır.
A uzunluğu 1, B uzunluğu 0, 21 veya 1.5	Hata: B uzunluğu 1 ile 20 arasında bir tamsayı olmalıdır.
$n = 1, m = 1$, A elemanı 2147483648 veya sayı	Hata: A elemanı int aralığında bir tamsayı olmalıdır.
$n = 1, m = 1$, A 0, B elemanı -2147483649 veya 1.5	Hata: B elemanı int aralığında bir tamsayı olmalıdır.
Geçerli uzunluklardan sonra A için boş satır veya giriş sonu	Hata: A elemanı int aralığında bir tamsayı olmalıdır.
$n = 1, m = 1$, A 0, sonra B için boş satır veya giriş sonu	Hata: B elemanı int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Ortak değer bulunmaması geçerli sonuçtur; adet 0 ve Yok yazılır.
- Aynı değer A'da, B'de veya ikisinde tekrar etse de sonuçta bir kez bulunur.
- 0 geçerli bir değerdir; sonuç dizisinin boş kapasitesindeki başlangıç sıfırlarından ayrı düşünülmelidir.
- Sonuç sırası A'nın ilk görülme sırasıdır; A ve B yer değiştirirse sıralama değişebilir, ortak değer kümesi değişmez.
- Sonuç adedi hem A'nın hem B'nin farklı değer sayısından büyük olamaz; kapasite n yeterlidir.
- Bütün işlemler eşitlik ve dizin denetimidir; elemanların tam int uçları üzerinde aritmetik yapılmaz.
- Geçersiz uzunluk, eleman biçimi, boş satır veya EOF durumunda sonraki okumalar durur ve sonuç üretilmez.

Kazanımlar

- İki farklı dizi bölümünde farklı amaçlarla doğrusal arama uygulama.
- Ortaklık denetimiyle tekrar denetiminin görevlerini ayırma.
- Dizi kapasitesi ile geçerli sonuç sayısını ayrı yönetme.
- 0 gibi başlangıç değeriyle aynı verileri yalnızca dolu bölümde arama.
- İlk dizinin ilk görülme sırasını koruyarak farklı ortak değerler üretme.

Alıřtırmalar

1. A ve B'nin yerini deęiřtirerek aynı girdiyi deneyin; ortak deęerlerin deęiřmedięini, çıktı sırasının deęiřebildięini gösterin.
2. A'da olup B'de olmayan farklı deęerleri aynı sonuç dizisi yaklařımıyla üretin.
3. Her ortak deęerin A ve B içindeki ayrı tekrar sayılarını da hesaplayıp deęerle beraber yazdırın; sonuçta deęer başına tek satır üretin.

050 Dizide tekrarları yerinde kaldırma

Düzey: İleri Önkoşullar: 042, 043, 049

Kavramlar: Yerinde güncelleme, Okuma ve yazma konumu, Geçerli dizi bölümü, Tekrar eleme, İlk görülme sırası

Problem tanımı

Bir tamsayı dizisindeki tekrarları, ek bir sonuç dizisi oluşturmadan kaldırın. Her farklı değeri ilk görülme sırasıyla dizinin başlangıcına yerleştirin ve kaç farklı değer kaldığını bulun. Örneğin 3, 1, 3, 2, 1 için geçerli sonuç bölümü 3, 1, 2 olur.

Buradaki kaldırma, dizinin fiziksel uzunluğunu değiştirmez. Aynı dizinin başlangıcındaki geçerli bölümü sıkıştırır; geride kalan konumlar sonuca dahil edilmez. Giriş tamamlandıktan sonra okuma konumuyla bütün eski elemanları dolaşın. Geçerli bölümde henüz bulunmayan değeri sıradaki yazma konumuna alın. Diziyi sıralamayın; değerlerin ilk görülme sırası korunmalıdır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; fiziksel dizi uzunluğu.
Girdi	a	Tam int aralığında n adet eleman.
Ara değer	adet	Geçerli farklı değer sayısı; aynı zamanda sonraki yazma dizini.
Çıktı	Farklı değerler	Farklı değerler: ardından başlangıçtaki geçerli bölümün değerleri.
Çıktı	Geçerli adet	Adet: N biçiminde farklı değer sayısı.

Eleman sayısı (1..50): isteminden sonra A[0]: ile başlayan istemlerde bütün elemanları ayrı satırlarda girin. Her girdi doğrulanmadan sıkıştırma başlamaz. Hatalı uzunluk veya elemanda tek hata satırı yazılır; kısmi farklı değer listesi üretilmez.

Algoritma

1. Uzunluğu doğrulayıp diziyi oluşturun ve bütün elemanları okuyarak doğrulayın.
2. Geçerli farklı değer adedini 0 yapın.
3. Okuma dizinini 0'dan n - 1 dahil olacak şekilde ilerletin.
4. Okunan değeri bir tamsayı değişkeninde saklayın.
5. Bu değeri dizinin 0..adet - 1 geçerli bölümünde arayın.
6. Zaten bulunuyorsa diziyi ve adedi değiştirmeden sıradaki eski elemana geçin.
7. Bulunmuyorsa a[adet] konumuna yazın ve adedi bir artırın.
8. Bütün okuma konumları bitince yalnızca 0..adet - 1 bölümünü ve adedi yazdırın.

Her adımın başında dizinin geçerli başlangıç bölümü, daha önce okunmuş değerlerin farklı olanlarını ilk görülme sırasıyla içerir. Okuma dizini okuma iken en fazla okuma farklı değer vardır; dolayısıyla yazma dizini adet <= okuma olur. Bir değer yalnızca okunmuş bir konuma veya mevcut okuma konumuna yazılır; ileride okunacak konumlar bozulmaz. Yerinde yazmadan önce mevcut değerlerin saklanması, hesaplama adımında okunacak değerlerin korunmasını sağlar. Okuma sayacı sınırlı uzunluk boyunca arttığından işlem sonlanır.

3, 1, 3, 2, 1 örneğinin geçerli bölümü:

Okuma dizini	Saklanan değer	Daha önce var mı?	Yazma işlemi	Geçerli bölüm
0	3	Hayır	a[0] = 3	3
1	1	Hayır	a[1] = 1	3, 1
2	3	Evet	Yok	3, 1
3	2	Hayır	a[2] = 2	3, 1, 2
4	1	Evet	Yok	3, 1, 2

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

int adet = 0;
for (int okuma = 0; okuma < n; okuma++)
{
    int deger = a[okuma];
    bool bulundu = false;
    for (int j = 0; j < adet; j++)
    {
        if (a[j] == deger)
        {
            bulundu = true;
            break;
        }
    }
    if (!bulundu)
    {
        a[adet] = deger;
        adet++;
    }
}
```

```

Console.Write("Farklı değerler:");
for (int i = 0; i < adet; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));

```

a.Length bütün işlem boyunca n olarak kalır. Yalnızca sonuçta kullanılan uzunluk adet olur. Örnekte işlem sonunda fiziksel içerik 3, 1, 2, 2, 1 olur; son iki konum geçerli bölüm dışındadır ve yazdırılmaz. Tekrar araması da yalnızca geçerli bölümde yapılır; işlenmemiş eski elemanlar daha önce bulunan değerler gibi değerlendirilmez.

Örnek çalıştırmalar

Uzunluk ve dizi elemanları ayrı satırlarda girilir. Metin blokları istemler dışındaki bütün sonuç satırlarını gösterir.

n = 5, dizi 3, 1, 3, 2, 1 için tam çıktı:

```

Farklı değerler: 3 1 2
Adet: 3

```

n = 1, dizi -2147483648 için tam çıktı:

```

Farklı değerler: -2147483648
Adet: 1

```

n = 4, dizi 8, 8, 8, 8 için tam çıktı:

```

Farklı değerler: 8
Adet: 1

```

n = 4, dizi -2147483648, 2147483647, -2147483648, 0 için tam çıktı:

```

Farklı değerler: -2147483648 2147483647 0
Adet: 3

```

Aşağıdaki hatalarda yalnızca belirtilen tek satır yazılır.

Girdi	Tam sonuç
Uzunluk 0, 51 veya iki	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
n = 2, ilk eleman 7, ikinci eleman sayı	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, eleman 2147483648 veya -2147483649	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, eleman 1.5	Hata: Eleman int aralığında bir tamsayı olmalıdır.
n = 1, sonra boş satır veya girişin sonu	Hata: Eleman int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Tek eleman için geçerli adet 1 olur ve değer aynı konuma yazılır.
- Bütün elemanlar farklıysa her yazma okuma konumuna denk gelir; geçerli adet n olur.
- Bütün elemanlar aynıysa ilk değer tutulur, diğerleri atlanır; geçerli adet 1 olur.
- 0, negatif değerler ve tam int uçları normal veridir; elemanlar üzerinde aritmetik yapılmaz.
- Fiziksel uzunluk küçülmez; geçerli bölüm dışındaki eski değerler çıktıya katılmaz.
- $\text{adet} \leq \text{okuma}$, yazma işleminin henüz okunmamış bir elemanı bozmamasını garanti eder. Ek bir sonuç dizisi oluşturulmaz.
- Geçersiz girdi, boş satır veya EOF halinde sıkıştırma başlamaz ve kısmi sonuç yazılmaz.

Kazanımlar

- Fiziksel dizi uzunluğu ile geçerli veri uzunluğunu ayrı yorumlama.
- Okuma ve yazma konumlarını farklı amaçlarla kullanma.
- Yazma konumunun okuma konumunu aşmadığını adım tablosuyla gösterme.
- Ek dizi oluşturmadan ilk görülme sırasıyla farklı değerleri sıkıştırma.
- Sonuç aramasını ve yazdırmayı yalnızca geçerli başlangıç bölümüyle sınırlama.

Alıştırmalar

1. Sonuçtan sonra $n - \text{adet}$ değerini kaldırılan tekrar sayısı olarak yazdırın; örnekte 2 bulunduğunu doğrulayın.
2. Geçerli bölüm dışındaki alanı 0 ile doldurun; bu işlemi bütün okumalar tamamlandıktan sonra yapmanın neden gerekli olduğunu açıklayın.
3. 049 numaralı sonuç dizisi yaklaşımıyla aynı dizinin farklı değerlerini ayrı bir dizide üretin; iki yöntemin çıktısını ve ek kapasite kullanımını karşılaştırın.

Bölüm 6: Sıralama ve algoritma karşılaştırmaları

051 Seçmeli sıralama ile diziyi sıralama

Düzey: İleri Önkoşullar: 041, 043, 046

Kavramlar: Seçmeli sıralama, Sıralı başlangıç bölümü, En küçük aday, Değer karşılaştırması, Takas sayısı

Problem tanımı

1 ile 50 arasında tamsayı içeren bir diziyi seçmeli sıralama ile azalmayan sıraya getirin. Azalmayan sırada her değer kendisinden sonraki değerden küçük veya ona eşittir; tekrarlar korunur. Her eleman -2147483648 ile 2147483647 arasında, yani tam int aralığında olabilir.

Her turda henüz sıralanmamış bölümün en küçük elemanını bulun ve bu bölümün ilk konumuna taşıyın. Örneğin 3, 1, 2, 1 dizisi 1, 1, 2, 3 olur. Hazır sıralama kullanmayın; aday seçimini ve takası görünür işlemlerle gerçekleştirin.

Algoritmanın yaptığı değer karşılaştırmalarını ve takasları ayrıca sayın. Bir karşılaştırma, $a[j] < a[enKucuk]$ ifadesinin bir kez değerlendirilmesidir. Döngü koşulları, girdi doğrulaması ve enKucuk $!= i$ dizin denetimi bu sayaca dahil değildir. En küçük zaten tur başındaysa takas yapmayın; üç atamadan oluşan bir yer değiştirme tek takastır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int; eleman sayısı.
Girdi	a	Tam int aralığında N eleman.
Çıktı	Sıralı dizi	Dizi: ardından her eleman için bir boşluk ve değer.
Çıktı	Sayaçlar	Ayrı satırlarda Karşılaştırma: X ve Takas: Y.

Önce Eleman sayısı (1..50): istemini yanıtlayın. Sonra $A[0]:$, $A[1]:$ gibi sıfır tabanlı istemlerde her elemanı ayrı satırda girin. Girdiler tamsayı olmalıdır; basamak ayırıcı veya ondalık bölüm kullanılmaz. Bütün elemanlar doğrulanmadan sıralama başlamaz.

Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., geçersiz elemanda Hata: Eleman geçerli bir int tamsayı olmalıdır. yazılır. Boş satır veya giriş sonu (EOF) da ilgili alanın hata satırını üretir. Hata halinde dizi ve sayaç sonuçları yazılmaz.

Algoritma

1. Uzunluğu doğrulayın ve bütün elemanları okuyup doğrulayın.
2. Karşılaştırma ve takas sayaçlarını 0 yapın.
3. Tur başı dizinini $i = 0$ ile başlatıp $n - 2$ dahil ilerletin.
4. En küçük adayının dizinini i yapın.
5. $j = i + 1$ ile başlayan taramada her eleman için karşılaştırma sayacını artırın.
6. $a[j] < a[i]$ ise aday dizinini j yapın; eşitlikte değiştirmeyin.
7. Aday dizini i değilse geçici değişkenle iki elemanı takas edip takas sayacını artırın.
8. Bütün turlar bitince diziyi ve iki sayacı yazdırın.

Tur başında $0..i - 1$ bölümü, bütün dizinin en küçük i değerini sıralı biçimde içerir. İç döngüde aday, i konumundan o ana kadar incelenen konuma kadarki en küçük değerdir. Tarama tamamlanınca kalan bölümün en küçük i konumuna yerleşir; sıralı bölüm bir eleman büyür. Takas elemanları kaybetmez; aynı değerlerin adetleri korunur. Sınırlı dizinler arttığından döngüler sonlanır.

3, 1, 2, 1 dizisinin turları; sayaçlar birikimli verilmiştir:

Tur başı i	Seçilen dizin / değer	Tur sonundaki dizi	Karşılaştırma	Takas
0	1 / 1	1, 3, 2, 1	3	1
1	3 / 1	1, 1, 2, 3	5	2
2	2 / 2	1, 1, 2, 3	6	2

Her turda kalan bölüm tamamen taranır. Karşılaştırma sayısı girdinin sırasından bağımsız olarak $(n - 1) + (n - 2) + \dots + 1 = n \times (n - 1) / 2$ olur. Bu formül sonuç sayacını önceden atamak için kullanılmaz; kod gerçekten yapılan karşılaştırmaları sayar.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}
```

```

int karsilastirma = 0, takas = 0;
for (int i = 0; i < n - 1; i++)
{
    int enKucuk = i;
    for (int j = i + 1; j < n; j++)
    {
        karsilastirma++;
        if (a[j] < a[enKucuk])
        {
            enKucuk = j;
        }
    }
    if (enKucuk != i)
    {
        int gecici = a[i];
        a[i] = a[enKucuk];
        a[enKucuk] = gecici;
        takas++;
    }
}

Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Karşılaştırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Takas: " + takas.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Elemanlar çıkarılarak karşılaştırılmadığı için iki int uç değeri arasında taşma oluşmaz. Takas için yalnız bir geçici tamsayı gerekir; ek sonuç dizisi oluşturulmaz. Eşit adayda kayıtlı dizin korunur; ancak uzaktaki elemanların takası, eşit değerlerin özgün konumlarını koruyan bir yöntem olarak yorumlanmamalıdır.

Örnek çalıştırmalar

Girdi sırası N ve ardından N elemandır; her girdi ayrı satırdadır. Metin blokları istemler dışındaki bütün sonuç satırlarını gösterir.

N = 4; elemanlar 3, 1, 2, 1:

```

Dizi: 1 1 2 3
Karşılaştırma: 6
Takas: 2

```

N = 1; tek eleman -2147483648:

```

Dizi: -2147483648
Karşılaştırma: 0
Takas: 0

```

N = 4; elemanlar 1, 2, 3, 4:

```

Dizi: 1 2 3 4
Karşılaştırma: 6
Takas: 0

```

N = 3; elemanlar 2147483647, 0, -2147483648:

```

Dizi: -2147483648 0 2147483647
Karşılaştırma: 3
Takas: 1

```

Aşağıdaki tablo büyük girdilerin tam çıktısı değildir; denetlenecek özellikleri belirtir.

Girdi	Beklenen özellik
N = 50; elemanlar 50'den 1'e doğru	Dizi 1'den 50'ye sıralanır; 1225 karşılaştırma, 25 takas.
N = 50; bütün elemanlar 2147483647	50 değer korunur; 1225 karşılaştırma, 0 takas.

N için 0, 51, -1, 1.5, metin, boş satır veya EOF verilirse tam hata çıktısı:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 3 sonrasında 7 ve iki girilirse tek hata satırı aşağıdadır. Herhangi bir elemanda 2147483648, -2147483649, 2.5, boş satır veya EOF da aynı hatayı üretir; sıralama sonucu ve sayaçlar verilmez.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- N = 1 için hiçbir sıralama turu yoktur; iki sayaç da 0 kalır.
- Sıralı veya tamamen eşit bir dizide takas yapılmaz; bütün değer karşılaştırmaları yine yapılır.
- Son konum için ayrı bir tur gerekmez; önceki konumlar doğruysa kalan tek eleman zaten yerindedir.
- N en fazla 50 olduğundan karşılaştırma sayısı en fazla 1225, takas sayısı en fazla 49'dur; sayaçlar int türüne sığar.
- Negatif değerler, 0 ve tam int uçları normal veridir; değerler üzerinde toplama veya çıkarma yapılmaz.
- Hatalı son eleman bile işlemi başlatmaz; eksik dizinin kısmi sıralaması yazılmaz.

Kazanımlar

- Sıralı başlangıç bölümü ile henüz sıralanmamış bölümü ayırma.
- Her turda en küçük adayını yeniden başlatıp kalan bölümün tamamını tarama.
- Gereksiz kendi kendine takası dizin karşılaştırmasıyla önleme.
- Değer karşılaştırması ile döngü denetiminin sayımını ayırt etme.
- Aynı N için farklı başlangıç sıralarının karşılaştırma sayısını değiştirmedeğini gösterme.

Alıştırmalar

1. Her turdan sonra diziyi ve o turdaki karşılaştırma sayısını yazdırın; 3, 1, 2, 1 girdisini izleme tablosuyla karşılaştırın.
2. Kalan bölümün en büyüğünü seçerek diziyi azalan sıraya getirin; sayaçların neyi saydığı sözleşmesini koruyun.
3. N = 5 için sıralı, ters sıralı ve eşit elemanlı üç dizi deneyin. Her birinde 10 karşılaştırma olduğunu doğrulayın ve takas farkını açıklayın.

052 Erken bitişli kabarcık sıralama

Düzey: İleri Önkoşullar: 051

Kavramlar: Kabarcık sıralama, Komşu takası, Daralan döngü sınırı, Erken sonlandırma, Tur ve işlem sayıları

Problem tanımı

Tam int aralığındaki 1 ile 50 elemanı kabarcık sıralama ile azalmayan sıraya getirin. Soldan sağa komşu elemanları karşılaştırın; soldaki büyükse yerlerini değiştirin. Bir geçişin sonunda henüz sıralanmamış bölümün en büyük değeri bu bölümün sonuna taşınır. Sonraki turda bu konumu yeniden incelemeyin.

Bir turda hiç takas yapılmadıysa dizi zaten azalmayan sıradadır; kalan turları çalıştırmadan bitirin. Örneğin 1, 2, 3, 4 dizisi için tek tur yeterlidir. Takas koşulu kesinlikle > olmalıdır; eşit komşuların yerleri değiştirilmez ve tekrarlar korunur.

Üç sayaç tutun. Karşılaştırma, iç döngünün bir gövdesinde $a[j] > a[j + 1]$ değerlendirmesidir. Takas, komşu çiftin bir kez yer değiştirmesidir. Tur, en az bir komşu karşılaştırması içeren bir dış döngü geçişidir; hiç takas yapılmayan son denetim turu da sayılır. Girdi doğrulaması ve döngü sınırı denetimleri karşılaştırma sayısına dahil değildir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında int.
Girdi	a	Her eleman -2147483648..2147483647 aralığında.
Çıktı	Sıralı dizi	Dizi: ardından boşlukla ayrılan N değer.
Çıktı	Sayaçlar	Ayrı satırlarda Karşılaştırma: X, Takas: Y, Tur: Z.

Eleman sayısı (1..50): istemini ve ardından $A[0]:$, $A[1]:$ gibi istemleri yanıtlayın. N ve her eleman ayrı satırda okunur; ondalık bölüm veya basamak ayırıcı kullanılmaz. Bütün elemanlar doğrulanmadan sıralama başlamaz.

Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., geçersiz elemanda Hata: Eleman geçerli bir int tamsayı olmalıdır. yazılır. Boş satır veya EOF ilgili alanın hatasıdır. Hata halinde dizi ve sayaç sonuçları üretilmez.

Algoritma

1. N değerini ve bütün dizi elemanlarını doğrulayarak okuyun.
2. Karşılaştırma, takas ve tur sayaçlarını 0 yapın.
3. Dış döngüyü $gecis = 0$ ile başlatıp $n - 2$ dahil ilerletin.
4. Her geçişte $degisti$ durumunu yanlış yapın.
5. İç döngüde $j < n - 1 - gecis$ olduğu sürece komşu çifti karşılaştırıp sayacı artırın.
6. Soldaki büyükse elemanları takas edin, takas sayacını artırın ve $degisti$ durumunu doğru yapın.
7. İç döngü tamamlanınca tur sayacını bir artırın.
8. Hiç takas yapılmadıysa dış döngüden çıkın; sonunda diziyi ve sayaçları yazdırın.

Bir iç döngü adımından sonra o ana kadar gezilen bölümün en büyük değeri sağdaki konuma ilerlemiştir. Geçiş sonunda kalan bölümün en büyüğü son konumuna ulaşır. Dış döngü başında en sağdaki gecis eleman artık sıralıdır; iç sınır bu bölümü dışarıda tutar. Bir geçişte takas yoksa gezilen bütün komşular doğru sıradadır; önceden tamamlanan sağ bölümle birlikte bütün dizi sıralıdır. İç ve dış sınırlar sonlu olduğu için erken çıkış olmasa da algoritma biter.

3, 1, 2, 1 dizisi için turlar; karşılaştırma ve takas sütunları birikimlidir:

Tur	İç döngüde karşılaştırılan j dizinleri	Tur sonundaki dizi	Karşılaştırma	Takas
1	0, 1, 2	1, 2, 1, 3	3	3
2	0, 1	1, 1, 2, 3	5	4
3	0	1, 1, 2, 3	6	4

Üçüncü turda takas olmaz ve çıkış yapılır. Bu denetim gerçek bir karşılaştırma içerdiği için tur sayısı 3'tür. N = 1 durumunda iç döngülü bir geçiş oluşmadığından tur sayısı 0'dır.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}
```

```

int karsilastirma = 0, takas = 0, tur = 0;
for (int gecis = 0; gecis < n - 1; gecis++)
{
    bool degisti = false;
    for (int j = 0; j < n - 1 - gecis; j++)
    {
        karsilastirma++;
        if (a[j] > a[j + 1])
        {
            int gecici = a[j];
            a[j] = a[j + 1];
            a[j + 1] = gecici;
            takas++;
            degisti = true;
        }
    }
    tur++;
    if (!degisti)
    {
        break;
    }
}

Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Karşılaştırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Takas: " + takas.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Tur: " + tur.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. $j < n - 1 - \text{gecis}$ koşulu hem sağdaki tamamlanmış bölümü atlar hem de $j + 1$ erişimini dizi sınırında tutar. Tur sayacının iç döngüden sonra artırılması, takassız son turun da kayda girmesini sağlar. Elemanlar üzerinde aritmetik yapılmadığı için tam int aralığı güvenlidir.

Örnek çalıştırmalar

Girdi sırası N ve N elemandır; her değer ayrı satırdadır. Metin blokları istemler dışındaki bütün sonuç satırlarıdır.

N = 4; elemanlar 3, 1, 2, 1:

```

Dizi: 1 1 2 3
Karşılaştırma: 6
Takas: 4
Tur: 3

```

N = 4; elemanlar 1, 2, 3, 4:

```

Dizi: 1 2 3 4
Karşılaştırma: 3
Takas: 0
Tur: 1

```

N = 1; tek eleman 2147483647:

```

Dizi: 2147483647
Karşılaştırma: 0
Takas: 0
Tur: 0

```


N = 2; elemanlar -2147483648, -2147483648:

Dizi: -2147483648 -2147483648
Karşılaştırma: 1
Takas: 0
Tur: 1

N = 4; elemanlar 4, 3, 2, 1:

Dizi: 1 2 3 4
Karşılaştırma: 6
Takas: 6
Tur: 3

Bu tablo büyük çıktının tamamı değildir; beklenen özellikleri tanımlar.

Girdi	Beklenen özellik
N = 50; elemanlar 50'den 1'e doğru	1'den 50'ye sıralı dizi; 1225 karşılaştırma, 1225 takas, 49 tur.
N = 50; bütün elemanlar 2147483647	50 eşit değer korunur; 49 karşılaştırma, 0 takas, 1 tur.

N için 0, 51, 1.5, metin, boş satır veya EOF verilirse tam hata çıktısı:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 2 ve ilk eleman 7 sonrasında iki girilirse tek hata satırı aşağıdadır. Herhangi bir elemenda 2147483648, -2147483649, 2.5, boş satır veya EOF da aynı hatayı verir; sayaçlar yazılmaz.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- Tek elemenda karşılaştırma, takas ve tur sayıları 0'dır.
- Sıralı veya eşit elemanlı $N > 1$ dizisinde ilk tur $N - 1$ karşılaştırmayla biter; takas 0, tur 1 olur.
- $>$ yerine $\geq$ kullanılması eşit komşularda gereksiz takas yapar ve erken çıkışı bozabilir.
- Ters sıralı farklı elemanlarda en fazla $n \times (n - 1) / 2$ karşılaştırma ve takas yapılır; $N = 50$ için üst sınır 1225'tir.
- Tur sayısı en fazla $N - 1$, yani 49'dur. int sayaçlar bu sınırların tamamını karşılar.
- Hatalı veya eksik eleman halinde ilk sıralama turu başlamaz; kısmi dizi sonucu yoktur.

Kazanımlar

- Komşu takaslarının en büyük değeri sağa nasıl taşıdığını adım adım açıklama.
- Tamamlanmış sağ bölüme göre iç döngü sınırını daraltma.
- Bir tur başında değişim durumunu yeniden başlatma.
- Takassız bir turdan bütün dizinin sıralı olduğunu çıkarma.
- Değer karşılaştırması, takas ve tur sayılarını ayrı anlamlarla ölçme.

Alıştırmalar

1. Erken çıkış koşulunu kaldırıp 1, 2, 3, 4 dizisi için sayaçları karşılaştırın. Sonuç aynı kalırken karşılaştırma ve tur sayılarının neden arttığını açıklayın.
2. Her turun sonunda diziyi ve o turdaki takas sayısını yazdırın; 3, 1, 2, 1 girdisini tabloyla karşılaştırın.
3. 051 ve bu çözümü aynı sıralı ve ters sıralı dizilerde çalıştırın. İki algoritmanın farklı işlemler saydığı takas sonuçlarını ve aynı anlamdaki değer karşılaştırmalarını karşılaştırın.

053 Eklemeli sıralama ile diziyi sıralama

Düzey: İleri Önkoşullar: 044, 051, 052

Kavramlar: Eklemeli sıralama, Sıralı başlangıç bölümü, Anahtar saklama, Geriye kaydırma, Değer karşılaştırması

Problem tanımı

Tam int aralığında 1 ile 50 eleman içeren diziyi eklemeli sıralama ile azalmayan sıraya getirin. İlk elemanı sıralı bir bölüm kabul edin. Her sonraki elemanı anahtar olarak saklayıp soldaki sıralı bölümde doğru yerine ekleyin. Anahtardan büyük değerleri bir konum sağa kaydırın; eşit değerleri kaydırmayın. Örneğin 3, 1, 2, 1 dizisi 1, 1, 2, 3 olur; tekrarlar korunur.

İki sayaç tutun. Karşılaştırma, mevcut sol eleman ile anahtar arasındaki $a[j] \leq \text{anahtar}$ değerlendirmesidir; aramayı bitiren doğru değerlendirme de sayılır. $j \geq 0$ dizin denetimi sayılmaz. Kaydırma, eski bir elemanın $a[j + 1] = a[j]$ ile sağa taşınmasıdır. Anahtarın son konumuna yazılması kaydırma değildir. Hazır sıralama veya ekleme aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1..50 aralığında int ; eleman sayısı.
Girdi	a	Tam int aralığında N eleman; sırasız olabilir.
Çıktı	Sıralı dizi	Dizi: ardından her değer için bir boşluk ve değer.
Çıktı	Sayaçlar	Karşılaştırma: X, ardından Kaydırma: Y.

Önce Eleman sayısı (1..50): , sonra $A[0]$: ile başlayan istemlerde her değeri ayrı satıra girin. Bütün elemanlar doğrulanmadan işlem başlamaz. Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., geçersiz elemanda Hata: Eleman geçerli bir int tamsayı olmalıdır. yazılır. Boş satır ve giriş sonu (EOF) ilgili alanın hatasını üretir; kısmi dizi veya sayaç yazılmaz.

Algoritma

1. Uzunluğu ve bütün elemanları okuyup doğrulayın; iki sayacı 0 yapın.
2. i dizinini 1'den $N - 1$ 'e ilerletin; $a[i]$ değerini anahtarda saklayın.
3. $j = i - 1$ ile soldaki sıralı bölümün sonundan başlayın.
4. $j \geq 0$ ise değer karşılaştırması sayacını artırın.
5. Sol eleman anahtardan küçük veya eşitse aramayı bitirin.
6. Aksi halde sol elemanı sağa kaydırın, kaydırma sayacını artırın ve J 'yi azaltın; 4. adıma dönün.
7. Anahtarı $a[j + 1]$ konumuna yazın; sonraki anahtara geçin.
8. Sonunda diziyi ve sayaçları yazdırın.

Dış tur başında ilk i eleman sıralıdır. Büyük değerleri sağa kaydırmak bu değerlerin sırasını korur ve anahtar için bir yer açar. Anahtar önceden saklandığı için eski konumunun üzerine yazılması onu kaybettirmez. Eşit değerde durmak yeni anahtarı eski eşitlerin sonrasına yerleştirir. J her kaydırmada azalır; dış dizin sonlu uzunluk boyunca artar. Döngüler sonlanır ve sıralı bölüm her turda bir eleman büyür.

3, 1, 2, 1 için sayaçlar birikimlidir:

Anahtarın eski dizini	Anahtar	Tur sonundaki dizi	Karşılaştırma	Kaydırma
1	1	1, 3, 2, 1	1	1
2	2	1, 2, 3, 1	3	2
3	1	1, 1, 2, 3	6	4

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

int karsilastirma = 0, kaydirma = 0;
for (int i = 1; i < n; i++)
{
    int anahtar = a[i];
    int j = i - 1;
    while (j >= 0)
    {
        karsilastirma++;
        if (a[j] <= anahtar)
        {
            break;
        }
        a[j + 1] = a[j];
        kaydirma++;
        j--;
    }
    a[j + 1] = anahtar;
}
```

```

Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Karşılaştırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Kaydırma: " + kaydirma.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. J -1 olursa artık eleman erişimi yapılmaz; anahtar 0. konuma yazılır. Değer karşılaştırması çıkarma kullanmaz. Sıralama ikinci bir dizi gerektirmez; anahtar ayrı bir tamsayı değişkenidir.

Örnek çalıştırmalar

Girdi sırası N, ardından N elemandır; her değer ayrı satırda girilir. Bloklar istemler dışındaki tam çıktılarıdır.

N = 4; dizi 3, 1, 2, 1:

```

Dizi: 1 1 2 3
Karşılaştırma: 6
Kaydırma: 4

```

N = 4; dizi 1, 2, 3, 4:

```

Dizi: 1 2 3 4
Karşılaştırma: 3
Kaydırma: 0

```

N = 4; dizi 4, 3, 2, 1:

```

Dizi: 1 2 3 4
Karşılaştırma: 6
Kaydırma: 6

```

N = 1; tek değer -2147483648:

```

Dizi: -2147483648
Karşılaştırma: 0
Kaydırma: 0

```

Aşağıdaki tablo büyük girdilerin tam çıktısı değildir.

Girdi	Beklenen özellik
N = 50; farklı değerler 50..1 sırasında	1225 karşılaştırma ve 1225 kaydırma; sonuç 1..50.
N = 50; bütün değerler 2147483647	49 karşılaştırma, 0 kaydırma; 50 eşit değer korunur.

N için 0, 51, metin, ondalık, boş satır veya EOF:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

N = 2; ilk eleman 7, ikinci eleman iki için aşağıdaki tek satır yazılır. Herhangi bir elemanda 2147483648, -2147483649, 2.5, boş satır veya EOF aynı hatayı verir.

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- Tek elemanda sıralama turu ve iki sayaç da 0'dır.
- Sıralı veya eşit elemanlı $N > 1$ dizisinde anahtar başına bir değer karşılaştırması, toplam $N - 1$ karşılaştırma yapılır; kaydırma olmaz.
- Anahtar önceki bütün değerlerden küçükse $J - 1$ 'e iner. Geçersiz dizine erişmeden anahtar başa yazılır.
- Eşit değerler kaydırılmaz; eşitlerin özgün görelî sırası korunur.
- En fazla 1225 kaydırma ve 1225 karşılaştırma vardır; int sayaçlar yeterlidir.
- Negatif değerler ve iki int ucu karşılaştırma ve atamayla güvenle işlenir.
- Hatalı son eleman bile sıralama başlamadan reddedilir; kısmi sonuç yoktur.

Kazanımlar

- Sıralı başlangıç bölümünü her turda bir eleman büyütme.
- Üzerine yazılacak değeri önceden saklama.
- Geriye ilerlerken dizi erişiminden önce dizin sınırını denetleme.
- Aramayı bitiren değer karşılaştırmasını da sayma.
- Kaydırma ile anahtarın son yerleştirme atamasını ayırma.

Alıştırmalar

1. Her anahtarın ardından diziye yazdırın; izleme tablosundaki üç durumu doğrulayın.
2. Karşılaştırmayı değiştirip azalan sıralama yapın; eşit anahtarları yine kaydırmadan yerleştirin.
3. 051 ve 052 ile aynı üç girdiyi deneyin. Değer karşılaştırması sayılarını karşılaştırın; takas ve kaydırmanın farklı hareketler olduğunu açıklayın.

054 Sayarak sıralama ile diziyi sıralama

Düzey: İleri Önkoşullar: 048, 053

Kavramlar: Sayarak sıralama, Sınırlı değer aralığı, Frekans dizisi, Değer ve dizin dönüşümü, Sonuç konumu

Problem tanımı

-10 ile 10 arasında değerler içeren, uzunluğu 1..50 olan diziyi azalmayan sıraya getirin. Komşuları veya minimum adaylarını karşılaştırarak sıralamak yerine her değerın adedini bulun; değerleri küçükten büyüğe kendi adetleri kadar yeniden yazın. Örneğin 3, -10, 3, 0, -10 sonucu -10, -10, 0, 3, 3 olur.

Değer aralığı bu problemin temel kısıtıdır: 21 olası değer için 21 sayaç yeterlidir. Veri değeri x için frekans dizini $x + 10$, frekans dizini J için değer $j - 10$ olur. Sonuçta tekrarlar korunmalıdır. Önce verinin tamamını doğrulayın; sonra frekansları üretip giriş dizisinin hücrelerini sıralı değerlerle yeniden doldurun. Hazır sıralama kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1..50 arasında int.
Girdi	a	Her elemanı -10..10 arasında olan N değer.
Ara değer	frekans	21 sayaç; J konumu J - 10 değerinin adedi.
Çıktı	Sıralı dizi	Dizi: ardından birer boşlukla bütün değerler.

Eleman sayısı (1..50): ve $A[0]$: ile başlayan istemlerde her tamsayı ayrı satırda girilir. Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.; geçersiz elemanda Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır. yazılır. Hata halinde Dizi: satırı üretilmez. Metin, ondalık, boş satır, EOF ve tür taşması reddedilir.

Algoritma

1. Uzunluğu ve bütün elemanları doğrulayarak okuyun.
2. Sıfırla başlatılmış 21 elemanlı frekans dizisi oluşturun.
3. Her eleman için $\text{frekans}[a[i] + 10]$ sayacını artırın.
4. Sonuç yazma konumunu 0 yapın.
5. Frekans dizisini 0'dan 20'ye ilerletin.
6. İlgili sayaç pozitifken $a[\text{konum}] = j - 10$ yazın, konumu artırın ve sayacı azaltın.
7. Bütün sayaçlar tüketilince giriş dizisini yazdırın.

Frekansların başlangıç toplamı N'dir. Her yazım bu toplamı bir azaltıp sonuç konumunu bir artırır; yazılan ve kalan adetlerin toplamı N kalır. Bu yüzden tam N değer yazılır ve hiçbir yazım N. dizine ulaşmaz. Küçük frekans dizinleri küçük değerlere karşılık geldiği için sıralı başlangıç bölümü sürekli büyür. Her iç döngü sayacını azaltır; dış döngü 21 konumla sınırlıdır.

3, -10, 3, 0, -10 için yalnız sıfır olmayan sayılar:

Değer	Frekans dizini	Başlangıç adedi	Yeniden yazılan değerler
-10	0	2	-10, -10
0	10	1	0
3	13	2	3, 3

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < -10 || deger > 10)
    {
        Console.WriteLine("Hata: Eleman -10 ile 10 arasında " +
            "bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

int[] frekans = new int[21];
for (int i = 0; i < n; i++)
{
    frekans[a[i] + 10]++;
}

int konum = 0;
for (int j = 0; j < frekans.Length; j++)
{
    while (frekans[j] > 0)
    {
        a[konum] = j - 10;
        konum++;
        frekans[j]--;
    }
}
```

```

Console.Write("Dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Sayısal dizinin ilk sayaçları otomatik 0'dır. Sayma tamamlandıktan sonra A'nın eski elemanlarına ihtiyaç kalmaz; yeniden yazım onları güvenle değiştirebilir. Bu sürüm frekans sayaçlarını tüketir; işlem sonunda bütün sayaçlar 0 olur. Ek bellek olarak 21 elemanlı frekans dizisi kullanılır.

Örnek çalıştırmalar

Önce N, ardından her eleman ayrı satırda girilir. Bloklar istemler dışındaki tam sonuçtur.

N = 5; değerler 3, -10, 3, 0, -10:

Dizi: -10 -10 0 3 3

N = 1; değer 10:

Dizi: 10

N = 4; değerler 0, 0, 0, 0:

Dizi: 0 0 0 0

N = 3; değerler 10, 0, -10:

Dizi: -10 0 10

Girdi	Beklenen özellik
N = 50; 25 kez 10, ardından 25 kez -10	Sonuç önce 25 kez -10, sonra 25 kez 10 içerir; toplam 50 değer vardır.
N = 21; değerler 10'dan -10'a doğru	Sonuç -10'dan 10'a kadar her değeri bir kez içerir.

Uzunluk 0, 51, metin, ondalık, boş satır veya EOF ise tek çıktı:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda -11, 11, 2147483647, 2147483648, metin, 1.5, boş satır veya EOF için:

Hata: Eleman -10 ile 10 arasında bir tamsayı olmalıdır.

Sınır durumları

- -10 ve 10 frekans dizisinin sırasıyla 0 ve 20. dizinlerini kullanır.
- 0 normal bir veri değeridir; bitiş işareti değildir.
- Bütün değerler eşitse yalnızca bir frekans konumu kullanılır; N kopya yeniden yazılır.
- Her sayaç en fazla 50'dir; int türü yeterlidir.
- Frekansların toplamı N olduğu için sonuç dizisine tam N yazım yapılır; son yazım N - 1 konumundadır.
- Bu 21 konumlu yöntem tam int aralığına doğrudan uygulanamaz. -10..10 sınırı kaldırılırsa dönüşüm ve bellek planı da değiştirilmelidir.
- Hatalı veri frekans dizini olarak kullanılmaz; bütün girişler önce doğrulanır.

Kazanımlar

- Bilinen dar değer aralığını bir sayaç dizisiyle temsil etme.
- Değerden dizine ve dizinden değere iki yönlü dönüşüm yapma.
- Frekans bilgisinden tekrarları koruyan sıralı dizi üretme.
- Sayaç azalmasıyla sonuç konumunun artışı arasındaki ilişkiyi kurma.
- Bir sıralama yönteminin hangi girdi kısıtına bağlı olduğunu açıklama.

Alıştırmalar

1. Frekansları sondan başa tarayarak azalan sıralama üretin.
2. Frekansları tüketmeden bir tekrar sayacıyla yazın; sıralama sonrasında özgün adetleri de gösterin.
3. Değer aralığını -20..20 yapın; 41 konum, kaydırma ve ters dönüşümü birlikte güncelleyin.

055 İki sıralı diziyi birleştirme

Düzy: İleri Önkoşullar: 046, 049, 054

Kavramlar: Sıralı birleştirme, İki okuma konumu, Sonuç konumu, Kalan bölüm, Sıralılık önkoşulu

Problem tanımı

Her biri 1..25 eleman içeren iki azalmayan tamsayı dizisini tek bir azalmayan dizide birleştirin. Elemanlar tam int aralığındadır. Bütün tekrarlar korunur; bu işlem farklı ortak değerleri bulma değildir. Örneğin A 1, 3, 3, B -2, 3, 8 için sonuç -2, 1, 3, 3, 3, 8 olur.

Her dizinin sıradaki kullanılmamış elemanını bir okuma diziniyle izleyin. Küçük olanı sonuç dizisine yazıp yalnız o dizinin konumunu ilerletin; eşitlikte A'dan alın. Bir dizi bitince diğerinin kalan bölümünü tamamlayın. Girişleri yeniden sıralamayın; sıralılık önkoşulunu doğrulayın. Hazır birleştirme veya kopyalama aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, m	A ve B uzunlukları; her biri 1..25.
Girdi	a, b	Tam int değerleri; her dizi kendi içinde azalmayan sırada.
Ara değer	sonuc	N + M uzunluğunda sonuç dizisi; en fazla 50 değer.
Çıktı	Birleşik dizi	Birleşik dizi: ardından birer boşlukla N + M değer.

Girdi sırası A uzunluğu (1..25): , B uzunluğu (1..25): , bütün A[0]: ile başlayan istemler, ardından bütün B[0]: ile başlayan istemlerdir. Her değer ayrı satırda girilir. Bütün elemanlar okunduktan sonra önce A'nın, sonra B'nin sıralılığı doğrulanır. Geçersiz uzunluk, eleman veya sıralılık için yalnız ilgili Hata: satırı yazılır; birleşik dizi üretilmez. Boş satır, EOF, metin, ondalık veya int dışı giriş reddedilir.

Algoritma

1. İki uzunluğu doğrulayın; dizileri oluşturup bütün elemanları doğrulayarak okuyun.
2. A ve B içindeki komşu çiftleri ayrı ayrı denetleyin; azalma varsa ilgili sıra hatasıyla bitirin.
3. N + M uzunluğunda sonuç dizisi oluşturun; iki okuma dizisini ve yazma konumunu 0 yapın.
4. İki dizide de eleman varken sıradaki elemanları karşılaştırın.
5. A'nın küçük veya eşitse A'dan, aksi halde B'den alın; seçilen dizini ve yazma konumunu artırın.
6. A'da kalan bütün elemanları sonuç dizisine yazın.
7. B'de kalan bütün elemanları sonuç dizisine yazın.
8. Sonuç dizisini yazdırın.

Her adımda yazılmış bölüm sıralıdır ve iki diziden tüketilmiş bütün elemanları içerir. Her kaynak kendi içinde sıralı olduğundan kalanların en küçüğü iki sıradaki değerden biridir. Küçüğünü almak bu özelliği korur. Yazma konumu iki okuma konumunun toplamıdır; her yazım bir kaynağı tüketir. Bir kaynak bittiğinde ötekinin sıralı kalanı sonuçtan küçük değildir. Toplam N + M yazım yapılır; hiçbir kaynak elemanı atlanmaz veya iki kez tüketilmez.

A 1, 3, 3, B -2, 3, 8 için:

Yazılan değer	Kaynak	Sonraki A dizini	Sonraki B dizini
-2	B	0	1
1	A	1	1
3	A; eşitlik	2	1
3	A; eşitlik	3	1
3, 8	B'nin kalanı	3	3

C# çözümü

```
using System;
using System.Globalization;

Console.Write("A uzunluğu (1..25): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 25)
{
    Console.WriteLine("Hata: A uzunluğu 1 ile 25 arasında bir tamsayı olmalıdır.");
    return;
}
Console.Write("B uzunluğu (1..25): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int m) || m < 1 || m > 25)
{
    Console.WriteLine("Hata: B uzunluğu 1 ile 25 arasında bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
int[] b = new int[m];
for (int i = 0; i < n; i++)
{
    Console.Write("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: A elemanı int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 0; i < m; i++)
{
    Console.Write("B[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: B elemanı int aralığında bir tamsayı olmalıdır.");
        return;
    }
    b[i] = deger;
}
```

```

for (int i = 1; i < n; i++)
{
    if (a[i - 1] > a[i])
    {
        Console.WriteLine("Hata: A dizisi azalmayan sırada olmalıdır.");
        return;
    }
}
for (int i = 1; i < m; i++)
{
    if (b[i - 1] > b[i])
    {
        Console.WriteLine("Hata: B dizisi azalmayan sırada olmalıdır.");
        return;
    }
}

int[] sonuc = new int[n + m];
int sol = 0, sag = 0, konum = 0;
while (sol < n && sag < m)
{
    if (a[sol] <= b[sag])
    {
        sonuc[konum++] = a[sol++];
    }
    else
    {
        sonuc[konum++] = b[sag++];
    }
}

while (sol < n)
{
    sonuc[konum++] = a[sol++];
}
while (sag < m)
{
    sonuc[konum++] = b[sag++];
}

Console.Write("Birleşik dizi:");
for (int i = 0; i < sonuc.Length; i++)
{
    Console.Write(" " + sonuc[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. konum++, mevcut konumu kullanıp sonra artırır; kaynak dizinlerinde de aynı sıra geçerlidir. İki kaynak dizi değiştirilmez. Döngü koşulları eleman erişiminden önce sınırları doğrular; sonucun kapasitesi tam $N + M$ 'dir. Elemanlar üzerinde aritmetik yapılmadığı için int uçları güvenlidir.

Örnek çalıştırmalar

N, M, bütün A elemanları ve bütün B elemanları ayrı satırlarda girilir. Bloklar istemler dışındaki tam çıktılarıdır.

N = 3, M = 3; A 1, 3, 3; B -2, 3, 8:

Birleşik dizi: -2 1 3 3 3 8

N = 1, M = 1; A -2147483648; B 2147483647:

Birleşik dizi: -2147483648 2147483647

N = 2, M = 3; A 7, 7; B 7, 7, 7:

Birleşik dizi: 7 7 7 7

N = 2, M = 1; A -5, -1; B 4:

Birleşik dizi: -5 -1 4

Girdi	Beklenen özellik
N = M = 25; A 0..24, B 25..49	Sonuç 50 değerle 0..49'dur; A bitince B'nin tamamı kopyalanır.

Aşağıdaki durumlar tek hata satırı üretir; sonuç yazılmaz.

Girdi	Tam sonuç
A uzunluğu 0, 26, metin, ondalık, boş veya EOF	Hata: A uzunluğu 1 ile 25 arasında bir tamsayı olmalıdır.
Geçerli A uzunluğundan sonra B uzunluğu 0, 26, metin, boş veya EOF	Hata: B uzunluğu 1 ile 25 arasında bir tamsayı olmalıdır.
N = M = 1; A elemanı 2147483648, metin, ondalık, boş veya EOF	Hata: A elemanı int aralığında bir tamsayı olmalıdır.
N = M = 1; A 0; B elemanı -2147483649, metin, ondalık, boş veya EOF	Hata: B elemanı int aralığında bir tamsayı olmalıdır.
N = 2, M = 1; A 3, 2; B 0	Hata: A dizisi azalmayan sırada olmalıdır.
N = 1, M = 2; A 0; B 3, 2	Hata: B dizisi azalmayan sırada olmalıdır.

Sınır durumları

- İki uzunluk da en az 1'dir; boş kaynaklar bu çalışmanın giriş sınırı dışındadır.
- Eşit değerlerde A önce tüketilir; B'deki aynı değerler sonradan ayrıca yazılır.
- Kaynaklardan biri erken biterse diğerinin kalanı atlanmaz. İki tamamlama döngüsünden biri hiç çalışmayabilir.
- 0, negatif ve tam int uçları normal veridir; sıralılığı çıkarma yerine doğrudan karşılaştırma denetler.
- Sonuçta N + M eleman vardır; farklı değer sayısı hesaplanmaz.
- Başlangıçtan sonra gelen herhangi bir azalma önkoşulu bozar; sıralama yaparak hata gizlenmez.
- Geçersiz veya eksik girişte hiçbir birleşik sonuç üretilmez.

Kazanımlar

- Sıralı veri önkoşulunu birleşim işleminden önce doğrulama.
- İki okuma dizinini ve bir yazma dizinini ayrı amaçlarla yönetme.
- Her adımda yalnız tüketilen kaynak dizinini artırma.
- Kaynaklar bittiğinde kalan bölümün tamamını yazma.
- Birleştirmede tekrarları koruma ve çıktı uzunluğunu kaynak uzunluklarından hesaplama.

Alıřtırmalar

1. Birleřtirme ařamasındaki deęer karřılařtırmalarını sayın; sıra doęrulama denetimlerini bu sayaca katmayın.
2. Eřitlikte B'yi seęin; tamsayı ıktısının aynı kaldıęını ve kaynak seęim sırasının deęiřtięini aıklayın.
3. Birleřik dizinin elemanlarını long ile toplayıp iki kaynak toplamının toplamıyla karřılařtırın.

056 Sıralı diziye eleman ekleme

Düzey: İleri Önkoşullar: 044, 046, 055

Kavramlar: Sıralı ekleme, Ek kapasite, Geriye kopyalama, Eşit değerler, Geçerli uzunluk

Problem tanımı

Azalmayan sıradaki bir tamsayı dizisine yeni bir tamsayı ekleyin ve sıralılığı koruyun. Başlangıçta 1 ile 49 arasında eleman bulunur. Yeni değer mevcut değerlerle eşitse bütün eşit değerlerin sonrasına yerleşmelidir. Örneğin 1, 2, 2, 4 dizisine 2 eklendiğinde sonuç 1, 2, 2, 2, 4, ekleme dizini 3 olur. Dizinler sıfırdan başlar.

Başlangıçta kapasitesi $n + 1$ olan tek bir dizi oluşturun; ilk n konumu giriş değerleriyle doldurun. Son konum ekleme için ayrılmış boş kapasitedir ve başlangıçta geçerli veri değildir. Dizinin sıralılığını doğrulayın. Sırasız veride hata verip yeni değeri istemeden bitirin. Geçerli veride ilk büyük değerini yerini bulun, sağdaki değerleri sondan başlayarak bir konum kaydırın ve yeni değeri açılan konuma yazın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 49 arasında int; başlangıçtaki geçerli eleman sayısı.
Girdi	a	Kapasitesi $n + 1$ olan dizi; ilk n eleman tam int aralığında ve azalmayan sırada.
Girdi	yeni	-2147483648..2147483647 aralığında eklenecek int.
Çıktı	Yeni dizi	Dizi: ardından birer boşlukla ayrılmış $n + 1$ değer.
Çıktı	Ekleme konumu	Ekleme dizini: I; yeni değerini sıfır tabanlı dizini.

Önce Eleman sayısı (1..49): istemini, ardından $A[0]$: ile başlayan eleman istemlerini ayrı satırlarda yanıtlayın. Bütün elemanlar doğru biçimde okunduktan sonra sıralılık denetlenir. Geçerli dizide son istem Yeni değer: olur. Hatalı uzunlukta, elemanda, sıralılıkta veya yeni değerde yalnızca ilgili hata satırı yazılır; sonuç dizisi üretilmez. Boş satır, girişin sonu (EOF), metin, ondalıklı giriş ve int taşması reddedilir.

Algoritma

1. Başlangıç uzunluğunu okuyup 1..49 aralığında doğrulayın.
2. $n + 1$ kapasiteli diziyi oluşturun; ilk n elemanı okuyup doğrulayın.
3. Komşuları karşılaştırın; bir azalma varsa sıra hatası yazıp bitirin.
4. Yeni değeri okuyup tam int aralığında doğrulayın.
5. Ekleme konumunu 0 yapın; mevcut değer yeni değerden küçük veya eşitken konumu ilerletin.
6. $i = n$ konumundan başlayıp $i > \text{konum}$ iken $a[i] = a[i - 1]$ ile geriye doğru kopyalayın.
7. Yeni değeri $a[\text{konum}]$ konumuna yazın.
8. Bütün $n + 1$ değeri ve ekleme dizinini yazdırın.

Konum aramasında geçilen bütün değerler yeni değerden küçük veya eşittir. Durulduğunda ya ilk büyük değere ulaşılmıştır ya da bütün eski değerler geçilmiştir. Bu nedenle eşitlerin sonrasına ekleme yapılır. Kaydırma sağ uçtan başlar; bir konuma yazmadan önce o konumdaki eski değer

daha sağı taşınmıştır. İleri yönde kopyalamak henüz taşınmamış değerleri ezebilir. Arama en fazla n adım ilerler; kaydırma sayacı her adımda azalır ve ekleme konumunda durur.

1, 2, 2, 4 ve yeni değer 2 için arama konumu 3'tür. Kullanılmayan son konumun başlangıçtaki 0 değeri aramaya katılmaz.

Aşama	İşlem	Fiziksel dizi içeriği
Giriş tamamlandı	Son konum henüz geçerli değildir.	1, 2, 2, 4, 0
Geriye kopyalama	a[4] = a[3]	1, 2, 2, 4, 4
Ekleme	a[3] = 2	1, 2, 2, 2, 4

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..49): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 49)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 49 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n + 1];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 1; i < n; i++)
{
    if (a[i - 1] > a[i])
    {
        Console.WriteLine("Hata: Dizi azalmayan sırada olmalıdır.");
        return;
    }
}

Console.WriteLine("Yeni değer: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int yeni))
{
    Console.WriteLine("Hata: Yeni değer int aralığında bir tamsayı olmalıdır.");
    return;
}
```



```

int konum = 0;
while (konum < n && a[konum] <= yeni)
{
    konum++;
}

for (int i = n; i > konum; i--)
{
    a[i] = a[i - 1];
}
a[konum] = yeni;

Console.Write("Dizi:");
for (int i = 0; i <= n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Ekleme dizini: " +
    konum.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. Fiziksel kapasite baştan ayrılmıştır; ekleme sırasında ikinci bir dizi oluşturulmaz. Sonuçta geçerli uzunluk $n + 1$ olur. Değerler yalnızca karşılaştırılır ve kopyalanır; iki `int` ucu arasında fark hesaplanmaz.

Örnek çalıştırmalar

Girdi sırası uzunluk, eski elemanlar ve yeni değerdir. Her girdi ayrı satırda verilir. Metin blokları istemlerden sonraki bütün sonuç satırlarını gösterir.

$n = 4$, dizi 1, 2, 2, 4, yeni değer 2 için tam çıktı:

```

Dizi: 1 2 2 2 4
Ekleme dizini: 3

```

$n = 1$, dizi 7, yeni değer -3 için tam çıktı:

```

Dizi: -3 7
Ekleme dizini: 0

```

$n = 2$, dizi -2147483648, 2147483647, yeni değer 2147483647 için tam çıktı:

```

Dizi: -2147483648 2147483647 2147483647
Ekleme dizini: 2

```

$n = 3$, dizi -5, -1, 0, yeni değer 6 için tam çıktı:

```

Dizi: -5 -1 0 6
Ekleme dizini: 3

```

Aşağıdaki tablo uzun çıktının tam eleman listesi değildir; denetlenecek özellikleri özetler.

Girdi	Beklenen özellik
$n = 49$, elemanlar sırasıyla 0..48, yeni değer 25	Sonuçta 50 değer vardır; 25 iki kez bulunur, sıra korunur ve ikinci satır Ekleme dizini: 26 olur.

Aşağıdaki örneklerde yalnızca ilgili tek hata satırı üretilir.

Girdi	Tam sonuç
Uzunluk 0, 50, iki, 1.5 veya 2147483648	Hata: Eleman sayısı 1 ile 49 arasında bir tamsayı olmalıdır.

Girdi	Tam sonuç
Uzunluk için boş satır veya girişin sonu	Hata: Eleman sayısı 1 ile 49 arasında bir tamsayı olmalıdır.
$n = 2$, ilk eleman 7, ikinci eleman sayı, 2.5 veya girişin sonu	Hata: Eleman int aralığında bir tamsayı olmalıdır.
$n = 1$, eleman 2147483648, -2147483649 veya boş satır	Hata: Eleman int aralığında bir tamsayı olmalıdır.
$n = 2$, dizi 3, 2	Hata: Dizi azalmayan sırada olmalıdır.; yeni değer istenmez.
$n = 1$, eleman 7, yeni değer yeni, 1.5 veya 2147483648	Hata: Yeni değer int aralığında bir tamsayı olmalıdır.
$n = 1$, eleman 7, yeni değer için boş satır veya girişin sonu	Hata: Yeni değer int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Yeni değer bütün eski değerlerden küçükse ekleme dizini 0 olur; bütün eski elemanlar kaydırılır.
- Yeni değer bütün eski değerlerden büyük veya eşitse dizin n olur; kaydırma döngüsü çalışmaz.
- Bütün eski değerler yeni değere eşitse yeni değer en sona eklenir.
- Tek eski eleman için en fazla bir kaydırma yapılır; komşu sıralılık döngüsü çalışmaz.
- Başlangıç uzunluğu 49 olduğunda fiziksel kapasite 50 olur. Uzunluk 50 bu problemin giriş sınırının dışındadır.
- 0 ve tam int uçları normal değerlerdir; kullanılmayan kapasitenin 0 içeriği eski eleman sayılmaz.
- Kaydırma $i >$ konum koşuluyla yapıldığından $i - 1$ negatif olmaz. Konum aramasında konum $< n$ denetimi dizi erişiminden önce gelir.
- Bütün gerekli girişler doğru olmadan sonuç yazılmaz. Sıra hatası, yeni değer okunmadan belirlenir.

Kazanımlar

- Fiziksel kapasite ile geçerli eleman sayısını ayırma.
- Eşit değerlerin sonrasındaki ekleme dizinini örnek üzerinde bulma.
- Geriye kopyalamanın henüz taşınmamış değerleri koruduğunu gösterme.
- Başa, ortaya ve sona eklemeyi aynı döngü sınırlarıyla gerçekleştirme.
- Sıralılık önkoşulunu kullanıcıdan yeni değer istemeden doğrulama.

Alıştırmalar

1. Eşitlerin öncesine ekleyen sürümü yazın; 1, 2, 2, 4 ve 2 için yeni dizinin aynı, ekleme dizininin farklı olduğunu gösterin.
2. Gerçek kaydırma adedini sayın; aynı diziye baştan, ortadan ve sondan eklemede $n -$ konum ile eşit olduğunu doğrulayın.
3. Başlangıçta iki boş konum ayırıp iki yeni değeri sırayla ekleyin; ikinci eklemede geçerli uzunluğun neden artırılması gerektiğini açıklayın.

057 Dizinin medyanını bulma

Düzey: İleri Önkoşullar: 008, 051, 053

Kavramlar: Seçmeli sıralama, Medyan, Tek ve çift uzunluk, Tür genişletme, Sabit çıktı biçimi

Problem tanımı

1 ile 50 tamsayı içeren bir diziyi küçükten büyüğe sıralayın ve medyanını bulun. Medyan, sıralı verinin orta değeridir: uzunluk tekse ortadaki eleman, çiftse ortadaki iki elemanın aritmetik ortalamasıdır. Örneğin 7, 1, 4, 2 sıralanınca 1, 2, 4, 7 olur; medyan $(2 + 4) / 2 = 3.0$ olarak yazılır. Medyan bütün elemanların aritmetik ortalaması değildir.

Sıralama için seçmeli sıralamayı döngülerle gerçekleştirin. Her turda kalan bölümün en küçük değerini bulup başına taşıyın. Elemanlar `int` türünün bütün aralığında olabilir. Çift uzunlukta orta değerleri toplarken toplamadan önce `long` türüne genişletin; sonucu `decimal` olarak hesaplayıp her zaman bir ondalık basamakla yazdırın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1 ile 50 arasında <code>int</code> ; dizi uzunluğu.
Girdi	a	Her biri -2147483648..2147483647 aralığında n tamsayı.
Ara değer	toplam	Çift uzunlukta iki orta değer <code>long</code> toplamı.
Çıktı	Sıralı değerler	Sıralı dizi: ardından birer boşlukla ayrılmış bütün değerler.
Çıktı	Medyan	Medyan: X; <code>decimal</code> sonuç, nokta ayırıcıyla bir ondalık basamak.

Eleman sayısı (1..50): istemini, ardından `A[0]:` ile başlayan istemleri ayrı satırlarda yanıtlayın. Hatalı uzunluk veya elemanda yalnızca ilgili hata yazılır. Boş satır, girişin sonu (EOF), metin, ondalıklı giriş ve `int` sınırını aşan değer reddedilir. Bütün elemanlar doğrulanmadan sıralama veya sonuç yazımı başlamaz.

Algoritma

- Uzunluğu okuyup 1..50 aralığında doğrulayın; diziyi oluşturun.
- Bütün elemanları ayrı satırlarda okuyup tam `int` aralığında doğrulayın.
- Her `i` için kalan `i..n - 1` bölümünün en küçük değerinin dizinini bulun.
- En küçük değer başka konumdaysa `a[i]` ile yerini değiştirin; ilk `n - 1` tur tamamlanınca dizi sıralıdır.
- Uzunluk tekse medyanı `a[n / 2]` olarak alın.
- Uzunluk çiftse `a[n / 2 - 1]` ile `a[n / 2]` değerlerini, ilk değeri toplamadan önce `long` yaparak toplayın; 2.0m ile bölün.
- Sıralı diziyi ve medyanı bir ondalık basamakla yazdırın.

Her sıralama turunun başında `i` konumunun solundaki bölüm doğru son sırasındadır. Kalan bölümün en küçük değeri `i` konumuna taşındığında bu bölüm bir eleman büyür. İç arama sonlu bölümde ilerler; dış döngü en fazla `n - 1` tur sürer. Sıralı dizi hazır olduğunda `n / 2` tamsayı bölümü tek uzunlukta ortayı, çift uzunlukta sağ orta dizini verir.

7, 1, 4, 2 için sıralama adımları:

Turun i dizini	En küçük değerin dizini	Tur sonrası dizi
0	1	1, 7, 4, 2
1	3	1, 2, 4, 7
2	2; takas gerekmez.	1, 2, 4, 7

İki orta dizin 1 ve 2'dir; değerleri 2 ve 4 olduğundan medyan 3.0 olur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında " +
        "bir tamsayı olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman int aralığında bir tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 0; i < n - 1; i++)
{
    int enKucuk = i;
    for (int j = i + 1; j < n; j++)
    {
        if (a[j] < a[enKucuk])
        {
            enKucuk = j;
        }
    }
    if (enKucuk != i)
    {
        int gecici = a[i];
        a[i] = a[enKucuk];
        a[enKucuk] = gecici;
    }
}
```

```

decimal medyan;
if (n % 2 == 1)
{
    medyan = a[n / 2];
}
else
{
    long toplam = (long)a[n / 2 - 1] + a[n / 2];
    medyan = toplam / 2.0m;
}

Console.Write("Sıralı dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Medyan: " +
    medyan.ToString("0.0", CultureInfo.InvariantCulture));

```

Kod bağımsız bir konsol projesinin Program.cs dosyasında çalışır. (long) dönüşümü toplamın ilk terimine uygulanır; böylece toplama long ile yapılır. Toplamı önce int olarak hesaplayıp daha sonra dönüştürmek taşmayı önlemez. 2.0m bir decimal sabitidir; tamsayı bölmesinin yarım birimi kaybetmesini engeller. "0.0" biçimi tamsayı medyanlarda da .0 yazdırır.

Örnek çalıştırmalar

Önce uzunluğu, sonra bütün elemanları ayrı satırlarda girin. Metin blokları istemlerden sonraki bütün sonuç satırlarını gösterir.

n = 4, dizi 7, 1, 4, 2 için tam çıktı:

```

Sıralı dizi: 1 2 4 7
Medyan: 3.0

```

n = 3, dizi 5, -1, 8 için tam çıktı:

```

Sıralı dizi: -1 5 8
Medyan: 5.0

```

n = 1, dizi -8 için tam çıktı:

```

Sıralı dizi: -8
Medyan: -8.0

```

n = 2, dizi -2147483648, 2147483647 için tam çıktı:

```

Sıralı dizi: -2147483648 2147483647
Medyan: -0.5

```

n = 2, dizi 2147483647, 2147483647 için tam çıktı:

```

Sıralı dizi: 2147483647 2147483647
Medyan: 2147483647.0

```

Aşağıdaki tablo uzun çıktıların tam eleman listesi değildir; denetlenecek özellikleri özetler.

Girdi	Beklenen özellik
n = 50, bütün değerler 2147483647	Sıralı satırda 50 kez aynı değer vardır; Medyan: 2147483647.0 yazılır. Ara toplam 4294967294'tür.
n = 50, bütün değerler -2147483648	Medyan satırı Medyan: -2147483648.0 olur. Ara toplam -4294967296'dır ve long içinde kalır.

Aşağıdaki örneklerde yalnızca ilgili tek hata satırı yazılır.

Girdi	Tam sonuç
Uzunluk 0, 51, iki, 1.5 veya 2147483648	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
Uzunluk için boş satır veya girişin sonu	Hata: Eleman sayısı 1 ile 50 arasında bir tamsayı olmalıdır.
$n = 2$, ilk eleman 7, ikinci eleman sayı veya 2.5	Hata: Eleman int aralığında bir tamsayı olmalıdır.
$n = 1$, eleman 2147483648 veya -2147483649	Hata: Eleman int aralığında bir tamsayı olmalıdır.
$n = 1$, eleman için boş satır veya girişin sonu	Hata: Eleman int aralığında bir tamsayı olmalıdır.

Sınır durumları

- Tek eleman hem sıralı dizinin hem medyanın tek değeridir; sıralama turu gerekmez.
- Tek uzunlukta orta dizin $n / 2$ olur. Çift uzunluk en az 2 olduğundan $n / 2 - 1$ negatif değildir.
- Tekrarlanan değerler medyan hesabında ayrı elemanlar olarak kalır; farklı değer sırası hesaplanmaz.
- İki orta değer toplamı -4294967296..4294967294 aralığındadır. long bu aralığı kapsar.
- İki int ucunun toplamı -1'dir; medyan -0.5 olur. Tamsayı bölmesi bu sonucu yanlışlıkla 0 yapardı.
- Eşit iki en küçük veya en büyük int değeri de güvenle işlenir; medyan giriş değerlerinin aralığında kalır.
- En fazla 50 eleman sıralanır; sıralama hazır bir araç çağrısıyla yapılmaz.
- Eksik, boş, biçimi yanlış veya tür sınırını aşan girişte sıralı dizi ve medyan satırları üretilmez.

Kazanımlar

- Seçmeli sıralamanın her turunda kalan bölümün en küçük değerini bulma.
- Tek ve çift uzunlukta medyan dizinlerini sıfır tabanlı olarak hesaplama.
- Medyan ile bütün değerlerin aritmetik ortalamasını örnek üzerinde ayırma.
- Tür dönüşümünün neden toplamadan önce yapılması gerektiğini uç değerlerle gösterme.
- Negatif yarım sonucu ve tamsayı sonucu aynı bir basamaklı biçimde yazdırma.

Alıştırmalar

1. Medyanın yanında bütün elemanların aritmetik ortalamasını da hesaplayın; 1, 2, 100 için iki sonucun neden farklı olduğunu açıklayın.
2. Çift uzunlukta iki orta değeri ayrı satırlarda yazdırın; -2147483648, 2147483647 için ara toplamı ve medyanı doğrulayın.
3. Aynı diziye kabarcık sıralamayla sıralayıp medyanı hesaplayın; tekrar içeren, tek uzunluklu ve çift uzunluklu üç örnekte sonuçları karşılaştırın.

058 K numaralı farklı küçük değeri bulma

Düzey: İleri Önkoşullar: 050, 051, 057

Kavramlar: Sıralama, Farklı değer sırası, Eşit komşular, Bir tabanlı sıra, Yetersiz farklı değer

Problem tanımı

Tam int aralığında 1..50 eleman içeren dizinin K numaralı farklı küçük değerini bulun. K sıra numarası 1'den başlar; tekrarlar yeni sıra oluşturmaz. Örneğin 7, 2, 7, -1, 2 dizisinin farklı değerleri küçükten büyüğe -1, 2, 7 olur. K = 2 için sonuç 2, K = 4 için yeterli farklı değer olmadığı sonucu verilir.

Girdi sırasız olabilir; seçmeli sıralamayı döngülerle uygulayın. Sıralı dizide eşit değerler komşu olacağı için yeni bir farklı değere yalnız ilk konumda veya önceki değerden farklı bir değerde rastlanır. K, 1..N aralığında doğrulanır. Geçerli K farklı değer sayısından büyükse bu bir giriş hatası değildir; normal bir bulunamadı sonucu üretin. Hazır sıralama veya farklı değer çıkarma aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1..50 arasında eleman sayısı.
Girdi	a	N adet tam int; tekrarlar olabilir.
Girdi	k	1..N arasında sıra numarası; sıfır tabanlı dizin değildir.
Çıktı	Sonuç	Değer: X ya da Sonuç: Yeterli farklı değer yok.

Önce Eleman sayısı (1..50): , sonra A[0]: ile başlayan N eleman istemi, en son Sıra (1..n): yanıtlanır. Her giriş ayrı satırdadır; son istemdeki N daha önce girilen uzunluktur. Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., elemanda Hata: Eleman geçerli bir int tamsayı olmalıdır., sırada Hata: K 1 ile n arasında bir tamsayı olmalıdır. yazılır. Hata halinde sonuç verilmez.

Algoritma

1. Uzunluğu, bütün elemanları ve K değerini okuyup doğrulayın.
 2. Seçmeli sıralamayla diziyi azalmayan sıraya getirin.
 3. Farklı değer sırasını 0, bulunma durumunu yanlış yapın.
 4. Sıralı diziyi baştan sona ilerletin.
 5. İlk konumda veya değer önceki elemandan farklıysa sıra sayacını artırın.
 6. Sıra K'ye ulaştıysa değeri saklayın, bulunma durumunu doğru yapıp taramayı bitirin.
 7. Bulunmuşsa değeri, aksi halde yetersiz farklı değer sonucunu yazdırın.
- 7, 2, 7, -1, 2 sıralanınca -1, 2, 2, 7, 7 olur. K = 2 için tarama:

Dizin	Değer	Yeni farklı değer mi?	Sıra	İşlem
0	-1	Evet; ilk konum	1	Devam
1	2	Evet	2	Değeri kaydet; bitir

Tarama başında sıra sayacı, incelenmiş bölümdeki farklı değer sayısıdır. Sıralılık, aynı değer bütünü kopyalarını yan yana getirir; komşu eşitleri atlamak her farklı değeri bir kez sayar. Sıra K'ye ulaştığında bu değer istenen küçük değerdir. Ulaşılmazsa dizide K kadar farklı değer yoktur. Sıralama ve tarama sonlu dizinlerle sınırlıdır.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

Console.WriteLine("Sıra (1..n): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int k) || k < 1 || k > n)
{
    Console.WriteLine("Hata: K 1 ile n arasında bir tamsayı olmalıdır.");
    return;
}

for (int i = 0; i < n - 1; i++)
{
    int enKucuk = i;
    for (int j = i + 1; j < n; j++)
    {
        if (a[j] < a[enKucuk])
        {
            enKucuk = j;
        }
    }
    if (enKucuk != i)
    {
        int gecici = a[i];
        a[i] = a[enKucuk];
        a[enKucuk] = gecici;
    }
}
```



```

int sıra = 0, sonuc = 0;
bool bulundu = false;
for (int i = 0; i < n; i++)
{
    if (i == 0 || a[i] != a[i - 1])
    {
        sıra++;
        if (sıra == k)
        {
            sonuc = a[i];
            bulundu = true;
            break;
        }
    }
}

if (bulundu)
{
    Console.WriteLine("Değer: " + sonuc.ToString(CultureInfo.InvariantCulture));
}
else
{
    Console.WriteLine("Sonuç: Yeterli farklı değer yok");
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. `i == 0 || ...` kısa devre değerlendirmesi ilk konumda `a[-1]` erişimini önler. Sonuç değişkeninin başlangıçtaki 0 değeri bulunamadı işareti değildir; ayrı bulundu durumu kullanılır. Böylece 0 ve iki int ucu da normal sonuç olabilir.

Örnek çalıştırmalar

N, N eleman ve K ayrı satırlarda girilir. Bloklar istemler dışındaki bütün çıktı satırlarıdır.

N = 5; dizi 7, 2, 7, -1, 2; K = 2:

Değer: 2

Aynı dizi için K = 4:

Sonuç: Yeterli farklı değer yok

N = 3; dizi 0, -2147483648, 2147483647; K = 2:

Değer: 0

N = 1; dizi -2147483648; K = 1:

Değer: -2147483648

Girdi	Beklenen özellik
N = 50; değerler 49..0; K = 50	50 farklı değer vardır; Değer: 49 yazılır.
N = 50; bütün değerler 7; K = 2	K geçerlidir; normal yetersiz farklı değer sonucu verilir.

Uzunluk 0, 51, biçimi yanlış, boş veya EOF ise:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda metin, ondalık, 2147483648, -2147483649, boş veya EOF verilirse K istenmeden:

Hata: Eleman geçerli bir int tamsayı olmalıdır.

$N = 2$ ve geçerli iki elemandan sonra K için 0, 3, 1.5, metin, boş veya EOF:

Hata: K 1 ile n arasında bir tamsayı olmalıdır.

Sınır durumları

- $K = 1$ her zaman dizinin en küçük değerini verir; N en az 1'dir.
- Tekrarlanan değerler korunarak sıralanır, ancak sıra sayacını yalnız bir kez artırır.
- $K = N$ yalnız bütün elemanlar farklıysa bulunabilir; aksi halde normal yetersizlik sonucu verilir.
- Tek elemanda $K = 1$ 'dir ve sıralama turu gerekmez.
- 0, negatif değerler ve iki `int` ucu bulunamadı işareti olarak kullanılmaz.
- K için hatalı biçim ve alan sınırı hatası, geçerli K için yetersiz farklı değer durumundan ayırdır.
- Bütün girdiler doğrulanmadan sıralama ve sonuç üretimi başlamaz.

Kazanımlar

- Bir tabanlı sıra numarası ile sıfır tabanlı dizini ayırt etme.
- Sıralı dizide eşit komşuları atlayarak farklı değerleri sayma.
- Değer alanından bir işaret seçmek yerine ayrı bulunma durumu kullanma.
- Geçersiz giriş ile geçerli fakat sonucu olmayan sorguyu ayırma.
- Önceki sıralama ve tekrar eleme becerilerini bir sıra sorgusunda birleştirme.

Alıştırmalar

1. Tarama sonunda toplam farklı değer sayısını da yazdırın; erken çıkışın bu sayımı neden eksik bırakacağını açıklayın.
2. K numaralı farklı büyük değeri bulun; tekrarları sıra hesabına katmayın.
3. Tekrarları atlamadan K numaralı elemanı bulun; 7, 2, 7, -1, 2 için $K = 3$ sonucunun neden değiştiğini açıklayın.

059 Dizinin modunu sıralama ile bulma

Düzey: İleri Önkoşullar: 048, 051, 058

Kavramlar: Mod, Sıralama, Ardışık eşit değer grupları, Grup sınırı, Eşit sıklık kuralı

Problem tanımı

1..50 elemanlı bir tamsayı dizisinin en sık görülen değerini, yani modunu bulun. Aynı en yüksek adede sahip birden çok değer varsa en küçük değeri seçin. Örneğin 3, 1, 3, 1, 2 için hem 1 hem 3 iki kez bulunur; sonuç mod 1, frekans 2'dir. Bütün değerler farklıysa bu çalışmanın eşitlik kuralına göre en küçük değer seçilir ve frekansı 1 olur.

Girdi sırasız olabilir; bütün elemanlar tam int aralığındadır. 048'deki 21 konumlu frekans dizisi bu alanı kapsamaz. Önce seçmeli sıralamayla aynı diziyi sıralayın; ardından ardışık eşit değer gruplarının uzunluklarını sayın. Hazır sıralama veya gruptama kullanmayın. Sonuç bu problemde tanımlanan eşitlik kuralına göre tek değer olarak verilir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n	1..50 arasında eleman sayısı.
Girdi	a	N adet tam int; tekrarlar olabilir.
Çıktı	Mod	Mod: X; en sık değer, eşit sıklıkta en küçük.
Çıktı	Frekans	Frekans: Y; modun kaç kez geçtiği.

Eleman sayısı (1..50): ve A[0]: ile başlayan istemler ayrı satırlarda yanıtlanır. Bütün elemanlar doğrulanmadan işlem başlamaz. Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., elemenda Hata: Eleman geçerli bir int tamsayı olmalıdır. yazılır. Metin, ondalık, tür taşması, boş ve EOF ilgili hatayı üretir; mod ve frekans satırları yazılmaz.

Algoritma

1. Uzunluğu ve bütün elemanları doğrulayarak okuyun.
 2. Diziyi seçmeli sıralamayla azalmayan sıraya getirin.
 3. Mod adayını ilk değer, en yüksek adedi 0 ve tarama dizinini 0 yapın.
 4. Tarama bitmediyse mevcut değeri saklayıp grup adedini 0 yapın.
 5. Dizin sınır içinde ve eleman saklanan değere eşitken grup adedini ve dizini artırın.
 6. Grup adedi kayıtlı en yüksek adetten büyükse mod ve adedi güncelleyin; eşitse değiştirmeyin.
 7. Tarama sürüyorsa 4. adıma dönün; bütün gruplar bittiğinde mod ve frekansı yazdırın.
- 3, 1, 3, 1, 2 sıralanınca 1, 1, 2, 3, 3 olur:

Grup değeri	Grup adedi	Sonraki dizin	Kayıtlı mod / frekans
1	2	2	1 / 2
2	1	3	1 / 2
3	2	5	1 / 2; eşitlikte değişmez.

Her dış tur başında önceki gruplar tamamen sayılmıştır ve kayıtlı mod onların kuralı sağlayan değeridir. Sıralılık her farklı değeri tek bir ardışık gruba toplar. İç döngü yalnız o grubun elemanlarını tüketir; eşit olmayan ilk değeri tüketmez. Gruplar küçükten büyüğe geldiğinden yalnız kesin büyük adette güncellemek eşit sıklıkta küçük değeri korur. Tarama dizini hiçbir zaman geri gitmez; toplam N eleman tüketilince son grup da değerlendirilmiş olur.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.Write("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}

for (int i = 0; i < n - 1; i++)
{
    int enKucuk = i;
    for (int j = i + 1; j < n; j++)
    {
        if (a[j] < a[enKucuk])
        {
            enKucuk = j;
        }
    }
    if (enKucuk != i)
    {
        int gecici = a[i];
        a[i] = a[enKucuk];
        a[enKucuk] = gecici;
    }
}
```

```

int mod = a[0], enYuksekAdet = 0, dizin = 0;
while (dizin < n)
{
    int deger = a[dizin], adet = 0;
    while (dizin < n && a[dizin] == deger)
    {
        adet++;
        dizin++;
    }
    if (adet > enYuksekAdet)
    {
        mod = deger;
        enYuksekAdet = adet;
    }
}

Console.WriteLine("Mod: " + mod.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Frekans: " +
    enYuksekAdet.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. İç koşul dizin denetimini önce yapar; son grubun ardından a[n] okunmaz. Mod gerçek bir elemanla başlatılır; 0 gibi dizide bulunmayabilecek bir aday kullanılmaz. En yüksek adedin 0 olması ilk grubun mutlaka kayda girmesini sağlar.

Örnek çalıştırmalar

N ve N eleman ayrı satırlarda girilir. Bloklar istemler dışındaki tam çıktılarıdır.

N = 5; dizi 3, 1, 3, 1, 2:

Mod: 1
Frekans: 2

N = 3; dizi 3, 1, 2:

Mod: 1
Frekans: 1

N = 4; dizi -2147483648, 2147483647, 2147483647, 0:

Mod: 2147483647
Frekans: 2

N = 1; tek eleman -8:

Mod: -8
Frekans: 1

Girdi	Beklenen özellik
N = 50; bütün değerler -2147483648	Bir grup vardır; mod -2147483648, frekans 50 olur.
N = 50; 25 kez 2147483647 ve 25 kez -2147483648	Eşit sıklıkta mod -2147483648, frekans 25 olur.

Uzunluk 0, 51, metin, ondalık, boş veya EOF ise:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda 2147483648, -2147483649, metin, ondalık, boş veya EOF için:

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- Tek eleman bir gruptur; mod kendisi, frekans 1 olur.
- Bütün elemanlar eşitse iç döngü bütün diziyi tüketir; dış döngü bir tur çalışır.
- Bütün değerler farklıysa bu çalışmanın kuralıyla en küçük değer seçilir. Birden çok mod tanımı yerine tek sonuç kuralı uygulanır.
- Son grubun adedi, iç döngü sınır nedeniyle bittiğinde de değerlendirilir.
- En yüksek frekans en fazla 50'dir; sayaçlar int türünde güvenlidir.
- Tam int aralığı için geniş bir frekans dizisi ayrılmaz; sıralama ve grublama kullanılır.
- Eksik veya hatalı girişte sonuç verilmez; bütün veriler önce doğrulanır.

Kazanımlar

- Sıralılığı aynı değerleri komşu gruplara toplamak için kullanma.
- Bir grubun bittiği ilk farklı elemanı sonraki gruba bırakma.
- Her grupta adedi yeniden başlatırken tarama dizisini sürdürme.
- Eşit sıklık kuralını sıralama yönü ve kesin karşılaştırmayla uygulama.
- Sınırlı alan frekans dizisiyle tam alan sıralama yaklaşımını karşılaştırma.

Alıştırmalar

1. En yüksek frekansa sahip bütün değerleri yazdırın; ilk taramada frekansı bulup ikinci taramada eşit grupları seçin.
2. Eşit sıklıkta en büyük değeri seçen sürümü yazın; güncelleme koşulunu açıklayın.
3. Her farklı değer için grup adedini yazdırın; toplamlarının N olduğunu ve son grubun da sayıldığını doğrulayın.

060 İki sıralama algoritmasını karşılaştırma

Düzey: İleri Önkoşullar: 051, 052, 053, 055

Kavramlar: Ortak girdi, Bağımsız kopyalar, İşlem sayısı, Karşılaştırma ölçütü, Girdiye bağlı davranış

Problem tanımı

Tam int aralığında 1..50 eleman içeren aynı veriyi seçmeli sıralama ve erken bitişli kabarcık sıralamayla ayrı ayrı sıralayın. İki sonuç dizisini, değer karşılaştırması ve gerçek takas sayılarını, ayrıca kabarcığın gerçek tur sayısını yazdırın. Son satırda hangi yöntemin daha az değer karşılaştırması yaptığını veya sayıların eşit olduğunu belirtin.

Özgün veriyi okuyup döngüyle iki ayrı diziye kopyalayın. İkinci yöntemi birincinin sıraladığı dizi üzerinde çalıştırmayın; iki yöntem aynı başlangıç sırasını görmelidir. Hazır sıralama, kopyalama veya süre ölçümü kullanmayın. Bu çalışma belirli bir girdide belirli işlemleri karşılaştırır; genel bir hız veya bütün girdiler için üstünlük iddiası üretmez.

Değer karşılaştırması seçmelide $secmeli[j] < secmeli[enKucuk]$, kabarcıkta $kabarcik[j] > kabarcik[j + 1]$ ifadesinin bir değerlendirmesidir. Dizin koşulları, doğrulama, kopyalama ve sayaçlar arasındaki son karar bu ölçüye dahil değildir. Takas iki değer gerçekten yer değiştirmesidir; seçmelide aynı dizine takas yapılmaz. Kabarcıkta takassız son denetim turu sayılır; tek elemanda tur yoktur.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, a	1..50 uzunluk ve N ayrı tam int değeri.
Ara değer	İki kopya	secmeli, kabarcik; başlangıçta özgün veriyle aynı.
Çıktı	Seçmeli sonuç	Seçmeli dizi:, Seçmeli karşılaştırma:, Seçmeli takas: satırları.
Çıktı	Kabarcık sonuç	Kabarcık dizi:, Kabarcık karşılaştırma:, Kabarcık takas:, Kabarcık tur: satırları.
Çıktı	Ortak ölçü kararı	Daha az değer karşılaştırması: Seçmeli, Kabarcık veya Eşit.

Eleman sayısı (1..50): ve A[0]: ile başlayan istemleri ayrı satırlarda yanıtlayın. Çıktı tam sekiz satırdır. İki dizi satırında başlıktan sonra her değer için bir boşluk vardır; sonunda fazladan boşluk yoktur. Geçersiz uzunlukta Hata: Eleman sayısı 1 ile 50 arasında olmalıdır., elemanda Hata: Eleman geçerli bir int tamsayı olmalıdır. yazılır. Bütün girişler doğru olmadan kopyalama ve sıralama başlamaz.

Algoritma

1. Uzunluğu ve bütün özgün elemanları doğrularak okuyun.
2. İki N uzunluklu dizi oluşturup özgün veriyi bir döngüyle ikisine de kopyalayın.
3. Seçmeli sayaçlarını 0 yapın; her turda kalan bölümün minimumunu bulun, gerekirse takas edin.
4. Kabarcık sayaçlarını 0 yapın; daralan bölümde komşu büyüklükleri takas edin.
5. Kabarcıkta her gerçek turu sayın; takassız tur tamamlanınca durun.
6. İki diziyi ve ayrı sayaçlarını yazdırın.
7. Değer karşılaştırması sayılarını karşılaştırıp son karar satırını yazın.

Dört elemanlı üç girdi için beklenen sayaçlar:

Girdi	Seçmeli karşılaştırma / takas	Kabarcık karşılaştırma / takas / tur
1, 2, 3, 4	6 / 0	3 / 0 / 1
4, 3, 2, 1	6 / 2	6 / 6 / 3
2, 1, 3, 4	6 / 1	5 / 1 / 2

Her kopyalama adımı sonunda iki kopyanın doldurulmuş bölümü özgün veriyle aynıdır. Sıralamalar yalnız kendi kopyalarını değiştirir; biri ötekini girdisini etkileyemez. Seçmelide sıralı başlangıç, kabarcıkta kesinleşmiş son bölüm büyür. Her değer karşılaştırması tam bir kez ilgili sayaçta kayda girer. Sonlu uzunluklar iki sıralamayı da bitirir. Son karar yalnız ortak ölçüyü karşılaştırır; daha az karşılaştırmadan daha az takas anlamına gelmesi gerekmez.

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Eleman sayısı (1..50): ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int n) || n < 1 || n > 50)
{
    Console.WriteLine("Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    Console.WriteLine("A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ");
    if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out int deger))
    {
        Console.WriteLine("Hata: Eleman geçerli bir int tamsayı olmalıdır.");
        return;
    }
    a[i] = deger;
}
```



```

int[] secmeli = new int[n];
int[] kabarcik = new int[n];
for (int i = 0; i < n; i++)
{
    secmeli[i] = a[i];
    kabarcik[i] = a[i];
}

int secimKarsilastirma = 0, secimTakas = 0;
for (int i = 0; i < n - 1; i++)
{
    int enKucuk = i;
    for (int j = i + 1; j < n; j++)
    {
        secimKarsilastirma++;
        if (secmeli[j] < secmeli[enKucuk])
        {
            enKucuk = j;
        }
    }
    if (enKucuk != i)
    {
        int gecici = secmeli[i];
        secmeli[i] = secmeli[enKucuk];
        secmeli[enKucuk] = gecici;
        secimTakas++;
    }
}

int kabarcikKarsilastirma = 0, kabarcikTakas = 0, kabarcikTur = 0;
for (int gecis = 0; gecis < n - 1; gecis++)
{
    bool degisti = false;
    for (int j = 0; j < n - 1 - gecis; j++)
    {
        kabarcikKarsilastirma++;
        if (kabarcik[j] > kabarcik[j + 1])
        {
            int gecici = kabarcik[j];
            kabarcik[j] = kabarcik[j + 1];
            kabarcik[j + 1] = gecici;
            kabarcikTakas++;
            degisti = true;
        }
    }
    kabarcikTur++;
    if (!degisti)
    {
        break;
    }
}

```

```

Console.Write("Seçmeli dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + secmeli[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Seçmeli karşılaştırma: " +
    secimKarsilastirma.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Seçmeli takas: " +
    secimTakas.ToString(CultureInfo.InvariantCulture));

Console.Write("Kabarcık dizi:");
for (int i = 0; i < n; i++)
{
    Console.Write(" " + kabarcik[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Kabarcık karşılaştırma: " +
    kabarcikKarsilastirma.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Kabarcık takas: " +
    kabarcikTakas.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Kabarcık tur: " +
    kabarcikTur.ToString(CultureInfo.InvariantCulture));

if (secimKarsilastirma < kabarcikKarsilastirma)
{
    Console.WriteLine("Daha az değer karşılaştırması: Seçmeli");
}
else if (kabarcikKarsilastirma < secimKarsilastirma)
{
    Console.WriteLine("Daha az değer karşılaştırması: Kabarcık");
}
else
{
    Console.WriteLine("Daha az değer karşılaştırması: Eşit");
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Özgün A dizisi korunur; iki sıralama için toplam iki ek dizi ayrılır. 051 ve 052'nin sayım kuralları aynen uygulanır. Değerleri çıkararak karşılaştırmak yerine doğrudan < ve > kullanılır; int uç değerleri taşma oluşturmaz.

Örnek çalıştırmalar

N ve ardından N eleman ayrı satırlarda girilir. Bloklar istemler dışındaki tam sekiz sonuç satırıdır.

N = 4; dizi 1, 2, 3, 4:

```

Seçmeli dizi: 1 2 3 4
Seçmeli karşılaştırma: 6
Seçmeli takas: 0
Kabarcık dizi: 1 2 3 4
Kabarcık karşılaştırma: 3
Kabarcık takas: 0
Kabarcık tur: 1
Daha az değer karşılaştırması: Kabarcık

```

N = 4; dizi 4, 3, 2, 1:

Seçmeli dizi: 1 2 3 4
Seçmeli karşılaştırma: 6
Seçmeli takas: 2
Kabarcık dizi: 1 2 3 4
Kabarcık karşılaştırma: 6
Kabarcık takas: 6
Kabarcık tur: 3
Daha az değer karşılaştırması: Eşit

N = 1; tek eleman -2147483648:

Seçmeli dizi: -2147483648
Seçmeli karşılaştırma: 0
Seçmeli takas: 0
Kabarcık dizi: -2147483648
Kabarcık karşılaştırma: 0
Kabarcık takas: 0
Kabarcık tur: 0
Daha az değer karşılaştırması: Eşit

Girdi	Beklenen özellik
N = 50; farklı değerler 50..1 sırasında	İki dizi 1..50 olur; karşılaştırmalar 1225/1225, takaslar 25/1225, kabarcık turu 49.
N = 50; bütün değerler 2147483647	Karşılaştırmalar 1225/49; iki takas sayısı 0, kabarcık turu 1; son karar Kabarcık.

Uzunluk 0, 51, metin, ondalık, boş veya EOF için tek çıktı:

Hata: Eleman sayısı 1 ile 50 arasında olmalıdır.

Herhangi bir elemanda metin, ondalık, 2147483648, -2147483649, boş veya EOF için:

Hata: Eleman geçerli bir int tamsayı olmalıdır.

Sınır durumları

- N = 1 için bütün sayaçlar 0'dır; sonuçlar eşittir.
- Sıralı N > 1 veride seçmeli bütün kalan bölümleri tarar; kabarcık N - 1 karşılaştırmadan sonra durur.
- Kabarcık en fazla seçmeli kadar değer karşılaştırması yapar; bu sürümlerde son karar Seçmeli olamaz. Üçlü karar, ölçü sayılarının genel karşılaştırmasını gösterir.
- Ters sıralı farklı değerlerde karşılaştırmalar aynı olmasına rağmen takas sayıları farklı olabilir.
- Tekrarlar iki sonuç dizisinde de aynı adetlerle korunur; eşit komşular kabarcıkta takas edilmez.
- Karşılaştırma ve kabarcık takası en fazla 1225; seçmeli takas ve kabarcık turu en fazla 49'dur.
- Sonuçlar seçilen veri, sayaç tanımı ve sürümlere aittir. Kopyalama, bellek, doğrulama ve diğer işlemler bu ölçünün dışındadır.
- Geçersiz veya eksik girişte sekiz satırlık sonuç üretilmez.

Kazanımlar

- İki yönteme aynı özgün verinin ayrı kopyalarını sağlama.
- Ortak ölçünün hangi işlemleri içerdiğini kesin tanımlama.
- Farklı anlamdaki sayaçları birbirinin yerine kullanmama.
- Sıralı, ters ve tekrarlı verinin sayaçları nasıl etkilediğini açıklama.
- Gözlenen işlem sayısından genel çalışma süresi üstünlüğü çıkarmanın sınırlarını açıklama.

Alıştırmalar

1. Aynı veri için üçüncü kopyada eklemeli sıralama çalıştırın; değer karşılaştırmasını ortak ölçü olarak tutup kaydırmaları ayrı verin.
2. Kopyalama atamalarını ayrı sayaçta sayın; iki kopya için $2N$ olduğunu doğrulayın, sıralama karşılaştırmalarına eklemeyin.
3. Kabarcığın erken çıkışını kaldırın; sıralı veride sayaçların ve son kararın nasıl değiştiğini açıklayın.

Bölüm 7: Metin işleme ve ayrıştırma

061 Karakter sınıflandırma ve harf frekansı

Düzy: İleri Önkoşullar: 048, 059, 060

Kavramlar: String ve char, Karakter aralığı, Harf normalleştirme, Frekans dizisi, Eşit sıklık kuralı

Problem tanımı

En fazla 200 karakterlik tek bir satırı inceleyin. Harf, rakam, normal boşluk ve diğer karakter adetlerini ayrı sayın. A–Z harflerini a–z ile aynı kabul ederek 26 elemanlı frekans dizisi oluşturun. En sık harfi ve adedini bulun; eşitlikte alfabetik olarak küçük harfi seçin. Hiç harf yoksa bunu ayrı sonuçla belirtin. Örneğin aA bB! 2 içinde dört harf vardır; a ve b ikiye kez geçer, seçilen harf a'dır.

Bu problemde veri alanı yazdırılabilir ASCII karakterleri, yani 32..126 kodlarıdır. Harfler yalnız A–Z ve a–z, rakamlar yalnız 0–9'dur; boşluk U+0020'dir. Türkçe harfler, emoji, sekme ve diğer karakterler giriş alanının dışındadır. Bu sınır sayesinde her kabul edilen karakter C# içinde bir char ve bir dizin konumudur. Boş satır geçerlidir; girişin sonu (EOF) boş satır değildir. Hazır karakter sınıflandırma, harf dönüştürme veya gruplama araçları kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 yazdırılabilir ASCII karakteri; tek satır.
Ara değer	frekans	26 sayaç; a dizini 0, z dizini 25.
Çıktı	Sınıflar	Harf: H, Rakam: R, Boşluk: B, Diğer: D.
Çıktı	En sık harf	En sık harf: c ve Frekans: F; harf yoksa En sık harf: Yok.

Metin: istemini bir satırla yanıtlayın. Bütün karakterler doğrulanmadan sayım veya sonuç üretimi başlamaz. EOF, 201 veya daha fazla karakter ve kabul edilmeyen herhangi bir karakter için yalnız Hata: Metin 0..200 yazdırılabilir ASCII karakteri içermelidir. yazılır.

Algoritma

1. Satırı okuyun; null değerini ve uzunluk sınırını denetleyin.
2. Her karakterin kodunun 32..126 aralığında olduğunu doğrulayın.
3. Dört sınıf sayacını ve 26 harf sayacını 0 yapın.
4. Büyük harfi kod farkıyla küçük harfe dönüştürün; harfi, rakamı, boşluğu veya diğer sınıfını sayın.
5. Harf için frekans[c - 'a'] sayacını artırın.
6. Frekans dizisini a'dan z'ye tarayın; yalnız kesin büyük adette en sık harfi güncelleyin.
7. Sınıfları ve harf bulunma durumuna uygun sonucu yazdırın.

Her kabul edilen karakter tam bir sınıfa girer; dört sayacın toplamı metnin uzunluğudur. Harf frekanslarının toplamı harf adedidir. Artan alfabetik dizinlerde yalnız büyük adette güncellemek eşitlikte ilk harfi korur. Tarama sonlu uzunlukla sınırlıdır; bütün sayaçlar en fazla 200 olur.

Karakter ile karakter kodu farklı değerlerdir. Console.Write('A') harfi, Console.Write((int)'A') ise 65 sayısını yazar. Harfi frekans dizinine dönüştürürken kullanılan c - 'a' ifadesi de karakter üretmez; a'ya göre sayısal konumu hesaplar.

aA bB!2 için birikimli sayım:

Okunan bölüm	Harf	Rakam	Boşluk	Diğer
aA	2	0	0	0
aA bB	4	0	1	0
aA bB!2	4	1	1	1

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    if (metin[i] < ' ' || metin[i] > '~')
    {
        Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}

int harf = 0, rakam = 0, bosluk = 0, diger = 0;
int[] frekans = new int[26];
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (c >= 'A' && c <= 'Z')
    {
        c = (char)(c + ('a' - 'A'));
    }
    if (c >= 'a' && c <= 'z')
    {
        harf++;
        frekans[c - 'a']++;
    }
    else if (c >= '0' && c <= '9')
    {
        rakam++;
    }
    else if (c == ' ')
    {
        bosluk++;
    }
    else
    {
        diger++;
    }
}
```

```

int enSikDizin = -1, enSikAdet = 0;
for (int i = 0; i < frekans.Length; i++)
{
    if (frekans[i] > enSikAdet)
    {
        enSikAdet = frekans[i];
        enSikDizin = i;
    }
}

Console.WriteLine("Harf: " + harf.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Rakam: " + rakam.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Boşluk: " + bosluk.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Diğer: " + diger.ToString(CultureInfo.InvariantCulture));
if (enSikDizin == -1)
{
    Console.WriteLine("En sık harf: Yok");
}
else
{
    Console.WriteLine("En sık harf: " + (char)('a' + enSikDizin));
    Console.WriteLine("Frekans: " +
        enSikAdet.ToString(CultureInfo.InvariantCulture));
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. string değiştirilemez; yerel c karakterini değiştirmek özgün metni değiştirmez. Karakterler çıkarıldığında sonuç sayısal kod farkıdır. -1 yalnız harf dizini için bulunamadı işaretidir; gerçek dizinler 0..25'tir. Sayısal çıktı biçimi kültürden bağımsızdır.

Örnek çalıştırmalar

Bloklar Metin: isteminden sonraki tam sonuçtur. Boş satır örneğinde Enter'a basılır.

aA bB!2:

```

Harf: 4
Rakam: 1
Boşluk: 1
Diğer: 1
En sık harf: a
Frekans: 2

```

ZzZ 09?:

```

Harf: 3
Rakam: 2
Boşluk: 1
Diğer: 1
En sık harf: z
Frekans: 3

```

Boş satır:

```

Harf: 0
Rakam: 0
Boşluk: 0
Diğer: 0
En sık harf: Yok

```


123!:

Harf: 0
Rakam: 3
Boşluk: 0
Diğer: 1
En sık harf: Yok

Girdi	Beklenen özellik
200 adet A	Harf adedi ve a frekansı 200; diğer sınıflar 0.
ASCII 32..126 karakterlerinin her biri bir kez	Harf 52, rakam 10, boşluk 1, diğer 32; seçilen harf a, frekans 2.
ç, emoji, sekme, 201 karakter veya EOF	Yalnız tanımlanan hata satırı; sınıf veya harf sonucu yok.

Sınır durumları

- Boş satırın uzunluğu 0'dır; hiçbir sayım döngüsü çalışmaz.
- A ile a aynı harf sayacına gider; özgün metin korunur.
- Rakam, noktalama ve boşluk harf yarışına katılmaz.
- Hiç harf yoksa frekans satırı yazılmaz.
- Eşit pozitif frekanslarda alfabetik küçük harf seçilir.
- C# char genel olarak UTF-16 kod birimidir; bu problemde ASCII sınırı çok birimli karakterleri dışlar.
- Hatalı karakter satırın sonunda olsa da önceki bölümün sayımı yazılmaz.

Kazanımlar

- string diziniyle karakter okuyup char kod aralıklarını karşılaştırma.
- Harf değerini 26 elemanlı frekans dizisinin dizinine dönüştürme.
- Normalleştirme ile özgün veriyi korumayı birlikte uygulama.
- Sınıf adetleri ve toplam uzunluk arasındaki değişmezi doğrulama.
- Harf olmayan veri ile hatalı karakteri problem kuralına göre ayırma.

Alıştırmalar

1. Pozitif frekanslı bütün harfleri alfabetik sırayla yazdırın; frekans toplamını harf adediyle karşılaştırın.
2. Eşit sıklıkta alfabetik büyük harfi seçin; güncelleme koşulunu ve harfsiz durumunu açıklayın.
3. Türkçe alfabeyi açık küçük/büyük harf eşleme dizileriyle destekleyen sürümü tasarlayın; İ, İ, ı ve i eşlemesini örnekleyin.

062 Boşlukları normalleştirme ve sözcük sayma

Düzey: İleri Önkoşullar: 050, 061

Kavramlar: Karakter tamponu, Okuma ve yazma konumu, Sözcük başlangıcı, Ertelenen boşluk, Geçerli uzunluk

Problem tanımı

En fazla 200 karakterlik bir satırın başındaki ve sonundaki normal boşlukları kaldırın; sözcükler arasındaki her boşluk grubunu tek boşluğa indirin. Aynı taramada sözcük adedini bulun. Sözcük, boşluk içermeyen en uzun kesintisiz harf/rakam bölümüdür. Örneğin başında iki, ortasında üç ve sonunda iki boşluk bulunan Ali 42 girdisi Ali 42 ve 2 sözcük üretir.

Yalnız A–Z, a–z, 0–9 ve normal boşluk U+0020 kabul edilir. Boş satır ve yalnız boşluklardan oluşan satır geçerlidir; sonuç metni boş ve sözcük adedi 0 olur. Harflerin büyüklüğü ve rakamlar aynen korunur. Trim, Split, Replace, düzenli ifade veya hazır sözcük sayma kullanmayın. Giriş uzunluğunda bir char tamponu ve ayrı yazma konumu kullanın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 karakter; yalnız belirtilen harf, rakam ve boşluklar.
Ara değer	tampon, yazma	Fiziksel kapasite ve üretilmiş geçerli karakter adedi.
Çıktı	Normal metin	Metin: ardından iki dikey çubuk arasında sonuç; çubuklar çıktı ayraçlarıdır.
Çıktı	Sözcük adedi	Sözcük: N.

Metin: istemini tek satırda yanıtlayın. EOF, 201 veya daha fazla karakter, noktalama, Türkçe harf, sekme veya diğer karakter için yalnız Hata: Metin 0..200 ASCII harf, rakam veya boşluk içermelidir. yazılır. Bütün giriş doğrulanmadan çıktı oluşturulmaz.

Algoritma

1. Satırı ve bütün karakterlerini doğrulayın; giriş uzunluğunda tampon oluşturun.
2. Yazma konumu ile sözcük sayısını 0, sözcük içinde olma durumunu yanlış yapın.
3. Boşlukta sözcük içinde olma durumunu yanlış yapın; henüz boşluk yazmayın.
4. Boşluk olmayan karakterde yeni sözcük başlıyorsa sayacı artırın; önceki çıktı varsa bir boşluk yazın.
5. Karakteri tampona yazıp yazma konumunu artırın; sözcük içinde olma durumunu doğru yapın.
6. Bütün giriş bitince yalnız tamponun geçerli bölümünden sonuç metnini üretilip sayacı yazdırın.

Başlangıç boşlukları çıktı yokken, son boşluklar yeni sözcük başlamadan atlanır. Araya boşluk ancak sonraki sözcüğün ilk karakteri geldiğinde yazılır. Yazılmış bölümde baş/son boşluk ve çift boşluk yoktur. Sonuç uzunluğu girişten büyük olamaz; kullanılmayan tampon hücreleri çıktı değildir.

Örnekte özgün satır 12 karakter, normal sonuç 6 karakter içerir. Sonuç yazılırken yalnız bu altı tampon hücresi kullanılır. Sözcük içinde olma durumu tarama geçmişini, yazma konumu ise üretilmiş karakter adedini taşır; birbirlerinin yerine kullanılamazlar.

Ali 42 için seçilmiş adımlar:

Okunan bölüm	Geçerli tampon	Sözcük adedi
İlk iki boşluk	Boş	0
Ali ve sonraki üç boşluk	Ali	1
4	Ali 4	2
2 ve son iki boşluk	Ali 42	2

C# çözümü

```
using System;
using System.Globalization;
```

```
Console.Write("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 ASCII harf, rakam " +
        "veya boşluk içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    bool gecerli = (c >= 'A' && c <= 'Z') ||
        (c >= 'a' && c <= 'z') || (c >= '0' && c <= '9') || c == ' ';
    if (!gecerli)
    {
        Console.WriteLine("Hata: Metin 0..200 ASCII harf, rakam " +
            "veya boşluk içermelidir.");
        return;
    }
}
```

```
char[] tampon = new char[metin.Length];
int yazma = 0, sozcuk = 0;
bool sozcukIcinde = false;
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (c == ' ')
    {
        sozcukIcinde = false;
    }

    else
    {
        if (!sozcukIcinde)
        {
            sozcuk++;
            if (yazma > 0)
            {
                tampon[yazma++] = ' ';
            }
        }
    }
}
```

```

        tampon[yazma++] = c;
        sozcukIcinde = true;
    }
}

```

```

string sonuc = new string(tampon, 0, yazma);
Console.WriteLine("Metin: |" + sonuc + "|");
Console.WriteLine("Sözcük: " + sozcuk.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. `new string(tampon, 0, yazma)` hazır normalleştirme yapmaz; algoritmanın ürettiği karakterleri metne paketler. Tamponun başlangıçtaki `\0` hücreleri yazma sınırının dışında kalır. `yazma++` atamada mevcut konumu kullanır, sonra ilerler.

Örnek çalıştırmalar

Bloklar giriş isteminden sonraki bütün sonuç satırlarıdır; çıktındaki çubukların arasındaki metin normalleştirilmiştir.

```

Ali  42 :
Metin: |Ali 42|
Sözcük: 2

```

A B C:

```

Metin: |A B C|
Sözcük: 3

```

Boş satır veya üç boşluk:

```

Metin: ||
Sözcük: 0

```

x7:

```

Metin: |x7|
Sözcük: 1

```

Girdi	Beklenen özellik
200 adet boşluk	Geçerli uzunluk 0; sözcük adedi 0.
200 adet a	Sonuç 200 adet a; tek sözcük.
a ve boşluk dönüşümlü 200 karakter	Son boşluk atlanır; sonuç uzunluğu 199, sözcük adedi 100.
Ali!, ç, sekme, 201 karakter veya EOF	Yalnız tanımlanan hata satırı.

Sınır durumları

- Boş girişte sıfır uzunluklu tampon ve boş string geçerlidir.
- Birden çok boşluk tek bir sözcük geçişi oluşturur.
- Sona boşluk yazılmasını önlemek için yazma bir sonraki sözcüğe ertelenir.
- Rakam içeren kesintisiz bölüm tek sözcüktür; `a1b2` parçalanmaz.
- Normalleştirilmiş metni tekrar işlemek aynı metni ve aynı sözcük adedini üretir.
- Geçerli veri sayısı tampon kapasitesinden küçük olabilir.

Kazanımlar

- Metinde sözcük başlangıcını önceki tarama durumuyla tanıma.
- Okuma ve yazma konumlarını ayrı tutarak veriyi sıkıştırma.
- Sonda istenmeyen ayraç üretmemek için yazma zamanını seçme.
- Karakter tamponunun yalnız geçerli bölümünü metne dönüştürme.
- Aynı girdinin ikinci işlenişinde değişmemesini kontrol etme.

Alıştırmalar

1. Her atlanan boşluğu sayın; giriş ve sonuç uzunluğu farkıyla karşılaştırın.
2. Boşluk yerine sözcükler arasına tek - yazın; boş ve tek sözcüklü girdileri koruyun.
3. Giriş tamponunu ayrı sonuç tamponu kullanmadan normalleştiren sürüm tasarlayın; okuma konumunun henüz okunmamış veriyi neden koruduğunu açıklayın.

063 En uzun sözcüğü ve başlangıç dizinini bulma

Düzey: İleri Önkoşullar: 041, 062

Kavramlar: Sözcük sınırı, Başlangıç ve uzunluk, İlk eşit aday, İki aşamalı tarama, Dizinlerin korunması

Problem tanımı

Yalnız A–Z, a–z ve normal boşluk U+0020 içeren, en fazla 200 karakterlik satırın en uzun sözcüğünü bulun. Sözcük, kesintisiz en uzun harf bölümüdür. Eşit uzunlukta birden çok sözcük varsa girişte ilk görüneni seçin. Sözcüğü, özgün satırdaki sıfır tabanlı başlangıç dizinini ve uzunluğunu yazdırın. Örneğin bir uzun iki için sonuç uzun, başlangıç dizini 6 ve uzunluk 4'tür.

Boş satır ve yalnız boşluklardan oluşan satır geçerlidir; sözcük bulunamadığını bildirin. Başlangıç dizini bulunmadan metni normalleştirmeyin; boşlukları kaldırmak özgün konumları değiştirir. Harf büyüklüğü korunur. Split, Trim, Substring, hazır en büyük değer veya sözcük bulma araçları kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 ASCII harf veya normal boşluk.
Ara değer	baslangic, uzunluk	Mevcut sözcüğün özgün konumu ve eleman adedi.
Çıktı	Bulunan sözcük	Sözcük: ardından dikey çubuklar arasında sözcük; Başlangıç dizini: I, Uzunluk: L.
Çıktı	Sözcük yok	Tek satır Sonuç: Sözcük yok.

Metin: istemini bir satırla yanıtlayın. EOF, uzunluk üst sınırının aşılması, rakam, noktalama, Türkçe harf veya sekme için yalnız Hata: Metin 0..200 ASCII harf veya boşluk içermelidir. yazılır. Satırın tamamı taramadan önce doğrulanır.

Algoritma

1. Satırı, uzunluğunu ve bütün karakterlerini doğrulayın.
2. Tarama dizinini 0, en iyi başlangıcı -1 ve en iyi uzunluğu 0 yapın.
3. Dizin sınır içinde ve karakter boşlukken ilerleyin.
4. Sözcüğün başlangıcını mevcut dizin olarak saklayın; boşluk gelene veya giriş bitene kadar ilerleyin.
5. Dizin ile başlangıcın farkından sözcük uzunluğunu hesaplayın.
6. Uzunluk mevcut en iyiden kesin büyükse başlangıcı ve uzunluğu güncelleyin; taramayı sürdürün.
7. Hiç sözcük yoksa bunu bildirin; aksi halde seçili karakterleri ve özgün konumu yazdırın.

Tarama dizini geri gitmez. Her dış turda ya bir boşluk bölümü ya bir sözcük tüketilir; sonunda son boşluklar da giriş sonuna ulaşır. En iyi aday yalnız kesin uzunluk artışında değiştiği için eşitlikte ilk sözcük kalır. Seçili bölüm özgün satırda [enYiBaslangic, enYiBaslangic + enYiUzunluk) aralığıdır.

bir uzun iki için:

Sözcük	Başlangıç	Uzunluk	Kayıtlı aday
bir	2	3	bir
uzun	6	4	uzun
iki	12	3	uzun

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 ASCII harf veya boşluk içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (!(c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') || c == ' ')
    {
        Console.WriteLine("Hata: Metin 0..200 ASCII harf veya boşluk içermelidir.");
        return;
    }
}

int dizin = 0, enIyiBaslangic = -1, enIyiUzunluk = 0;
while (dizin < metin.Length)
{
    while (dizin < metin.Length && metin[dizin] == ' ')
    {
        dizin++;
    }
    int baslangic = dizin;
    while (dizin < metin.Length && metin[dizin] != ' ')
    {
        dizin++;
    }
    int uzunluk = dizin - baslangic;
    if (uzunluk > enIyiUzunluk)
    {
        enIyiBaslangic = baslangic;
        enIyiUzunluk = uzunluk;
    }
}
```

```

if (enIyiBaslangic == -1)
{
    Console.WriteLine("Sonuç: Sözcük yok");
}
else
{
    Console.Write("Sözcük: |");
    for (int i = enIyiBaslangic; i < enIyiBaslangic + enIyiUzunluk; i++)
    {
        Console.Write(metin[i]);
    }
    Console.WriteLine("|");
    Console.WriteLine("Başlangıç dizini: " +
        enIyiBaslangic.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Uzunluk: " +
        enIyiUzunluk.ToString(CultureInfo.InvariantCulture));
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Dizin sınırı && içinde karakter erişiminden önce değerlendirilir. Son boşluklar tüketildiğinde uzunluk 0 olur; var olan aday değişmez. Çıktı döngüsü yeni sözcük aramaz, kaydedilmiş aralığı aynen yazar.

Örnek çalıştırmalar

Bloklar Metin: isteminden sonraki tam sonuçtur.

bir uzun iki:

Sözcük: |uzun|
 Başlangıç dizini: 6
 Uzunluk: 4

Ali Veli Ece:

Sözcük: |Veli|
 Başlangıç dizini: 4
 Uzunluk: 4

ab cd:

Sözcük: |ab|
 Başlangıç dizini: 0
 Uzunluk: 2

Boş satır veya üç boşluk:

Sonuç: Sözcük yok

Girdi	Beklenen özellik
200 adet z	Tek sözcük; başlangıç 0, uzunluk 200.
199 boşluk ve A	Başlangıç 199, uzunluk 1.
abc123, ç, sekme, 201 karakter veya EOF	Yalnız tanımlanan hata satırı.

Sınır durumları

- Sözcük yokken -1 gerçek bir dizin olarak kullanılmaz.
- Son sözcükten sonra boşluk bulunması gerekmez.
- Başlangıç ve uzunluk karakter konumudur; sözcük sıra numarası değildir.
- Eşit uzunlukta güncelleme yapılmaması ilk sonucu korur.
- Yalnız boşluklar giriş hatası değildir.
- Başlangıç boşlukları özgün dizinde sayılır.

Kazanımlar

- Ayırıştırılacak bölümün başlangıcını ve bitişini farklı anlarda belirleme.
- Sözcük üretmeden sınırları saklayarak aday seçme.
- Ön işleme adımının özgün konum bilgisini nasıl değiştirdiğini açıklama.
- Eşit aday kuralını karşılaştırmanın kesinliğiyle uygulama.
- İç döngülerin dış taramayı neden ilerlettiğini gösterme.

Alıştırmalar

1. Eşit uzunlukta son sözcüğü seçen sürümü yazın; değişen koşulu $ab \leq cd$ ile doğrulayın.
2. En kısa sözcüğü bulun; son boşlukların 0 uzunluklu aday olarak seçilmesini önleyin.
3. Bütün en uzun sözcüklerin özgün başlangıç dizinlerini ikinci taramada yazdırın.

064 Metinde iki uçtan palindrom denetimi

Düzye: İleri Önkoşullar: 024, 043, 061, 063

Kavramlar: İki uçtan tarama, Karakter atlama, Normalleştirme, Erken sonlandırma, Karşılaştırma sayımı

Problem tanımı

En fazla 200 yazdırılabilir ASCII karakterinden oluşan satırın, yalnız harf ve rakamları dikkate alınca palindrom olup olmadığını bulun. A–Z ile a–z aynı harf kabul edilir. Boşluk ve noktalama karşılaştırmaya katılmaz. Örneğin A!b!a anlamlı karakterlerle aba olur ve palindromdur. a1b palindrom değildir.

Normalleştirilmiş metin veya ters kopya oluşturmada iki uçtan ilerleyin. Önce her uçtaki harf/rakam olmayan karakterleri atlayın, ardından iki anlamlı karakteri karşılaştırın. Gerçek karakter karşılaştırması adedini sayın; atlama ve dizin sınırı denetimleri bu sayaca dahil değildir. Anlamlı karakter sayısı 0 veya 1 ise palindrom kabul edilir ve karşılaştırma sayısı 0'dır. Reverse, Replace, hazır harf/rakam denetimi, harf dönüştürme veya palindrom aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 karakter; ASCII kodları 32..126.
Ara değer	sol, sag	Özgün metinde içeri doğru ilerleyen dizinler.
Çıktı	Karar	Palindrom: Evet veya Palindrom: Hayır.
Çıktı	Sayaç	Karşılaştırma: C; ilk farklı çift de sayılır.

Metin: istemini tek satırla yanıtlayın. EOF, 201 veya daha fazla karakter, Türkçe harf, emoji veya sekme için yalnız Hata: Metin 0..200 yazdırılabilir ASCII karakteri içermelidir. yazılır. Boş satır geçerlidir; bütün veri karar vermeden önce doğrulanır.

Algoritma

1. Satırı ve bütün karakterlerini doğrulayın.
2. Sol dizini 0, sağ dizini son konum, kararı doğru ve sayacı 0 yapın.
3. Sol uçta harf/rakam olmayanları içeri doğru atlayın; sağ uçta da aynı işlemi yapın.
4. Sol dizin sağa ulaşmış veya geçmişse karşılaştırmadan bitirin.
5. İki uçtaki büyük harfleri yerel küçük harf değerlerine dönüştürün.
6. Sayacı artırıp iki karakteri karşılaştırın; farklıysa kararı yanlış yapıp bitirin.
7. Eşitse iki dizini içeri ilerletin; taramayı sürdürün ve sonucu yazdırın.

İki uç dışında kalmış anlamlı karakter çiftleri eşittir. Anlamsız karakteri atlamak bu özelliği değiştirmez. Yeni eşit çift, doğrulanmış bölümü büyütür; farklı çift normal palindrom sonucunu yanlış yapar. Her adımda sol artar veya sağ azalır; dizinler sonlu aralıkta buluşur.

İki uç içeri ilerler; atlama döngüleri bütün karakter çiftlerini denemez. Her karakter sabit sayıda incelenir, bu yüzden tarama metin uzunluğuyla orantılıdır. Yeni bir anlamlı karakter dizisi veya ters metin için tampon ayrılmaz.

A!b!a için:

Aşama	Sol	Sağ	Sonuç
İlk anlamlı çift	0; A	4; a	Eşit; sayaç 1
Noktalama atlama	2; b	2; b	Dizinler buluştu
Bitiş	2	2	Ortadaki karakter karşılaştırılmaz

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    if (metin[i] < ' ' || metin[i] > '~')
    {
        Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}

int sol = 0, sag = metin.Length - 1, karsilastirma = 0;
bool palindrom = true;
while (sol < sag)
{
    while (sol < sag)
    {
        char c = metin[sol];
        if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
            (c >= '0' && c <= '9'))
        {
            break;
        }
        sol++;
    }
}
```

```

while (sol < sag)
{
    char c = metin[sag];
    if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9'))
    {
        break;
    }
    sag--;
}
if (sol >= sag)
{
    break;
}

char a = metin[sol], b = metin[sag];
if (a >= 'A' && a <= 'Z')
{
    a = (char)(a + ('a' - 'A'));
}
if (b >= 'A' && b <= 'Z')
{
    b = (char)(b + ('a' - 'A'));
}
karsilastirma++;
if (a != b)
{
    palindrom = false;
    break;
}
sol++;
sag--;
}

Console.WriteLine(palindrom ? "Palindrom: Evet" : "Palindrom: Hayır");
Console.WriteLine("Karşılaştırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Büyük harf dönüşümü metni değiştirmez. Boş girişte sağ dizin -1 olur, dış döngü çalışmaz ve karakter erişimi yapılmaz. Çıktıdaki koşullu ifade yalnız iki sabit sonuçtan birini seçer.

Örnek çalıştırmalar

Bloklar istem dışındaki bütün sonuç satırlarını gösterir.

A!b!a:

```

Palindrom: Evet
Karşılaştırma: 1

```

a1b:

```

Palindrom: Hayır
Karşılaştırma: 1

```

A man, a plan, a canal: Panama!:

```

Palindrom: Evet
Karşılaştırma: 10

```

Boş satır, ... veya Z:

```

Palindrom: Evet
Karşılaştırma: 0

```

Girdi	Beklenen özellik
200 adet A	Palindrom; 100 karşılaştırma.
a ardından 198 boşluk ardından b	İlk anlamlı çift farklı; 1 karşılaştırma.
Türkçe harf, sekme, 201 karakter veya EOF	Yalnız tanımlanan hata satırı.

Sınır durumları

- Anlamlı karakteri olmayan metin bu problem kuralıyla palindromdur.
- Rakamlar atlanmaz; 1a2 palindrom değildir.
- İlk farklı çiftte durulur; kalan çiftlerin sayımı yapılmaz.
- Büyük/küçük harf eşitliği yalnız ASCII harfleri için tanımlanmıştır.
- Atlama sonrasında dizinlerin buluşması yeniden denetlenir.
- Gerçek karakter karşılaştırması en fazla 100'dür.

Kazanımlar

- Birden fazla iç taramayı ortak iki dizinle yönetme.
- Atlanacak veri ile karşılaştırılacak veriyi ayırma.
- Dönüştürülmüş kopya oluşturmada normalleştirilmiş karşılaştırma yapma.
- Sonlanma ve sınır güvenliğini dizinlerin hareketiyle açıklama.
- Erken sonlandırmanın işlem sayısına etkisini ölçme.

Alıştırmalar

1. Karşılaştırılan özgün sol/sağ dizin çiftlerini yazdıran izleme sürümü hazırlayın.
2. Noktalama atlanmasını kaldırıp yalnız boşluk atlayan sürümü yazın; A!b!a ve a!a? sonuçlarını karşılaştırın.
3. Anlamlı karakter sayısı 0 olan girdiyi ayrı sonuç yapın; tek karakterli metinlerin davranışını koruyun.

065 Harf frekanslarıyla anagram denetimi

Düzey: İleri Önkoşullar: 049, 061, 064

Kavramlar: İki veri kümesi, Frekans farkı, Tekrarların korunması, Normalleştirme, Sıfır denetimi

Problem tanımı

Yalnız ASCII harfleri ve normal boşluk içeren iki satırın anagram olup olmadığını bulun. Büyük/küçük harf ayrımı yapmayın ve boşlukları yok sayın. Anagram olmak, aynı harflerin aynı adetlerle bulunmasıdır; sıraları farklı olabilir. Örneğin Dormitory ile Dirty room anagramdır. aab ile abb aynı farklı harfleri içerir ama adetleri farklı olduğundan anagram değildir.

Her satır 0..200 karakter içerir. Kabul edilen harfler A–Z ve a–z, boşluk U+0020'dir. İki boş satır veya yalnız boşluklardan oluşan iki satır bu kuralla anagramdır. Rakam ve noktalama kabul edilmez. Bir 26 elemanlı fark dizisi kullanın: birinci metin harfleri sayacı artırır, ikinci metin harfleri azaltır. Hazır sıralama, gruptama, harf dönüştürme veya anagram aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	İki metin	Her biri 0..200 ASCII harf veya normal boşluk; ayrı satırlar.
Ara değer	fark	Her harfin birinci ve ikinci metindeki adetlerinin farkı.
Çıktı	Karar	Tek satır Anagram: Evet veya Anagram: Hayır.

İstem sırası Birinci metin: ve İkinci metin: olur. İlk satır hatalıysa ikinci satır istenmez. Herhangi bir satırda EOF, fazla uzunluk, Türkçe harf, rakam, noktalama veya sekme için yalnız Hata: Metin 0..200 ASCII harf veya boşluk içermelidir. yazılır. İki giriş de doğru olmadan anagram sonucu üretilmez.

Algoritma

- 26 elemanlı fark dizisini sıfırla başlatın.
- Birinci satırı okuyup tamamını doğrulayın; ikinci satır için aynı doğrulamayı uygulayın.
- Her doğrulanmış satırda boşlukları atlayıp büyük harfleri küçük harfe dönüştürün.
- Birinci metinde ilgili harf farkını 1 artırın, ikincide 1 azaltın.
- Bütün farkları tarayın; ilk sıfır olmayan değerde kararı yanlış yapıp durun.
- Kararı yazdırın.

Birinci metnin taraması bitince fark dizisi o metnin harf adetleridir. İkinci metnin her harfi kendi adedini düşürür; sonunda bütün hücreler 0 ise her harfin adedi eşittir. Farkların toplamının 0 olması tek başına yetmez: aab ve abb için a farkı 1, b farkı -1'dir. Her fark -200..200 aralığındadır.

aab ve abb için:

Aşama	a farkı	b farkı	Yorum
Birinci metin bitti	2	1	Birinci metnin adetleri
İkinci metin bitti	1	-1	Aynı uzunluk yeterli değil
Son denetim	1	-1	Anagram değil

C# çözümü

```
using System;

int[] fark = new int[26];
for (int tur = 0; tur < 2; tur++)
{
    Console.Write(tur == 0 ? "Birinci metin: " : "İkinci metin: ");
    string? metin = Console.ReadLine();
    if (metin is null || metin.Length > 200)
    {
        Console.WriteLine("Hata: Metin 0..200 ASCII harf veya boşluk içermelidir.");
        return;
    }
    for (int i = 0; i < metin.Length; i++)
    {
        char c = metin[i];
        if (!((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') || c == ' '))
        {
            Console.WriteLine("Hata: Metin 0..200 ASCII harf " +
                "veya boşluk içermelidir.");
            return;
        }
    }
    for (int i = 0; i < metin.Length; i++)
    {
        char c = metin[i];
        if (c == ' ')
        {
            continue;
        }
        if (c >= 'A' && c <= 'Z')
        {
            c = (char)(c + ('a' - 'A'));
        }
        fark[c - 'a'] += tur == 0 ? 1 : -1;
    }
}

bool anagram = true;
for (int i = 0; i < fark.Length; i++)
{
    if (fark[i] != 0)
    {
        anagram = false;
        break;
    }
}

Console.WriteLine(anagram ? "Anagram: Evet" : "Anagram: Hayır");
```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Dış döngü yalnız iki giriş turu yapar; tur farkın işaretini belirler. Birinci metnin iç sayaçları ikinci girişten önce hazırlanabilir, ancak sonuç iki giriş doğrulanıp işlendikten sonra yazılır. continue mevcut boşluğu atlayıp sonraki karaktere geçer.

Örnek çalıştırmalar

Her çift iki ayrı giriş satırıdır. Bloklar istemler dışındaki tam sonuçtur.

Dormitory ve Dirty room:

Anagram: Evet

aab ve abb:

Anagram: Hayır

Ab ve b a:

Anagram: Evet

Boş satır ve üç boşluk:

Anagram: Evet

a ve boş satır:

Anagram: Hayır

Girdi	Beklenen özellik
Birinci 200 adet A, ikinci 200 adet a	Bütün farklar 0; anagram.
Aynı harf adetleri farklı sıralarda	Sıra farkı sonucu değiştirmez.
İlk satırda !, 1, ç, sekme, 201 karakter veya EOF	Tek hata; ikinci istem verilmez.
İlk satır geçerli, ikinci hatalı veya EOF	Tek hata; anagram sonucu verilmez.

Sınır durumları

- Boşluk adetlerinin veya özgün uzunlukların eşit olması gerekmez.
- Aynı farklı harflerin bulunması yeterli değildir; tekrar adetleri korunur.
- Fark dizisinin her hücresi ayrı denetlenir; toplam fark tek karar ölçüsü değildir.
- Harfsiz iki geçerli metin anagramdır.
- Büyük/küçük harf dönüşümü kültüre bağlı araçla yapılmaz.
- İkinci satır hatalıysa birinci satırın sayımı dışarıya sonuç olarak yazılmaz.

Kazanımlar

- İki veri kümesinin adetlerini tek fark dizisinde karşılaştırma.
- Sıra eşitliği, farklı değer eşitliği ve adet eşitliğini ayırt etme.
- Normalleştirme kuralını iki girdiye aynı biçimde uygulama.
- Toplam değişimin bütün hücre eşitliğini neden garanti etmediğini örnekleme.
- İç hesaplama ile sonuç üretiminin zamanını ayrı yönetme.

Alıştırmalar

1. Sıfır olmayan bütün harf farklarını yazdırın; işaretin hangi metinde fazlalık gösterdiğini açıklayın.
2. Harfsiz metinleri ayrı Har-f yok sonucu yapın; boşluklu anagram örneklerini koruyun.
3. İkinci metne eklenmesi ve ikinci metinden çıkarılması gereken toplam harf adetlerini farklardan hesaplayın.

066 Örtüşen alt metin eşleşmelerini bulma

Düzey: İleri Önkoşullar: 042, 064, 065

Kavramlar: Aday başlangıç, İç karşılaştırma döngüsü, Örtüşen eşleşme, Başarısız aday, Karşılaştırma sayımı

Problem tanımı

Bir metin içinde verilen desenin bütün başlangıç dizinlerini bulun. Karakterleri birebir karşılaştırın; büyük/küçük harf ve boşluk anlamlıdır. Örtüşen eşleşmeleri de sayın. Örneğin aaaa içinde aa deseni 0, 1 ve 2 dizinlerinde bulunur. Bir eşleşmeden sonra desen uzunluğu kadar atlamak bu sonucu eksik bırakır.

Metin 0..200, desen 1..50 karakter uzunluğundadır; iki girdi de yalnız yazdırılabilir ASCII 32..126 karakterlerini içerir. Boş metin geçerlidir, boş desen geçersizdir. Her uygun başlangıçta deseni soldan sağa karşılaştırın ve ilk farkta o adayı bırakın. Gerçek karakter eşitliği değerlendirmelerini sayın; ilk farklı karakter de sayılır. IndexOf, Contains, Substring, düzenli ifade veya hazır arama aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 yazdırılabilir ASCII karakteri.
Girdi	desen	1..50 yazdırılabilir ASCII karakteri; ayrı satır.
Çıktı	Dizinler	Dizinler: ardından artan dizinler; bulunamazsa Dizinler: Yok.
Çıktı	Sayaçlar	Eşleşme: N, ardından Karşılaştırma: C.

İstemler Metin: ve Desen: olur. İlk giriş hatalıysa desen istenmez. Metin hatasında Hata: Metin 0..200 yazdırılabilir ASCII karakteri içermelidir., desen hatasında Hata: Desen 1..50 yazdırılabilir ASCII karakteri içermelidir. yazılır. EOF ilgili alanın hatasıdır. İki giriş doğru olmadan arama sonucu yazılmaz.

Algoritma

1. Metni ve deseni kendi uzunluk/karakter kurallarıyla doğrulayın.
2. Eşleşme ve karşılaştırma sayaçlarını 0 yapın; dizin sonuçları için tampon ayırın.
3. Aday başlangıcını 0'dan metin.Length - desen.Length dahil son başlangıca ilerletin.
4. Desen dizinini 0 yapın; her karakter eşitliği değerlendirmesinden önce sayacı artırın.
5. İlk farklı karakterde iç aramayı bitirin; eşitlikte desen dizinini artırın.
6. Desenin tamamı eşitse aday başlangıcını sonuç tamponuna ekleyin.
7. Adayı her durumda yalnız 1 artırarak sürdürün; sonunda bütün dizinleri ve sayaçları yazdırın.

Her adayın yalnız karşılaştırılmış desen başlangıcı eşittir. İlk fark o adayı eler; tamamının eşitliği bir eşleşmedir. Son uygun başlangıç sınırı $i + j$ erişiminin metin içinde kalmasını sağlar. Her başlangıç bir kez denenir; bu yüzden örtüşmeler korunur ve dizinler artan sıradadır. En fazla 200 eşleşme vardır; gevşek karşılaştırma üst sınırı $200 \times 50 = 10000$ 'dir.

ababa ve aba için:

Başlangıç	Bu adayda karşılaştırma	Eşleşme
0	3	Evet
1	1; b ile a farklı	Hayır
2	3	Evet

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    if (metin[i] < ' ' || metin[i] > '~')
    {
        Console.WriteLine("Hata: Metin 0..200 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}

Console.Write("Desen: ");
string? desen = Console.ReadLine();
if (desen is null || desen.Length < 1 || desen.Length > 50)
{
    Console.WriteLine("Hata: Desen 1..50 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < desen.Length; i++)
{
    if (desen[i] < ' ' || desen[i] > '~')
    {
        Console.WriteLine("Hata: Desen 1..50 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}
```

```

int[] dizinler = new int[metin.Length];
int eslesme = 0, karsilastirma = 0;
for (int i = 0; i <= metin.Length - desen.Length; i++)
{
    int j = 0;
    while (j < desen.Length)
    {
        karsilastirma++;
        if (metin[i + j] != desen[j])
        {
            break;
        }
        j++;
    }
    if (j == desen.Length)
    {
        dizinler[eslesme++] = i;
    }
}

Console.Write("Dizinler:");
if (eslesme == 0)
{
    Console.Write(" Yok");
}
else
{
    for (int i = 0; i < eslesme; i++)
    {
        Console.Write(" " + dizinler[i].ToString(CultureInfo.InvariantCulture));
    }
}
Console.WriteLine();
Console.WriteLine("Eşleşme: " + eslesme.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Karşılaştırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Desen metinden uzunsa son başlangıç negatiftir ve aday döngüsü çalışmaz. Sonuç tamponunun kapasitesi metin uzunluğudur, ancak yalnız eslesme kadar hücre yazdırılır. Sayaç desen/metin uzunluğu karşılaştırmalarını içermez.

Örnek çalıştırmalar

Metin ve desen ayrı satırlarda girilir; bloklar istemler dışındaki tam çıktıdır.

aaaa ve aa:

```

Dizinler: 0 1 2
Eşleşme: 3
Karşılaştırma: 6

```

ababa ve aba:

```

Dizinler: 0 2
Eşleşme: 2
Karşılaştırma: 7

```

abc ve d:

```

Dizinler: Yok
Eşleşme: 0
Karşılaştırma: 3

```

abc ve abcd; boş metin ve a için de aynı tam çıktı:

Dizinler: Yok

Eşleşme: 0

Karşılaştırma: 0

Girdi	Beklenen özellik
200 adet a ve 50 adet a	Dizinler 0..150; eşleşme 151, karşılaştırma 7550.
200 adet a ve a	200 eşleşme ve 200 karşılaştırma.
Tek boşluk metni ve tek boşluk deseni	Dizin 0; eşleşme 1 ve karşılaştırma 1.
Hatalı metin veya desen; desen için boş satır	Yalnız ilgili hata satırı.

Sınır durumları

- Boş metin geçerli arama alanıdır; hiç aday yoktur.
- Boş desenin eşleşme tanımı bu problemin dışında bırakılmıştır.
- Büyük/küçük harf ayrımı vardır; A ile a eşit değildir.
- Son uygun başlangıç döngüye dahildir.
- Eşleşmeden sonra 1 ilerlemek örtüşen sonuçları korur.
- İlk farkta durmak yalnız mevcut adayı eler; dış arama sürer.
- Hatalı ikinci girişten önce hiçbir eşleşme veya sayaç yazılmaz.

Kazanımlar

- Başlangıç konumu ve desen içi konumu birlikte yönetme.
- Pencere sınırından güvenli karakter erişimi türetme.
- Örtüşen eşleşme ile ayrık eşleşmenin farkını örnekleme.
- İç döngüde erken çıkışın dış döngüyü sonlandırmadığını gösterme.
- Aynı karar için yapılan gerçek karşılaştırmaları sayma.

Alıştırmalar

1. Örtüşmeyen eşleşmeleri bulan sürümü yazın; aaaa ve aa için 0, 2 sonucunu doğrulayın.
2. Yalnız ilk eşleşmeyi bulan sürümde erken çıkışı iki döngü arasında yönetin.
3. ASCII büyük/küçük harf farkını yok sayın; noktalama ve boşluk eşitliğini koruyun.

067 Ardışık harfleri adetleriyle kodlama

Düzey: İleri Önkoşullar: 059, 063, 066

Kavramlar: Ardışık grup, Kodlama kuralı, Çok basamaklı adet, Çıktı uzunluğu, Sıkıştırma sınırı

Problem tanımı

Yalnız A–Z ve a–z harflerinden oluşan, en fazla 200 karakterlik satırı ardışık grup uzunluklarıyla kodlayın. Her grubun harfini hemen ardından pozitif ondalık adediyle yazın; adet 1 olsa da yazılmalıdır. Büyük/küçük harf farklıdır. Örneğin AAAbbA sonucu A3b2A1 olur. Grup sayısını ve kodun karakter uzunluğunu da bildirin.

Boş satır geçerlidir ve boş kod üretir. Rakam, boşluk veya diğer karakterler giriş değildir; böylece harf ile adet sınırı belirsiz olmaz. Grupların sırası korunur; aba içindeki iki a birleştirilmez. Bu temsil ardışık tekrarları kısaltabilir, ancak her girdiyi küçültmez: ab sonucu a1b1 dört karakterdir. Hazır gruplama veya sıkıştırma aracı kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 ASCII harfi; harf büyüklüğü korunur.
Çıktı	Kod	Kod: ardından dikey çubuklar arasında harf-adet çiftleri; çubuklar koda dahil değildir.
Çıktı	Sayaçlar	Gruplar: G, ardından Kod uzunluğu: L; ayraçlar L'ye dahil değildir.

Metin: istemini tek satırla yanıtlayın. EOF, fazla uzunluk, rakam, boşluk, Türkçe harf veya diğer karakter için yalnız Hata: Metin 0..200 ASCII harfi içermelidir. yazılır. Satırın tamamı kod çıktısı başlamadan önce doğrulanır.

Algoritma

1. Satırı ve bütün karakterlerini doğrulayın.
2. Tarama dizini, grup sayısı ve kod uzunluğunu 0 yapın.
3. Mevcut harfi saklayın; aynı harf sürerken adedi ve dizini artırın.
4. Harfi ve grubun adedini yan yana yazın; grup sayısını artırın.
5. Adedin bir kopyasını 10'a bölerek basamak adedini bulun; kod uzunluğuna 1 harf ve basamak adedini ekleyin.
6. Sonraki farklı harfle yeni gruba başlayın; giriş bitince kodu kapatıp sayaçları yazdırın.

Grup taramasında yalnız mevcut harfe eşit karakterler tüketilir; ilk farklı karakter sonraki grubun başlangıcı olarak bırakılır. Her karakter bir gruba girer, grup adetlerinin toplamı giriş uzunluğudur. Bir grup kodda 1 harf ve adedin ondalık basamak sayısı kadar karakter üretir. Tarama dizini sürekli arttığı için sonlanır.

Adet 9'dan 10'a veya 99'dan 100'e çıktığında koddaki basamak adedi bir artar. Bu nedenle grup sayısı aynı kalsa da kod uzunluğu değişebilir. Grup adedi ile kodun karakter uzunluğunu ayrı sayaçlarda tutmak bu iki bilgiyi karıştırmayı önler.

AAAbba için:

Grup	Adet	Kod parçası	Birikimli kod uzunluğu
A	3	A3	2
b	2	b2	4
A	1	A1	6

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 ASCII harfi içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (!((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')))
    {
        Console.WriteLine("Hata: Metin 0..200 ASCII harfi içermelidir.");
        return;
    }
}

int dizin = 0, grup = 0, kodUzunlugu = 0;

Console.WriteLine("Kod: |");

while (dizin < metin.Length)
{
    char harf = metin[dizin];
    int adet = 0;
    while (dizin < metin.Length && metin[dizin] == harf)
    {
        adet++;
        dizin++;
    }
    Console.WriteLine(harf);
    Console.WriteLine(adet.ToString(CultureInfo.InvariantCulture));
    grup++;
}
```

```

int kopya = adet, basamak = 1;
while (kopya >= 10)
{
    kopya /= 10;
    basamak++;
}
kodUzunlugu += 1 + basamak;
}
Console.WriteLine("|");
Console.WriteLine("Gruplar: " + grup.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Kod uzunluđu: " +
    kodUzunlugu.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Doğrulama ayrı bir ilk geçiş olduğu için sonradan görülen hatalı karakter yarım kod çıktısı üretmez. Sayıların ToString kullanımı yalnız çıktı biçimlendirmesidir; grup bulma ve uzunluk hesabı döngülerle yapılır. En uzun kod 200 tekli grup için 400 karakterdir.

Örnek çalıştırmalar

Bloklar Metin: isteminden sonraki tam sonuçtur.

AAAbbA:

```

Kod: |A3b2A1|
Gruplar: 3
Kod uzunluđu: 6

```

abab:

```

Kod: |a1b1a1b1|
Gruplar: 4
Kod uzunluđu: 8

```

11 adet A:

```

Kod: |A11|
Gruplar: 1
Kod uzunluđu: 3

```

Boş satır:

```

Kod: ||
Gruplar: 0
Kod uzunluđu: 0

```

Girdi	Beklenen özellik
200 adet z	Kod z200; 1 grup, kod uzunluđu 4.
a ve b dönüşümlü 200 harf	200 grup, kod uzunluđu 400; temsil daha uzun.
Aa	A1a1; büyük/küçük harf iki ayrı grup.
A1, boşluk, ç, 201 karakter veya EOF	Yalnız tanımlanan hata satırı; kod başlangıcı bile yazılmaz.

Sınır durumları

- Adet 1 için rakam atlanmaz; açma kuralı bunu gerektirir.
- Boş kod ile hatalı giriş farklı durumlardır.
- Aynı harfin ayrı grupları yer değiştirmez veya birleştirilmez.
- 9, 10, 99, 100 ve 200 adetlerinde kod uzunluğu basamak sayısına bağlıdır.
- Kod ayraçları ve başlıklar ölçülen kod uzunluğuna dahil değildir.
- Bu temsil her metinde daha az yer kullanma garantisi vermez.

Kazanımlar

- Önceki grup sayma yaklaşımını sıralama yapmadan ardışık veriye uygulama.
- Bir kodlama biçiminin açık ve çözülebilir kurallarını tanımlama.
- Grubun bitiş karakterini tüketmeden sonraki gruba bırakma.
- Çıktı uzunluğunu algoritmanın ürettiği bileşenlerden hesaplama.
- Tekrar biçiminin sıkıştırmanın yararını nasıl değiştirdiğini açıklama.

Alıştırmalar

1. Giriş uzunluğu ile kod uzunluğu arasındaki farkı yazdırın; farkın negatif olabildiğini gösterin.
2. Kod yerine her grubun başlangıç dizini, harfi ve adedini birer satırda yazdırın.
3. A ile a'yı aynı kabul eden sürüm yazın; bunun özgün büyük/küçük harf bilgisini neden kaybettirdiğini açıklayın.

068 Metni taşma denetimiyle elle tamsayıya dönüştürme

Düzey: İleri Önkoşullar: 022, 057, 061, 067

Kavramlar: Giriş dilbilgisi, İsteğe bağlı işaret, Basamak biriktirme, Çarpmadan önce sınır, Asimetrik int aralığı

Problem tanımı

Bir satırı hazır sayı dönüştürme aracı kullanmadan C# int değerine dönüştürün. Kabul edilen biçim, isteğe bağlı tek + veya - işaretinin ardından en az bir ASCII rakamıdır. Satır 1..11 karakter olmalıdır; boşluk, ondalık ayraç ve başka karakter kabul edilmez. Baştaki sıfırlar geçerlidir. Örneğin +0042 sonucu 42, -0 sonucu 0 olur.

Sayısal değer -2147483648..2147483647 aralığında olmalıdır. Negatif sınırın büyüklüğü pozitif sınırdan 1 fazladır. Basamakları negatif olmayan long büyüklükte biriktirin; işarete göre sınırı belirleyin ve buyukluk * 10 + basamak işleminden önce yeni değerın sınırı aşmayacağını doğrulayın. Parse, TryParse, Convert, hazır sayısal ayrıştırma veya taşmayı yakalamak için istisna kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	1..11 karakter; belirtilen işaret ve rakam biçimi.
Ara değer	buyukluk	İşaretsiz sayısal büyüklük; long, en fazla 2147483648.
Çıktı	Tamsayı	Tek satır Değer : X; işaret ve baştaki sıfırlar normalleşir.

Tamsayı metni: istemini bir satırla yanıtlayın. EOF, boş satır, yalnız işaret, uzunluk ihlali, biçim ihlali veya int aralığı dışındaki değer için yalnız Hata: Metin geçerli bir int tamsayı belirtmelidir. yazılır. Girdinin tamamı doğru olmadan değer yazılmaz.

Algoritma

1. Satırı okuyup 1..11 uzunluğunu doğrulayın.
2. İlk karakter işaretse negatiflik durumunu belirleyip rakam başlangıcını 1 yapın.
3. İşareten sonra hiç karakter kalmadıysa hata verin.
4. Sınırı negatif girişte 2147483648, diğer girişte 2147483647; büyüklüğü 0 yapın.
5. Her karakter için 0..9 rakamını doğrulayın ve kod farkından basamağı hesaplayın.
6. Büyüklük (sınır - basamak) / 10 değerinden büyükse çarpmadan önce hata verin.
7. Aksi halde büyüklüğü 10 ile çarpıp basamağı ekleyin; bütün basamaklar bitince işareti uygulayıp int sonucu yazın.

Kullanılan eşitsizlik, negatif olmayan sayılarda $10 * buyukluk + basamak \leq sınır$ koşuluyla eşdeğerdir. Tamsayı bölümünün aşağı yuvarlaması sınırı doğru daraltır. Her adımda büyüklük kabul edilen sınırdan kalır; son işaret uygulaması long üzerinde yapıldığı için en küçük int güvenle oluşur.

-2147483648 için son iki basamak:

Yeni basamak	Önceki büyüklük	İzin verilen en büyük önceki değer	Yeni büyüklük
4	21474836	214748364	214748364
8	214748364	214748364	2147483648

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Tamsayı metni: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length < 1 || metin.Length > 11)
{
    Console.WriteLine("Hata: Metin geçerli bir int tamsayı belirtmelidir.");
    return;
}

int baslangic = 0;
bool negatif = false;
if (metin[0] == '+' || metin[0] == '-')
{
    negatif = metin[0] == '-';
    baslangic = 1;
}
if (baslangic == metin.Length)
{
    Console.WriteLine("Hata: Metin geçerli bir int tamsayı belirtmelidir.");
    return;
}

long sinir = negatif ? 2147483648L : 2147483647L;
long buyukluk = 0;
for (int i = baslangic; i < metin.Length; i++)
{
    char c = metin[i];
    if (c < '0' || c > '9')
    {
        Console.WriteLine("Hata: Metin geçerli bir int tamsayı belirtmelidir.");
        return;
    }
    int basamak = c - '0';
    if (buyukluk > (sinir - basamak) / 10)
    {
        Console.WriteLine("Hata: Metin geçerli bir int tamsayı belirtmelidir.");
        return;
    }
    buyukluk = buyukluk * 10 + basamak;
}

long isaretli = negatif ? -buyukluk : buyukluk;
int sonuc = (int)isaretli;
Console.WriteLine("Değer: " + sonuc.ToString(CultureInfo.InvariantCulture));
```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Son int dönüşümü ancak bütün rakamlar ve aralık doğrulandıktan sonra yapılır. ToString yalnız sonucu yazdırır, giriş ayırtmaz.

Önceki derslerde kullanılan TryParse içindeki temel basamak ve aralık mantığı burada açık dögüyle görünür hale gelir; bu örnek o aracın bütün kabul kurallarını taklit etmez.

Örnek çalıştırmalar

Bloklar istemden sonraki tek ve tam sonuç satırıdır.

+0042:

Değer: 42

-2147483648:

Değer: -2147483648

2147483647:

Değer: 2147483647

-0:

Değer: 0

2147483648, -2147483649, 12a, 1.5, +, boş satır veya EOF:

Hata: Metin geçerli bir int tamsayı belirtmelidir.

Girdi	Beklenen özellik
+2147483647	İşaret dahil 11 karakter; pozitif üst sınır geçerli.
00000000000	11 karakter, değeri 0; geçerli.
000000000000	Değeri küçük olsa da 12 karakter uzunluk kuralını bozar.
Başta/sonda boşluk, çift işaret, Arapça rakam veya sekme	Biçim hatası; yalnız hata satırı.

Sınır durumları

- İşaret sadece ilk konumda ve en fazla bir kez bulunabilir.
- İşaretin ardından en az bir rakam gerekir.
- Negatif sınırın büyüklüğü int içinde pozitif olarak temsil edilemez; long kullanılır.
- -0, +0 ve başında sıfırlar bulunan geçerli biçimler aynı değeri verir.
- Baştaki sıfırlar uzunluk sınırını kaldırmaz.
- Taşma denetimi çarpma ve toplama işleminden önce yapılır.
- Hatalı son karakter önceki basamakların kısmi değerini çıktı yapmaz.

Kazanımlar

- Metin biçimini işaret ve rakam bölümlerinden oluşan bir giriş dili olarak tanımlama.
- Karakter kod farkını sayısal basamağa dönüştürme.
- Aritmetik sınır koşulunu işlem öncesine taşıma.
- Pozitif ve negatif tamsayı sınırlarının asimetrisini yönetme.
- Biçim doğrulaması, aralık doğrulaması ve çıktı biçimini ayırma.

Alıřtırmalar

1. Hata nedenini boş giriř, biçim ve aralık olarak ayrı mesajlarla bildirin.
2. İşaret dışındaki en fazla rakam adedini ayrıca sınırlayın; baştaki sıfırların davranışını açıklayın.
3. -9999..9999 aralığını kullanan sürümü yazın; sınır eşitsizliğini yeni aralığa uyarlayın.

069 Harf ve adet kodunu doğrulayarak açma

Düzey: İleri Önkoşullar: 062, 067, 068

Kavramlar: Harf-adet dilbilgisi, Değişken uzunluklu alan, İşlem öncesi sınır, Toplam kapasite, Kısmi çıktıyı önleme

Problem tanımı

067'deki harf-adet biçimini açın. Kod, bir ASCII harfi ve hemen ardından pozitif ondalık adetten oluşan çiftlerin art arda yazılmasıdır. Örneğin A3b2A1 sonucu AAAbbA olur. Adet en az 1, en fazla 200'dür ve başında sıfır olamaz. Kod satırı 0..400 karakter, açılmış metin en fazla 200 karakter olmalıdır.

Boş kod geçerlidir ve boş metin üretir. Rakamlar yalnız 0–9, harfler yalnız A–Z ve a–z'dir; boşluk, işaret veya başka karakter kabul edilmez. Aynı harfin ardışık çiftlerde bulunması geçerlidir: A1A1 sonucu AA olur; kodun en kısa biçimde yazılması şart değildir. Çok basamaklı adedi elle ayrıştırın. Parse, TryParse, düzenli ifade, hazır tekrar veya açma aracı kullanmayın.

Girişin sonundaki hata daha önce açılmış bölümü çıktı yapmamalıdır. Bu nedenle sonucu 200 karakterlik tamponda hazırlayın; ancak bütün çiftler doğru ve toplam kapasite yeterliyse yazdırın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	kod	0..400 karakter; tanımlanan harf-adet çiftleri.
Ara değer	adet, yazma	Mevcut grubun adedi ve toplam üretilmiş karakter sayısı.
Çıktı	Açılmış metin	Metin: ardından dikey çubuklar arasında sonuç; çubuklar metne dahil değildir.
Çıktı	Uzunluk	Uzunluk: L; ayraçlar L'ye dahil değildir.

Kod: isteminde yalnız kodu girin; 067 çıktısındaki başlık ve çubuklar giriş değildir. EOF, uzunluk ihlali, yanlış çift biçimi, sıfır/başında sıfırlı adet, 200'den büyük adet veya toplamın 200'ü aşması için yalnız Hata: Kod geçersiz veya açılmış metin 200 karakteri aşıyor. yazılır.

Algoritma

1. Kod satırını okuyup 0..400 uzunluğunu doğrulayın; 200 karakterlik tampon ayırın.
2. Kod dizini ve yazma konumunu 0 yapın.
3. Sıradaki karakteri ASCII harfi olarak doğrulayıp saklayın; dizini ilerletin.
4. Sonraki karakterin 1..9 rakamı olduğunu denetleyin; eksik veya başında sıfırlı adedi reddedin.
5. Ardışık rakamları tüketin; her basamak öncesinde $adet \leq (200 - basamak) / 10$ sınırını denetleyip adedi biriktirin.
6. Adet kalan tampon kapasitesinden büyükse hata verin; aksi halde harfi adet kadar tampona yazın.
7. Sonraki harf-adet çiftini işleyin; bütün kod bitince geçerli tampon bölümünü ve uzunluğu yazdırın.

Her çift sonrası tampon, o ana kadar doğrulanmış kodun tam açılımıdır. Adet sınırı ve toplam kapasite sınırı farklı denetimlerdir: iki ayrı 150'lik grup tek başlarına geçerli ama birlikte fazla uzundur. Tamponun hiçbir yazımı 200. dizine ulaşmaz. Kod dizini harfi ve bütün rakamlarını tüketir; sonraki karakter bir sonraki çiftin başlangıcıdır.

A3b2A1 için:

Çift	Adet	Yazma konumu sonrası	Tamponun geçerli bölümü
A3	3	3	AAA
b2	2	5	AAAbb
A1	1	6	AAAbbA

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Kod: ");
string? kod = Console.ReadLine();
if (kod is null || kod.Length > 400)
{
    Console.WriteLine("Hata: Kod geçersiz veya açılmış metin " +
        "200 karakteri aşıyor.");
    return;
}

char[] tampon = new char[200];
int dizin = 0, yazma = 0;
while (dizin < kod.Length)
{
    char harf = kod[dizin];
    if (!((harf >= 'A' && harf <= 'Z') || (harf >= 'a' && harf <= 'z')))
    {
        Console.WriteLine("Hata: Kod geçersiz veya açılmış metin " +
            "200 karakteri aşıyor.");
        return;
    }
    dizin++;
    if (dizin == kod.Length || kod[dizin] < '1' || kod[dizin] > '9')
    {
        Console.WriteLine("Hata: Kod geçersiz veya açılmış metin " +
            "200 karakteri aşıyor.");
        return;
    }

    int adet = 0;
    while (dizin < kod.Length && kod[dizin] >= '0' && kod[dizin] <= '9')
    {
        int basamak = kod[dizin] - '0';
        if (adet > (200 - basamak) / 10)
        {
            Console.WriteLine("Hata: Kod geçersiz veya açılmış metin " +
                "200 karakteri aşıyor.");
            return;
        }
        adet = adet * 10 + basamak;
        dizin++;
    }
}
```

```

    if (adet > tampon.Length - yazma)
    {
        Console.WriteLine("Hata: Kod geçersiz veya açılmış metin " +
            "200 karakteri aşıyor.");
        return;
    }
    for (int i = 0; i < adet; i++)
    {
        tampon[yazma++] = harf;
    }
}

string sonuc = new string(tampon, 0, yazma);
Console.WriteLine("Metin: |" + sonuc + "|");
Console.WriteLine("Uzunluk: " + yazma.ToString(CultureInfo.InvariantCulture));

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. `dizin == kod.Length || ...` kısa devre koşulu eksik adette dizi dışı erişimi önler. Başlangıç rakamı 1..9 kuralı A0 ve A01 biçimlerini reddeder; sonraki rakamlar 0 olabilir. Tampon, sonradan hata görülse bile yalnız bellekte kalır; başarılı sonuç ancak dış döngü bittikten sonra yazılır.

Örnek çalıştırmalar

Bloklar Kod: isteminden sonraki tam sonuçtur.

A3b2A1:

```

Metin: |AAAbbA|
Uzunluk: 6

```

a1b1a1b1:

```

Metin: |abab|
Uzunluk: 4

```

A1A1:

```

Metin: |AA|
Uzunluk: 2

```

Boş satır:

```

Metin: ||
Uzunluk: 0

```

A0, A01, A, 3A, A201, A200B1 veya EOF:

Hata: Kod geçersiz veya açılmış metin 200 karakteri aşıyor.

Girdi	Beklenen özellik
z200	200 adet z; uzunluk 200.
A100b100	İki grup, toplam 200; geçerli.
a1 çiftinin 200 tekrarı	Kod uzunluğu 400, sonuç 200 adet a; geçerli.
A150b150	Her adet geçerli; toplam uzunluk nedeniyle hata.
A2b, A2!, A2b01	Sonraki çift hatalı; AA şeklinde kısmi çıktı yok.
A ardından 399 adet 9; 401 karakter; sekme veya Türkçe harf	Hata; büyük sayıyı biriktirirken taşma yapılmaz.

Sınır durumları

- Boş kod tek geçerli harfsiz girdidir; EOF değildir.
- Her harfin ardından en az bir rakam gerekir.
- Adet 0 ve başında sıfırlı biçimler geçersizdir; adet 10 ve 100 geçerlidir.
- Harfin büyüklüğü açılmış metinde korunur.
- Aynı harfin komşu çiftlerde bulunması hata değildir.
- Grup adedi uygun olsa da toplam kapasite aşılabılır.
- 067 ile kodlanıp bu algoritmayla açılan kabul edilmiş metin aynen geri gelir.
- Her geçerli kodu açıp yeniden kodlamak özgün kodu vermeyebilir; A1A1 yeniden A2 olur.

Kazanımlar

- Uzunlukları değişen harf ve sayı alanlarını ortak dizinle ayrıştırma.
- Alan sınırı ile toplam çıktı kapasitesini ayrı doğrulama.
- Çarpmadan önce sınır denetimini daha küçük bir sayı alanına uyarlama.
- Kısmi çalışma sonucunu kullanıcıya yazmadan hata yönetme.
- Kodlama ve açma ilişkisini iki yönün farklı özellikleriyle açıklama.

Alıştırmalar

1. Hatalı çiftin başlangıç dizinini hata mesajına ekleyin; çok basamaklı adetlerde bu dizini koruyun.
2. Aynı harfin komşu çiftlerde bulunmasını reddeden sürüm yazın; A1A1 ile A1a1 davranışını ayırın.
3. Önce yalnız doğrulayıp toplam uzunluğu hesaplayan, sonra tam uzunlukta tampon ayırarak açan iki geçişli sürüm hazırlayın.

070 Yığınla farklı parantez türlerini eşleştirme

Düzey: İleri Önkoşullar: 050, 064, 069

Kavramlar: Diziyle yığın, Son giren ilk çıkar, İç içe eşleşme, İlk hata konumu, En yüksek derinlik

Problem tanımı

En fazla 200 yazdırılabilir ASCII karakteri içeren satırda (), [] ve { } parantezlerinin doğru tür ve sırayla eşleşmesini denetleyin. Açılan son parantez önce kapanmalıdır. Dengeli girdide en yüksek iç içe parantez derinliğini yazdırın. Dengesiz girdide ilk sorunlu dizini ve nedenini bildirin. Örneğin ([)] içinde adetler eşit olsa da 2. dizindeki) en son açılmış [ile uyuşmaz.

Bu problem yalnız parantez eşleştirmesidir: parantez olmayan bütün kabul edilmiş karakterler yok sayılır. Tırnak, yorum veya kaçış dizileri için ayrı dil kuralı yoktur; tırnak içindeki parantezler de denetlenir. Boş satır ve hiç parantez içermeyen satır dengeli, derinliği 0 kabul edilir. Hazır Stack, düzenli ifade veya ifade ayrıştırıcı kullanmayın; char dizisi ve geçerli eleman adediyle yığın kurun.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 karakter; ASCII 32..126.
Ara değer	yığın, tepe	Henüz kapanmamış açılışlar ve geçerli eleman adedi.
Çıktı	Dengeli	Dengeli: Evet, ardından En yüksek derinlik: D.
Çıktı	Dengesiz	Dengeli: Hayır, Hata dizini: I, Neden: açıklama.

İfade: istemini tek satırla yanıtlayın. Bir kapanış için yığın boşsa neden Açılış yok, tepesi başka türdeyse Tür uyuşmazlığı olur; dizin o kapanışın özgün sıfır tabanlı konumudur. Giriş bittiğinde açık parantez varsa neden Kapanış eksik, hata dizini metin uzunluğudur; bu değer giriş sonu konumunu belirtir ve karakter erişiminde kullanılmaz.

EOF, uzunluk ihlali, sekme, Türkçe harf veya diğer kabul edilmeyen karakter için yalnız Hata: İfade 0..200 yazdırılabilir ASCII karakteri içermelidir. yazılır. Bu giriş hatası, geçerli karakterlerden oluşan dengesiz ifadenin normal sonucundan ayrıdır. Girdinin tamamı önce doğrulanır.

Algoritma

1. Satırı, uzunluğunu ve bütün karakterlerini doğrulayın.
2. Giriş uzunluğunda yığın dizisi oluşturun; geçerli adet ve en yüksek derinliği 0 yapın.
3. Açılış karakterini yığının sonraki boş hücreğine yazıp adedi artırın; en yüksek adedi güncelleyin.
4. Kapanış karakterinde yığın boşsa açılış yok sonucuyla taramayı durdurun.
5. Beklenen açılış türünü hesaplayın; tepedeki tür farklıysa uyuşmazlık sonucuyla durdurun.
6. Tür doğruysa geçerli adedi 1 azaltıp son açılışı çıkarın; diğer karakterleri atlayın.
7. Tarama hatasız bittiğinde yığında eleman kaldıysa giriş sonu için kapanış eksik sonucunu üretin.
8. Hata durumunu veya dengeli kararını ve en yüksek derinliği yazdırın.

Her adım başında yığında, işlenmiş bölümün henüz kapanmamış açılışları giriş sırasıyla bulunur. Doğru kapanış yalnız en son açılışı tüketir; farklı bir alt açılışı seçmek iç içe düzeni bozar. tepe

kapasite değil geçerli adettir; boş değilken üst karakter yığın[tepe - 1] olur. Bir karakter en fazla bir kez yığına girer; giriş uzunluğunda kapasite yeterlidir.

([])] için:

Dizin	Karakter	Yığın; soldan sağa alt-üst	İşlem
0	(	(	Ekle; derinlik 1
1	[	([	Ekle; derinlik 2
2	)	([	Tepedeki [uymaz; dur

C# çözümü

```
using System;
using System.Globalization;

Console.Write("İfade: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: İfade 0..200 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    if (metin[i] < ' ' || metin[i] > '~')
    {
        Console.WriteLine("Hata: İfade 0..200 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}

char[] yigin = new char[metin.Length];
int tepe = 0, enYuksek = 0, hataDizini = -1;
string neden = "";
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (c == '(' || c == '[' || c == '{')
    {
        yigin[tepe++] = c;
        if (tepe > enYuksek)
        {
            enYuksek = tepe;
        }
    }
    else if (c == ')' || c == ']' || c == '}')
    {
        if (tepe == 0)
        {
            hataDizini = i;
            neden = "Açılış yok";
            break;
        }
    }
}
```

```

        char beklenen = c == ')' ? '(' : c == ']' ? '[' : '{';
        if (yigin[tepe - 1] != beklenen)
        {
            hataDizini = i;
            neden = "Tür uyuşmazlığı";
            break;
        }
        tepe--;
    }
}

if (hataDizini == -1 && tepe > 0)
{
    hataDizini = metin.Length;
    neden = "Kapanış eksik";
}
if (hataDizini == -1)
{
    Console.WriteLine("Dengeli: Evet");
    Console.WriteLine("En yüksek derinlik: " +
        enYuksek.ToString(CultureInfo.InvariantCulture));
}
else
{
    Console.WriteLine("Dengeli: Hayır");
    Console.WriteLine("Hata dizini: " +
        hataDizini.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Neden: " + neden);
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Çıkarma eski hücreyi silmez; tepe azaldığı için o hücre geçerli yığının dışına çıkar. Sonraki ekleme aynı hücreyi yeniden kullanabilir. `hataDizini == -1` kontrolü, ilk kapanış hatasının giriş sonu hatasıyla değiştirilmesini önler. Dengesiz sonuçta derinlik satırı yazılmaz.

Örnek çalıştırmalar

Bloklar İfade: isteminden sonraki tam sonuçtur.

`(a+[b*c]):`

```

Dengeli: Evet
En yüksek derinlik: 2

```

`([])`:

```

Dengeli: Hayır
Hata dizini: 2
Neden: Tür uyuşmazlığı

```

`a)`:

```

Dengeli: Hayır
Hata dizini: 1
Neden: Açılış yok

```

`(a`:

```

Dengeli: Hayır
Hata dizini: 2
Neden: Kapanış eksik

```

Boş satır veya `abc 123!`:

```

Dengeli: Evet
En yüksek derinlik: 0

```

Girdi	Beklenen özellik
()[]{}	Dengeli; en yüksek derinlik 1.
100 adet (ardından 100 adet)	Dengeli; en yüksek derinlik 100.
200 adet (	Kapanış eksik; hata dizini 200, kapasite aşılmaz.
([) ve ardından sekme	Karakter doğrulama hatası; önceki uyumsuzluk sonucu yazılmaz.
201 karakter, Türkçe harf veya EOF	Yalnız tanımlanan giriş hata satırı.

Sınır durumları

- Eşit açılış/kapanış adetleri doğru tür ve sıra için yeterli değildir.
- Boş yığında tepe - 1 erişimi yapılmaz.
- Sonradan gelen başka hatalar ilk kapanış hatasını değiştirmez.
- Giriş sonu dizini bir karakterin dizini değildir.
- Boş veya parantezsiz geçerli satır dengelidir.
- Yığında kalan eski hücreler geçerli eleman adedine dahil edilmez.
- Tırnak ve yorum ayrıştırılması bu problemin kuralı değildir; tüm parantezler aynı değerlendirilir.
- En yüksek derinlik anlık açık adetlerinin en büyüğüdür; toplam çift sayısı değildir.

Kazanımlar

- Son giren ilk çıkar davranışını hazır koleksiyon olmadan diziyle gerçekleştirme.
- Fiziksel kapasite, geçerli adet ve tepe dizinini ayırt etme.
- Birden fazla parantez türünün iç içe düzenini koruma.
- Geçersiz giriş ile geçerli fakat dengesiz ifade sonucunu ayırma.
- İlk hata konumunu ve giriş sonu hatasını farklı anlamlarla raporlama.

Alıştırmalar

1. Yığına açılışların özgün dizinlerini de koyun; kapanış eksikse en son açık parantezin dizinini ayrıca yazdırın.
2. Dengeli ifadede toplam eşleşmiş çift sayısını sayın; derinlikle aynı olmadığını ()() ve (()) ile gösterin.
3. Yalnız tek tür parantez için bir sayaçla çalışan sürüm yazın; bunun ([)] durumunu neden çözemediğini açıklayın.

Bölüm 8: Metotlar ve özyineleme

071 Metotlarla aralıktaki asal sayıları özetleme

Düzy: İleri Önkoşullar: 027, 037, 060

Kavramlar: Metot sözleşmesi, Parametre, Dönüş değeri, Yerel değişken, Yan etkisiz hesaplama

Problem tanımı

İki uç dahil olmak üzere verilen aralıktaki asal sayıların adedini, toplamını, ilkin ve sonuncusunu bulun. Uçlar 0..10000 arasında olmalı ve alt uç üst uçtan büyük olmamalıdır. Hiç asal sayı yoksa adet ve toplam 0, ilk ve son asal Yok olur.

Asal sayı denetimini `AsalMi(int sayi)` metoduna ayırın. Metot yalnız parametresini inceleyip `bool` döndürmelidir; konsoldan veri okumamalı, sonuç yazmamalı ve dışarıdaki sayaçları değiştirmemelidir. Böylece aynı denetim aralık taramasında ve tek sayı üzerinde kullanılabilir. Önceki asal sayı algoritması korunur; yeni görev, hesaplama ile kullanıcı etkileşiminin sınırlarını belirlemektir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	alt, ust	Ayrı satırlarda 0..10000 tamsayı; alt <= ust.
Metot	AsalMi	Her int için asal ise true, diğer durumlarda false.
Çıktı	Özet	Adet: N, Toplam: T, İlk asal: X, Son asal: Y.

İstemler sırayla Alt uç: ve Üst uç: olur. İlk uç geçersizse ikinci uç istenmez. Geçersiz tamsayı, EOF veya aralık ihlaliinde yalnız Hata: Uçlar 0..10000 aralığında ve sıralı olmalıdır. yazılır. Sayısal girişte çevre boşlukları ve işaret kabul edilir; ondalıklı değer kabul edilmez. Çıktı sayıları kültürden bağımsızdır.

Algoritma

1. Uçları okuyup doğrulayın; sayaçları ve toplamı 0, ilk/son asalı -1 yapın.
2. Her adayı `AsalMi` metoduna gönderin.
3. Metotta 2'den küçük sayıyı reddedin; 2'den başlayarak olası bölenleri deneyin.
4. Bir bölen bulunursa false, bölen bulunmazsa true döndürün.
5. Asal adayda adedi ve toplamı artırın; ilk asal henüz yoksa saklayın, son asalı güncelleyin.
6. Dört sonuç satırını yazdırın.

Metodun parametresi değer olarak aktarılır; `sayi`, çağırının değişkeninden bağımsız yerel değerdir. `return`, bulunduğu metot çağırısını bitirir; aralık taraması sonraki adayla devam eder. Bölen koşulu `bolen <= sayi / bolen`, kare çarpımı oluşturmadan gerekli sınırı sağlar. Toplam `long` tutulur; tarama en fazla 10001 aday içerir.

Metot, üst düzey programın `static` yerel fonksiyonudur. `static`, dış yerel değişkenleri yakalamasını engeller; tek başına yan etkisizlik garantisi değildir. Bu örnekte yan etkisizlik, metotta konsol veya paylaşılan durum kullanılmamasıyla sağlanır. Tanımın çağrıdan sonra yazılması yürütme sırasını değiştirmez. Yeni bir sınıf tanımlamak gerekmez.

2..10 aralığındaki ilk dört aday için:

Aday	Dönüş	Adet	Toplam
2	true	1	2
3	true	2	5
4	false	2	5
5	true	3	10

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Alt uç: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int alt) || alt < 0 || alt > 10000)
{
    Console.WriteLine("Hata: Uçlar 0..10000 aralığında ve sıralı olmalıdır.");
    return;
}
Console.Write("Üst uç: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int ust) || ust < alt || ust > 10000)
{
    Console.WriteLine("Hata: Uçlar 0..10000 aralığında ve sıralı olmalıdır.");
    return;
}

int adet = 0, ilk = -1, son = -1;
long toplam = 0;
for (int aday = alt; aday <= ust; aday++)
{
    if (AsalMi(aday))
    {
        adet++;
        toplam += aday;
        if (ilk == -1)
        {
            ilk = aday;
        }
        son = aday;
    }
}

Console.WriteLine("Adet: " + adet.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("İlk asal: " +
    (ilk == -1 ? "Yok" : ilk.ToString(CultureInfo.InvariantCulture)));
Console.WriteLine("Son asal: " +
    (son == -1 ? "Yok" : son.ToString(CultureInfo.InvariantCulture)));
```

```

static bool AsalMi(int sayi)
{
    if (sayi < 2)
    {
        return false;
    }

    for (int bolen = 2; bolen <= sayi / bolen; bolen++)
    {
        if (sayi % bolen == 0)
        {
            return false;
        }
    }
    return true;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır.

Örnek çalıştırmalar

Uçları belirtilen sırayla ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

alt = 2, ust = 10:

```

Adet: 4
Toplam: 17
İlk asal: 2
Son asal: 7

```

alt = 14, ust = 16:

```

Adet: 0
Toplam: 0
İlk asal: Yok
Son asal: Yok

```

alt = 17, ust = 17:

```

Adet: 1
Toplam: 17
İlk asal: 17
Son asal: 17

```

alt = 10, ust = 2:

Hata: Uçlar 0..10000 aralığında ve sıralı olmalıdır.

Sınır durumları

- 0 ve 1 asal değildir; 0..1 aralığında bütün özet değerleri boş duruma karşılık gelir.
- Eşit uçlar geçerlidir; yalnız bir aday değerlendirilir.
- 10000 üst uçtur ve bileşik sayıdır; döngü onu da dener.
- Metot negatif bir parametre alırsa false döndürür; arayüz negatif uç kabul etmez.
- İlk adayın asal olmaması ilk asalın belirlenmesini engellemez.
- Giriş hatasında hiçbir özet satırı yazılmaz.

Kazanımlar

- Tek bir hesaplamayı parametre ve dönüş değeriyle tanımlama.
- Metot içindeki return ile ana programdaki return etkisini ayırma.
- Çağıranın sayaçlarını hesaplama metodunun dışında yönetme.
- Metot sözleşmesini arayüzün giriş kısıtından ayrı açıklama.
- Önceki algoritmayı tekrar yazmadan farklı bağlamlarda kullanma.

Alıştırmalar

1. Aralıktaki asal sayıları da yazdırın; AsalMi içine çıktı eklemeyin.
2. İki komşu asal arasındaki en büyük farkı bulun; tek asal durumunu tanımlayın.
3. AsalMi metodunu 0, 1, 2, 49 ve 97 ile doğrudan deneyin; beklenen dönüşleri açıklayın.

072 Metot zinciriyle kesri sadeleştirme

Düzey: İleri Önkoşullar: 028, 068, 071

Kavramlar: Metot bileşimi, Önkoşul, İşaret normalleştirme, Tür genişletme, Kanonik gösterim

Problem tanımı

Tam int aralığındaki pay ve sıfır olmayan paydadan oluşan kesri sadeleştirin. Son payda pozitif, pay ile payda aralarında asal olmalıdır. Sıfır paylı her geçerli kesir $0/1$ biçiminde yazılır. Kesri ondalıklı sayıya çevirmeyin; tamsayı değerini koruyun.

İşaret normalleştirmesinden sonra payın mutlak değerini Mutlak metoduyla, ortak böleni Ebob metoduyla bulun. İki metot da konsola erişmeden long döndürür. `int.MinValue` değerini önce long yapın: bu değer pozitif büyüklüğü int aralığında değildir. Sonuç pay veya payda da int sınırını aşabilir; örneğin $-2147483648 / -1$, $2147483648/1$ olur.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	Pay, payda	Ayrı satırlarda tam int; payda sıfır olamaz.
Metot	Mutlak	Bu problemde $-2147483648..2147483648$ değerlerinin büyüklüğü.
Metot	Ebob	$a \geq 0, b > 0$; pozitif ortak bölen döndürür.
Çıktı	Kesir ve bölen	Sade kesir: P/Q , ardından EBOB: G .

İstemler Pay: ve Payda: olur. Pay hatalıysa payda istenmez. Geçersiz tamsayı, EOF veya sıfır paydada yalnız Hata: Pay ve payda int olmalı, payda sıfır olmamalıdır. yazılır. Sayısal giriş çevre boşlukları ve işaret kabul eder. Çıktıda boşluksuz P/Q gösterimi ve kültürden bağımsız sayılar kullanılır.

Algoritma

1. Pay ve paydayı doğrulayıp ayrı long değişkenlere aktarın.
2. Payda negatifse hem payın hem paydanın işaretini değiştirin.
3. Mutlak(payload) sonucunu pozitif payda ile Ebob metoduna verin.
4. EBOB içinde (a, b) çiftini $(b, a \% b)$ ile küçültün; $b == 0$ olunca a döndürün.
5. Pay ve paydayı ortak bölenle bölüp kesri ve böleni yazdırın.

İki değeri aynı sıfır olmayan sayıyla bölmek kesrin değerini korur. EBOB bütün ortak çarpanları kaldırır. Sıfır payda dışlandığından Ebob(0, b) sonucu b olur ve bölme güvenlidir. Mutlak genel `long.MinValue` için sözleşme vermez; çağıran yalnız belirtilen dar aralıktaki değerleri gönderir.

$-18/-24$ işaret normalleştirmesinden sonra $18/24$ olur. EBOB çağrısı:

Başlangıç a	Başlangıç b	Kalan
18	24	18
24	18	6
18	6	0

C# çözümü

```
using System;
using System.Globalization;

Console.Write("Pay: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int girilenPay))
{
    Console.WriteLine("Hata: Pay ve payda int olmalı, " +
        "payda sıfır olmamalıdır.");
    return;
}
Console.Write("Payda: ");
if (!int.TryParse(Console.ReadLine(), NumberStyles.Integer,
    CultureInfo.InvariantCulture, out int girilenPayda) || girilenPayda == 0)
{
    Console.WriteLine("Hata: Pay ve payda int olmalı, " +
        "payda sıfır olmamalıdır.");
    return;
}

long pay = girilenPay, payda = girilenPayda;
if (payda < 0)
{
    pay = -pay;
    payda = -payda;
}
long ortak = Ebob(Mutlak(pay), payda);
pay /= ortak;
payda /= ortak;
Console.WriteLine("Sade kesir: " +
    pay.ToString(CultureInfo.InvariantCulture) + "/" +
    payda.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("EBOB: " + ortak.ToString(CultureInfo.InvariantCulture));

static long Mutlak(long deger)
{
    return deger < 0 ? -deger : deger;
}

static long Ebob(long a, long b)
{
    while (b != 0)
    {
        long kalan = a % b;
        a = b;
        b = kalan;
    }
    return a;
}
```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Mutlak(pay) önce hesaplanır; dönen değer Ebob çağrısının ilk parametresi olur. Her metodun yerel değişkenleri kendi çağrısına aittir. Ebob içindeki atamalar ana programın pay ve payda değişkenlerini değiştirmez. Son kesir, dönen ortak bölen kullanılarak çağıranda oluşturulur.

Örnek çalıştırmalar

Önce payı, sonra paydayı ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

pay = -18, payda = -24:

Sade kesir: 3/4

EBOB: 6

pay = 0, payda = -7:

Sade kesir: 0/1

EBOB: 7

pay = 2147483647, payda = -2147483648:

Sade kesir: -2147483647/2147483648

EBOB: 1

pay = 5, payda = 0:

Hata: Pay ve payda int olmalı, payda sıfır olmamalıdır.

Sınır durumları

- İki negatif işaret birbirini kaldırır; yalnız payın işareti negatif kalabilir.
- `int.MinValue` önce genişletilir; negatifini `int` içinde hesaplamak taşar.
- Payın büyüklüğü paydadadan büyük olabilir; bileşik kesir geçerlidir.
- Aralarında asal değerlerde EBOB 1'dir; kesir zaten sade olabilir.
- -2147483648 / -2147483648 sonucu 1/1, EBOB 2147483648 olur.
- İki girdinin de sıfır olması payda kuralıyla reddedilir.

Kazanımlar

- Bir metodun dönüşünü başka metodun parametresi olarak kullanma.
- Metod önkoşullarını çağıran programda sağlama.
- Tür genişletmesini işaret değişiminden önce yapma.
- Sayısal değer korunurken tek bir standart gösterim üretme.
- Değer parametrelerinin çağırının değişkenlerini değiştirmediğini gösterme.

Alıştırmalar

1. Sade kesrin tamsayı ve kalan pay bölümünü ayrıca yazdırın; negatif pay kuralını tanımlayın.
2. Aynı kesri tekrar sadeleştirmenin neden sonucu değiştirmediğini açıklayın.
3. İki sade kesri toplamak için ortak payda hesabını metotlara ayırın; tür sınırlarını yeniden belirleyin.

073 Tekrar kullanılabilir giriş metoduyla not raporu

Düzey: İleri Önkoşullar: 010, 021, 071, 072

Kavramlar: out parametresi, Try sözleşmesi, Sınırlı yeniden deneme, EOF, Doğrulama ve hesaplama ayrımı

Problem tanımı

1..10 öğrenci için 0..100 aralığındaki notları okuyup ortalamayı ve en az 60 alan öğrenci sayısını bulun. Her sayısal alan için en fazla üç giriş denemesi yapın. Yanlış girişten sonra aynı alan tekrar sorulur; EOF durumunda yeni deneme yapılmadan program sonlandırılır. Üç deneme de başarısızsa rapor oluşturulmaz.

Okuma ve aralık doğrulamasını Oku(string istem, int alt, int ust, out int deger) metodunda birleştirin. Başarı true dönüşüyle, okunan değer out parametresiyle verilir. Başarısızlıkta değer 0 olarak atanır ve kullanılmaz. Raporun hesaplama metotları konsola erişmez. Böylece giriş metodunun bilinçli kullanıcı etkileşimi ile hesaplama metotlarının yan etkisiz davranışı ayrılır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	Öğrenci sayısı	1..10; her alan en fazla üç deneme.
Girdi	Notlar	Her öğrenci için ayrı satırda 0..100 tamsayı.
Ara çıktı	Yeniden deneme	Uyarı: Geçersiz giriş.; hatalı her denemede yazılır.
Sonuç	Rapor	Ortalama: X iki ondalık; Geçen: N.

İstemler Öğrenci sayısı: , sonra Not[0]: ile başlayan istemlerdir. Son hatalı deneme de uyarı üretir; ardından Hata: Giriş tamamlanamadı. yazılır. EOF yalnız bu hata satırını üretir; uyarı yoktur. Önceden alınmış notlar için kısmi rapor yazılmaz. Tamsayı okuma çevre boşlukları ve işaret kabul eder; ondalıklı değer kabul etmez.

Algoritma

1. Oku ile öğrenci sayısını alın; başarısızsa programı bitirin.
2. Her not için aynı metodu farklı istem ve sınırlarla çağırın.
3. Oku içinde üç denemeyi sayın; EOF'ta veya başarılı ayrıştırmada hemen dönün.
4. Başarısız bir denemede uyarı yazıp kalan deneme varsa aynı alanı yeniden sorun.
5. Bütün notlar alınınca Ortalama ve GecenSayisi metotlarını çağırın.
6. Nokta ayırıcıyla iki ondalıklı ortalamayı ve geçen sayısını yazdırın.

out, metodun normal dönüş yollarının hepsinde bir değer atamasını gerektirir. 0 geçerli bir not da olabilir; başarıyı değer 0 olmasıyla değil bool dönüşle kontrol edin. Ortalama metodunun önkoşulu boş olmayan, doğrulanmış not dizisidir. En fazla 33 okuma yapılır: öğrenci sayısı için üç ve on notun her biri için üç deneme.

İki öğrenci için x, 2, 101, 50, 80 girişlerinin akışı:

Alan	Deneme girdisi	İşlem
Öğrenci sayısı	x, sonra 2	Uyarı; ikinci deneme kabul edilir.
Not 0	101, sonra 50	Uyarı; ikinci deneme kabul edilir.
Not 1	80	İlk denemede kabul edilir.

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Öğrenci sayısı: ", 1, 10, out int n))
{
    Console.WriteLine("Hata: Giriş tamamlanamadı.");
    return;
}
int[] notlar = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "Not[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, 0, 100, out notlar[i]))
    {
        Console.WriteLine("Hata: Giriş tamamlanamadı.");
        return;
    }
}

Console.WriteLine("Ortalama: " +
    Ortalama(notlar).ToString("0.00", CultureInfo.InvariantCulture));
Console.WriteLine("Geçen: " +
    GecenSayisi(notlar).ToString(CultureInfo.InvariantCulture));

static bool Oku(string istem, int alt, int ust, out int deger)
{
    deger = 0;
    for (int deneme = 0; deneme < 3; deneme++)
    {
        Console.Write(istem);
        string? satir = Console.ReadLine();
        if (satir is null)
        {
            return false;
        }
        if (int.TryParse(satir, NumberStyles.Integer,
            CultureInfo.InvariantCulture, out int aday) &&
            aday >= alt && aday <= ust)
        {
            deger = aday;
            return true;
        }
        Console.WriteLine("Uyarı: Geçersiz giriş.");
    }
    return false;
}
```

```

static decimal Ortalama(int[] notlar)
{
    int toplam = 0;
    for (int i = 0; i < notlar.Length; i++)
    {
        toplam += notlar[i];
    }
    return (decimal)toplam / notlar.Length;
}

static int GecenSayisi(int[] notlar)
{
    int adet = 0;
    for (int i = 0; i < notlar.Length; i++)
    {
        if (notlar[i] >= 60)
        {
            adet++;
        }
    }
    return adet;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. out notlar[i], kabul edilen değeri doğrudan o dizi hücreğine aktarır. Başarısız dönüşte program hücreyi rapora katmaz. Oku yalnız giriş işi yapar; ortalama veya geçme kuralını bilmez. Her çağrıda yerel deneme sayacı yeniden 0'dan başlar.

Örnek çalıştırmalar

İstemler konsolda aynı satıra yazılır; aşağıdaki bloklar istemler çıkarıldığında kalan bütün uyarı ve sonuç satırlarıdır. Verilen giriş sırası ayrı satırlarda uygulanır.

2, 50, 80:

```

Ortalama: 65.00
Geçen: 1

```

x, 2, 101, 50, 80:

```

Uyarı: Geçersiz giriş.
Uyarı: Geçersiz giriş.
Ortalama: 65.00
Geçen: 1

```

1, 60:

```

Ortalama: 60.00
Geçen: 1

```

Öğrenci sayısına 0, 11, x:

```

Uyarı: Geçersiz giriş.
Uyarı: Geçersiz giriş.
Uyarı: Geçersiz giriş.
Hata: Giriş tamamlanamadı.

```

2, 100, ardından EOF:

```

Hata: Giriş tamamlanamadı.

```

Sınır durumları

- Not 0 geçerlidir; başarısızlığın göstergesi değildir.
- Not 60 geçer; 59 geçmez. Not 100 üst sınırdır.
- Üçüncü denemede geçerli giriş kabul edilir; dördüncü deneme yoktur.
- Boş satır geçersiz bir denemedir; EOF hemen bitişir.
- Öğrenci sayısı en az 1 olduğundan ortalamada sıfıra bölme oluşmaz.
- Toplam en fazla 1000 olur; decimal bölme küsuratı korur.

Kazanımlar

- Başarı bilgisini dönüş değeriyle, veriyi out ile aktarma.
- Parametrelerle aynı giriş davranışını farklı alanlarda kullanma.
- EOF ile yanlış kullanıcı girdisini ayrı yönetme.
- Yeniden deneme sınırını her alan için bağımsız tutma.
- Doğrulanmamış veya eksik veriyle rapor üretmemeyi sağlama.

Alıştırmalar

1. Deneme sınırını parametre yapın; sıfır veya negatif sınır için sözleşme belirleyin.
2. GecenSayisi metoduna geçme eşiği parametresi ekleyin.
3. İlk denemede kabul edilen alanları sayın; bu bilgiyi ek bir out parametresiyle döndürün.

074 ref ve dizi parametreleriyle yerinde döndürme

Düzey: İleri Önkoşullar: 043, 044, 071, 073

Kavramlar: ref parametresi, Dizi referansı, Yerinde değiştirme, Alt aralık sözleşmesi, Üç ters çevirme

Problem tanımı

0..30 elemanlı tamsayı dizisini, verilen negatif olmayan k adımı kadar sağa döndürün. Son elemanlar başa geçer; elemanların birbirlerine göre döngüsel sırası korunur. Dizi boşsa sonuç boşdur. k , 0..2147483647 arasında olabilir; etkin adım dolu dizide $k \% n$ olarak bulunur.

Ek dizi oluşturmadan üç ters çevirme kullanın: dizinin tamamı, ilk $k \% n$ elemanı ve kalan bölüm. İşleri SagaDondur, TersCevir ve Takas metotlarına ayırın. Takas(ref int a, ref int b) çağırının iki hücreğini değiştirir. Dizi parametresinde ref kullanmayın: dizi referansının değeri olarak aktarılması mevcut dizinin elemanlarını değiştirmek için yeterlidir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, k	Ayrı satırlarda 0..30 uzunluk ve negatif olmayan int adım.
Girdi	Elemanlar	Tam int aralığında n ayrı satır.
Metot	TersCevir	Dolu alt aralıkta dahil uçlar geçerli dizindir; boş aralıkta erişim yoktur.
Çıktı	Dizi	Dizi: ardından değerler; boşsa Dizi: Boş.

İstemler Eleman sayısı: , Döndürme: , sonra A[0]: ile başlayan istemlerdir. İlk hatalı alanda Hata: Geçersiz giriş. yazılır; sonraki alanlar istenmez. EOF, boş satır, ondalıklı giriş ve tür/aralık dışı değeri reddedilir. Bütün giriş doğrulanmadan dizi değiştirilmez. Sayısal giriş çevre boşlukları ve işaret kabul eder.

Algoritma

1. Uzunluğu, döndürmeyi ve bütün elemanları doğrulayın.
2. SagaDondur içinde boş dizide hemen dönün; aksi halde etkin adımı kalanla bulun.
3. Etkin adım 0 ise diziyi değiştirmeden dönün.
4. TersCevir ile tamamını, sonra ilk etkin adımı, sonra kalan bölümü ters çevirin.
5. TersCevir, iki ucu Takas ile değiştirip uçları birbirine yaklaştırsın.
6. Ana programda oluşan diziyi yazdırın.

Takas iki ref parametresine atama yapar; çağrıda da ref yazılmalıdır. TersCevir içindeki dizin parametreleri değeri olarak aktarılır; onları artırıp azaltmak çağırının değişkenlerini değiştirmez. Üç ters çevirme toplam doğrusal iş yapar ve sabit ek bellek kullanır. Çağırın aynı dizi nesnesini görür.

Dizi referansı değeri olarak aktarıldığı için metot içinde parametreye yeni dizi atamak çağırının dizi değişkenini yeniden bağlamaz. Mevcut nesnenin hücrelerini değiştirmek ise çağırından görülür. Diziyi yeniden bağlamak için gerekli olabilecek ref int[], bu problemde gerekli değildir. SagaDondur metoduna negatif olmayan adım ve null olmayan dizi gönderilir; boş dizi kabul edilir.

1, 2, 3, 4, 5 dizisini iki adım sağa döndürme:

Metot çağırısı	Sonraki dizi
Tamamını ters çevir	5, 4, 3, 2, 1
İlk iki elemanı ters çevir	4, 5, 3, 2, 1
Kalan üç elemanı ters çevir	4, 5, 1, 2, 3

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Eleman sayısı: ", out int n) || n < 0 || n > 30)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
if (!Oku("Döndürme: ", out int k) || k < 0)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, out a[i]))
    {
        Console.WriteLine("Hata: Geçersiz giriş.");
        return;
    }
}

SagaDondur(a, k);
Console.Write("Dizi:");
if (n == 0)
{
    Console.Write(" Boş");
}
for (int i = 0; i < n; i++)
{
    Console.Write(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();

static bool Oku(string istem, out int deger)
{
    Console.Write(istem);
    return int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out deger);
}
```

```

static void SagaDondur(int[] a, int k)
{
    if (a.Length == 0)
    {
        return;
    }
    k %= a.Length;
    if (k == 0)
    {
        return;
    }

    TersCevir(a, 0, a.Length - 1);
    TersCevir(a, 0, k - 1);
    TersCevir(a, k, a.Length - 1);
}

static void TersCevir(int[] a, int sol, int sag)
{
    while (sol < sag)
    {
        Takas(ref a[sol], ref a[sag]);
        sol++;
        sag--;
    }
}

static void Takas(ref int a, ref int b)
{
    int gecici = a;
    a = b;
    b = gecici;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. void metot veri döndürmez; bu örnekte sonuç, dizinin değişmiş elemanlarıdır.

Örnek çalıştırmalar

Önce n, sonra k, ardından elemanları ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

n = 5, k = 2, dizi 1, 2, 3, 4, 5:

Dizi: 4 5 1 2 3

n = 3, k = 2147483647, dizi 7, 8, 9:

Dizi: 9 7 8

n = 0, k = 5:

Dizi: Boş

n = 3, k = 3, dizi -1, 0, 1:

Dizi: -1 0 1

Sınır durumları

- Boş dizi için kalan işlemi yapılmaz; sıfıra bölme önlenir.
- Tek elemanlı dizide her etkin adım 0'dır.
- $k = 0$ veya uzunluğun katları diziye değiştirmez.
- Çok büyük k , tekrar tekrar tek adım döndürülmez; önce azaltılır.
- Takas aynı hücreyle çağrılırsa değer korunur.
- Negatif döndürme bu arayüzün kabul alanında değildir.

Kazanımlar

- Değer parametresi, ref parametresi ve dizi referansını ayırt etme.
- Büyük işi birbirini çağıran küçük void metotlarla kurma.
- Alt aralık uçlarını ve mutasyon etkisini sözleşmede belirtme.
- Boş veri durumunu aritmetik işlemden önce ele alma.
- Geçerli veri üzerinde ek dizi oluşturmada dönüşüm yapma.

Alıştırmalar

1. Aynı metotları kullanarak sola döndürme ekleyin; etkin adımı açıklayın.
2. Takas çağrılarını sayan bir ref sayaç ekleyin; adım 0 durumunu koruyun.
3. Metot içinde `a = new int[a.Length]` ataması yapınca çağırının neden değişmediğini küçük örnekle gösterin.

075 out parametrelerle tek geçişte dizi özeti

Düzey: İleri Önkoşullar: 041, 057, 073, 074

Kavramlar: Birden çok sonuç, out sözleşmesi, Boş veri, Okuma amacıyla dizi parametresi, Güvenli türetilmiş ölçüler

Problem tanımı

0..30 elemanlı bir tamsayı dizisini değiştirmeden en küçük değeri, en büyük değeri ve toplamı tek geçişte hesaplayın. Bu üç değeri `Özetle` metodunun out parametreleriyle döndürün. Metot, dizi boşsa `false`, doluysa `true` döndürür. Dolu dizide ana program ayrıca ortalamayı ve değer aralığının genişliğini hesaplar.

Boş dizi için minimum veya maksimum 0 kabul edilmez: bu ölçüler tanımsızdır. Başarısız dönüşte out değerler 0'a atanır ancak sonuç olarak kullanılmaz. Sıralama, hazır minimum/maksimum/toplam metotları veya yeni veri sınıfı kullanmayın. Amaç, başarının ayrı bildirildiği çok sonuçlu bir prosedürel sözleşme kurmaktır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, elemanlar	0..30 uzunluk; tam int aralığında ayrı satırlar.
Metot çıktısı	min, max, toplam	Dolu dizide geçerli iki int ve bir long.
Sonuç	Beş ölçü	Minimum, maksimum, toplam, iki ondalıklı ortalama ve genişlik.
Boş sonuç	Durum	Tek satır Sonuç: Dizi boş.

İstemler Eleman sayısı: , sonra `A[0]:` ile başlayan istemlerdir. İlk hatalı alanda yalnız Hata: Geçersiz giriş. yazılır. EOF, boş satır, ondalıklı değer veya aralık ihlali geçersizdir. Sayısal giriş çevre boşluklarını ve işareti kabul eder; bütün değerler alınmadan özet üretilmez.

Algoritma

1. Uzunluğu ve bütün elemanları doğrulayın.
2. `Özetle` girişinde bütün out parametrelerine 0 atayın; boş dizide `false` dönün.
3. Minimum ve maksimumu ilk elemanla başlatın.
4. Her elemanda iki sınırı güncelleyip long toplamı artırın.
5. `true` dönüşte ortalamayı decimal bölmeyle, genişliği long farkla hesaplayın.
6. Beş ölçüyü yazdırın; `false` dönüşte yalnız boş durumunu yazdırın.

İlk elemanla başlatmak bütün değerlerin negatif veya pozitif olduğu dizilerde doğrudur. Toplamın long olması en fazla 30 tam int değerinin toplamını taşmadan tutar. Genişlik hesaplanırken çıkarmadan önce tür genişletilir. Metot sadece okur; dizi parametresinin referans olması elemanları değiştirme zorunluluğu doğurmaz.

Minimum veya maksimumun 0 olması başarı hakkında bilgi vermez; yalnız metodun `bool` dönüşü kontrol edilir. Dolu dizide bütün çıktı parametreleri aynı taramanın sonuçlarıdır. Ortalama ve genişlik, bu sonuçlardan çağırılan tarafından türetilir; hesaplama metodu yazdırma biçimini bilmez.

4, -2, 4 için kısmi sonuçlar:

İşlenen değer	Minimum	Maksimum	Toplam
4	4	4	4
-2	-2	4	2
4	-2	4	6

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Eleman sayısı: ", out int n) || n < 0 || n > 30)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, out a[i]))
    {
        Console.WriteLine("Hata: Geçersiz giriş.");
        return;
    }
}

if (!Ozetle(a, out int min, out int max, out long toplam))
{
    Console.WriteLine("Sonuç: Dizi boş");
    return;
}
decimal ortalama = (decimal)toplam / n;
long genislik = (long)max - min;
Console.WriteLine("Minimum: " + min.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Maksimum: " + max.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Ortalama: " +
    ortalama.ToString("0.00", CultureInfo.InvariantCulture));
Console.WriteLine("Genişlik: " + genislik.ToString(CultureInfo.InvariantCulture));

static bool Oku(string istem, out int deger)
{
    Console.Write(istem);
    return int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out deger);
}
```

```

static bool Ozetle(int[] a, out int min, out int max, out long toplam)
{
    min = 0;
    max = 0;
    toplam = 0;
    if (a.Length == 0)
    {
        return false;
    }

    min = a[0];
    max = a[0];
    for (int i = 0; i < a.Length; i++)
    {
        if (a[i] < min)
        {
            min = a[i];
        }
        if (a[i] > max)
        {
            max = a[i];
        }
        toplam += a[i];
    }
    return true;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Diziye yeni değer atanmaz; bütün sonuçlar çıktı parametreleri üzerinden ana programa aktarılır.

Örnek çalıştırmalar

Önce n, sonra elemanları ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

n = 3, dizi 4, -2, 4:

```

Minimum: -2
Maksimum: 4
Toplam: 6
Ortalama: 2.00
Genişlik: 6

```

n = 2, dizi -2147483648, 2147483647:

```

Minimum: -2147483648
Maksimum: 2147483647
Toplam: -1
Ortalama: -0.50
Genişlik: 4294967295

```

n = 1, dizi 0:

```

Minimum: 0
Maksimum: 0
Toplam: 0
Ortalama: 0.00
Genişlik: 0

```

n = 0:

Sonuç: Dizi boş

Sınır durumları

- Boş dizide ilk elemana erişilmez; ortalama hesabı da yapılmaz.
- Tek elemanda minimum ve maksimum eşittir; genişlik 0'dır.
- Bütün değerler negatifken minimum/maksimum ilk elemandan belirlenir.
- `int` uçları arasındaki fark 4294967295'tir ve `long` gerektirir.
- Otuz `int.MaxValue` toplamı 64424509410'dur; `int` yeterli değildir.
- Başarısız giriş ile geçerli boş dizi farklı sonuçlar üretir.

Kazanımlar

- Birden fazla hesaplama sonucunu `out` parametreleriyle aktarma.
- Boş veri durumunu sahte sayısal sonuçlarla temsil etmemeyi öğrenme.
- Dizi parametresini değiştirmeden ortak tarama yapma.
- Genişletilmiş toplamdan güvenli ortalama ve fark türetme.
- Sonuç geçerliliği ile çıktı biçimini farklı sorumluluklarda tutma.

Alıştırmalar

1. En küçük değerin kaç kez geçtiğini yeni `out` parametresiyle döndürün.
2. İlk minimum ve maksimum dizinlerini de aynı geçişte bulun.
3. Girdi dizisinin bütün elemanlarının metot çağrısından sonra korunduğunu denetleyin.

076 Metotlarla sözcük frekansı raporu oluşturma

Düzey: İleri Önkoşullar: 053, 061, 062, 065, 075

Kavramlar: İşlem hattı, Geçerli eleman adedi, Paralel diziler, İki ölçütlü sıralama, Metot sorumlulukları

Problem tanımı

0..200 karakterlik, yalnız ASCII harfleri ve normal boşluk içeren bir satırın sözcük frekanslarını bulun. Büyük/küçük harf aynı kabul edilir. Sözcük, boşluklar arasında kalan kesintisiz harf bölümüdür. Farklı sözcükleri önce azalan frekansa, eşitlikte alfabetik sıraya göre yazdırın. Örneğin Ali veli ALI için ali: 2, ardından veli: 1 gelir.

İş üç hesaplama metoduna ayırın: sözcüklere ayırma, farklı sözcükleri sayma ve sözcük/adet çiftlerini birlikte sıralama. Her metot yalnız parametreleriyle çalışır; konsola erişmez. Split, düzenli ifade, sözlük, LINQ veya hazır sıralama kullanmayın. Metin için ASCII sınırı burada da geçerlidir; Türkçe alfabe sırası desteklendiği varsayılmaz.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	metin	0..200 ASCII harfi veya U+0020 boşluk; tek satır.
Ara değer	Sözcük ve adet dizileri	Yalnız döndürülen geçerli eleman adedi işlenir.
Çıktı	Özet	Toplam sözcük: N, Farklı sözcük: K.
Çıktı	Frekanslar	Frekanslar: ve her farklı sözcüğün satırı; boşsa Frekanslar: Yok.

Metin: istemini yanıtlayın. EOF, 201 veya daha fazla karakter, rakam, noktalama, Türkçe harf veya sekme için yalnız Hata: Metin 0..200 ASCII harf veya boşluk içermelidir. yazılır. Boş satır ve yalnız boşluklar geçerlidir. Bütün karakterler, herhangi bir rapor oluşturulmadan doğrulanır.

Algoritma

1. Satırı ve karakter kümesini doğrulayın; çalışma dizilerini giriş uzunluğunda oluşturun.
2. Ayır içinde boşlukları atlayın; her sözcüğü küçük harfe dönüştürüp sözcük dizisine yazın, adedi döndürün.
3. Say içinde her sözcüğü farklı sözcük dizisinde arayın; varsa sayacı artırın, yoksa yeni çift ekleyin.
4. Sırala içinde eklemeli sıralama yapın; frekansı büyük olan, eşit frekansta sözcüğü alfabetik küçük olan önce gelsin.
5. Her kaydırmada sözcük ile adet hücrelerini birlikte taşıyın.
6. Yalnız geçerli çiftleri ana programda yazdırın.

Her sözcük en az bir karakter içerdiği için giriş uzunluğundaki diziler yeterlidir. Ayır ve Say dönüşleri fiziksel kapasiteyi değil geçerli veri adedini bildirir. string.CompareOrdinal, küçük ASCII harfleri için kültürden bağımsız alfabetik karşılaştırma sağlar. Sayma ve sıralama en kötü durumda sözcük adedinin karesiyle orantılıdır; 200 karakter sınırı bu eğitim örneğinin iş yükünü sınırlar.

B a b C a b için metotlar arasındaki veri:

Aşama	Geçerli veri
Ayır	b, a, b, c, a, b; adet 6
Say	b: 3, a: 2, c: 1; farklı adet 3
Sirala	b: 3, a: 2, c: 1

C# çözümü

```
using System;
using System.Globalization;

Console.WriteLine("Metin: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: Metin 0..200 ASCII harf veya boşluk içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    char c = metin[i];
    if (!((c >= 'A' && c <= 'Z') ||
        (c >= 'a' && c <= 'z') || c == ' '))
    {
        Console.WriteLine("Hata: Metin 0..200 ASCII harf veya boşluk içermelidir.");
        return;
    }
}

string[] sozcukler = new string[metin.Length];
string[] farkli = new string[metin.Length];
int[] adetler = new int[metin.Length];
int n = Ayir(metin, sozcukler);
int k = Say(sozcukler, n, farkli, adetler);
Sirala(farkli, adetler, k);
Console.WriteLine("Toplam sözcük: " + n.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Farklı sözcük: " + k.ToString(CultureInfo.InvariantCulture));
Console.WriteLine(k == 0 ? "Frekanslar: Yok" : "Frekanslar:");
for (int i = 0; i < k; i++)
{
    Console.WriteLine(farkli[i] + ": " +
        adetler[i].ToString(CultureInfo.InvariantCulture));
}
```

```

static int Ayir(string metin, string[] sozcukler)
{
    char[] tampon = new char[metin.Length];
    int i = 0, adet = 0;
    while (i < metin.Length)
    {
        while (i < metin.Length && metin[i] == ' ')
        {
            i++;
        }
        int uzunluk = 0;
        while (i < metin.Length && metin[i] != ' ')
        {
            char c = metin[i++];
            tampon[uzunluk++] = c >= 'A' && c <= 'Z'
                ? (char)(c + ('a' - 'A')) : c;
        }
        if (uzunluk > 0)
        {
            sozcukler[adet++] = new string(tampon, 0, uzunluk);
        }
    }
    return adet;
}

static int Say(string[] sozcukler, int n, string[] farkli, int[] adetler)
{
    int k = 0;
    for (int i = 0; i < n; i++)
    {
        int j = 0;
        while (j < k && farkli[j] != sozcukler[i])
        {
            j++;
        }
        if (j == k)
        {
            farkli[k] = sozcukler[i];
            adetler[k] = 0;
            k++;
        }
        adetler[j]++;
    }
    return k;
}

```

```

static void Siralala(string[] sozcukler, int[] adetler, int n)
{
    for (int i = 1; i < n; i++)
    {
        string sozcuk = sozcukler[i];
        int adet = adetler[i], j = i - 1;
        while (j >= 0 && (adetler[j] < adet ||
            (adetler[j] == adet &&
            string.CompareOrdinal(sozcukler[j], sozcuk) > 0)))
        {
            sozcukler[j + 1] = sozcukler[j];
            adetler[j + 1] = adetler[j];
            j--;
        }
        sozcukler[j + 1] = sozcuk;
        adetler[j + 1] = adet;
    }
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Hesaplama metotlarının önkoşulları, doğrulanmış ASCII metni ve yeterli kapasiteli dizilerdir. Say tarafından oluşturulan çiftler aynı dizinle ilişkilidir; yalnız adetleri sıralamak yanlış sözcüğe yanlış frekans bağlar. Kullanılmayan string hücrelerine erişilmez. Özgün giriş metni korunur; küçük harf dönüşümü yalnız sözcük kopyalarında yapılır.

Örnek çalıştırmalar

Bloklar Metin: isteminden sonraki tam sonuçtur.

Ali veli ALI:

```

Toplam sözcük: 3
Farklı sözcük: 2
Frekanslar:
ali: 2
veli: 1

```

z A z a b:

```

Toplam sözcük: 5
Farklı sözcük: 3
Frekanslar:
a: 2
z: 2
b: 1

```

Boş satır veya yalnız boşluk:

```

Toplam sözcük: 0
Farklı sözcük: 0
Frekanslar: Yok

```

Sınır durumları

- Başta, sonda veya arada birden fazla boşluk boş sözcük üretmez.
- Aynı harfin farklı büyüklükleri aynı sözcükte birleştirilir.
- Frekans eşitliğinde a, z sözcüğünden önce gelir.
- a ve aa farklıdır; örnek olan kısa sözcük alfabetik olarak önce gelir.
- 200 harflik tek sözcük geçerlidir; tamponun tamamı kullanılabilir.
- 200 karakterde en fazla 100 sözcük bulunur; bu üst sınırı tek harf/boşluk düzeniyle ulaşılır.

Kazanımlar

- Bir metin raporunu birbirini tamamlayan üç metotla kurma.
- Dizi kapasitesi ile metotların döndürdüğü geçerli adetleri ayırma.
- Paralel dizilerdeki ilişkileri kaydırma boyunca koruma.
- İki sıralama ölçütünü açık öncelikle birleştirme.
- Hesaplama metotlarını konsol bağımlılığı olmadan yeniden kullanma.

Alıştırmalar

1. Yalnız ilk üç farklı sözcüğü raporlayın; üçten az sözcük durumunu koruyun.
2. En yüksek frekansa sahip bütün sözcükleri ayrı bir metotla bulun.
3. Sözcüklerin ilk görüldükleri dizinlerini üçüncü paralel diziyle izleyin ve sıralarken birlikte taşıyın.

077 Özyinelemeli ikili aramayla ilk eşleşmeyi bulma

Düzey: İleri Önkoşullar: 046, 047, 071, 074

Kavramlar: Özyineleme, Taban durum, Küçülen aralık, İlk eşleşme, ref çağrı sayacı

Problem tanımı

Küçükten büyüğe, azalmayan sıradaki 0..30 elemanlı bir dizide aranan tamsayının ilk dizinini bulun. Tekrarlı değerlerde soldaki ilk eşleşme seçilir; yoksa -1 yazılır. Diziyi sıralamayın: verilen sırayı doğrulayın, sırasızsa hata üretin.

İkili aramayı döngü yerine kendini çağıran İlkBul metoduyla gerçekleştirin. Her çağrı dahil sol..sag aralığını inceler ve daha küçük bir aralıkla devam eder. sol > sag, hiçbir elemana erişilmeden -1 dönen taban durumdur. Bir ref sayaçla bütün çağrıları, boş aralık çağrıları dahil sayın. Sayaç değer karşılaştırması veya çalışma süresi değildir.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, dizi	0..30 uzunluk; tam int elemanları, azalmayan sıra.
Girdi	Aranan	Tam int; sıra doğrulandıktan sonra istenir.
Çıktı	Dizin	Dizin: I; ilk eşleşme veya -1.
Çıktı	Çağrı sayısı	Çağrı: C; taban durum dahil her metot girişi.

İstemler Eleman sayısı: , A[0]: ile başlayan istemler, ardından Aranan: olur. İlk hatalı sayısal alanda Hata: Geçersiz giriş. yazılır. Sırasız dizide yalnız Hata: Dizi azalmayan sırada olmalıdır. yazılır ve aranan değer istenmez. EOF veya boş satır geçersizdir; sayısal giriş çevre boşluklarını ve işareti kabul eder.

Algoritma

1. Diziyi okuyup SıralıMi ile önkoşulu doğrulayın; sonra arananı okuyun.
2. Sayacı 0 yapıp İlkBul(a, aranan, 0, n - 1, ref cagri) çağırın.
3. Her çağrı girişinde sayacı artırın; boş aralıkta -1 dönün.
4. Ortayı sol + (sag - sol) / 2 ile hesaplayın.
5. Orta değer aranan değerden küçükse sağ alt aralığın sonucunu döndürün.
6. Aksi halde sol alt aralıkta ilk eşleşmeyi arayın; varsa onu döndürün.
7. Solda yoksa orta değer eşitse orta dizini, değilse -1 döndürün.

Her alt aralık ortayı dışlar; uzunluk kesin küçülür ve özyineleme sonlanır. Eşit orta değer solunu aramak ilk eşleşmeyi korur. Her düzey en fazla bir alt çağrı yapar; bütün diziyi dallanarak gezmez. Dizi değiştirilmez; her çağrının sol, sag ve orta değerleri kendine aittir.

Çağrılar yürütme yığnında ayrı yerel durumlar taşır; ref cagri ise hepsinin aynı sayaç hücrecini artırmasını sağlar. Arama metodunun önkoşulu sıralı dizi ve geçerli bir alt aralıktır; başlangıçta boş dizi için 0..-1 boş aralığı geçerlidir.

1, 2, 2, 2, 5 dizisinde 2 arama:

Çağrı	Aralık	Karar
1	0..4; orta 2	Eşit; solu araştır.
2	0..1; orta 0	1 küçüktür; sağa git.
3	1..1; orta 1	Eşit; solu araştır.
4	1..0	Boş; -1 dön. Sonra çağrı 3 dizin 1 döndürür.

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Eleman sayısı: ", out int n) || n < 0 || n > 30)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, out a[i]))
    {
        Console.WriteLine("Hata: Geçersiz giriş.");
        return;
    }
}
if (!SiraliMi(a))
{
    Console.WriteLine("Hata: Dizi azalmayan sırada olmalıdır.");
    return;
}
if (!Oku("Aranan: ", out int aranan))
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}

int cagri = 0;
int dizin = IlkBul(a, aranan, 0, n - 1, ref cagri);
Console.WriteLine("Dizin: " + dizin.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Çağrı: " + cagri.ToString(CultureInfo.InvariantCulture));

static bool Oku(string istem, out int deger)
{
    Console.Write(istem);
    return int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out deger);
}
```

```

static bool SiraliMi(int[] a)
{
    for (int i = 1; i < a.Length; i++)
    {
        if (a[i] < a[i - 1])
        {
            return false;
        }
    }
    return true;
}

static int IlkBul(int[] a, int aranan, int sol, int sag, ref int cagri)
{
    cagri++;
    if (sol > sag)
    {
        return -1;
    }

    int orta = sol + (sag - sol) / 2;
    if (a[orta] < aranan)
    {
        return IlkBul(a, aranan, orta + 1, sag, ref cagri);
    }

    int soldaki = IlkBul(a, aranan, sol, orta - 1, ref cagri);
    if (soldaki != -1)
    {
        return soldaki;
    }
    return a[orta] == aranan ? orta : -1;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Metot konsola erişmez. Dönen dizin çağrılarının dönüşleri boyunca ana programa taşınır.

Örnek çalıştırmalar

Önce n, elemanlar ve arananı ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

n = 5, dizi 1, 2, 2, 2, 5, aranan 2:

Dizin: 1
Çağrı: 4

Aynı dizide aranan 3:

Dizin: -1
Çağrı: 4

n = 0, aranan 7:

Dizin: -1
Çağrı: 1

n = 2, dizi 2, 1; aranan istenmez:

Hata: Dizi azalmayan sırada olmalıdır.

Sınır durumları

- Tek elemanda eşleşme olsa bile ilk dizin kuralı için boş sol aralık çağrılır; toplam iki çağrı olur.
- Bütün elemanlar eşitse arama soldaki ilk dizine kadar sürer.
- Aranan küçük veya büyük uç dışında olabilir; -1 normal sonuçtur.
- Boş aralıkta orta dizin hesaplanmaz ve dizi okunmaz.
- int uçları karşılaştırılır; farkları alınmaz, sayısal taşma oluşmaz.
- En fazla 30 eleman, özyineleme derinliğini sınırlı tutar; çok büyük problemler için yığın kapasitesi ayrıca düşünülmelidir.

Kazanımlar

- Özyinelemeli metoda taban durum ve kesin küçülen alt problem koyma.
- Eşitlikte aramayı sürdürerek ilk sonucu koruma.
- Yerel çağrı durumu ile ortak ref sayacı ayırma.
- Sıralılık önkoşulunu algoritmadan önce doğrulama.
- Çağrı sayısının hangi işlemi ölçtüğünü açıkça tanımlama.

Alıştırmalar

1. Son eşleşmeyi bulan sürümü yazın; eşitlikte hangi tarafa gidileceğini değiştirin.
2. Yalnız en yüksek özyineleme derinliğini ölçün; bunu toplam çağrı sayısı ile karşılaştırın.
3. Aynı ilk eşleşmeyi döngülü sürümle bulun; dizin sonuçlarının aynı olduğunu farklı girdilerde denetleyin.

078 Özyinelemeli birleştirmeli sıralama

Düzey: İleri Önkoşullar: 055, 060, 074, 077

Kavramlar: Böl ve yönet, İki alt çağrı, Paylaşılan çalışma tamponu, Kararlı birleştirme, Karşılaştırma sayacı

Problem tanımı

0..30 tamsayıyı birleştirmeli sıralamayla küçükten büyüğe sıralayın. Sırala metodu aralığı ikiye bölsün, iki alt aralığı özyinelemeli sıralasın ve Birleştirmeli metodu bunları birleştirsün. Çalışma tamponu ana programda bir kez oluşturulsun; her çağrıda yeni dizi oluşturulmasın.

Sıralama boyunca iki eleman değeri arasında yapılan $\leq$ karşılaştırmalarını sayın. Döngü/dizin koşulları, veri kopyalama ve giriş doğrulama bu sayaca dahil değildir. Eşit değerlerde soldaki elemanı önce alarak kararlı birleştirme kurun. Yalnız tamsayı çıktısı eşit değerlerin kimliğini göstermez; kararlılık, eşitlikteki seçim kuralıyla açıklanmalıdır.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, elemanlar	0..30 uzunluk; tam int değerleri ayrı satırlarda.
Ara değer	tampon	Diziyle aynı kapasite; alt çağrılar kendi aralıklarını kullanır.
Çıktı	Sıralı dizi	Dizi: ardından değerler; boşsa Dizi: Boş.
Çıktı	Sayaç	Karşılaştırma: C; yalnız iki değer arasındaki karşılaştırmalar.

İstemler Eleman sayısı: , sonra $A[0]:$ ile başlayan istemlerdir. EOF, geçersiz tamsayı veya uzunluk ihlalinde yalnız Hata: Geçersiz giriş. yazılır. İlk hata sonraki okumaları durdurur. Bütün elemanlar doğrulanmadan sıralama başlamaz. Sayısal giriş çevre boşluklarını ve işareti kabul eder.

Algoritma

1. Diziyi okuyun; bir çalışma tamponu ve 0 karşılaştırma sayacı oluşturun.
2. Sırala içinde sıfır veya tek elemanlı aralıkta dönün.
3. Ortayı hesaplayıp sol ve sağ aralıkları sıralayın.
4. Birleştirmeli içinde iki sıralı bölümün başlarını karşılaştırın; küçük olanı tampona alın.
5. Eşitlikte soldan alın; iki taraftan biri bitince ötekinin kalanını karşılaştırmaz kopyalayın.
6. Tamponun yalnız bu aralığını asıl diziye aktarın; sonuç ve sayacı yazdırın.

Alt çağrılar döndüğünde iki bölüm sıralıdır. Birleştirme her adımda kalanların en küçüğünü seçer; çıktı sıralı olur ve elemanlar kaybolmaz. Her çağrı daha kısa aralıklarla ilerler. Genel iş miktarı $n \log n$, çalışma tamponu n , çağrı yığını derinliği $\log n$ ile orantılıdır. Bu bellek kullanımı, önceki yerinde sıralamaların farklı bir tercihidir.

Sırala sözleşmesi geçerli dahil uçları, aynı uzunluktaki tamponu ve başlangıç sayacını ister; boş aralık da kabul edilir. Birleştirmeli iki bölümün zaten sıralı olduğu önkoşuluyla çağrılır. Tampondaki diğer bölümlere ait eski değerler okunmaz. Soldan ve sağdan alınan elemanların toplamı aralık uzunluğudur.

4, 1, 3, 2 için birleřtirmeler:

Birleřtirilen bölümler	Sonuç	Yeni karřılařtırma
4 ve 1	1, 4	1
3 ve 2	2, 3	1
1, 4 ve 2, 3	1, 2, 3, 4	3

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Eleman sayısı: ", out int n) || n < 0 || n > 30)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}

int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, out a[i]))
    {
        Console.WriteLine("Hata: Geçersiz giriş.");
        return;
    }
}

int[] tampon = new int[n];
int karsilastirma = 0;
Siralama(a, tampon, 0, n - 1, ref karsilastirma);
Console.WriteLine("Dizi:");
if (n == 0)
{
    Console.WriteLine(" Boř");
}
for (int i = 0; i < n; i++)
{
    Console.WriteLine(" " + a[i].ToString(CultureInfo.InvariantCulture));
}
Console.WriteLine();
Console.WriteLine("Karřılařtırma: " +
    karsilastirma.ToString(CultureInfo.InvariantCulture));

static bool Oku(string istem, out int deger)
{
    Console.WriteLine(istem);
    return int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out deger);
}
```

```

static void Sirala(int[] a, int[] tampon, int sol, int sag, ref int sayac)
{
    if (sol >= sag)
    {
        return;
    }

    int orta = sol + (sag - sol) / 2;
    Sirala(a, tampon, sol, orta, ref sayac);
    Sirala(a, tampon, orta + 1, sag, ref sayac);
    Birlestir(a, tampon, sol, orta, sag, ref sayac);
}

static void Birlestir(int[] a, int[] tampon,
    int sol, int orta, int sag, ref int sayac)
{
    int i = sol, j = orta + 1, yazma = sol;
    while (i <= orta && j <= sag)
    {
        sayac++;
        if (a[i] <= a[j])
        {
            tampon[yazma++] = a[i++];
        }
        else
        {
            tampon[yazma++] = a[j++];
        }
    }

    while (i <= orta)
    {
        tampon[yazma++] = a[i++];
    }
    while (j <= sag)
    {
        tampon[yazma++] = a[j++];
    }
    for (int k = sol; k <= sag; k++)
    {
        a[k] = tampon[k];
    }
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Tampon parametresi her çağrıda aynı diziye gösterir; aralık parametreleri hangi hücrelerin işleneceğini sınırlar.

Örnek çalıştırmalar

Önce n, sonra elemanları ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

n = 4, dizi 4, 1, 3, 2:

Dizi: 1 2 3 4

Karşılaştırma: 5

n = 3, dizi 1, 1, 1:

Dizi: 1 1 1

Karşılaştırma: 3

n = 0:

Dizi: Boş

Karşılaştırma: 0

n = 1, dizi -2147483648:

Dizi: -2147483648

Karşılaştırma: 0

Sınır durumları

- Sıfır ve tek elemanda birleştirme çağrılmaz.
- Tek uzunlukta iki alt bölüm eşit boyda olmayabilir; dahil uçlar kapsama boşluğu bırakmaz.
- Eşit değerler atılmaz; soldan önce alınır.
- Negatif değerler ve int uçları karşılaştırılır; eleman değerleriyle aritmetik yapılmaz.
- Kalan bölüm kopyalanırken karşılaştırma sayacı artırılmaz.
- Tampon her aralık için yeniden yazılır; önceki çağrıdan kalan hücreler sonucu etkilemez.

Kazanımlar

- İki alt problemin dönüşünden sonra birleştirme yapma.
- Paylaşılan tamponu aralık sınırlarıyla güvenli kullanma.
- Birleştirme metodunun önkoşulunu özyinelemeyle sağlama.
- Veri değerleri karşılaştırmasını kopyalama ve dizin koşullarından ayırma.
- Kararlılık, ek bellek ve iş miktarı arasındaki tercihleri açıklama.

Alıştırmalar

1. Azalan sıralama ekleyin; eşitlikte soldan alma kuralını koruyun.
2. Veri kopyalamalarını ayrı sayaçla ölçün; karşılaştırmayla karıştırmayın.
3. Elemanların özgün dizinlerini paralel dizide taşıyarak eşit değerlerin kararlılığını görünür kılın.

079 Geri izlemeyle hedef toplamalı alt kümeleri sayma

Düzy: İleri Önkoşullar: 040, 045, 075, 077, 078

Kavramlar: Geri izleme, Seç veya dışla, Budama, Kalan toplam, Değerle aktarılan arama durumu

Problem tanımı

0..16 elemanlı, her değeri 1..20 olan dizide toplamı verilen hedefe eşit alt kümelerin sayısını bulun. Hedef 0..320 arasında olabilir. Bir eleman en fazla bir kez seçilir. Aynı değere sahip farklı dizinler ayrı seçimlerdir: 1, 1 dizisinde hedef 1 için iki alt küme vardır. Hedef 0 için tek çözüm boş alt kümedir.

Her konumda elemanı seçme ve dışlama dalları oluşturan özyinelemeli arama kullanın. Toplam hedefi aştıysa veya kalan bütün elemanları eklemek bile hedefe ulaşamıyorsa dalı budayın. kalan[i], i konumundan sona kadar toplamdır. Pozitif değer kısıtı bu budamaları güvenli yapar. Hazır alt küme üretimi veya dinamik programlama kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	n, hedef	Ayrı satırlarda 0..16 uzunluk ve 0..320 hedef.
Girdi	Elemanlar	Ayrı satırlarda 1..20 pozitif tamsayı.
Çıktı	Çözüm adedi	Alt küme: N; dizin seçimleri ayrı sayılır.
Çıktı	Arama ölçüsü	Ziyaret: Z; budanmış ve başarılı çağrılar dahil her giriş.

İstemler Eleman sayısı: , Hedef: , ardından A[0]: ile başlayan istemlerdir. İlk geçersiz alanda yalnız Hata: Geçersiz giriş. yazılır; diğer alanlar istenmez. EOF, boş satır veya aralık ihlali geçersizdir. Sayısal giriş çevre boşluklarını ve işareti kabul eder. Tam veri alınmadan arama başlamaz.

Algoritma

1. Uzunluk, hedef ve pozitif elemanları doğrulayın.
2. Uzunluğu n + 1 olan kalan toplam dizisini sondan başa kurun; kalan[n] = 0 olsun.
3. Say metodunu konum 0 ve toplam 0 ile çağırın; her girişte ziyaret sayısını artırın.
4. Toplam hedefe eşitse 1 dönün: kalan pozitif elemanların hepsi dışlanmalıdır.
5. Dizi bittiyse, toplam fazlaysa veya toplam ile kalan toplam hedefe yetmiyorsa 0 dönün.
6. Elemanı seçen ve dışlayan iki çağrının çözüm sayılarını toplayıp döndürün.
7. Çözüm adedini ve ziyaret sayısını yazdırın.

İki dal birbirinden ayrıdır ve her dizin için seçim bir kez yapılır. Konum her çağrıda artar; en fazla n + 1 düzey vardır. Durum değer parametreleriyle taşındığından seçimden dönerken ortak bir toplamı geri almak gerekmez. Bu, geri izlemeyi yerel çağrı durumuyla kurar. En fazla 65536 seçim ve 131071 çağrı bulunur; budama genellikle daha azını inceler. Sayılar int aralığına sığar.

Arama metodunun önkoşulları pozitif dizi, onunla uyumlu kalan toplam dizisi ve geçerli konumdur. Başlangıç toplamı 0'dır. İki dal ayrı yerel toplam değerleri alır; yalnız ziyaret sayacı ortaktır. Çözüm listesi tutulmaz.

1, 1, hedef 1 için bütün çağrılar:

Ziyaret	Konum ve toplam	Sonuç
1	0, 0	İlk elemanı seçme ve dışlama dalları.
2	1, 1	Başarılı; 1 çözüm.
3	1, 0	İkinci eleman için iki dal.
4	2, 1	Başarılı; 1 çözüm.
5	2, 0	Dizi bitti; 0 çözüm.

C# çözümü

```
using System;
using System.Globalization;

if (!Oku("Eleman sayısı: ", out int n) || n < 0 || n > 16)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
if (!Oku("Hedef: ", out int hedef) || hedef < 0 || hedef > 320)
{
    Console.WriteLine("Hata: Geçersiz giriş.");
    return;
}
int[] a = new int[n];
for (int i = 0; i < n; i++)
{
    string istem = "A[" + i.ToString(CultureInfo.InvariantCulture) + "]: ";
    if (!Oku(istem, out a[i]) || a[i] < 1 || a[i] > 20)
    {
        Console.WriteLine("Hata: Geçersiz giriş.");
        return;
    }
}

int[] kalan = new int[n + 1];
for (int i = n - 1; i >= 0; i--)
{
    kalan[i] = a[i] + kalan[i + 1];
}
int ziyaret = 0;
int adet = Say(a, kalan, 0, 0, hedef, ref ziyaret);
Console.WriteLine("Alt küme: " + adet.ToString(CultureInfo.InvariantCulture));
Console.WriteLine("Ziyaret: " + ziyaret.ToString(CultureInfo.InvariantCulture));

static bool Oku(string istem, out int deger)
{
    Console.Write(istem);
    return int.TryParse(Console.ReadLine(), NumberStyles.Integer,
        CultureInfo.InvariantCulture, out deger);
}
```

```

static int Say(int[] a, int[] kalan,
    int i, int toplam, int hedef, ref int ziyaret)
{
    ziyaret++;
    if (toplam == hedef)
    {
        return 1;
    }
    if (i == a.Length || toplam > hedef || toplam + kalan[i] < hedef)
    {
        return 0;
    }

    int sec = Say(a, kalan, i + 1, toplam + a[i], hedef, ref ziyaret);
    int disla = Say(a, kalan, i + 1, toplam, hedef, ref ziyaret);
    return sec + disla;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Genel iş miktarı üstel olabilir; 16 eleman sınırı ve budama, bu öğretim örneğinin kapsamını belirler.

Örnek çalıştırmalar

Önce n, hedef ve elemanları ayrı satırlarda girin. Bloklar istemlerden sonraki tam sonuçtur.

n = 3, hedef 3, dizi 1, 2, 3:

Alt küme: 2
Ziyaret: 13

n = 2, hedef 1, dizi 1, 1:

Alt küme: 2
Ziyaret: 5

n = 2, hedef 7, dizi 2, 4:

Alt küme: 0
Ziyaret: 1

n = 0, hedef 0:

Alt küme: 1
Ziyaret: 1

Sınır durumları

- Boş dizi ve pozitif hedef için 0 çözüm, bir ziyaret vardır.
- Hedef 0 her pozitif dizide boş seçimin tek çözümüdür; ilk çağrıda dönülür.
- Eşit elemanlar ayrı dizinlerdeyse ayrı alt kümeler üretir; tekrarlar silinmez.
- Bütün elemanların toplamı hedeften küçükse kökte budama yapılır.
- Sıfır veya negatif eleman bu sürümde kabul edilmez; erken başarı ve budama gerekçesini değiştirir.
- On altı adet 1 için hedef 8, 12870 ayrı dizin seçimi üretir.

Kazanımlar

- Seçim problemini birbirini dışlayan iki dala ayırma.
- Bir budamanın dayandığı veri önkoşulunu açıklama.
- Dönüş değerleriyle çözüm sayılarını birleştirme.
- Değer parametreleriyle dalların durumlarını bağımsız tutma.
- Üstel aramanın giriş boyutu ve ölçüm sınırlarını tanımlama.

Alıřtırmalar

1. Budamaları kaldırıp aynı özüm sayısını ve ziyaret farkını ölçün.
2. Seçilmiş dizinleri tamponda tutup özümleri de yazdırın; geri dönüşte geçerli seçili adedini koruyun.
3. Sıfır değeri kabul etmek için erken başarı kuralını değıştirin; hedef 0 için birden fazla özümü gösterin.

080 Metotlarla postfix ifade değerlendirme

Düzey: İleri Önkoşullar: 068, 069, 070, 073, 075

Kavramlar: Birleşik prosedürel uygulama, Belirteç ayrıştırma, Diziyle yığın, Try ve out sözleşmeleri, Güvenli tamsayı aritmetiği

Problem tanımı

Boşluklarla ayrılmış postfix ifadeyi tamsayı aritmetiğiyle değerlendirin. Postfix gösterimde işlem iki işlenenden sonra gelir: $8\ 3\ -\ 2\ *$, $(8\ -\ 3)\ *\ 2$ anlamındadır ve 10 üretir. Desteklenen işlemler $+$, $-$, $*$, $/$ olur. Tek karakterlik işlem belirteçleri ile işaretli sayıları ayırın: $-$ işlemdir, -3 sayıdır.

Sayı belirteci, isteğe bağlı $+$ veya $-$ işaretinin ardından en az bir ASCII rakam içermelidir; tam `int` aralığına sığmalıdır. Başta sıfırlar kabul edilir. Belirteç ayırma, elle sayı ayrıştırma ve aritmetiği ayrı metotlara bölün. Diziyle yığın kullanın; hazır `Stack`, `Split`, düzenli ifade, `Parse` veya `TryParse` kullanmayın.

Girdi ve çıktı

Tür	Ad	Açıklama
Girdi	İfade	0..200 yazdırılabilir ASCII karakteri; ayraç yalnız U+0020 boşluk.
Ara değer	Yığın	<code>int</code> işlenenler; geçerli adet, kapasiteden ayırır.
Başarı	Sonuç	Tek satır Sonuç: N.
Hata	Neden	Boş ifade, eksik/artan işlenen, geçersiz belirteç, sıfıra bölme veya taşma.

İfade: istemini yanıtlayın. EOF, fazla uzunluk veya ASCII 32..126 dışı karakter için Hata: İfade 0..200 yazdırılabilir ASCII karakteri içermelidir. yazılır. Tam satır karakter kümesi açısından önce doğrulanır. Sonra belirteçler soldan sağa değerlendirilir; ilk değerlendirme hatası raporlanır. Sonunda tam bir işlenen kalmalıdır. Hiçbir ara sonuç yazdırılmaz.

Aralık dışı sayı belirteci Hata: Geçersiz belirteç. üretir; geçerli işlenenlerin işlemi `int` aralığını aşarsa Hata: Taşma. olur. Bölme sıfıra doğru kesilir: $-7\ 2\ /\$ sonucu -3 'tür. Parantez, ondalık sayı veya örtük işlem desteklenmez; her belirteç normal boşlukla ayrılmalıdır.

Algoritma

1. Giriş uzunluğunu ve bütün karakterlerini doğrulayın.
2. Hesapla içinde boşlukları atlayıp her belirtecin başlangıç ve uzunluğunu bulun.
3. Tek karakterlik işlemi tanıyın; yığında iki işlenen yoksa eksik işlenen hatası dönün.
4. Önce sağ, sonra sol işleneni çıkarın; İşlem ile hesaplayıp sonucu yığına koyun.
5. Diğer belirteçleri Sayı metoduyla elle ayrıştırın; geçersizse hata, geçerliyse yığına ekleme yapın.
6. Bitişte sıfır işlenen için boş ifade, birden fazla için artan işlenen hatası dönün.
7. Bir sonuç kaldıysa out ile döndürün ve ana programda yazdırın.

Yığının tepesi, sıradaki işlemde kullanılacak sağ işlenendir; çıkarma ve bölmede bu sıra önemlidir. İşlem `long` içinde yapılır, ardından `int` sınırı denetlenir. İki `int` değerinin çarpımı `long` aralığına sığar. Sayı ayrıştırmada çarpma/toplamadan önce büyüklük sınırı denetlenir. Giriş uzunluğundaki yığın kapasitesi belirteç sayısı için yeterlidir.

8 3 - 2 * için yığın; soldan sağa alt-üst sırası:

Belirteç	Yığın	İşlem
8	8	Sayıyı ekle.
3	8, 3	Sayıyı ekle.
-	5	Sol 8, sağ 3; fark 5.
2	5, 2	Sayıyı ekle.
*	10	Çarpım 10.

C# çözümü

```
using System;
using System.Globalization;

Console.Write("İfade: ");
string? metin = Console.ReadLine();
if (metin is null || metin.Length > 200)
{
    Console.WriteLine("Hata: İfade 0..200 yazdırılabilir ASCII " +
        "karakteri içermelidir.");
    return;
}
for (int i = 0; i < metin.Length; i++)
{
    if (metin[i] < ' ' || metin[i] > '~')
    {
        Console.WriteLine("Hata: İfade 0..200 yazdırılabilir ASCII " +
            "karakteri içermelidir.");
        return;
    }
}

if (Hesapla(metin, out int sonuc, out string neden))
{
    Console.WriteLine("Sonuç: " + sonuc.ToString(CultureInfo.InvariantCulture));
}
else
{
    Console.WriteLine("Hata: " + neden + ".");
}
```

```

static bool Hesapla(string metin, out int sonuc, out string neden)
{
    sonuc = 0;
    neden = "";
    int[] yigin = new int[metin.Length];
    int i = 0, tepe = 0;
    while (i < metin.Length)
    {
        while (i < metin.Length && metin[i] == ' ')
        {
            i++;
        }
        if (i == metin.Length)
        {
            break;
        }

        int baslangic = i;
        while (i < metin.Length && metin[i] != ' ')
        {
            i++;
        }
        string belirtec = metin.Substring(baslangic, i - baslangic);
        if (IslemMi(belirtec))
        {
            if (tepe < 2)
            {
                neden = "Eksik işlenen";
                return false;
            }
            int sag = yigin[--tepe];
            int sol = yigin[--tepe];
            if (!Islem(belirtec[0], sol, sag, out int deger, out neden))
            {
                return false;
            }
            yigin[tepe++] = deger;
        }
        else
        {
            if (!Sayi(belirtec, out int deger))
            {
                neden = "Geçersiz belirteç";
                return false;
            }
            yigin[tepe++] = deger;
        }
    }

    if (tepe != 1)
    {
        neden = tepe == 0 ? "Boş ifade" : "Artan işlenen";
        return false;
    }
    sonuc = yigin[0];
    return true;
}

```

```

static bool IslemMi(string belirtec)
{
    return belirtec.Length == 1 && (belirtec[0] == '+' ||
        belirtec[0] == '-' || belirtec[0] == '*' || belirtec[0] == '/');
}

static bool Sayi(string metin, out int deger)
{
    deger = 0;
    int i = 0;
    bool negatif = metin[0] == '-';
    if (metin[0] == '-' || metin[0] == '+')
    {
        i++;
    }
    if (i == metin.Length)
    {
        return false;
    }

    long sinir = negatif ? 2147483648L : 2147483647L;
    long buyukluk = 0;
    for (; i < metin.Length; i++)
    {
        if (metin[i] < '0' || metin[i] > '9')
        {
            return false;
        }
        int rakam = metin[i] - '0';
        if (buyukluk > (sinir - rakam) / 10)
        {
            return false;
        }
        buyukluk = buyukluk * 10 + rakam;
    }
    deger = (int)(negatif ? -buyukluk : buyukluk);
    return true;
}

static bool Islem(char islem, int sol, int sag,
    out int sonuc, out string neden)
{
    sonuc = 0;
    neden = "";
    if (islem == '/' && sag == 0)
    {
        neden = "Sıfıra bölme";
        return false;
    }

    long genis = islem == '+' ? (long)sol + sag
        : islem == '-' ? (long)sol - sag
        : islem == '*' ? (long)sol * sag : (long)sol / sag;
    if (genis < int.MinValue || genis > int.MaxValue)
    {
        neden = "Taşma";
        return false;
    }
    sonuc = (int)genis;
    return true;
}

```

Kod bağımsız konsol projesinin Program.cs dosyasında çalışır. Substring yalnız zaten belirlenmiş karakter aralığını kopyalar; belirteç sınırlarını döngü bulur. Sayı boş olmayan belirteç, İşlem dört desteklenen işlem önkoşuluyla çağrılır. Bu koşullar Hesapla tarafından sağlanır. Hata halinde sonuc kullanılmaz; neden yazdırılır. Hesaplama metotları konsola erişmez ve dış değişken yakalamaz.

Örnek çalıştırmalar

Bloklar İfade: isteminden sonraki tam sonuçtur.

8 3 - 2 *:

Sonuç: 10

-7 2 /:

Sonuç: -3

+0042:

Sonuç: 42

3 -:

Hata: Eksik işlenen.

1 0 /:

Hata: Sıfıra bölme.

2147483647 1 +:

Hata: Taşma.

Sınır durumları

- Boş satır veya yalnız normal boşluk için Hata: Boş ifade. yazılır.
- 2 3 için Hata: Artan işlenen. yazılır; iki bağımsız sayı sonuç oluşturmaz.
- 2147483648 veya 2.5 için Hata: Geçersiz belirteç. yazılır.
- -2147483648 -1 / bölmesi int aralığını aşar; taşma hatasıdır.
- 3 8 - sonucu -5'tir; yığından çıkarma sırası ters çevrilemez.
- Birden fazla, baştaki veya sondaki boşluk boş belirteç üretmez.
- Satır sonundaki sekme veya Türkçe harf, değerlendirme başlamadan giriş hatası üretir.
- İzin verilen uzunluk içinde baştaki sıfırlar sayı büyüklüğünü değiştirmez.

Kazanımlar

- Birleşik bir uygulamayı ayrıştırma, değerlendirme ve aritmetik sözleşmelerine bölme.
- Yığınla işlem sırasını ve işlenen yönünü koruma.
- Aynı başarı/veri/hata aktarım biçimini metotlar arasında kullanma.
- Sayı belirteci hatası ile hesaplama taşmasını ayırma.
- Giriş doğrulamasını ve sonuç yazdırmayı hesaplamanın dışında yönetme.

Alıştırmalar

1. Hatalı belirtecin sıfır tabanlı başlangıç dizinini ek bir out parametresiyle bildirin.
2. Kalan işlem % desteği ekleyin; sıfır sağ işlenen ve negatif değer davranışını tanımlayın.
3. Her işlemden sonraki yığın durumunu ayrı izleme modunda yazdırın; normal modda yalnız sonucun yazılmasını koruyun.

Bölüm 9: Dosyalar ve ilişkili veri kümeleri

081 Dosyanın satır envanteri

Düzey: İleri Önkoşullar: 062, 073

Kavramlar: ReadLine ve EOF, Fiziksel satır, UTF-8, using ile kaynak kapatma

Problem tanımı

Bir metin dosyasını inceleyen araç yazın. Fiziksel satır sayısını, uzunluğu sıfır olan boş satırları ve en uzun satırın uzunluğu ile ilk satır numarasını bulun. Boşluk içeren satır boş sayılmaz. Metni değiştirmeyin; amaç dosyanın yapısını ölçmektir.

Girdi ve çıktı

Tek yol: ornek-081/metin.txt. Dosyada serbest UTF-8 metin bulunabilir. Çıktı dört satırdır: Satır, Boş, En uzun, İlk konum. Boş dosyada ilk konum 0 olur; bu gerçek bir satır numarası değildir.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Görelî yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Algoritma

1. Tek yol argümanını denetleyip dosyayı Oku ile okuyun.
2. Her fiziksel satır için boşluk ve uzunluk ölçülerini güncelleyin.
3. Yalnız daha uzun satır geldiğinde adayın konumunu değiştirin.
4. Dosyanın tamamı doğrulanınca dört ölçüyü yazın.

Satır	Metin	Boş sayısı	En uzun	İlk konum
1	Ada	0	3	1
2	boş	1	3	1
3	Deniz	1	5	3
4	Ekin	1	5	3

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] satirlar = Oku(args[0], out int n);
    int bos = 0, uzun = 0, ilk = 0;
    for (int i = 0; i < n; i++)
    {
        if (satirlar[i].Length == 0) bos++;
        if (ilk == 0 || satirlar[i].Length > uzun)
        {
            uzun = satirlar[i].Length;
            ilk = i + 1;
        }
    }
    Console.WriteLine("Satır: " + n.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Boş: " + bos.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("En uzun: " + uzun.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("İlk konum: " + ilk.ToString(CultureInfo.InvariantCulture));
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}
```

```
static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}
```

Örnek çalıştırmalar

dotnet run -- ornek-081/metin.txt; dosya (2. satır boştur):

Ada

Deniz

Ekin

Konsolun tam çıktısı:

Satır: 4

Boş: 1

En uzun: 5

İlk konum: 3

Aynı yol, sıfır baytlık dosya:

Konsolun tam çıktısı:

Satır: 0

Boş: 0

En uzun: 0

İlk konum: 0

Dosyada yalnız bir boş satır (tek LF):

Konsolun tam çıktısı:

Satır: 1

Boş: 1

En uzun: 0

İlk konum: 1

Dosyanın ilk satırı 121 adet A:

Konsolun tam çıktısı:

Hata: metin.txt: satır 1: Satır en fazla 120 karakter.

Sınır durumları

- Yinelenen satırlar geçerlidir; bu problemde anahtar yoktur. Eşit uzunlukta ilk satır korunur.
- Uzunluk görsel harf sayısı değildir: emoji gibi bir simge iki UTF-16 kod birimi kullanılabilir.
- Sayaçlar en fazla 100, uzunluk en fazla 120 olduğundan int taşması olmaz.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- EOF ile boş fiziksel satırı ayrı ele alma.
- Dosya okuma ve ölçüm işlemlerini static yerel fonksiyonlarla ayırma.
- Hata halinde using ile kaynağın kapandığını açıklama.

Alıřtırmalar

1. Bořluklardan oluřan satırları ayrı bir sayaçla raporlayın.
2. Tamamen boş satırlardan oluřan dosyada ilk konumun neden 1 olduđunu açıklayın.

082 Sensör dosyasında artış serisi

Düzey: İleri Önkoşullar: 021, 075, 081

Kavramlar: Satır doğrulama, Ardışık ölçümler, out ile okunan adet, Kültürden bağımsız çıktı

Problem tanımı

Her satırında bir tamsayı ölçüm olan sensör dosyasından toplamı, ortalamayı ve ardışık kesin artan en uzun ölçüm serisinin uzunluğunu bulun. Eşit veya azalan ölçüm seriyi yeniden 1 yapar. Sonuç bir zaman serisinin sürekliliğini ölçer; değerleri sıralamayın.

Girdi ve çıktı

Tek yol: ornek-082/olcum.txt. Her satır -1000..1000 tamsayıdır; boş satır geçersizdir. Boş dosya kabul edilir: toplam 0, ortalama Yok, seri uzunluğu 0. Ortalama iki ondalık basamakla yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Ölçüm isteğe bağlı + veya - işareti ve ASCII rakamlardan oluşur; baştaki sıfırlar kabul edilir. Çevre boşluğu, binlik ayırıcı ve ondalık ölçüm geçersizdir. Taşan sayı ayrıştırma hatasıdır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır.

Algoritma

1. Dosyayı okuyun; her satırı aralığıyla doğrulayın.
2. Toplama ekleyin. İlk ölçümde veya artış bozulduğunda seriyi 1 yapın.
3. Artışta seriyi uzatıp en uzun adayı güncelleyin.
4. EOF sonrasında boş veri durumunu ayırıp özeti yazın.

Ölçüm	Seri uzunluğu	En uzun	Toplam
2	1	1	2
5	2	2	7
5	1	2	12
8	2	2	20
9	3	3	29

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int toplam = 0, onceki = 0, seri = 0, enUzun = 0;
    for (int i = 0; i < n; i++)
    {
        int x = Tam(args[0], i + 1, s[i], -1000, 1000);
        toplam += x;
        seri = i > 0 && x > onceki ? seri + 1 : 1;
        if (seri > enUzun) enUzun = seri;
        onceki = x;
    }
    Console.WriteLine("Toplam: " + toplam.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Ortalama: " + (n == 0 ? "Yok" :
        ((decimal)toplam / n).ToString("0.00", CultureInfo.InvariantCulture)));
    Console.WriteLine("Artan seri: " + enUzun.ToString(CultureInfo.InvariantCulture));
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}
```

```
static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}
```

```
static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}
```

Örnek çalıştırmalar

dotnet run -- ornek-082/olcum.txt; dosya:

```
2
5
5
8
9
```

Konsolun tam çıktısı:

```
Toplam: 29
Ortalama: 5.80
Artan seri: 3
```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

```
Toplam: 0
Ortalama: Yok
Artan seri: 0
```

Tek ölçüm:

```
-1000
```

Konsolun tam çıktısı:

```
Toplam: -1000
Ortalama: -1000.00
Artan seri: 1
```

Geçersiz ikinci satır:

```
1
x
```

Konsolun tam çıktısı:

```
Hata: olcum.txt: satır 2: Geçersiz tamsayı.
```

Sınır durumları

- Tekrar eden değerler geçerlidir fakat kesin artışı bozar; anahtar yoktur.
- Boş satır EOF değildir ve tamsayı hatasıdır. 1001 veya int sınırını aşan belirteç de aynı hatayı verir.
- Toplamın mutlak değeri en fazla 100000, seri uzunluğu en fazla 100 olur.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Dosya sırasını anlamlı bir veri ilişkisi olarak koruma.
- Sonuçları bütün satırlar doğrulandıktan sonra yazma.
- Önceki değer ile geçerli seri durumunu birlikte izleme.

Alıştırmalar

1. En uzun serinin ilk başlangıç satırını da raporlayın.
2. Eşit değerlere izin verilen azalmayan seri için koşulu değiştirin.

083 Stok dosyasında kayıt doğrulama

Düzey: İleri Önkoşullar: 068, 075, 082

Kavramlar: Noktalı virgülle alan ayırma, Kayıt sözleşmesi, Yinelenen anahtar, decimal tutar

Problem tanımı

Küçük bir stok dosyasında ürünlerin adet ve birim bedellerini doğrulayın; ürün sayısını ve toplam stok değerini hesaplayın. Bir kayıt yalnız bütün alanları ve benzersiz ürün kodu kabul edildiğinde hesaplamaya girsin.

Girdi ve çıktı

Tek yol: örnek-083/stok.txt. Başlık yoktur; her satır kod;adet;bedel biçimindedir. Kod 1..9999 benzersiz tamsayı, adet 0..1000 tamsayı, bedel 0..10000 ondalıktır. Tırnak ve kaçış yoktur. Kodun metinsel biçimi değil sayısal değeri anahtardır: 01 ile 1 aynı koddur. Boş dosyada ürün 0 ve değer 0.00 yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göreler.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağlı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağlı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır. Taşan sayı ayrıştırma hatasıdır.

Algoritma

1. Her satırı tam üç alana ayırın; eksik veya fazla ayırıcıyı reddedin.
2. Kod, adet ve bedeli sınırlarıyla doğrulayın.
3. Önceki kodlar dizisinde doğrusal arama yapın; tekrar varsa durun.
4. Kodu saklayıp adet × bedeli toplama ekleyin; sonunda iki sonucu yazın.

Satır	Kod	Adet × bedel	Birikmiş değer
1	12	3 × 2.50	7.50
2	7	2 × 4.00	15.50

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int[] kodlar = new int[100];
    decimal toplam = 0;
    for (int i = 0; i < n; i++)
    {
        string[] a = Alanlar(args[0], i + 1, s[i], 3);
        int kod = Tam(args[0], i + 1, a[0], 1, 9999);
        int adet = Tam(args[0], i + 1, a[1], 0, 1000);
        decimal bedel = Ondalik(args[0], i + 1, a[2], 0, 10000);
        if (Bul(kodlar, i, kod) >= 0)
            Hata(args[0], i + 1, "Yinelenen kod.");
        kodlar[i] = kod;
        toplam += adet * bedel;
    }
    Console.WriteLine("Ürün: " + n.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Değer: " + toplam.ToString("0.00", CultureInfo.InvariantCulture));
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
```

```

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

```

```

static decimal Ondalik(string yol, int satir, string metin,
    decimal alt, decimal ust)
{
    int bas = metin.StartsWith('-') || metin.StartsWith('+') ? 1 : 0;
    int nokta = -1, rakam = 0;
    for (int i = bas; i < metin.Length; i++)
    {
        if (metin[i] >= '0' && metin[i] <= '9') rakam++;
        else if (metin[i] == '.' && nokta == -1) nokta = i;
        else Hata(yol, satir, "Geçersiz ondalık.");
    }
    if (rakam == 0 || nokta == bas ||
        (nokta >= 0 && (metin.Length - nokta - 1 < 1 ||
            metin.Length - nokta - 1 > 2)))
        Hata(yol, satir, "Geçersiz ondalık.");
    if (!decimal.TryParse(metin,
        NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint,
        CultureInfo.InvariantCulture, out decimal deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz ondalık.");
    return deger;
}

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

```

Örnek çalıştırmalar

dotnet run -- ornek-083/stok.txt; dosya:

```

12;3;2.50
7;2;4

```

Konsolun tam çıktısı:

```

Ürün: 2
Değer: 15.50

```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

```

Ürün: 0
Değer: 0.00

```

Kod tekrarı:

```

1;0;0
01;1;1

```

Konsolun tam çıktısı:

```

Hata: stok.txt: satır 2: Yinelenen kod.

```

Virgüllü bedel:

```

1;1;2,50

```

Konsolun tam çıktısı:

```

Hata: stok.txt: satır 1: Geçersiz ondalık.

```

Sınır durumları

- Boş satır alan sayısı hatasıdır. Boş alan, çevre boşluğu, üçten fazla alan veya geçersiz sayı kabul edilmez.
- Sıfır adet ve sıfır bedel geçerlidir; boş dosyayla tek sıfır kayıt aynı ürün sayısını vermez.
- En büyük toplam $100 \times 1000 \times 10000 = 1000000000$ olur; decimal güvenle taşır.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Alan sayısını, değer aralığını ve anahtar benzersizliğini ayrı doğrulama.
- Paralel veri yerine yalnız gereken kod dizisini saklama.
- Ondalık dilbilgisini kültürden bağımsız tutma.

Alıştırmalar

1. Hata mesajlarına 1 tabanlı alan numarası ekleyin.
2. En değerli ürünün ilk kodunu da raporlayın.

084 Tarihli ölçümlerde eksik günler

Düzey: İleri Önkoşullar: 019, 020, 083

Kavramlar: TryParseExact, Kesin artan tarih, Gün farkı, Eksik gözlem

Problem tanımı

Günde bir kez alınan sıcaklık kayıtlarında ilk ve son tarihi, kayıt sayısını ve arada ölçüm yapılmayan gün sayısını bulun. Tarihler kesin artan sırada olmalıdır. Aynı güne iki kayıt kabul etmeyin; dosyayı sıralamak veri üretim hatasını gizlemesin.

Girdi ve çıktı

Tek yol: `ornek-084/gunler.txt`. Her satır `yyyy-MM-dd;sıcaklık` biçimindedir; tarih `0001-01-01..9999-12-31` arasında geçerli Gregoryen tarih, sıcaklık `-100..100` ondalıktır. Başlık ve boş satır yoktur. Boş dosyada Kayıt: 0 ve Aralık: Yok, Eksik gün: 0 yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin `Program.cs` dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen `dotnet run --` sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. `ReadLine` satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. `using` blokları hata halinde de akışı ve okuyucuyu kapatır.

Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağlı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağlı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı `CultureInfo.InvariantCulture` kullanır. Taşan sayı ayrıştırma hatasıdır.

Algoritma

1. Satırı iki alana ayırıp tarihi tam `yyyy-MM-dd` biçimiyle ve sıcaklığı doğrulayın.
2. İlk kaydın tarihini saklayın. Sonrakilerde tarih önceki tarihten büyük olmalıdır.
3. İki tarih arasındaki gün farkından 1 çıkarıp eksik güne ekleyin.
4. Son tarihi güncelleyin ve tüm dosya kabul edildiğinde aralığı yazın.

Tarih	Öncekiyle fark	Yeni eksik	Birikmiş eksik
2024-02-28	ilk kayıt	0	0
2024-03-01	2	1	1
2024-03-04	3	2	3

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    DateTime ilk = default, onceki = default;
    int eksik = 0;
    for (int i = 0; i < n; i++)
    {
        string[] a = Alanlar(args[0], i + 1, s[i], 2);
        if (!DateTime.TryParseExact(a[0], "yyyy-MM-dd",
            CultureInfo.InvariantCulture, DateTimeStyles.None,
            out DateTime tarih)) Hata(args[0], i + 1, "Geçersiz tarih.");
        Ondalik(args[0], i + 1, a[1], -100, 100);
        if (i == 0) ilk = tarih;
        else
        {
            if (tarih <= onceki)
                Hata(args[0], i + 1, "Tarihler kesin artmalı.");
            eksik += (tarih - onceki).Days - 1;
        }
        onceki = tarih;
    }
    Console.WriteLine("Kayıt: " + n.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Aralık: " + (n == 0 ? "Yok" :
        ilk.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture) + " / " +
        onceki.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)));
    Console.WriteLine("Eksik gün: " + eksik.ToString(CultureInfo.InvariantCulture));
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
```

```

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

```

```

static decimal Ondalik(string yol, int satir, string metin,
    decimal alt, decimal ust)
{
    int bas = metin.StartsWith('-') || metin.StartsWith('+') ? 1 : 0;
    int nokta = -1, rakam = 0;
    for (int i = bas; i < metin.Length; i++)
    {
        if (metin[i] >= '0' && metin[i] <= '9') rakam++;
        else if (metin[i] == '.' && nokta == -1) nokta = i;
        else Hata(yol, satir, "Geçersiz ondalık.");
    }
    if (rakam == 0 || nokta == bas ||
        (nokta >= 0 && (metin.Length - nokta - 1 < 1 ||
            metin.Length - nokta - 1 > 2)))
        Hata(yol, satir, "Geçersiz ondalık.");
    if (!decimal.TryParse(metin,
        NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint,
        CultureInfo.InvariantCulture, out decimal deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz ondalık.");
    return deger;
}

```

Örnek çalıştırmalar

dotnet run -- ornek-084/gunler.txt; dosya:

```

2024-02-28;1.5
2024-03-01;-2
2024-03-04;0

```

Konsolun tam çıktısı:

```

Kayıt: 3
Aralık: 2024-02-28 / 2024-03-04
Eksik gün: 3

```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

```

Kayıt: 0
Aralık: Yok
Eksik gün: 0

```

Tek tarih:

```

0001-01-01;100

```

Konsolun tam çıktısı:

```

Kayıt: 1
Aralık: 0001-01-01 / 0001-01-01
Eksik gün: 0

```

Geçersiz artık gün:

```

2023-02-29;4

```

Konsolun tam çıktısı:

```

Hata: gunler.txt: satır 1: Geçersiz tarih.

```


Sınır durumları

- Aynı tarih ve geriye giden tarih aynı sıralılık hatasıdır. 2024-2-01, çevre boşluğu ve saat alanı kabul edilmez.
- Eksik gün yalnız ilk ve son kayıt arasında sayılır; dosya öncesi ve sonrası için gözlem dönemi varsayılmaz.
- DateTime farkı tüm kabul edilen aralıkta en fazla 3652058 gündür; toplam eksik de bu aralığı aşmaz. Son tarihe gün eklenmediği için üst tarih sınırında taşma yoktur.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Takvim doğrulamasını sabit bir metin biçimine bağlama.
- Sıralılık önkoşulunu dosya kaynağında denetleme.
- Eksik gözlem ile sıfır sıcaklık ölçümünü ayırt etme.

Alıştırmalar

1. En uzun eksik gün aralığını başlangıç ve bitiş tarihleriyle raporlayın.
2. Aynı gün tekrarı ile geriye gidiş için farklı hata mesajları üretin.

085 Dosyada ardışık veri paketleri

Düzey: İleri Önkoşullar: 073, 075, 083

Kavramlar: Çoklu veri kümesi, Başlık ve gövde, Beklenen kayıt sayısı, Eksik EOF

Problem tanımı

Tek dosyada art arda yer alan ölçüm paketlerini çözümleyin. Her paket kendi kimliği, ölçüm adedi ve ölçümleriyle bağımsızdır. Paket toplamalarını dosyadaki sırayla üretin; önceki paketin toplamı yeni pakete taşınmasın.

Girdi ve çıktı

Tek yol: ornek-085/paketler.txt. Paket başlığı kimlik;n, ardından tam n satır ölçüm gelir. Kimlik 1..9999 benzersiz, n 0..20, ölçüm -1000..1000 tamsayıdır. En fazla 20 paket ve ortak 100 fiziksel satır sınırı vardır. Sıfır ölçümlü paket yalnız başlıktan oluşur. Başlık veya ölçümde boş satır hatadır. Boş dosyanın çıktısı Paket: Yok olur.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Tamsayılar isteğe bağlı + veya - işareti ve ASCII rakamlardan oluşur; baştaki sıfırlar kabul edilir. Çevre boşluğu, binlik ayırıcı ve ondalık sayı geçersizdir. Taşan sayı ayrıştırma hatasıdır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır.

Algoritma

1. Okuma konumu p sıfırdan başlasın; EOF sonrasında yeni başlık beklenmez.
2. Başlığı doğrulayın; kodun benzersizliğini ve n ölçüm için yeterli satırı denetleyin.
3. Yeni toplamı sıfırlayıp sonraki n satırı okuyun; ölçüm hatasında gerçek fiziksel konumu bildirin.
4. Paket sonucunu bir çıktı tamponuna ekleyin; p sonraki başlığa ilerlesin.
5. Bütün paketler geçerliyse tamponu yazın.

Başlık satırı	Başlık	Ölçüm satırları	Sonraki başlık konumu
1	7;2	2, 3	4
4	9;0	yok	5
5	4;1	6	EOF

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int[] kodlar = new int[20];
    int p = 0, paket = 0;
    var sonuc = new StringBuilder();
    while (p < n)
    {
        int baslik = p + 1;
        if (paket == 20) Hata(args[0], baslik, "En fazla 20 paket.");
        string[] a = Alanlar(args[0], baslik, s[p++], 2);
        int kod = Tam(args[0], baslik, a[0], 1, 9999);
        int adet = Tam(args[0], baslik, a[1], 0, 20);
        if (Bul(kodlar, paket, kod) >= 0)
            Hata(args[0], baslik, "Yinelenen kod.");
        if (n - p < adet)
            Hata(args[0], n + 1, "Paket tamamlanmadan EOF.");
        int toplam = 0;
        for (int j = 0; j < adet; j++)
        {
            toplam += Tam(args[0], p + 1, s[p], -1000, 1000);
            p++;
        }
        kodlar[paket++] = kod;
        sonuc.AppendLine(kod.ToString(CultureInfo.InvariantCulture) +
            ";" + adet.ToString(CultureInfo.InvariantCulture) +
            ";" + toplam.ToString(CultureInfo.InvariantCulture));
    }
    if (paket == 0) Console.WriteLine("Paket: Yok");
    else Console.Write(sonuc.ToString());
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
```

```

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

```

Örnek çalıştırmalar

dotnet run -- ornek-085/paketler.txt; dosya:

```
7;2
4
-1
9;0
4;1
10
```

Konsolun tam çıktısı:

```
7;2;3
9;0;0
4;1;10
```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

Paket: Yok

Eksik paket:

```
7;2
4
```

Konsolun tam çıktısı:

Hata: paketler.txt: satır 3: Paket tamamlanmadan EOF.

Boş ölçüm satırı (dosya tam iki LF ile biter):

```
7;1
```

Konsolun tam çıktısı:

Hata: paketler.txt: satır 2: Geçersiz tamsayı.

Sınır durumları

- Başlıkta $n=0$ EOF anlamına gelmez; sonraki satır yeni bir başlıktır. $n=20$ geçerlidir.
- Eksik ölçümde hata konumu EOF sonrası beklenen ilk satırdır. Eksik adet önce denetlendiği için yetersiz dosyada içteki olası sayı hatalarından önce EOF hatası gelir.
- Başlık fazla alanlıysa başlık hatasıdır; ölçüm satırında noktalı virgül tamsayı hatasıdır. Paket kodları sayısal olarak benzersiz olmalıdır.
- Paket toplamı mutlak en fazla 20000; toplamlar paketler arasında birleştirilmez.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Başlıkla bildirilen veri uzunluğunu fiziksel satırlarla eşleştirme.
- Paket başında durum değişkenlerini sıfırlama.
- Son pakette eksik EOF ile normal EOF farkını gösterme.

Alıştırmalar

1. Her paket için boş değilse minimum ve maksimumu da üretin.
2. Paket adedini ilk satırda bildiren alternatif bir biçim tasarlayın.

086 Tırnaklı CSV kayıtlarını çözömlleme

Düzey: İleri Önkoşullar: 066, 068, 080, 083

Kavramlar: Durumla ayrıştırma, Kaçırılmış tırnak, Sütun konumu, CSV alt kümesi

Problem tanımı

Bir ürün açıklama dosyasında virgöl içeren ve çift tırnakla kaçırılmış açıklamaları çözömlleyin. Her kaydın kodunu ve açılmış açıklamasını yazın. Basit Split virgölü alan ayırıcıyla içerikteki virgölden ayıramaz; bu ayrımı karakter karakter kurun.

Girdi ve çıktı

Tek yol: ornek-086/aciklamalar.csv. Desteklenen CSV alt kümesi tam kod,"açıklama" biçimindedir. Kod 1..9999 benzersiz tamsayı; açıklama 1..60 yazdırılabilir ASCII karakteridir (32..126). Açıklama içindeki çift tırnak dosyada iki çift tırnakla yazılır. Çok satırlı alan, başlık, üçüncü alan, alan çevresinde fazladan boşluk ve boş satır desteklenmez. Çıktı kod: açılmış açıklama; boş dosyada Kayıt: Yok olur. UTF-8 dosya kullanılmasına rağmen açıklama dilbilgisi bu problem için ASCII ile sınırlıdır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değıştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğeri I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır. Taşan sayı ayrıştırma hatasıdır.

Algoritma

1. İlk virgöl karakter döngüsüyle bulun; ondan önceki kodu doğrulayın.
2. Virgülden hemen sonra açılış tırnağı bekleyin.
3. Normal karakteri tampona ekleyin; çift tırnak çifti tek tırnak üretir.
4. Eşlenmemiş tırnak kapanıştır; ardından satır sonu gelmelidir.
5. Kapanış, açıklama uzunluğu ve kod benzersizliği doğrulanınca kayıt sonucunu tampona alın.
6. Bütün dosya kabul edildiğinde sonuçları yazın.

Okunan bölüm	Eylem	Açılmış tampon
"	açılış	boş
A,	içerik	A,
""	kaçırılmış tırnak	A,"
B	içerik	A,"B
""	kaçırılmış tırnak	A,"B"
" ve satır sonu	kapanış	A,"B"

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int[] kodlar = new int[100];

    var sonuc = new StringBuilder();
    for (int i = 0; i < n; i++)
    {
        Coz(args[0], i + 1, s[i], out int kod, out string aciklama);
        if (Bul(kodlar, i, kod) >= 0)
            Hata(args[0], i + 1, "Yinelenen kod.");
        kodlar[i] = kod;
        sonuc.AppendLine(kod.ToString(CultureInfo.InvariantCulture) +
            ": " + aciklama);
    }
}
```

```

        if (n == 0) Console.WriteLine("Kayıt: Yok");
        else Console.Write(sonuc.ToString());
    }
    catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
    catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
    catch (UnauthorizedAccessException)
    { Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
    catch (ArgumentException)
    { Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
    catch (NotSupportedException)
    { Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

```



```

static void Coz(string yol, int satir, string s,
    out int kod, out string aciklama)
{
    int p = 0;
    while (p < s.Length && s[p] != ',') p++;
    if (p == s.Length) CsvHata(yol, satir, p + 1);
    kod = Tam(yol, satir, s.Substring(0, p), 1, 9999);
    p++;
    if (p == s.Length || s[p] != '"') CsvHata(yol, satir, p + 1);
    p++;
    var metin = new StringBuilder();
    bool kapandi = false;

    while (p < s.Length)
    {
        char c = s[p++];
        if (c == '"')
        {
            if (p < s.Length && s[p] == '"')
            {
                metin.Append('"');
                p++;
            }
            else { kapandi = true; break; }
        }
        else
        {
            if (c < 32 || c > 126) CsvHata(yol, satir, p);
            metin.Append(c);
        }
        if (metin.Length > 60) CsvHata(yol, satir, p);
    }

    if (!kapandi || p != s.Length || metin.Length == 0)
        CsvHata(yol, satir, p + 1);
    aciklama = metin.ToString();
}

static void CsvHata(string yol, int satir, int sutun)
{
    Hata(yol, satir, "CSV sütun " +
        sutun.ToString(CultureInfo.InvariantCulture) + ": Geçersiz kayıt.");
}

```

Örnek çalıştırmalar

dotnet run -- ornek-086/aciklamalar.csv; dosya:

```

7,"Vida, büyük"
2,"A,""B""""

```

Konsolun tam çıktısı:

```

7: Vida, büyük
2: A,"B"

```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

Kayıt: Yok

Açılmayan tırnak:

1,abc

Konsolun tam çıktısı:

Hata: aciklamalar.csv: satır 1: CSV sütun 3: Geçersiz kayıt.

Kapanış sonrası üçüncü alan:

1,"a",b

Konsolun tam çıktısı:

Hata: aciklamalar.csv: satır 1: CSV sütun 6: Geçersiz kayıt.

Sınır durumları

- Hata sütunu 1 tabanlı UTF-16 konumudur. Beklenen karakter EOF ise uzunluk + 1 verilir; kod sayısı ve tekrar hataları yalnız satır bildirir.
- Boş açıklama geçersizdir; açıklama içindeki boşluk veya virgöl geçerlidir. Açılmış uzunluk 60, fiziksel satır 120 sınırlarını ayrı düşünün: tümü tırnak olan 60 karakterin kaçırılmış biçimi fiziksel sınırı aşabilir.
- İki art arda tırnak içerikte tek tırnaktır; kapanış için ek bir tırnak gerekir.
- Anahtarlar sayısal olarak benzersizdir. Sayaçlar ve tampon bu sınırlarla taşmaz.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Alan ayırıcı ile içerikteki ayırıcıyı durum üzerinden ayırt etme.
- İlk sözdizimi hatasını sütun konumuyla raporlama.
- CSV standardının açıkça tanımlanmış bir alt kümesini uygulama.

Alıştırmalar

1. Açıklama dışında isteğe bağlı boşluk için ayrı kurallar ekleyin.
2. Bir satır sonunu tırnaklı alan içinde desteklemek için fiziksel ve mantıksal kayıt ayrımını açıklayın.

087 Hesap hareketlerini sırayla denetleme

Düzey: İleri Önkoşullar: 080, 083

Kavramlar: Sıralı durum geçişi, İşlem kimliği, İş kuralı doğrulama, Kısmi sonuçtan kaçınma

Problem tanımı

Başlangıç bakiyesi sıfır olan bir deneme hesabının hareket dosyasını denetleyin. Para yatırma ve çekme işlemlerini dosya sırasıyla uygulayın; hiçbir ara bakiye negatif veya bir milyonun üstünde olmasın. İşlem kodu tekrarlanırsa aynı hareket iki kez uygulanmasın.

Girdi ve çıktı

Tek yol: ornek-087/hareket.txt. Satır işlemKodu;tür;tutar: kod 1..9999 benzersiz, tür tam D (yatırma) veya W (çekme), tutar 0.01..10000 ondalıktır. Başlık yoktur. Bakiye her adımda 0..1000000 aralığındadır. Gerçek banka hesabına erişim yoktur; yalnız verilen deneme dosyası hesaplanır. Boş dosyada hareket 0, bakiye 0.00 yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağlı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağlı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır. Taşan sayı ayrıştırma hatasıdır.

Algoritma

1. Alanları ve işlem kimliğini doğrulayın; kodun önceki hareketlerde bulunmadığını denetleyin.
2. Türü ve tutarı okuyun; D için ekleyip W için çıkararak aday bakiye oluşturun.
3. Aday sınırı ihlal ediyorsa o satırda durun; bakiye değişmesin.
4. Adayı kabul edin; bütün dosya geçince hareket adedi ve son bakiyeyi yazın.

Satır	Hareket	Önceki bakiye	Aday	Karar
1	D 20.00	0.00	20.00	kabul
2	W 7.50	20.00	12.50	kabul
3	W 12.50	12.50	0.00	kabul

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 1) throw new FormatException("Bir giriş yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int[] kodlar = new int[100];
    decimal bakiye = 0;
    for (int i = 0; i < n; i++)
    {
        string[] a = Alanlar(args[0], i + 1, s[i], 3);
        int kod = Tam(args[0], i + 1, a[0], 1, 9999);
        if (Bul(kodlar, i, kod) >= 0)
            Hata(args[0], i + 1, "Yinelenen kod.");
        if (a[1] != "D" && a[1] != "W")
            Hata(args[0], i + 1, "Tür D veya W olmalı.");
        decimal tutar = Ondalık(args[0], i + 1, a[2], 0.01m, 10000);
        decimal aday = a[1] == "D" ? bakiye + tutar : bakiye - tutar;
        if (aday < 0 || aday > 1000000)
            Hata(args[0], i + 1, "Bakiye sınırı aşıldı.");
        bakiye = aday;
        kodlar[i] = kod;
    }
    Console.WriteLine("Hareket: " + n.ToString(CultureInfo.InvariantCulture));
    Console.WriteLine("Bakiye: " + bakiye.ToString("0.00", CultureInfo.InvariantCulture));
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
```

```

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

```

```

static decimal Ondalik(string yol, int satir, string metin,
    decimal alt, decimal ust)
{
    int bas = metin.StartsWith('-') || metin.StartsWith('+') ? 1 : 0;
    int nokta = -1, rakam = 0;
    for (int i = bas; i < metin.Length; i++)
    {
        if (metin[i] >= '0' && metin[i] <= '9') rakam++;
        else if (metin[i] == '.' && nokta == -1) nokta = i;
        else Hata(yol, satir, "Geçersiz ondalık.");
    }
    if (rakam == 0 || nokta == bas ||
        (nokta >= 0 && (metin.Length - nokta - 1 < 1 ||
            metin.Length - nokta - 1 > 2)))
        Hata(yol, satir, "Geçersiz ondalık.");
    if (!decimal.TryParse(metin,
        NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint,
        CultureInfo.InvariantCulture, out decimal deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz ondalık.");
    return deger;
}

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

```

Örnek çalıştırmalar

dotnet run -- ornek-087/hareket.txt; dosya:

```

10;D;20
11;W;7.50
12;W;12.50

```

Konsolun tam çıktısı:

```

Hareket: 3
Bakiye: 0.00

```

Sıfır baytlık dosya:

Konsolun tam çıktısı:

```

Hareket: 0
Bakiye: 0.00

```

Yetersiz bakiye:

```

1;D;5
2;W;5.01

```

Konsolun tam çıktısı:

```

Hata: hareket.txt: satır 2: Bakiye sınırı aşıldı.

```

Yinelenen hareket:

```

1;D;5
1;D;3

```

Konsolun tam çıktısı:

```

Hata: hareket.txt: satır 2: Yinelenen kod.

```

Sınır durumları

- W ilk satırda geçerli pozitif tutarla daima bakiye hatasıdır. Tam bakiyeyi çekmek geçerlidir; sıfır tutar geçersizdir.
- Boş satır alan sayısı hatasıdır. d veya w küçük harfleri kabul edilmez.
- En fazla $100 \times 10000 = 1000000$ yatırılabilir; decimal ara hesapları taşmaz. Üst iş kuralı sınırı da açıkça denetlenir.
- Önceki geçerli hareketlerin özeti hata halinde yazılmaz; giriş dosyası hiçbir durumda güncellenmez.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Kayıt doğruluğu ile iş kuralı doğruluğunu ayrı ele alma.
- Önce aday durumu hesaplayıp kabul edildiğinde atama yapma.
- İşlem sırasının sonuç üzerindeki etkisini açıklama.

Alıştırmalar

1. Kabul edilen ara bakiyeleri yalnız tam başarıda yazılan bir izleme raporuna ekleyin.
2. Başlangıç bakiyesini ikinci argümandan almak için sözleşme oluşturun.

088 İki sayım dosyasını uzlaştırma

Düzey: İleri Önkoşullar: 046, 055, 075, 083

Kavramlar: İki dosyada anahtar eşleme, Sıralılık sözleşmesi, Dış birleştirme, Dosya kaynağına göre hata

Problem tanımı

İki bağımsız depo sayımını ürün koduna göre uzlaştırın. Her iki dosyada bulunan üründe sağ adet eksi sol adet farkını, yalnız bir dosyada bulunan üründe SOL veya SAG durumunu üretin. Aynı adede sahip ürünleri de ORTAK 0 olarak gösterin; tek dosyada görünmeyen ürünün adedini sıfır varsaymayın.

Girdi ve çıktı

İki yol: ornek-088/sol.txt ornek-088/sag.txt. Her satır kod;adet: kod 1..9999, adet 0..1000. Her dosyada kodlar kesin artmalı ve benzersiz olmalıdır. İki dosyada aynı kod eşleştirilir. Çıktı artan kod sırasıyla kod;ORTAK;fark, kod;SOL;adet veya kod;SAG;adet olur. İki dosya boşsa Kayıt: Yok yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen dotnet run -- sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göreler.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

İsteğe bağlı işaret ve ASCII rakamlar kabul edilir. Boşluk, binlik ayırıcı ve ondalık sayı geçersizdir. Biçim kültürden bağımsızdır.

Algoritma

1. Önce sol sonra sağ dosyayı tamamen doğrulayıp paralel dizilere okuyun.
2. İki okuma konumu sıfırdan başlasın.
3. Kodlar eşitse farkı üretilen iki konumu ilerletin.
4. Küçük kodun bulunduğu tarafta tek taraflı satır üretilen yalnız o konumu ilerletin.
5. Biten tarafın ardından diğer tarafın kalan kayıtlarını tamamlayın.

Sol kod	Sağ kod	Üretilen kayıt	İlerleyen konum
1	2	1;SOL;3	sol
3	2	2;SAG;4	sağ

Sol kod	Sağ kod	Üretilen kayıt	İlerleyen konum
3	3	3;ORTAK;-1	ikisi

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 2) throw new FormatException("İki giriş yolu gerekli.");
    Ciftler(args[0], out int[] sol, out int[] sa, out int sn);
    Ciftler(args[1], out int[] sag, out int[] ga, out int gn);
    var sonuc = new StringBuilder();
    int i = 0, j = 0;
    while (i < sn || j < gn)
    {
        if (i < sn && j < gn && sol[i] == sag[j])
        {
            Satir(sonuc, sol[i], "ORTAK", ga[j] - sa[i]);
            i++; j++;
        }
        else if (j == gn || (i < sn && sol[i] < sag[j]))
        {
            Satir(sonuc, sol[i], "SOL", sa[i]); i++;
        }
        else { Satir(sonuc, sag[j], "SAG", ga[j]); j++; }
    }
    if (sn + gn == 0) Console.WriteLine("Kayıt: Yok");
    else Console.Write(sonuc.ToString());
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
```

```

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

```

```

static void Ciftler(string yol, out int[] kod, out int[] adet,
    out int n)
{
    string[] s = Oku(yol, out n);
    kod = new int[100]; adet = new int[100];
    for (int i = 0; i < n; i++)
    {
        string[] a = Alanlar(yol, i + 1, s[i], 2);
        kod[i] = Tam(yol, i + 1, a[0], 1, 9999);
        adet[i] = Tam(yol, i + 1, a[1], 0, 1000);
        if (i > 0 && kod[i] <= kod[i - 1])
            Hata(yol, i + 1, "Kodlar kesin artmalı.");
    }
}

static void Satir(StringBuilder s, int kod, string durum, int deger)
{
    s.AppendLine(kod.ToString(CultureInfo.InvariantCulture) +
        ";" + durum + ";" + deger.ToString(CultureInfo.InvariantCulture));
}

```

Örnek çalıştırmalar

dotnet run -- ornek-088/sol.txt ornek-088/sag.txt; sol.txt:

```

1;3
3;5

```

sag.txt:

```

2;4
3;4

```

Konsolun tam çıktısı:

```

1;SOL;3
2;SAG;4
3;ORTAK;-1

```

Her iki dosya sıfır bayt:

Konsolun tam çıktısı:

```

Kayıt: Yok

```

sol.txt boş; sag.txt:

```

9;0

```

Konsolun tam çıktısı:

```

9;SAG;0

```

sol.txt boş; sag.txt tekrar içeriyor:

```

3;1
3;2

```

Konsolun tam çıktısı:

```

Hata: sag.txt: satır 2: Kodlar kesin artmalı.

```

Sınır durumları

- Dosya içindeki tekrar hata, dosyalar arasındaki tekrar eşleşmedir. Sayısal eşitlik 01 ve 1 biçimlerini de kapsar.
- Sıralılık doğrulanmadan birleştirme başlamaz; sağ dosyanın hatası sol dosyanın geçerli sonuçlarının yazılmasını önler.
- Tek boş dosya diğerinin tüm kayıtlarını tek taraflı üretir; boş satır geçersizdir.
- Çıktı en fazla 200 kayıt; fark -1000..1000 aralığındadır. Okuma ve birleştirme doğrusal, saklama kapasitesi dosya başına 100 kayıttır.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Tam dış eşleştirme ile yalnız kesişim arasındaki farkı açıklama.
- Kısa devre koşullarıyla bitmiş diziyi okumadan yönetme.
- Hatanın hangi dosyadan geldiğini koruma.

Alıştırmalar

1. Yalnız farkı sıfır olmayan ortak ürünleri yazan bir seçenek ekleyin.
2. Aynı kaynağın iki argümanda verilmesi halinde beklenen çıktıyı açıklayın.

089 Katalog ve satışlardan rapor

Düzey: İleri Önkoşullar: 042, 053, 074, 083, 088

Kavramlar: İlişkili dosyalar, Referans doğrulama, Paralel dizilerle sıralama, CreateNew ile rapor

Problem tanımı

Ürün kataloğundaki birim bedelleri satış dosyasındaki adetlerle birleştirip ürün başına toplam satış adedi ve gelir raporu üretin. Katalogda satılmayan ürün de sıfır gelikle yer alsın. Bilinmeyen ürün kodunu sessizce atmayın; satış dosyasındaki konumuyla bildirin.

Girdi ve çıktı

Üç yol: `ornek-089/katalog.txt` `ornek-089/satis.txt` `ornek-089/rapor.txt`. Katalog satırı `kod;bedel: benzersiz kod 1..9999, bedel 0..10000`. Satış satırı `kod;adet: katalogda mevcut kod, adet 1..1000`. Satış kodları tekrarlanabilir ve birleştirilir. Her iki dosya sırasız olabilir. Rapor kod sırasıyla `kod;toplamAdet;gelir` biçiminde, başlıksızdır. Konsola yalnız Rapor yazıldı. yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin `Program.cs` dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen `dotnet run --` sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. `ReadLine` satır sonunu çıkarır: `null` EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. `using` blokları hata halinde de akışı ve okuyucuyu kapatır.

Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağlı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağlı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı `CultureInfo.InvariantCulture` kullanır. Taşan sayı ayrıştırma hatasıdır.

Çıktı yolu üçüncü argümandır; dosya henüz var olmamalıdır ve üst klasörünü siz önceden oluşturmalsınız. `FileMode.CreateNew` mevcut bir dosyayı girişle aynı yol olsa bile ezmez; bu durumda I/O hatası oluşur. Başarı mesajı yalnız yazıcı ve akış kapandıktan sonra verilir. Giriş doğrulama hatasında çıktı dosyası hiç açılmaz. Yazım sırasında disk dolması gibi I/O hatasında yeni dosya kısmi kalabilir; program başarı bildirmez ve bu yeni dosyayı siz denetleyip başka adla tekrar deneyebilirsiniz. Bu küçük öğretim örneği atomik yayın sözü vermez. Çıktı UTF-8 BOM olmadan, LF satır sonlarıyla yazılır; her kayıt son LF ile biter. Bu örnekte kayıtsız rapor sıfır bayttır.

Algoritma

1. Kataloğu kod ve bedel dizilerine okuyun; kod tekrarını reddedin.
2. Satışları doğrulayın; her kodu katalogda doğrusal arayın, bulunamazsa durun.
3. Ürün başına adet dizisini güncelleyin.
4. Eklemeli sıralamada kod, bedel ve adet dizilerini birlikte kaydırın.
5. Rapor metnini oluşturup yeni dosyaya yazın; kaynaklar kapandıktan sonra başarıyı bildirin.

Satış	Katalog dizini	Birikmiş adet	Gelir
2;3	1	3	12.00
7;2	0	2	5.00
2;1	1	4	16.00

Oku yalnız dosya sınırlarını ve kodlamayı denetler, out n ile geçerli satır adedini döndürür. 100 hücrelik dizinin yalnız ilk n hücresi okunabilir; kalan hücreler veri değildir. Hesaplama ana akışta ve göreve özgü static yerel fonksiyonlarda yapılır.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 3) throw new FormatException("İki giriş ve bir çıktı yolu gerekli.");
    string[] k = Oku(args[0], out int kn);
    int[] kod = new int[100], adet = new int[100];
    decimal[] bedel = new decimal[100];

    for (int i = 0; i < kn; i++)
    {
        string[] a = Alanlar(args[0], i + 1, k[i], 2);
        kod[i] = Tam(args[0], i + 1, a[0], 1, 9999);
        bedel[i] = Ondalik(args[0], i + 1, a[1], 0, 10000);
        if (Bul(kod, i, kod[i]) >= 0) Hata(args[0], i + 1, "Yinelenen kod.");
    }

    string[] s = Oku(args[1], out int sn);
    for (int i = 0; i < sn; i++)
    {
        string[] a = Alanlar(args[1], i + 1, s[i], 2);
        int kimlik = Tam(args[1], i + 1, a[0], 1, 9999);
        int miktar = Tam(args[1], i + 1, a[1], 1, 1000);
        int p = Bul(kod, kn, kimlik);
        if (p < 0) Hata(args[1], i + 1, "Katalogda olmayan kod.");
        adet[p] += miktar;
    }
}
```

```

        Sirala(kod, bedel, adet, kn);
        var rapor = new StringBuilder();
        for (int i = 0; i < kn; i++)
            rapor.Append(kod[i].ToString(CultureInfo.InvariantCulture) + ";" +
                adet[i].ToString(CultureInfo.InvariantCulture) + ";" +
                (adet[i] * bedel[i]).ToString("0.00", CultureInfo.InvariantCulture) + "\n");
        Yaz(args[2], rapor.ToString());
        Console.WriteLine("Rapor yazıldı.");
    }
    catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
    catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
    catch (UnauthorizedAccessException)
    { Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
    catch (ArgumentException)
    { Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
    catch (NotSupportedException)
    { Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }

    static string[] Oku(string yol, out int n)
    {
        string[] satirlar = new string[100];
        n = 0;
        using var akis = new FileStream(yol, FileMode.Open,
            FileAccess.Read, FileShare.Read);
        if (akis.Length > 32768) throw new FormatException(
            Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
        using var okuyucu = new StreamReader(akis,
            new UTF8Encoding(false, true), false);
        string? satir;
        while ((satir = okuyucu.ReadLine()) is not null)
        {
            if (n == 0 && satir.StartsWith('\uFEFF'))
                satir = satir.Substring(1);
            if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
            if (satir.Length > 120) Hata(yol, n + 1,
                "Satır en fazla 120 karakter.");
            satirlar[n++] = satir;
        }
        return satirlar;
    }

    static void Hata(string yol, int satir, string mesaj)
    {
        throw new FormatException(Path.GetFileName(yol) +
            ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
            ": " + mesaj);
    }

    static string[] Alanlar(string yol, int satir, string metin,
        int adet)
    {
        string[] alan = metin.Split(';');
        if (alan.Length != adet)
            Hata(yol, satir, "Alan sayısı yanlış.");
        return alan;
    }

```

```

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

static decimal Ondalik(string yol, int satir, string metin,
    decimal alt, decimal ust)
{
    int bas = metin.StartsWith('-') || metin.StartsWith('+') ? 1 : 0;
    int nokta = -1, rakam = 0;
    for (int i = bas; i < metin.Length; i++)
    {
        if (metin[i] >= '0' && metin[i] <= '9') rakam++;
        else if (metin[i] == '.' && nokta == -1) nokta = i;
        else Hata(yol, satir, "Geçersiz ondalık.");
    }
    if (rakam == 0 || nokta == bas ||
        (nokta >= 0 && (metin.Length - nokta - 1 < 1 ||
            metin.Length - nokta - 1 > 2)))
        Hata(yol, satir, "Geçersiz ondalık.");
    if (!decimal.TryParse(metin,
        NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint,
        CultureInfo.InvariantCulture, out decimal deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz ondalık.");
    return deger;
}

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.NewLine = "\n";
    yazici.Write(metin);
}

```



```

static void Sirala(int[] kod, decimal[] bedel, int[] adet, int n)
{
    for (int i = 1; i < n; i++)
    {
        int k = kod[i], a = adet[i], j = i - 1;
        decimal b = bedel[i];
        while (j >= 0 && kod[j] > k)
        {
            kod[j + 1] = kod[j]; bedel[j + 1] = bedel[j];
            adet[j + 1] = adet[j]; j--;
        }
        kod[j + 1] = k; bedel[j + 1] = b; adet[j + 1] = a;
    }
}

```

Örnek çalıştırmalar

dotnet run -- ornek-089/katalog.txt ornek-089/satis.txt ornek-089/rapor.txt;
katalog.txt:

```

7;2.50
2;4
9;10

```

satis.txt:

```

2;3
7;2
2;1

```

Yeni rapor.txt dosyasının tam içeriği:

```

2;4;16.00
7;2;5.00
9;0;0.00

```

Konsolun tam çıktısı:

Rapor yazıldı.

İki giriş dosyası boş, rapor.txt yok:

Yeni rapor.txt sıfır bayttır.

Konsolun tam çıktısı:

Rapor yazıldı.

katalog.txt yalnız 7;2.50; satis.txt:

```

2;1

```

rapor.txt oluşturulmaz.

Konsolun tam çıktısı:

Hata: satis.txt: satır 1: Katalogda olmayan kod.

Geçerli girişler, rapor.txt önceden var:

Mevcut raporun baytları korunur.

Konsolun tam çıktısı:

Hata: Dosya işlemi başarısız.

Sınır durumları

- Katalog boşken satış kaydı varsa bilinmeyen kod hatasıdır; satış boşken tüm katalog ürünleri sıfır gelirle yazılır. Boş satır alan sayısı hatasıdır.
- Katalog kodu tekrarı hata; satış kodu tekrarı normal birikimdir.
- Ürün adedi en fazla 100000, ürün geliri ve bütün raporun gelir toplamı en fazla 10000000000; int adet ve decimal gelir taşmaz.
- Kaydırma sırasında üç paralel dizi aynı kaydın alanları olarak birlikte taşınır. Hazır LINQ, GroupBy veya Sort kullanılmaz.
- Ortak dosya boyutu, fiziksel satır, kodlama, eksik dosya ve erişim davranışları Girdi ve çıktı bölümündeki sözleşmeye tabidir.

Kazanımlar

- Bir dosyadaki anahtar diğer dosyadaki referansla ilişkilendirme.
- Paralel dizilerde kayıt bütünlüğünü sıralama boyunca koruma.
- Doğrulama ve yeni rapor oluşturmayı ayrı aşamalarda yürütme.

Alıştırmalar

1. Ürün başına satış satırı sayısını dördüncü paralel diziye ekleyin.
2. Toplam gelire göre azalan, eşitlikte koda göre artan sıralama yazın.

090 İş ağacında birikimli süre raporu

Düzey: İleri Önkoşullar: 042, 053, 077, 078, 085, 089

Kavramlar: Üst kayıt ilişkisi, Üç durumlu döngü denetimi, Özyinelemeli toplam, Çoklu veri kaynağı, Yeni dosyada rapor

Problem tanımı

Bir proje iş ağacındaki alt işlerin sürelerini üst işlere taşıyan rapor yazın. İş tanımlarıyla süre kayıtları ayrı dosyalardadır. Her işin kendi süresini ve bütün alt işleri dahil birikimli süresini hesaplayın. Eksik üst iş veya döngü varsa rapor üretmeyin. Tanımların üst işten önce gelmesi gerekmez; dosya sırası ilişki sırası değildir.

Girdi ve çıktı

Üç yol: `ornek-090/isler.txt` `ornek-090/sureler.txt` `ornek-090/rapor.txt`. İş satırı kod;üstKod: benzersiz kod 1..9999; üstKod 0 kök, aksi halde mevcut iş kodudur. Birden çok kök kabul edilir. Süre satırı kod;dakika: mevcut iş kodu ve 1..1000 tamsayı dakika; tekrarlanan süre kodları kendi süreye eklenir. Dosya başına ortak 100 satır sınırı vardır. Çıktı artan kod sırasıyla kod;kendiDakika;birikimliDakika biçiminde başlıksızdır. Konsola yalnız Rapor yazıldı. yazılır.

Yollar komut satırı argümanlarıdır; program istem yazmaz. Kod bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Örnekleri yalnız kendinizin oluşturduğu ayrı bir örnek klasöründe hazırlayın; program giriş dosyalarını değiştirmez. Proje klasöründen `dotnet run --` sonrasında belirtilen yolları verin. Göreli yollar programın çalışma klasörüne göredir.

Her giriş dosyası isteğe bağlı UTF-8 BOM ile UTF-8 metindir. LF ve CRLF kabul edilir. Dosya başına en fazla 32768 bayt, 100 fiziksel satır ve satır başına 120 UTF-16 kod birimi kabul edilir. Sınırlar dosya açıldıktan sonra ve her satır saklanmadan önce denetlenir. Bu küçük sınırlar öğretim amaçlıdır; büyük dosya akışı ayrı bir tasarım gerektirir. ReadLine satır sonunu çıkarır: null EOF, boş metin ise gerçek boş satırdır. Sondaki tek satır sonu ek bir boş kayıt oluşturmaz; art arda iki satır sonu aradaki boş satırı oluşturur.

İlk hata işlemi durdurur; doğrulama bitmeden sonuç yazılmaz. Biçim hatasında dosyanın adı ve 1 tabanlı fiziksel satır numarası bildirilir. Eksik dosya, klasörün bulunamaması veya diğer I/O sorunlarında Hata: Dosya işlemi başarısız.; yetki sorununun ayrı yakalanması halinde Hata: Dosyaya erişim izni yok. yazılır. Geçersiz yol veya bozuk UTF-8 için Hata: Yol veya UTF-8 kodlaması geçersiz. yazılır. using blokları hata halinde de akışı ve okuyucuyu kapatır.

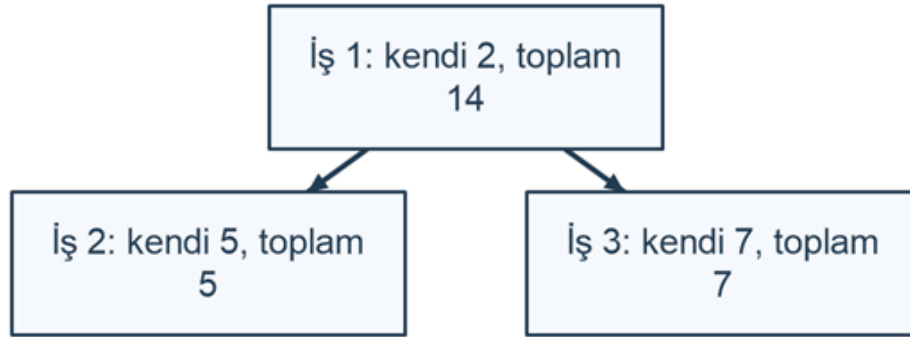
Sayısal alanlarda çevre boşluğu, binlik ayırıcı, üs gösterimi ve virgüllü ondalık kabul edilmez. Tamsayıda isteğe bağlı + veya - işareti ve ASCII rakamlar kullanılır; baştaki sıfırlar kabul edilir. Ondalıkta isteğe bağlı işaret, en az bir tam kısım rakamı ve varsa noktadan sonra 1..2 rakam vardır. Girdi ve çıktı CultureInfo.InvariantCulture kullanır. Taşan sayı ayrıştırma hatasıdır.

Çıktı yolu üçüncü argümandır; dosya henüz var olmamalıdır ve üst klasörünü siz önceden oluşturmalsınız. FileMode.CreateNew mevcut bir dosyayı girişle aynı yol olsa bile ezmez; bu durumda I/O hatası oluşur. Başarı mesajı yalnız yazıcı ve akış kapandıktan sonra verilir. Giriş doğrulama hatasında çıktı dosyası hiç açılmaz. Yazım sırasında disk dolması gibi I/O hatasında yeni dosya kısmi kalabilir; program başarı bildirmez ve bu yeni dosyayı siz denetleyip başka adla tekrar deneyebilirsiniz. Bu küçük öğretim örneği atomik yayın sözü vermez. Çıktı UTF-8 BOM olmadan, LF satır sonlarıyla yazılır; her kayıt son LF ile biter. Bu örnekte kayıtsız rapor sıfır bayttır.

Algoritma

1. İş tanımlarını okuyun; benzersiz kodları ve üst kod aralıklarını doğrulayın.
2. Bütün tanımlar okunduktan sonra üst kodları dizi dizinlerine dönüştürün; bulunmayanı tanım satırında bildirin.
3. Her işin üst zincirini üç durumla gezin: 0 görülmedi, 1 bu çağrı zincirinde, 2 tamamlandı. Durum 1 ziyaretinde döngü vardır.
4. Süre dosyasını doğrulayın; kod başına kendi dakika toplamını biriktirin.
5. Bir işin toplamı kendi süresi ile çocuklarının özyinelemeli toplamıdır; tamamlanan toplamı ayrı bool dizisiyle ön belleğe alın.
6. Kodları değiştirmeden dizin dizisini eklemeli sıralayın, raporu oluşturun ve yeni dosyaya yazın.

İş	Üst iş	Kendi dakika	Alt iş toplamları	Birikimli dakika
3	1	7	yok	7
2	1	5	yok	5
1	kök	2	7 + 5	14



Şekil 8 Üst iş ve alt işlerin süreleri

Oku, out n ile satır adedini verir; dizilerin yalnız ilk n hücresi veridir.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 3) throw new FormatException("İki giriş ve bir çıktı yolu gerekli.");
    string[] s = Oku(args[0], out int n);
    int[] kod = new int[100], ustKod = new int[100], ust = new int[100];
    for (int i = 0; i < n; i++)
    {
        string[] a = Alanlar(args[0], i + 1, s[i], 2);
        kod[i] = Tam(args[0], i + 1, a[0], 1, 9999);
        ustKod[i] = Tam(args[0], i + 1, a[1], 0, 9999);
        if (Bul(kod, i, kod[i]) >= 0) Hata(args[0], i + 1, "Yinelenen kod.");
    }
    for (int i = 0; i < n; i++)
    {
        ust[i] = ustKod[i] == 0 ? -1 : Bul(kod, n, ustKod[i]);
        if (ustKod[i] != 0 && ust[i] < 0)
            Hata(args[0], i + 1, "Üst iş bulunamadı.");
    }
    int[] durum = new int[100];
    for (int i = 0; i < n; i++) Denetle(i, ust, durum, args[0]);
    int[] kendi = new int[100];
    string[] t = Oku(args[1], out int tn);
    for (int i = 0; i < tn; i++)
    {
        string[] a = Alanlar(args[1], i + 1, t[i], 2);
        int kimlik = Tam(args[1], i + 1, a[0], 1, 9999);
        int dakika = Tam(args[1], i + 1, a[1], 1, 1000);
        int p = Bul(kod, n, kimlik);
        if (p < 0) Hata(args[1], i + 1, "İş bulunamadı.");
        kendi[p] += dakika;
    }
    int[] toplam = new int[100], sıra = new int[100];
    bool[] hazır = new bool[100];
    for (int i = 0; i < n; i++)
    {
        Topla(i, ust, kendi, toplam, hazır, n);
        sıra[i] = i;
    }
    Sirala(sıra, kod, n);
    var rapor = new StringBuilder();
    for (int i = 0; i < n; i++)
    {
        int p = sıra[i];
        rapor.Append(kod[p].ToString(CultureInfo.InvariantCulture) + ";" +
            kendi[p].ToString(CultureInfo.InvariantCulture) + ";" +
            toplam[p].ToString(CultureInfo.InvariantCulture) + "\n");
    }
    Yaz(args[2], rapor.ToString());
    Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
```

```

catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }

static string[] Oku(string yol, out int n)
{
    string[] satirlar = new string[100];
    n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768) throw new FormatException(
        Path.GetFileName(yol) + ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? satir;
    while ((satir = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && satir.StartsWith('\uFEFF'))
            satir = satir.Substring(1);
        if (n == 100) Hata(yol, n + 1, "En fazla 100 satır.");
        if (satir.Length > 120) Hata(yol, n + 1,
            "Satır en fazla 120 karakter.");
        satirlar[n++] = satir;
    }
    return satirlar;
}

static void Hata(string yol, int satir, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static string[] Alanlar(string yol, int satir, string metin,
    int adet)
{
    string[] alan = metin.Split(';');
    if (alan.Length != adet)
        Hata(yol, satir, "Alan sayısı yanlış.");
    return alan;
}

static int Tam(string yol, int satir, string metin,
    int alt, int ust)
{
    if (!int.TryParse(metin, NumberStyles.AllowLeadingSign,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, "Geçersiz tamsayı.");
    return deger;
}

```

```

static int Bul(int[] kodlar, int n, int kod)
{
    for (int i = 0; i < n; i++)
        if (kodlar[i] == kod) return i;
    return -1;
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.NewLine = "\n";
    yazici.Write(metin);
}

static void Denetle(int i, int[] ust, int[] durum, string yol)
{
    if (durum[i] == 2) return;
    if (durum[i] == 1) Hata(yol, i + 1, "Üst iş döngüsü.");
    durum[i] = 1;
    if (ust[i] >= 0) Denetle(ust[i], ust, durum, yol);
    durum[i] = 2;
}

static int Topla(int i, int[] ust, int[] kendi, int[] toplam,
    bool[] hazır, int n)
{
    if (hazır[i]) return toplam[i];
    int sonuc = kendi[i];
    for (int j = 0; j < n; j++)
        if (ust[j] == i) sonuc += Topla(j, ust, kendi, toplam, hazır, n);
    toplam[i] = sonuc; hazır[i] = true;
    return sonuc;
}

static void Sirala(int[] siras, int[] kod, int n)
{
    for (int i = 1; i < n; i++)
    {
        int p = siras[i], j = i - 1;
        while (j >= 0 && kod[siras[j]] > kod[p])
            { siras[j + 1] = siras[j]; j--; }
        siras[j + 1] = p;
    }
}

```

Örnek çalıştırmalar

dotnet run -- ornek-090/isler.txt ornek-090/sureler.txt ornek-090/rapor.txt;
 isler.txt:

```

3;1
1;0
2;1

```

sureler.txt:

1;2
2;5
3;4
3;3

Yeni rapor.txt dosyasının tam içeriği:

1;2;14
2;5;5
3;7;7

Konsolun tam çıktısı:

Rapor yazıldı.

Her iki giriş boş ve rapor.txt yoksa yeni rapor.txt sıfır bayttır. Konsolun tam çıktısı:

Rapor yazıldı.

sureler.txt boş; isler.txt döngü:

1;2
2;1

rapor.txt oluşturulmaz. Konsolun tam çıktısı:

Hata: isler.txt: satır 1: Üst iş döngüsü.

sureler.txt boş; isler.txt eksik üst:

1;9

rapor.txt oluşturulmaz. Konsolun tam çıktısı:

Hata: isler.txt: satır 1: Üst iş bulunamadı.

Sınır durumları

- Kendi üstü olan iş de döngüdür. Döngü hatası, dosya sırasıyla yapılan üst zinciri ziyaretinde ilk yeniden girilen işin tanım satırını verir; bu sayı zincirin son satırı olmayabilir.
- İş tanım kodları benzersizdir; süre kayıtlarında tekrar normaldir. Boş süre dosyasında bütün iş süreleri 0; boş iş dosyasında süre kaydı varsa hata olur.
- Döngü denetimi süre okumadan önce yapılır. Önce iş kaynağı hatası raporlanır.
- En fazla 100 iş nedeniyle çağrı derinliği en fazla 100; her işin alt ağacı dosyanın toplam 100000 dakikasını aşmaz. Topamlar int aralığına sığar. Farklı üst işlerin rapor toplamalarını yeniden toplamak süreleri birden fazla sayar.
- Çocuk araması n elemanlık taramadır; önbellek her işi bir kez tamamlar. Denetim ve toplam hesabı bu temsil ile en kötü n^2 iş yapar. Sıralanan dizinlerdir; üst ilişkilerinin dizinleri değişmez.

Kazanımlar

- Dosya sırası ile veri ilişkilerinin sırasını ayırt etme.
- Döngü denetimini özyinelemeli toplamın önkoşulu olarak kurma.
- Hiyerarşi ve ayrı süre kümesini birleştirip kaynak konumlarını koruma.
- Dizin dizisini sıralayarak ilişkilerin bozulmasını önleme.

Alıştırmalar

1. Her iş için alt iş sayısını ikinci bir özyinelemeli sonuç olarak hesaplayın.
2. 100 iş sınırını kaldırırken çağrı yığını ve çocuk arama maliyetinin nasıl değişeceğini tartışın.

Bölüm 10: Birleşik uygulamalar ve optimizasyon

091 Ders denemelerinden ağırlıklı başarı

Düzyey: İleri Önkoşullar: 053, 073, 083, 089

Kavramlar: Bileşik anahtar, En iyi deneme, Ağırlıklı ortalama, Dizinle raporlama

Problem tanımı

Öğrencilerin bir dersi birden fazla denemede alabileceği bir programda, her öğrenci için ders başına en yüksek puanı seçin ve kredi ağırlıklı ortalamayı hesaplayın. Üç dosya öğrenci, ders ve deneme verilerini ayrı tutar. Satış toplamından farklı olarak burada aynı ilişkiyi tek bir en iyi sonuç temsil eder; bir denemeyi iki kez saymayın.

Girdi ve çıktı

```
dotnet run -- ornek-091/ogrenciler.txt ornek-091/dersler.txt ornek-091/denemeler.txt ornek-091/rapor.txt
```

ogrenciler.txt: öğrenciKod (1..9999). dersler.txt: dersKod;kredi (kod 1..9999, kredi 1..10). denemeler.txt: öğrenciKod;dersKod;deneme;puan (deneme 1..3, puan 0..100). Öğrenci ve ders kodları benzersizdir; deneme anahtarı öğrenci/ders/deneme üçlüsüdür. Farklı denemeler normaldir. Her denemede iki ana kayıt da bulunmalıdır.

Rapor öğrenci kodu artan sırada kod;farklıDers;krediToplamı;ortalama biçimindedir. Ortalamada en iyi puan × kredi toplamını kredi toplamına bölün; iki basamakla, midpoint to even yuvarlama ile yazın. Eşit en iyi puanlarda ilk deneme korunur, sonuç değişmez. Denemesi olmayan öğrencide kod;0;0;VERI_YOK yazın; hiç öğrenci yoksa rapor boştur. Bir öğrencinin almadığı katalog dersi eksik kayıt değildir.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağılı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtarı belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayırıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.;

desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taştı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Üç tabloyu ayrıştırın ve benzersizlikleri doğrulayın.
2. Denemelerin öğrenci ve ders kodlarını doğrusal aramayla dizi dizinlerine dönüştürün.
3. Her öğrenci/ders hücrelerini -1 ile başlatın; puan daha yüksekse değiştirin. Sıfır puanı veri yok durumundan ayırın.
4. Her öğrenci için dolu hücrelerde kredi ve ağırlıklı puan biriktirin; önce bütün hesapları tamamlayın.
5. Öğrenci dizinlerini eklemeli sıralayıp raporu oluşturun; yeni dosyaya yazın.

Öğrenci	Ders	Denemeler	Seçilen puan	Ağırlıklı katkı
2	7	40, 80	80	$80 \times 3 = 240$
2	9	60	60	$60 \times 1 = 60$
2	toplam	2 ders	4 kredi	$300 / 4 = 75.00$

Oku yalnız geçerli satırları içeren diziyi döndürür. Tekil seçilen sütunları birlikte anahtar sayar; Bul ilk sütunda doğrusal arar, yoksa -1 verir. Sıra kaynak kayıtları değiştirmeden kod artan eklemeli sıralamayla dizin dizisi döndürür. Satır yalnız bellek raporunu değiştirir; Yaz dış dosyayı oluşturur. Hazır LINQ, GroupBy, Sort veya arama algoritması kullanılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 4) throw new FormatException(
        "4 argüman gerekli.");
    checked
    {
        int[][] ogr = Oku(args[0], new[] { 1 }, new[] { 9999 });
        int[][] ders = Oku(args[1], new[] { 1, 1 }, new[] { 9999, 10 });
        int[][] den = Oku(args[2], new[] { 1, 1, 1, 0 },
            new[] { 9999, 9999, 3, 100 });
        Tekil(ogr, args[0], 0); Tekil(ders, args[1], 0);
        Tekil(den, args[2], 0, 1, 2);
        int[,] enIyi = new int[ogr.Length, ders.Length];
        for (int i = 0; i < ogr.Length; i++)
            for (int j = 0; j < ders.Length; j++) enIyi[i, j] = -1;
    }
}
```

```

for (int i = 0; i < den.Length; i++)
{
    int o = Bul(ogr, den[i][0]), d = Bul(ders, den[i][1]);
    if (o < 0) Hata(args[2], i + 1, 1, "Öğrenci bulunamadı.");
    if (d < 0) Hata(args[2], i + 1, 2, "Ders bulunamadı.");
    if (den[i][3] > enIyi[o, d]) enIyi[o, d] = den[i][3];
}

var rapor = new StringBuilder();
foreach (int o in Sira(ogr))
{
    int kredi = 0, agirlik = 0, adet = 0;
    for (int d = 0; d < ders.Length; d++)
        if (enIyi[o, d] >= 0)
        {
            kredi += ders[d][1];
            agirlik += enIyi[o, d] * ders[d][1]; adet++;
        }
    string ort = kredi == 0 ? "VERI_YOK" :
        Math.Round((decimal)agirlik / kredi, 2,
            MidpointRounding.ToEven).ToString("F2",
            CultureInfo.InvariantCulture);
    Satir(rapor, ogr[o][0], adet, kredi, ort);
}

Yaz(args[3], rapor.ToString());
}
Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taşıtı."); }

```

```

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -1)
                veri[n][j] = Tarih(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

```

```

static int Tarih(string yol, int satir, int alan, string metin)
{
    if (!DateOnly.TryParseExact(metin, "yyyy-MM-dd",
        CultureInfo.InvariantCulture, DateTimeStyles.None,
        out DateOnly tarih) || tarih.Year < 2000 || tarih.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz tarih.");
    return tarih.DayNumber;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] siras = new int[veri.Length];
    for (int i = 0; i < siras.Length; i++) siras[i] = i;
    for (int i = 1; i < siras.Length; i++)
    {
        int p = siras[i], j = i - 1;
        while (j >= 0 && veri[siras[j]][0] > veri[p][0])
        { siras[j + 1] = siras[j]; j--; }
        siras[j + 1] = p;
    }
    return siras;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

```

```
static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}
```

Örnek çalıştırmalar

Normal örnek için yukarıdaki komutu kullanın; rapor.txt başlangıçta yoktur.

ogrenciler.txt dosyasının tam içeriği:

```
3
2
```

dersler.txt dosyasının tam içeriği:

```
9;1
7;3
```

denemeler.txt dosyasının tam içeriği:

```
2;7;1;40
2;7;2;80
2;9;1;60
```

Yeni rapor.txt dosyasının tam içeriği:

```
2;2;4;75.00
3;0;0;VERI_YOK
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Sınır örneği: aynı dosya adlarını ayrı boş bir klasörde hazırlayın; aynı komutu çalıştırın.

ogrenciler.txt dosyasının tam içeriği:

```
9999
```

dersler.txt dosyasının tam içeriği:

```
9999;10
```

denemeler.txt dosyasının tam içeriği:

```
9999;9999;3;0
```

Yeni rapor.txt dosyasının tam içeriği:

```
9999;1;10;0.00
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Geçersiz örnek: ayrı bir klasörde, aynı komut.

ogrenciler.txt dosyasının tam içeriği:

```
2
```

dersler.txt dosyasının tam içeriği:

```
7;3
```

denemeler.txt dosyasının tam içeriği:

```
2;8;1;60
```

Rapor oluşturulmaz. Konsolun tam çıktısı:

```
Hata: denemeler.txt: satır 1, alan 2: Ders bulunamadı.
```

Bütün giriş dosyaları sıfır bayt ve rapor.txt yokken yeni rapor sıfır bayttır.

Konsolun tam çıktısı:

Rapor yazıldı.

Geçerli normal girdilerle rapor.txt önceden varsa mevcut baytlar korunur. Konsolun tam çıktısı:

Hata: Dosya işlemi başarısız.

Sınır durumları

- En fazla 100 öğrenci, 100 ders ve 100 deneme satırı vardır. Öğrenci başına en fazla 1000 kredi ve 100000 ağırlıklı puan; int güvenlidir, bölme decimal ile yapılır. Ders dosyası boşken deneme varsa eksik ders hatasıdır. Boş deneme dosyası tüm öğrencileri VERI_YOK yapar. Aynı üçlü ikinci kez görünürse, puan farklı olsa da hata olur.
- Satır kapasitesinin tam dolması kabul edilir; bir sonraki satır saklanmadan hata olur. Tek bir boş satır sıfır kayıt sayılmaz; alan/sayı hatasıdır. Çok büyük veya taşan sayısal alan ayrıştırma hatasıdır.
- Aynı sayıyı farklı baştaki sıfırlarla yazmak anahtar tekrarını gizlemez. Sıralama sırasında dizinler taşındığı için dosya satırı ile hata konumu ve ilişkiler değişmez.
- Hatalı son kayıt dahil bütün giriş doğrulanmadan rapor açılmaz. Ortak I/O ve çıktı güvenliği sözleşmesi yukarıda tanımlıdır.

Kazanımlar

- Veri yok durumunu gerçek sıfırdan ayırma; bileşik anahtar ile tekrar denemeyi ayırt etme; seçim ve ağırlıklı özetlemeyi farklı metot sorumlulukları olarak açıklama.
- Öğrenciye ait ders sonuçlarını paralel hücrelerde tutarken aynı doğrulama ve ortalama davranışları tekrar eder. Sonraki derste bu verileri bir öğrenci nesnesinde toplamak, tutarsız puan/kredi eşleşmelerini önleyebilir.

Alıştırmalar

1. En iyi deneme numarasını rapora ekleyin; eşit puanda küçük deneme numarasını seçin.
2. Sıfır puan ile VERI_YOK değerinin neden farklı olduğunu ve Öğrenci/Ders verisini bir arada korumanın yararını açıklayın.

092 Salon rezervasyonlarında çakışma taraması

Düzey: İleri Önkoşullar: 053, 084, 088, 089

Kavramlar: Yarı açık zaman aralığı, İkili çakışma, İlişki doğrulama, Çok ölçütlü sıralama

Problem tanımı

Salon kapasitesi ile rezervasyonları birleştirin. Geçerli rezervasyonların aynı salonda ve günde çakışan bütün çiftlerini bulun. Çakışma, raporlanacak normal bir sonuçtur; bozuk girdi değildir. Her çifti yalnız bir kez yazın. Aralıkların bitişi hariç olduğu için art arda toplantılar çakışmaz.

Girdi ve çıktı

```
dotnet run -- ornek-092/salonlar.txt ornek-092/rezervasyonlar.txt ornek-092/rapor.txt
```

salonlar.txt: salonKod;kapasite (kod 1..9999, kapasite 1..1000). rezervasyonlar.txt: rezKod;salonKod;tarih;başlangıçDakika;bitişDakika;katılımcı (kodlar 1..9999, başlangıç 0..1439, bitiş 1..1440, katılımcı 1..1000). Tarih ortak ISO sözleşmesindedir. Salon ve rezervasyon kodları benzersizdir. Salon bulunmalı, başlangıç bitişten küçük ve katılımcı salon kapasitesini aşmamalıdır.

Rapor küçükRezKod;büyükRezKod;ortakDakika biçimindedir. Önce küçük, sonra büyük rezervasyon koduna göre artar. $\max(\text{başlangıçlar}) < \min(\text{bitişler})$ ise çakışma vardır. Farklı salon veya tarihte aralıklar karşılaştırılmaz. Çakışma yoksa tam içerik ÇAKISMA_YOK ve LF'dir; iki dosya boşken de aynıdır. Sıralama eşitliğinde ikinci kod kullanılır; aynı çift bir kez vardır.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtarı belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun

tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşıtı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Tabloları ve tekil kodları doğrulayın.
2. Her rezervasyonun salonunu, zaman aralığını ve kapasitesini doğrulayın.
3. Rezervasyon dizinlerini kodlarına göre sıralayın; asıl kayıtları taşımayın.
4. Sıralı dizinlerde $i < j$ bütün çiftlerini tarayın; salon ve tarih aynıysa yarı açık kesişimi hesaplayın.
5. Tüm çakışmaları bellek raporunda biriktirip yeni dosyaya yazın.

Çift	Aralıklar	Kesişim	Sonuç
2 ve 3	[60,120), [90,150)	[90,120)	30 dakika
2 ve 4	[60,120), [120,180)	boş	çakışmaz
3 ve 4	[90,150), [120,180)	[120,150)	30 dakika

Oku yalnız geçerli satırları içeren diziye döndürür. Tekil seçilen sütunları birlikte anahtar sayar; Bul ilk sütunda doğrusal arar, yoksa -1 verir. Sıra kaynak kayıtları değiştirmeden kod artan eklemeli sıralamayla dizin dizisi döndürür. Satir yalnız bellek raporunu değiştirir; Yaz dış dosyayı oluşturur. Hazır LINQ, GroupBy, Sort veya arama algoritması kullanılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 3) throw new FormatException(
        "3 argüman gerekli.");
    checked
    {
        int[][] salon = Oku(args[0], new[] { 1, 1 },
            new[] { 9999, 1000 });
        int[][] rez = Oku(args[1], new[] { 1, 1, 0, 0, 1, 1 },
            new[] { 9999, 9999, -1, 1439, 1440, 1000 });
        Tekil(salon, args[0], 0); Tekil(rez, args[1], 0);
        for (int i = 0; i < rez.Length; i++)
        {
            int p = Bul(salon, rez[i][1]);
            if (p < 0) Hata(args[1], i + 1, 2, "Salon bulunamadı.");
            if (rez[i][3] >= rez[i][4])
                Hata(args[1], i + 1, 5, "Bitiş başlangıçtan büyük olmalı.");
            if (rez[i][5] > salon[p][1])
                Hata(args[1], i + 1, 6, "Salon kapasitesi yetersiz.");
        }
    }
}
```

```

int[] sıra = Sira(rez);
var rapor = new StringBuilder();
for (int i = 0; i < sıra.Length; i++)
    for (int j = i + 1; j < sıra.Length; j++)
    {
        int[] a = rez[sıra[i]], b = rez[sıra[j]];
        if (a[1] != b[1] || a[2] != b[2]) continue;
        int ortak = Kesisim(a[3], a[4], b[3], b[4]);
        if (ortak > 0) Satir(rapor, a[0], b[0], ortak);
    }
if (rapor.Length == 0) rapor.Append("CAKISMA_YOK\n");

Yaz(args[2], rapor.ToString());
}
Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taşıtı."); }

static int Kesisim(int a, int b, int c, int d)
{
    int bas = a > c ? a : c, son = b < d ? b : d;
    return son > bas ? son - bas : 0;
}

```

```

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -1)
                veri[n][j] = Tarih(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

```

```

static int Tarih(string yol, int satir, int alan, string metin)
{
    if (!DateOnly.TryParseExact(metin, "yyyy-MM-dd",
        CultureInfo.InvariantCulture, DateTimeStyles.None,
        out DateOnly tarih) || tarih.Year < 2000 || tarih.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz tarih.");
    return tarih.DayNumber;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

```

```
static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}
```

Örnek çalıştırmalar

Normal örnek için yukarıdaki komutu kullanın; rapor.txt başlangıçta yoktur.

salonlar.txt dosyasının tam içeriği:

```
7;10
```

rezervasyonlar.txt dosyasının tam içeriği:

```
4;7;2026-10-05;120;180;8
2;7;2026-10-05;60;120;10
3;7;2026-10-05;90;150;5
```

Yeni rapor.txt dosyasının tam içeriği:

```
2;3;30
3;4;30
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Sınır örneği: aynı dosya adlarını ayrı boş bir klasörde hazırlayın; aynı komutu çalıştırın.

salonlar.txt dosyasının tam içeriği:

```
9999;1000
```

rezervasyonlar.txt dosyasının tam içeriği:

```
9999;9999;2099-12-31;0;1440;1000
```

Yeni rapor.txt dosyasının tam içeriği:

```
CAKISMA_YOK
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Geçersiz örnek: ayrı bir klasörde, aynı komut.

salonlar.txt dosyasının tam içeriği:

```
7;10
```

rezervasyonlar.txt dosyasının tam içeriği:

```
1;7;2026-10-05;60;120;11
```

Rapor oluşturulmaz. Konsolun tam çıktısı:

```
Hata: rezervasyonlar.txt: satır 1, alan 6: Salon kapasitesi yetersiz.
```

Bütün giriş dosyaları sıfır bayt ve rapor.txt yokken yeni raporun tam içeriği:

```
CAKISMA_YOK
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Geçerli normal girdilerle rapor.txt önceden varsa mevcut baytlar korunur. Konsolun tam çıktısı:

```
Hata: Dosya işlemi başarısız.
```

Sınır durumları

- 100 rezervasyonda en fazla 4950 çift üretilir; kayıt kapasitesi 100 olsa da sonuç satırı sayısı 4950 olabilir. Dakika farkı en fazla 1440; int taşmaz. Çapraz gece aralığı kabul edilmez, iki güne bölünmelidir. Salon boş ve rezervasyon doluysa eksik ilişki hatasıdır. Boş rezervasyon dosyası CAKISMA_YOK üretir.
- Satır kapasitesinin tam dolması kabul edilir; bir sonraki satır saklanmadan hata olur. Tek bir boş satır sıfır kayıt sayılmaz; alan/sayı hatasıdır. Çok büyük veya taşan sayısal alan ayrıştırma hatasıdır.
- Aynı sayıyı farklı baştaki sıfırlarla yazmak anahtar tekrarını gizlemez. Sıralama sırasında dizinler taşındığı için dosya satırı ile hata konumu ve ilişkiler değişmez.
- Hatalı son kayıt dahil bütün giriş doğrulanmadan rapor açılmaz. Ortak I/O ve çıktı güvenliği sözleşmesi yukarıda tanımlıdır.

Kazanımlar

- Yarı açık aralıkların uç eşitliğini açıklama; her çifti bir kez tarama; fiziksel dosya sırasından bağımsız rapor düzeni kurma.
- Rezervasyonun salon, tarih ve iki zaman alanı ile çakışma davranışı birlikte kullanılır. Sonraki derste bunları aynı nesnede tutmak, bitişi başlangıçtan önce olan bir rezervasyonun uygulamada dolaşmasını engellemeyi kolaylaştırabilir.

Alıştırmalar

1. Çakışan çift başına toplam katılımcı sayısını ekleyin; bunun salon kapasitesi aşımı kararından nasıl ayrıldığını açıklayın.
2. Çakışma metodunun konsoldan bağımsız sözleşmesini yazın ve nesneye taşındığında hangi verileri kullanacağını belirtin.

093 Ödünç ve iadelerden gecikme raporu

Düzey: İleri Önkoşullar: 042, 053, 084, 089

Kavramlar: Birden çok ilişki, İade eşleştirme, Kesim tarihi, Açık kayıt

Problem tanımı

Üye ve kitap kataloglarını, ödünç kayıtlarını ve ayrı iade kayıtlarını birleştirin. Her ödünç için verilen kesim tarihinde gecikme gününü ve durumunu üretin. Kesim tarihinden sonra gerçekleşen bir iade veride geçerli olsa bile o tarihte kitap hâlâ açıktır. Böylece bugünkü veriyle geçmiş bir rapor hazırlanabilir.

Girdi ve çıktı

```
dotnet run -- ornek-093/uyeler.txt ornek-093/kitaplar.txt ornek-093/oduncler.txt  
ornek-093/iadeler.txt 2026-10-10 ornek-093/rapor.txt
```

uyeler.txt: üyeKod; kitaplar.txt: kitapKod; kodlar 1.9999. oduncler.txt: ödünçKod; üyeKod; kitapKod; alışTarihi; sonTarih. iadeler.txt: ödünçKod; iadeTarihi. Tüm tarihler ISO sözleşmesindedir. Her katalog kodu, ödünç kodu ve iadedeki ödünç kodu kendi dosyasında benzersizdir. Ödünçte üye ve kitap, iadede ödünç bulunmalıdır. Son tarih alıştan, iade tarihi alıştan önce olamaz. Rapor kesim tarihi beşinci argümandır, çıktı altıncıdır.

Rapor kesim tarihine kadar başlayan ödünçler için artan ödünç koduyla ödünçKod; üyeKod; kitapKod; durum; gecikmeGün yazar. Kesime kadar iade varsa durum İADE, gecikme max(0, iade-son); yoksa ACIK, gecikme max(0, kesim-son) olur. Kesimden sonraki alışlar raporda yer almaz. Aynı kitap için birden fazla ödünç kabul edilir; bu çalışma fiziksel kitap nüshası veya çakışma denetimi yapmaz. İade eksikliği hata değil ACIK durumudur.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Göreli yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtar belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşttı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Dört dosyayı okuyun, ardından kesim tarihini doğrulayın.
2. Tekil anahtarları ve ödünçlerin katalog ilişkilerini denetleyin.
3. İadeleri ödünç dizinleriyle eşleyin; alıştan önce iadeyi reddedin.
4. Her ödünç için kesimde görünür olup olmadığını ve etkin bitiş gününü belirleyin.
5. Gecikmeyi negatif olmayacak biçimde hesaplayıp kod sırasıyla yazın.

Ödünç	Son tarih	İade	Kesim	Durum ve gecikme
1	2026-10-07	2026-10-09	2026-10-10	IADE, 2
2	2026-10-08	2026-10-12	2026-10-10	ACIK, 2

Oku yalnız geçerli satırları içeren diziyi döndürür. Tekil seçilen sütunları birlikte anahtar sayar; Bu1 ilk sütunda doğrusal arar, yoksa -1 verir. Sıra kaynak kayıtları değiştirmeden kod artan eklemeli sıralamayla dizin dizisi döndürür. Satır yalnız bellek raporunu değiştirir; Yaz dış dosyayı oluşturur. Hazır LINQ, GroupBy, Sort veya arama algoritması kullanılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 6) throw new FormatException(
        "6 argüman gerekli.");
    checked
    {
        int[][] uye = Oku(args[0], new[] { 1 }, new[] { 9999 });
        int[][] kitap = Oku(args[1], new[] { 1 }, new[] { 9999 });
        int[][] odunc = Oku(args[2], new[] { 1, 1, 1, 0, 0 },
            new[] { 9999, 9999, 9999, -1, -1 });
        int[][] iade = Oku(args[3], new[] { 1, 0 }, new[] { 9999, -1 });
        int kesim = Tarih("kesim", 1, 1, args[4]);
        Tekil(uye, args[0], 0); Tekil(kitap, args[1], 0);
        Tekil(odunc, args[2], 0); Tekil(iade, args[3], 0);
        for (int i = 0; i < odunc.Length; i++)
        {
            if (Bul(uye, odunc[i][1]) < 0)
                Hata(args[2], i + 1, 2, "Üye bulunamadı.");
            if (Bul(kitap, odunc[i][2]) < 0)
                Hata(args[2], i + 1, 3, "Kitap bulunamadı.");
            if (odunc[i][4] < odunc[i][3])
                Hata(args[2], i + 1, 5, "Son tarih alıştan önce.");
        }

        int[] iadeGun = new int[odunc.Length];
        for (int i = 0; i < iade.Length; i++)
        {
            int p = Bul(odunc, iade[i][0]);
            if (p < 0) Hata(args[3], i + 1, 1, "Ödünç bulunamadı.");
            if (iade[i][1] < odunc[p][3])
                Hata(args[3], i + 1, 2, "İade alıştan önce.");
            iadeGun[p] = iade[i][1];
        }

        var rapor = new StringBuilder();
        foreach (int p in Sira(odunc))
        {
            if (odunc[p][3] > kesim) continue;
            bool dondu = iadeGun[p] != 0 && iadeGun[p] <= kesim;
            int son = dondu ? iadeGun[p] : kesim;
            int gecikme = son > odunc[p][4] ? son - odunc[p][4] : 0;
            Satir(rapor, odunc[p][0], odunc[p][1], odunc[p][2],
                dondu ? "IADE" : "ACIK", gecikme);
        }
    }
}
```

```

        Yaz(args[5], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -1)
                veri[n][j] = Tarih(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

```

```

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static int Tarih(string yol, int satir, int alan, string metin)
{
    if (!DateOnly.TryParseExact(metin, "yyyy-MM-dd",
        CultureInfo.InvariantCulture, DateTimeStyles.None,
        out DateOnly tarih) || tarih.Year < 2000 || tarih.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz tarih.");
    return tarih.DayNumber;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

```

```

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

Örnek çalıştırmalar

Normal örnek için yukarıdaki komutu kullanın; rapor.txt başlangıçta yoktur.

uyeler.txt dosyasının tam içeriği:

2

kitaplar.txt dosyasının tam içeriği:

7

oduncler.txt dosyasının tam içeriği:

2;2;7;2026-10-02;2026-10-08
1;2;7;2026-10-01;2026-10-07

iadeler.txt dosyasının tam içeriği:

1;2026-10-09
2;2026-10-12

Yeni rapor.txt dosyasının tam içeriği:

1;2;7;IADE;2
2;2;7;ACIK;2

Konsolun tam çıktısı:

Rapor yazıldı.

Sınır örneği: aynı dosya adlarını ayrı boş bir klasörde hazırlayın; aynı komutu, kesim tarihini 2000-02-29 yaparak çalıştırın.

uyeler.txt dosyasının tam içeriği:

9999

kitaplar.txt dosyasının tam içeriği:

9999

oduncler.txt dosyasının tam içeriği:

9999;9999;9999;2000-02-29;2000-02-29

iadeler.txt sıfır bayttır.

Yeni rapor.txt dosyasının tam içeriği:

9999;9999;9999;ACIK;0

Konsolun tam çıktısı:

Rapor yazıldı.

Geçersiz örnek: ayrı bir klasörde, aynı komut, normal örneğin kesim tarihiyle.

uyeler.txt dosyasının tam içeriği:

2

kitaplar.txt dosyasının tam içeriği:

7

oduncler.txt dosyasının tam içeriği:

1;2;7;2026-10-01;2026-10-07

iadeler.txt dosyasının tam içeriği:

1;2026-09-30

Rapor oluşturulmaz. Konsolun tam çıktısı:

Hata: iadeler.txt: satır 1, alan 2: İade alıştan önce.

Bütün giriş dosyaları sıfır bayt ve rapor.txt yokken, kesim tarihi normal örnekteki gibi geçerliyen yeni rapor sıfır bayttır.

Konsolun tam çıktısı:

Rapor yazıldı.

Geçerli normal girdilerle rapor.txt önceden varsa mevcut baytlar korunur. Konsolun tam çıktısı:

Hata: Dosya işlemi başarısız.

Sınır durumları

- Dört dosyanın her biri en fazla 100 satırdır. Gün numarası farkı 2000–2099 içinde en fazla 36524; int yeterlidir. Her dosya boşken rapor boşdur. Üye veya kitap boşken ödünç kaydı hata olur. İade dosyası boş olabilir; olmayan ödünç için iade hata olur. Kesim tarihinden sonraki kayıtlar da rapor dışında kalsa bile doğrulanır.
- Satır kapasitesinin tam dolması kabul edilir; bir sonraki satır saklanmadan hata olur. Tek bir boş satır sıfır kayıt sayılmaz; alan/sayı hatasıdır. Çok büyük veya taşan sayısal alan ayrıştırma hatasıdır.
- Aynı sayıyı farklı baştaki sıfırlarla yazmak anahtar tekrarını gizlemez. Sıralama sırasında dizinler taşındığı için dosya satırı ile hata konumu ve ilişkiler değişmez.
- Hatalı son kayıt dahil bütün giriş doğrulanmadan rapor açılmaz. Ortak I/O ve çıktı güvenliği sözleşmesi yukarıda tanımlıdır.

Kazanımlar

- Eksik ilişki ile isteğe bağlı ilişkiyi ayırma; kesim tarihiyle geçmiş görünüm kurma; dosyadaki sıra ile kimlik sırasını ayırt etme.
- Ödünçün alış, son tarih ve iade durumu aynı gecikme davranışında birleşir. Sonraki derste bu alanları bir neshede korumak, tarih kurallarının farklı raporlarda tekrar edilmesini azaltabilir.

Alıştırmalar

1. Üye başına açık ve gecikmiş ödünç sayısını ikinci bir bellek raporunda üretin.
2. Kesim tarihinden sonraki iadenin neden ACIK sayıldığını açıklayın; bir Ödünç nesnesi hangi tarih tutarlılığını koruyabilir?

094 Depolar arası transfer ve stok korunumu

Düzey: İleri Önkoşullar: 053, 083, 087, 089

Kavramlar: İki boyutlu stok, Transfer güncellemesi, Korunum denetimi, Bileşik anahtar

Problem tanımı

Depo ve ürün tanımlarından bir stok matrisi kurun. Açılış sayımlarını yükledikten sonra transferleri dosya sırasında uygulayın. Her transfer bir depodan eksiltirken diğerine aynı miktarı eklemelidir. Sadece son bakiyeleri denetlemek yeterli değildir; ara adımda kaynak stok yetmezse işlem hattını durdurun.

Girdi ve çıktı

dotnet run -- ornek-094/depolar.txt ornek-094/urunler.txt ornek-094/acilis.txt
ornek-094/transferler.txt ornek-094/rapor.txt

depolar.txt: depoKod; urunler.txt: ürünKod; kodlar 1..9999 ve tekildir. acilis.txt: depoKod;ürünKod;adet (adet 0..1000000); depo/ürün çifti tekildir, verilmemiş çiftin açılışı sıfırdır. transferler.txt: transferKod;kaynakDepo;hedefDepo;ürünKod;adet (kodlar 1..9999, adet 1..1000000). Transfer kodu tekil; depo ve ürünler bulunmalı, kaynak ve hedef farklı olmalıdır.

Transfer sırası dosya sırasındır; transfer koduna göre işlem sıralaması yapılmaz. Kaynak stok yetmiyorsa hata; hedef stokun en fazla 1000000 sınırını aşması da hatadır. İki koşul denetlenmeden hücrelerin hiçbirisi değiştirilmez. Başarılı rapor tüm depo/ürün çiftlerini, stok sıfır olsa da depoKod;ürünKod;sonAdet biçiminde yazar; depo, ardından ürün kodu artar. Ürün bazında bütün depoların toplamı açılış toplamına eşit kalmalıdır.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtarı belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-

8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Dört tabloyu ve tekil/bileşik anahtarları doğrulayın.
2. Açılış ilişkilerini eşleyip stok ve ürün başına long açılış toplamalarını kurun.
3. Transferi iki depo ve bir ürüne eşleyin; kaynak yeterliliği ve hedef üst sınırını birlikte kontrol edin.
4. Eksiltme ve eklemeyi uygulayın; her hücre için 0..1000000 değişmezini koruyun.
5. Ürün toplamını yeniden hesaplayıp korunum denetleyin; depo ve ürün dizinlerini ayrı sıralayıp matrisi raporlayın.

İşlem	Depo 1 / ürün 7	Depo 2 / ürün 7	Toplam
Açılış	10	0	10
1 → 2, adet 4	6	4	10
2 → 1, adet 1	7	3	10

Oku yalnız geçerli satırları içeren diziyi döndürür. Tekil seçilen sütunları birlikte anahtar sayar; Bu1 ilk sütunda doğrusal arar, yoksa -1 verir. Sıra kaynak kayıtları değiştirmeden kod artan eklemeli sıralamayla dizin dizisi döndürür. Satır yalnız bellek raporunu değiştirir; Yaz dış dosyayı oluşturur. Hazır LINQ, GroupBy, Sort veya arama algoritması kullanılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 5) throw new FormatException(
        "5 argüman gerekli.");
    checked
    {
        int[][] depo = Oku(args[0], new[] { 1 }, new[] { 9999 });
        int[][] urun = Oku(args[1], new[] { 1 }, new[] { 9999 });
        int[][] acilis = Oku(args[2], new[] { 1, 1, 0 },
            new[] { 9999, 9999, 1000000 });
        int[][] aktar = Oku(args[3], new[] { 1, 1, 1, 1, 1 },
            new[] { 9999, 9999, 9999, 9999, 1000000 });
        Tekil(depo, args[0], 0); Tekil(urun, args[1], 0);
        Tekil(acilis, args[2], 0, 1); Tekil(aktar, args[3], 0);
        int[,] stok = new int[depo.Length, urun.Length];
        long[] ilk = new long[urun.Length];
        for (int i = 0; i < acilis.Length; i++)
        {
            int d = Bul(depo, acilis[i][0]), u = Bul(urun, acilis[i][1]);
            if (d < 0) Hata(args[2], i + 1, 1, "Depo bulunamadı.");
            if (u < 0) Hata(args[2], i + 1, 2, "Ürün bulunamadı.");
            stok[d, u] = acilis[i][2]; ilk[u] += acilis[i][2];
        }

        for (int i = 0; i < aktar.Length; i++)
        {
            int a = Bul(depo, aktar[i][1]), b = Bul(depo, aktar[i][2]);
            int u = Bul(urun, aktar[i][3]), miktar = aktar[i][4];
            if (a < 0) Hata(args[3], i + 1, 2, "Depo bulunamadı.");
            if (b < 0) Hata(args[3], i + 1, 3, "Depo bulunamadı.");
            if (u < 0) Hata(args[3], i + 1, 4, "Ürün bulunamadı.");
            if (a == b) Hata(args[3], i + 1, 3, "Depolar farklı olmalı.");
            if (stok[a, u] < miktar)
                Hata(args[3], i + 1, 5, "Kaynak stok yetersiz.");
            if (stok[b, u] > 1000000 - miktar)
                Hata(args[3], i + 1, 5, "Hedef stok sınırı aşıldı.");
            stok[a, u] -= miktar; stok[b, u] += miktar;
        }

        for (int u = 0; u < urun.Length; u++)
        {
            long son = 0;
            for (int d = 0; d < depo.Length; d++) son += stok[d, u];
            if (son != ilk[u]) throw new FormatException("Korunum bozuldu.");
        }
        var rapor = new StringBuilder();
        int[] ds = Sira(depo), us = Sira(urun);
        foreach (int d in ds)
            foreach (int u in us) Satir(rapor, depo[d][0], urun[u][0], stok[d, u]);
    }
}
```

```

        Yaz(args[4], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -1)
                veri[n][j] = Tarih(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

```

```

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static int Tarih(string yol, int satir, int alan, string metin)
{
    if (!DateOnly.TryParseExact(metin, "yyyy-MM-dd",
        CultureInfo.InvariantCulture, DateTimeStyles.None,
        out DateOnly tarih) || tarih.Year < 2000 || tarih.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz tarih.");
    return tarih.DayNumber;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

```

```

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

Örnek çalıştırmalar

Normal örnek için yukarıdaki komutu kullanın; rapor.txt başlangıçta yoktur.

depolar.txt dosyasının tam içeriği:

```

2
1

```

urunler.txt dosyasının tam içeriği:

```

7

```

acilis.txt dosyasının tam içeriği:

```

1;7;10

```

transferler.txt dosyasının tam içeriği:

```

9;1;2;7;4
3;2;1;7;1

```

Yeni rapor.txt dosyasının tam içeriği:

```

1;7;7
2;7;3

```

Konsolun tam çıktısı:

```

Rapor yazıldı.

```

Sınır örneği: aynı dosya adlarını ayrı boş bir klasörde hazırlayın; aynı komutu çalıştırın.

depolar.txt dosyasının tam içeriği:

```

9999
1

```

urunler.txt dosyasının tam içeriği:

```

9999

```

acilis.txt dosyasının tam içeriği:

```

1;9999;1000000

```

transferler.txt dosyasının tam içeriği:

```

9999;1;9999;9999;1000000

```

Yeni rapor.txt dosyasının tam içeriği:

```

1;9999;0
9999;9999;1000000

```

Konsolun tam çıktısı:

Rapor yazıldı.

Geçersiz örnek: ayrı bir klasörde, aynı komut.

depolar.txt dosyasının tam içeriği:

1
2

urunler.txt dosyasının tam içeriği:

7

acilis.txt dosyasının tam içeriği:

1;7;10

transferler.txt dosyasının tam içeriği:

1;2;1;7;1

Rapor oluşturulmaz. Konsolun tam çıktısı:

Hata: transferler.txt: satır 1, alan 5: Kaynak stok yetersiz.

Bütün giriş dosyaları sıfır bayt ve rapor.txt yokken yeni rapor sıfır bayttır.

Konsolun tam çıktısı:

Rapor yazıldı.

Geçerli normal girdilerle rapor.txt önceden varsa mevcut baytlar korunur. Konsolun tam çıktısı:

Hata: Dosya işlemi başarısız.

Sınır durumları

- 100 depo × 100 ürün en fazla 10000 rapor satırıdır. Açılış ve transfer dosyaları ayrı ayrı 100 satırıdır. Her hücre en fazla 1000000; ürün toplamı en fazla 100000000, long ile tutulur. Depo veya ürün boşsa matris raporu boştur; dolu açılış/transferde eksik ilişki hatasıdır. Açılış dosyası boşken bütün stoklar sıfırdır. Sıfır transfer miktarı hata olur.
- Satır kapasitesinin tam dolması kabul edilir; bir sonraki satır saklanmadan hata olur. Tek bir boş satır sıfır kayıt sayılmaz; alan/sayı hatasıdır. Çok büyük veya taşan sayısal alan ayırıştırma hatasıdır.
- Aynı sayıyı farklı baştaki sıfırlarla yazmak anahtar tekrarını gizlemez. Sıralama sırasında dizinler taşındığı için dosya satırı ile hata konumu ve ilişkiler değişmez.
- Hatalı son kayıt dahil bütün giriş doğrulanmadan rapor açılmaz. Ortak I/O ve çıktı güvenliği sözleşmesi yukarıda tanımlıdır.

Kazanımlar

- Tek hareketi iki bağlantılı güncellemeye ayırıştırma; ara durum ile son durumu ayırma; toplamın korunmasını ek güvence olarak kullanma.
- Stok hücresinin depo, ürün ve adet ilişkisi ile transferin iki taraflı davranışı birlikte değişir. Sonraki derste bu veriyi koruyan nesneler, bir tarafı güncelleyip diğerini unutma riskini azaltabilir; bu çözümde matrisi ve metot sözleşmelerini izleyin.

Alıştırmalar

1. Her depo için kaç farklı üründe pozitif stok kaldığını rapora ekleyin.
2. Yalnız son stokların kontrol edilmesinin hangi hatalı transfer dizisini kaçıracağını örnekleyin; davranışı bir nesnede korumanın gerekçesini yazın.

095 Reçete ve siparişlerden üretim planı

Düzey: İleri Önkoşullar: 053, 079, 083, 089

Kavramlar: Paylaşılan kaynak, Öncelikli tahsis, İhtiyaç matrisi, Tam sipariş kabulü

Problem tanımı

Malzeme stoklarını, ürün reçetelerini ve siparişleri birleştirerek öncelikli bir üretim planı çıkarın. Her sipariş ya bütünüyle karşılanır ya da hiç malzeme tüketmez. Kısmi stok azaltımı yapmayın. Bu politika dosya sırasından bağımsız bir açgözlü tahsistir; toplam üretimi en büyük yapan planı bulduğu iddia edilmez.

Girdi ve çıktı

```
dotnet run -- ornek-095/malzemeler.txt ornek-095/receeteler.txt ornek-095/siparisler.txt ornek-095/rapor.txt
```

malzemeler.txt: malzemeKod;stok (kod 1..9999, stok 0..1000000), kod tekildir. receeteler.txt: ürünKod;malzemeKod;birimİhtiyaç (kodlar 1..9999, ihtiyaç 1..1000), ürün/malzeme çifti tekildir. Ürünler reçete dosyasında tanımlanır; her reçete malzemesi katalogda bulunmalıdır. siparisler.txt: siparişKod;ürünKod;adet;öncelik (kodlar 1..9999, adet 1..1000, öncelik 1..9), sipariş kodu tekildir, ürünün en az bir reçete satırı bulunmalıdır.

İşlem sırası öncelik azalan, eşitlikte sipariş kodu artandır. Her malzemenin ihtiyaç × adet miktarı eldeki stokla karşılaştırılır. Bir malzeme yetmezse bütün sipariş BEKLER, aksi halde tüm gereksinimler düşülür ve URET olur. Rapor işlem sırasıyla S;siparişKod;durum, ardından malzeme kodu artan M;malzemeKod;kalanStok satırlarıdır. Yinelenen ürün farklı siparişlerde kabul edilir. Sipariş yokken yalnız stok bölümü yazılır.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağılı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtar belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşdı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Tabloları, anahtarları ve reçete malzeme ilişkilerini doğrulayın.
2. Siparişlerde ürün için reçete varlığını doğrulayın.
3. Sipariş dizinlerini öncelik azalan ve kod artan eklemeli sıralayın.
4. Her siparişte önce tüm gereksinimleri kontrol edin; ancak yeterliyse ikinci geçişte stok düşürün.
5. Karar satırlarını ve son malzeme stoklarını bellek raporuna ekleyip yeni dosyaya yazın.

Sipariş	Öncelik	İhtiyaç	Önceki stok	Karar / sonraki stok
2	9	$2 \times 2 = 4$	5	URET / 1
1	1	$1 \times 2 = 2$	1	BEKLER / 1

Oku yalnız geçerli satırları içeren diziyi döndürür. Tekil seçilen sütunları birlikte anahtar sayar; Bu ilk sütunda doğrusal arar, yoksa -1 verir. Sıra kaynak kayıtları değiştirmeden kod artan eklemeli sıralamayla dizin dizisi döndürür. Satır yalnız bellek raporunu değiştirir; Yaz dış dosyayı oluşturur. Hazır LINQ, GroupBy, Sort veya arama algoritması kullanılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 4) throw new FormatException(
        "4 argüman gerekli.");
    checked
    {
        int[][] mal = Oku(args[0], new[] { 1, 0 },
            new[] { 9999, 1000000 });
        int[][] rec = Oku(args[1], new[] { 1, 1, 1 },
            new[] { 9999, 9999, 1000 });
        int[][] sip = Oku(args[2], new[] { 1, 1, 1, 1 },
            new[] { 9999, 9999, 1000, 9 });
        Tekil(mal, args[0], 0); Tekil(rec, args[1], 0, 1);
        Tekil(sip, args[2], 0);
        int[] mi = new int[rec.Length], kalan = new int[mal.Length];
        for (int i = 0; i < mal.Length; i++) kalan[i] = mal[i][1];
        for (int i = 0; i < rec.Length; i++)
        {
            mi[i] = Bul(mal, rec[i][1]);
            if (mi[i] < 0) Hata(args[1], i + 1, 2, "Malzeme bulunamadı.");
        }
        for (int i = 0; i < sip.Length; i++)
            if (Bul(rec, sip[i][1]) < 0)
                Hata(args[2], i + 1, 2, "Reçete bulunamadı.");

        int[] sira = Sira(sip);
        for (int i = 1; i < sira.Length; i++)
        {
            int p = sira[i], j = i - 1;
            while (j >= 0 && sip[sira[j]][3] < sip[p][3])
            { sira[j + 1] = sira[j]; j--; }
            sira[j + 1] = p;
        }
        var rapor = new StringBuilder();
        foreach (int p in sira)
        {
            bool yeter = true;
            for (int r = 0; r < rec.Length; r++)
                if (rec[r][0] == sip[p][1] &&
                    kalan[mi[r]] < rec[r][2] * sip[p][2]) yeter = false;
            if (yeter)
                for (int r = 0; r < rec.Length; r++)
                    if (rec[r][0] == sip[p][1])
                        kalan[mi[r]] -= rec[r][2] * sip[p][2];
            Satir(rapor, "S", sip[p][0], yeter ? "URET" : "BEKLER");
        }
        foreach (int p in Sira(mal)) Satir(rapor, "M", mal[p][0], kalan[p]);
    }
}
```



```

        Yaz(args[3], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}
catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -1)
                veri[n][j] = Tarih(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

```

```

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static int Tarih(string yol, int satir, int alan, string metin)
{
    if (!DateOnly.TryParseExact(metin, "yyyy-MM-dd",
        CultureInfo.InvariantCulture, DateTimeStyles.None,
        out DateOnly tarih) || tarih.Year < 2000 || tarih.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz tarih.");
    return tarih.DayNumber;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

```

```

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

Örnek çalıştırmalar

Normal örnek için yukarıdaki komutu kullanın; rapor.txt başlangıçta yoktur.

malzemeler.txt dosyasının tam içeriği:

7;5

receteler.txt dosyasının tam içeriği:

9;7;2

siparisler.txt dosyasının tam içeriği:

1;9;1;1

2;9;2;9

Yeni rapor.txt dosyasının tam içeriği:

S;2;URET

S;1;BEKLER

M;7;1

Konsolun tam çıktısı:

Rapor yazıldı.

Sınır örneği: aynı dosya adlarını ayrı boş bir klasörde hazırlayın; aynı komutu çalıştırın.

malzemeler.txt dosyasının tam içeriği:

9999;1000000

receteler.txt dosyasının tam içeriği:

9999;9999;1000

siparisler.txt dosyasının tam içeriği:

9999;9999;1000;9

Yeni rapor.txt dosyasının tam içeriği:

S;9999;URET

M;9999;0

Konsolun tam çıktısı:

Rapor yazıldı.

Geçersiz örnek: ayrı bir klasörde, aynı komut.

malzemeler.txt dosyasının tam içeriği:

7;5

receteler.txt dosyasının tam içeriği:

9;7;2

siparisler.txt dosyasının tam içeriği:

1;8;1;1

Rapor oluşturulmaz. Konsolun tam çıktısı:

Hata: siparisler.txt: satır 1, alan 2: Reçete bulunamadı.

Bütün giriş dosyaları sıfır bayt ve rapor.txt yokken yeni rapor sıfır bayttır.

Konsolun tam çıktısı:

Rapor yazıldı.

Geçerli normal girdilerle rapor.txt önceden varsa mevcut baytlar korunur. Konsolun tam çıktısı:

Hata: Dosya işlemi başarısız.

Sınır durumları

- Her dosya 100 satırdır. Tek malzemenin tek sipariş gereksinimi en fazla 1000000; stok ve çarpım int aralığındadır. Bileşik anahtar tekil olduğu için bir malzemenin aynı ürün gereksinimi gizlice iki satırda toplanmaz. Malzeme yokken reçete, reçete yokken sipariş hata olur. Tümü boşsa rapor boştur. Üretilmeyen sipariş normal sonuçtur.
- Satır kapasitesinin tam dolması kabul edilir; bir sonraki satır saklanmadan hata olur. Tek bir boş satır sıfır kayıt sayılmaz; alan/sayı hatasıdır. Çok büyük veya taşan sayısal alan ayrıştırma hatasıdır.
- Aynı sayıyı farklı baştaki sıfırlarla yazmak anahtar tekrarını gizlemez. Sıralama sırasında dizinler taşındığı için dosya satırı ile hata konumu ve ilişkiler değişmez.
- Hatalı son kayıt dahil bütün giriş doğrulanmadan rapor açılmaz. Ortak I/O ve çıktı güvenliği sözleşmesi yukarıda tanımlıdır.

Kazanımlar

- Kontrol et ve uygula geçişlerini ayırma; sınırlı kaynakta sıranın etkisini açıklama; açgözlü politikanın en iyi plan garantisi vermediğini gösterme.
- Bir siparişin miktarı, önceliği ve karşılanabilirlik davranışı; reçetenin ürün/malzeme ilişkileri tekrar birlikte kullanılır. Sonraki derste bu birliktelikleri nesnelerde toplamak, karar ve stok güncellemesinin farklı yerlerde ayrışmasını önleyebilir.

Alıştırmalar

1. BEKLER siparişinde ilk yetersiz malzeme kodunu, malzeme kodu artan sırada seçerek ekleyin.
2. Daha fazla toplam ürün veren başka bir sıra bulun; Sipariş nesnesinin hangi doğrulama ve hesap davranışını bir arada tutacağını tartışın.

096 Vardiyaları ücret dilimlerine bölerek maliyet hesaplama

Düzey: İleri Önkoşullar: 084, 089, 092

Kavramlar: Aralık parçalama, Takvim dakikası, Gün sınırı, Süre korunumu, Çoklu veri kaynağı

Problem tanımı

Çalışan, vardiya ve her gün tekrarlanan dakika ücretlerini birleştirin. Vardiyaları ücret değişimlerinde ve gece yarısında parçalayıp çalışan başına toplam süreyi ve maliyeti bulun. Aynı çalışanın çalıştığı vardiyaları hatadır. Bu problemde verilen dakika ücretleri kullanılır; mola, fazla mesai veya başka ücret kuralı uygulanmaz.

Girdi ve çıktı

```
dotnet run -- ornek-096/calisanlar.txt ornek-096/vardiyalar.txt ornek-096/tarife.txt ornek-096/rapor.txt
```

Çalışan satırı kod (1..9999, en fazla 30 çalışan); vardiya satırı vardiyaKod;çalışanKod;başlangıç;bitiş (kodlar 1..9999, en fazla 100 vardiya); tarife satırı başlangıçDakika;bitişDakika;dakikaÜcretiKuruş (0..1439, 1..1440, 1..100000, en fazla 24 dilim). Çalışan ve vardiya kodları kendi dosyalarında benzersizdir. Tarifeler dosyada sırasız olabilir; sıralandıktan sonra 0..1440 gününü boşluk ve çakışma olmadan kapsamalıdır.

Vardiya zamanları tam yyyy-MM-dd HH:mm, 2000..2099 yılları arasındadır. Süre 1..1440 dakikadır. Aralık [başlangıç,bitiş) biçimindedir; bitiş ile sonraki başlangıcın eşitliği çakışma değildir. Hesap takvim dakikalarıyla yapılır; saat dilimi veya yaz saati dönüşümü uygulanmaz. 24:00 yerine sonraki günün 00:00 değeri yazılır.

Rapor çalışan kodu artan sırada kod;toplamDakika;maliyetKuruş biçimindedir. Vardiyası olmayan çalışan sıfırlarla bulunur. Tarife zorunludur; çalışan ve vardiya dosyaları boş ve tarife geçerliyse rapor boştur. Boşluk/kapsama gibi bütün dosyayı ilgilendiren tarife hatasında satır ve alan 0 olabilir.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtarları belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayırıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik

dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşıtı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Üç dosyayı ve benzersiz anahtarları doğrulayın; tarife dizinlerini başlangıca göre sıralayın.
2. İlk tarifenin 0'dan başlayıp her bitişin sonraki başlangıç olduğunu ve son bitişin 1440 olduğunu doğrulayın.
3. Çalışan ilişkisini, vardiya süresini ve aynı çalışanın bütün önceki vardiyalarıyla çakışmayı denetleyin.
4. Her vardiyada bulunduğunuz dakikanın günlük tarife dilimini bulun.
5. Vardiya bitiş ile dilim bitişinin erken olanına kadar süre ve maliyet ekleyin; zamanı bu sınıra taşıyın.
6. Çalışan koduna göre rapor üretin ve yeni dosyaya yazın.

Takvim dakikası, yıl 1'in başından bu yana tam dakika sayısıdır; kabul edilen tarihlerde int aralığına sığar. an % 1440 gün içi konumu, an - gunIci gün başlangıcını verir. Günün son diliminden sonra yeni günün ilk dilimi bulunur. Her parça pozitifdir; toplam parçalar vardiya süresini tam korur. Maliyet en fazla $100 \times 1440 \times 100000 = 14400000000$ kuruştur ve long gerektirir.

21:00–ertesi gün 01:00 için tarife 00:00–08:00 ve 22:00–24:00 arasında 2, diğer saatlerde 1 kuruş/dakika olsun:

Parça	Dakika	Birim ücret	Maliyet
21:00–22:00	60	1	60
22:00–24:00	120	2	240
00:00–01:00	60	2	120

Zaman kesin biçimdeki metni takvim dakikasına dönüştürür. Oku zaman sütunlarını -2 işaretiyle tanır; bu işaret dosya verisi değildir. Sıra yalnız dizinleri taşır, hata satırları özgün dosyayı gösterir. Doğrulama bittikten sonra her parçanın maliyeti biriktirilir.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 4) throw new FormatException("4 argüman gerekli.");
    checked
    {
        int[][] cal = Oku(args[0], new[] { 1 }, new[] { 9999 }, 30);
        int[][] vard = Oku(args[1], new[] { 1, 1, 0, 0 },
            new[] { 9999, 9999, -2, -2 });
        int[][] tar = Oku(args[2], new[] { 0, 1, 1 },
            new[] { 1439, 1440, 100000 }, 24);
        Tekil(cal, args[0], 0); Tekil(vard, args[1], 0);
        int[] ts = Sira(tar);
        int beklenen = 0;
        foreach (int p in ts)
        {
            if (tar[p][0] != beklenen || tar[p][1] <= tar[p][0])
                Hata(args[2], p + 1, 0, "Tarife boşluklu veya çakışıyor.");
            beklenen = tar[p][1];
        }
        if (beklenen != 1440)
            Hata(args[2], 0, 0, "Tarife günü kapsamıyor.");

        int[] ci = new int[vard.Length];
        for (int i = 0; i < vard.Length; i++)
        {
            ci[i] = Bul(cal, vard[i][1]);
            if (ci[i] < 0)
                Hata(args[1], i + 1, 2, "Çalışan bulunamadı.");
            int sure = vard[i][3] - vard[i][2];
            if (sure < 1 || sure > 1440)
                Hata(args[1], i + 1, 0, "Süre 1..1440 olmalıdır.");
            for (int j = 0; j < i; j++)
                if (ci[j] == ci[i] &&
                    Math.Max(vard[i][2], vard[j][2]) <
                    Math.Min(vard[i][3], vard[j][3]))
                    Hata(args[1], i + 1, 3, "Vardiyalar çakışıyor.");
        }
    }
}
```

```

long[] dakika = new long[cal.Length], maliyet = new long[cal.Length];
for (int i = 0; i < vard.Length; i++)
{
    int an = vard[i][2];
    while (an < vard[i][3])
    {
        int gunIci = an % 1440, p = 0;
        while (gunIci >= tar[ts[p]][1]) p++;
        int t = ts[p];
        int son = Math.Min(vard[i][3], an - gunIci + tar[t][1]);
        int parca = son - an;
        dakika[ci[i]] += parca;
        maliyet[ci[i]] += (long)parca * tar[t][2];
        an = son;
    }
}
var rapor = new StringBuilder();
foreach (int p in Sira(cal))
    Satir(rapor, cal[p][0], dakika[p], maliyet[p]);
Yaz(args[3], rapor.ToString());
}
Console.WriteLine("Rapor yazıldı.");
}

catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

```



```

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            if (ust[j] == -2)
                veri[n][j] = Zaman(yol, n + 1, j + 1, a[j]);
            else
                veri[n][j] = Tam(yol, n + 1, j + 1,
                    a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

```

```

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

```

static int Zaman(string yol, int satir, int alan, string metin)
{
    if (metin.Length != 16) Hata(yol, satir, alan, "Geçersiz zaman.");
    for (int i = 0; i < metin.Length; i++)
    {
        char beklenen = i == 4 || i == 7 ? '-' :
            i == 10 ? ' ' : i == 13 ? ':' : '\0';
        if (beklenen != '\0' ? metin[i] != beklenen :
            metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz zaman.");
    }
    if (!DateTime.TryParseExact(metin, "yyyy-MM-dd HH:mm",
        CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime an) ||
        an.Year < 2000 || an.Year > 2099)
        Hata(yol, satir, alan, "Geçersiz zaman.");
    return (int)(an.Ticks / TimeSpan.TicksPerMinute);
}

```

Örnek çalıştırmalar

Yukarıdaki komut için giriş dosyalarının tam içerikleri aşağıdadır; rapor başlangıçta yoktur.

calisanlar.txt:

```

2
1

```

vardiyalar.txt:

```

7;1;2026-10-05 21:00;2026-10-06 01:00

```

tarife.txt:

```

1320;1440;2
0;480;2
480;1320;1

```

Yeni rapor.txt:

```

1;240;420
2;0;0

```

Konsolun tam çıktısı:

```

Rapor yazıldı.

```

Sınır örneğinde çalışan dosyası 1, vardiya dosyası 1;1;2099-12-30 23:59;2099-12-31 23:59, tarife dosyası 0;1440;100000 satırıdır. Yeni rapor 1;1440;144000000 satırı, konsol yine Rapor yazıldı. olur. Her satır LF ile biter.

Normal örnekte vardiya dosyasına 8;1;2026-10-05 23:00;2026-10-06 02:00 ikinci satırını ekleyin. Rapor oluşmaz; tam konsol çıktısı:

```

Hata: vardiyalar.txt: satır 2, alan 3: Vardiyalar çakışıyor.

```

Sınır durumları

- Bir dakikalık veya tam 1440 dakikalık vardiya kabul edilir; sıfır, negatif ve daha uzun süre reddedilir.
- Farklı çalışanlar aynı anda çalışabilir; aynı çalışanın bitişte başlayan vardiyası kabul edilir.
- Tarifede boşluk, örtüşme ve eksik son bölüm hata olur. Tek 0;1440;ücret satırı geçerlidir.
- Ay, yıl ve artık gün geçişleri kesin takvim ayrıştırmasıyla doğrulanır; saat veya dakika taşırılmaz.
- Satırların farklı sırada olması sonucu değiştirmez. Çakışma denetimi ilk hatalı fiziksel vardiya satırını bildirir.

Kazanımlar

- Aralığı doğru sınırdaki parçalarla süreyi ve maliyeti koruma.
- Günlük konumu genel takvim konumundan ayırma; sayısal türü toplam sınırına göre seçme.
- Çalışan kimliği, vardiya aralığı ve ücret diliminin veri/davranış ilişkisini belirleme; sonraki derste bu doğrulamaların hangi nesnelerde toplanabileceğini tartışma.

Alıştırmalar

1. Tarife dilimi başına dakika toplamalarını da raporlayın; vardiya süreleriyle toplamalarının eşitliğini denetleyin.
2. Çakışma hatasında önceki vardiyanın kodunu da bildirin.
3. Bir Vardiya nesnesinde süre, çakışma ve parçalama davranışlarının nasıl birlikte korunabileceğini açıklayın.

097 Bütçe kısıtlı proje seçimi

Düzey: İleri Önkoşullar: 053, 075, 079, 095

Kavramlar: 0/1 seçim, Dinamik programlama, Tam maliyet durumu, Çözümü geri kurma, Sözlük sırası

Problem tanımı

Her projeyi en fazla bir kez seçerek verilen bütçeyle toplam kazanımı en büyük yapan kümeyi bulun. Eşit kazanımda en düşük harcamayı; yine eşitlikte artan proje kodları listesinin sözlük sırasına göre ilkini seçin. Kazanım/maliyet oranına göre seçim her zaman doğru değildir; tüm geçerli seçimler için en iyi sonucu garanti eden dinamik programlama kurun.

Girdi ve çıktı

dotnet run -- ornek-097/projeler.txt 8 ornek-097/rapor.txt

Proje satırı kod;maliyet;kazanım biçimindedir: kod 1..9999 benzersiz; maliyet 1..500, kazanım 1..10000, en fazla 30 proje. İkinci argüman bütçedir, 0..500 tamsayıdır. maliyet ve kazanım bu problemin soyut birimleridir.

Rapor önce T;harcama;kazanım, ardından seçilen projeler için kod artan P;kod satırlarıdır. Boş küme geçerlidir; hiçbir proje seçilmediğinde yalnız T;0;0 yazılır. Sözlük sırası kodları sayısal olarak karşılaştırır: ilk farklı kodu küçük olan liste önce gelir; bir liste diğerinin tam başlangıcıysa kısa liste önce gelir. Pozitif kazanım nedeniyle eşit optimumda bir listenin diğerinin gerçek başlangıcı olması mümkün değildir.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtarları belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun

tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşıtı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Projeleri ve bütçeyi doğrulayın; proje dizinlerini kod artan sıralayın.
2. $dp[i, b]$ durumunu i 'den sonraki projelerle tam b harcayarak erişilen en yüksek kazanım olarak tanımlayın; erişilemez durum -1'dir.
3. Taban olarak projeler bitmişken yalnız sıfır harcama/sıfır kazanımı kabul edin.
4. Son projeden başa gidin. i projesini atlama ile bir kez seçme adaylarının kazanımlarını karşılaştırın.
5. Bütçeyi aşmayan harcamaları küçükten büyüğe tarayın; yalnız daha yüksek kazanımda güncelleyerek en düşük harcama eşitliğini koruyun.
6. Kod artan sırada, mevcut proje kalan tam maliyet ve kazanımı sağlayabiliyorsa onu seçin; kalan değerleri azaltın.
7. Toplamı ve seçimi raporlayın.

Dinamik programlama, aynı alt problemi tekrar tekrar çözmek yerine sonucunu bir durum tablosunda saklar. Seçme ve atlama dalları bütün olasılıkları kapsar; her iki dal $i+1$ satırına baktığından bir proje tekrar kullanılamaz. Taban $dp[n, 0] = 0$, diğer durumlar -1'dir. 0 ile -1 birbirinden ayrılmalıdır.

Geri kurmada küçük kodu seçmek ancak geriye kalan tablonun tamamlayabildiği bir optimum olduğunda yapılır. Böylece sayısal sözlük sırası korunur. Pozitif maliyet kalan bütçeyi azaltır; en fazla n seçim denetlenir. Durum sayısı en fazla 31×501 , kazanım en fazla 300000'dir. Süre $O(n \times \text{bütçe})$, bellek $O(n \times \text{bütçe})$ olup bütçenin sayısal büyüklüğüne bağlıdır.

Kodlar 10:(5,10), 20:(4,7), 30:(4,7), bütçe 8 için:

Aday küme	Harcama	Kazanım	Karar
10	5	10	Geçerli, en iyi değil
20	4	7	Geçerli
20, 30	8	14	En iyi
10, 20	9	17	Bütçeyi aşıyor

10 en yüksek kazanım/maliyet oranına sahipken tek başına seçilmesi 14 kazanımı kaçıırır. Tablo, 079'daki seç/dışla yaklaşımının optimum kazanım hedefi için yeniden kullanılmasını sağlar; bu kez hedef toplam kümelerin adedi sayılmaz.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 3) throw new FormatException("3 argüman gerekli.");
    checked
    {
        int[][] pro = Oku(args[0], new[] { 1, 1, 1 },
            new[] { 9999, 500, 10000 }, 30);
        int butce = Tam("argüman", 0, 2, args[1], 0, 500);
        Tekil(pro, args[0], 0);
        int[] sira = Sira(pro);
        int n = pro.Length;
        int[,] dp = new int[n + 1, butce + 1];
        for (int i = 0; i <= n; i++)
            for (int b = 0; b <= butce; b++) dp[i, b] = -1;
        dp[n, 0] = 0;

        for (int i = n - 1; i >= 0; i--)
        {
            int p = sira[i], tutar = pro[p][1], kazanc = pro[p][2];
            for (int b = 0; b <= butce; b++)
            {
                dp[i, b] = dp[i + 1, b];
                if (b >= tutar && dp[i + 1, b - tutar] >= 0)
                {
                    int aday = kazanc + dp[i + 1, b - tutar];
                    if (aday > dp[i, b]) dp[i, b] = aday;
                }
            }
        }
        int harcama = 0, enIyi = 0;
        for (int b = 0; b <= butce; b++)
            if (dp[0, b] > enIyi)
                { harcama = b; enIyi = dp[0, b]; }

        var rapor = new StringBuilder();
        Satir(rapor, "T", harcama, enIyi);
        int kalan = harcama, hedef = enIyi;
        for (int i = 0; i < n; i++)
        {
            int p = sira[i], tutar = pro[p][1], kazanc = pro[p][2];
            if (kalan >= tutar && dp[i + 1, kalan - tutar] >= 0 &&
                kazanc + dp[i + 1, kalan - tutar] == hedef)
            {
                Satir(rapor, "P", pro[p][0]);
                kalan -= tutar; hedef -= kazanc;
            }
        }
        Yaz(args[2], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}
```

```

catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            veri[n][j] = Tam(yol, n + 1, j + 1,
                a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

```



```

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

Örnek çalıştırmalar

Yukarıdaki komutta projeler.txt dosyasının tam içeriği:

```
30;4;7
10;5;10
20;4;7
```

Yeni rapor.txt:

```
T;8;14
P;20
P;30
```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

Aynı girdi ve bütçe 4 için tam rapor:

```
T;4;7
P;20
```

20 ve 30 eşit maliyet/kazanımdadır; küçük kod seçilir. Aynı dosya ile bütçe 0 veya sıfır baytlık dosya ile bütçe 8 için rapor yalnız T;0;0 satırıdır; konsol yine Rapor yazıldı. olur.

Geçersiz örnekte proje dosyası tek 1;0;9 satırıdır; bütçe 8. Rapor açılmaz; konsolun tam çıktısı:

```
Hata: projeler.txt: satır 1, alan 2: Geçersiz tamsayı.
```

Sınır durumları

- Bir projeyi bütçe yetse bile iki kez seçemezsiniz; aynı kodlu kayıt hata olur.
- Aynı kazanım daha düşük harcamayla elde edilebiliyorsa düşük harcama önce gelir; sözlük sırası yalnız bundan sonra uygulanır.
- Erişilemez tam harcama -1'dir; boş kümenin kazanımı 0'dır. Erişilemez duruma kazanım eklenmez.
- Başlangıç kodu sırası veya eşit maliyetli projelerin dosya sırası çıktıyı değiştirmez.
- 30 proje ve bütçe 500 tablo sınırıdır. Daha büyük bütçe veya proje sayısına genel performans sözü verilmez.

Kazanımlar

- Durum, taban ve geçişi tanımlayarak optimumu bütün seçimler üzerinden kurma.
- Optimum sayısal değeri bulmak ile somut çözümü geri kurmayı ayırma.
- Proje maliyeti, kazanımı ve bütçe uygunluğunun neden birlikte doğrulanabileceğini açıklama; sonraki nesneye yönelik derste bu sözleşmeleri modelleme.

Alıştırmalar

1. Bütçesi çok büyük, proje sayısı küçükken geri izlemeyle çözümü karşılaştırın; durum tablosunun hangi sınırda pahalılaştığını açıklayın.
2. Seçilen proje sayısına da bir üst sınır koymak için durumun hangi boyutunu eklemeniz gerektiğini belirleyin.
3. Eşit kazanımda en düşük harcama kuralını kaldırınca geri kurmanın nasıl değişeceğini örnekleyin.

098 Yönlü ulaşım ağında en düşük maliyetli rota

Düzey: İleri Önkoşullar: 041, 047, 078, 089, 090

Kavramlar: Yönlü çizge, Komşuluk matrisi, Dijkstra, İki ölçütlü uzaklık, Rota geri kurma

Problem tanımı

Bir yönlü ağda başlangıçtan hedefe toplam maliyeti en küçük rotayı bulun. Aynı maliyette daha az bağlantıyı; yine eşitlikte düğüm kodları listesinin sayısal sözlük sırasına göre ilkini seçin. Ulaşılamayan hedef normal sonuçtur. Ağı hazır çizge kitaplığı veya öncelik kuyruğuyla çözmeyin; dizilerle Dijkstra algoritmasını kurun.

Girdi ve çıktı

```
dotnet run -- ornek-098/dugumler.txt ornek-098/baglantilar.txt 1 4 ornek-098/rapor.txt
```

Düğüm satırı kod (1..9999, benzersiz, en fazla 20 düğüm); bağlantı satırı kaynakKod;hedefKod;maliyet (kodlar 1..9999, maliyet 1..10000, en fazla 100 bağlantı). İki uç mevcut olmalıdır. Aynı yönlü çift tekrar edemez; düğümün kendisine bağlantısı yasaktır. Ters yöndeki bağlantı ayrı kayıttır. Başlangıç ve hedef üçüncü/dördüncü argümandaki mevcut düğüm kodlarıdır.

Rapor M;toplamMaliyet, K;bağlantıSayısı, Y;kod1;kod2;... olmak üzere üç satırdır. Yol başlangıç ve hedefi içerir. Yol yoksa tek ROTA_YOK satırıdır. Başlangıç hedefse maliyet ve bağlantı sayısı 0, yol tek düğümdür. Sözlük sırası ilk farklı düğüm kodunu sayısal karşılaştırır.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtar belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun

tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Katalog, bağlantılar, benzersiz çiftler ve bütün ilişkileri doğrulayın; yönlü maliyet matrisini kurun.
2. Hedefin uzaklığını ve bağlantı sayısını 0, diğerlerini sonsuz yapın.
3. Kesinleşmemiş düğümlerden uzaklık/bağlantı sayısı en küçük olanı seçin.
4. Seçilen v düğümüne gelen $u \rightarrow v$ bağlantılarıyla u'nun hedefe gidiş çiftini iyileştirin.
5. Ulaşılabilir düğümler bitene kadar devam edin.
6. Başlangıçtan itibaren en iyi maliyet ve bağlantı çiftini koruyan komşuların en küçük kodlusunu seçerek yolu geri kurun.
7. Yol veya ulaşılama sonucunu raporlayın.

Çizge, düğümler ve onları birleştiren bağlantılardan oluşur. maliyet[u,v], $u \rightarrow v$ bağlantısının değeridir; 0 bağlantı yok demektir ve girdi maliyetleri pozitifdir. Dijkstra, geçici uzaklıkların en küçükünü kesinleştirir. Pozitif bağlantılarla daha sonra daha pahalı bir düğüm üzerinden bu kesin değeri düşürmek mümkün değildir.

Bu çözüm hedefe doğru uzaklıkları bulur: hedefte başlayıp gelen bağlantıları inceler. Bu, yönleri ters çevrilmiş ağda Dijkstra uygulamaktır; asıl yolun yönü değiştirilmez. Eşit maliyette en az bağlantı da saklanır. Sonsuz değere toplama yapılmaz. En iyi yol pozitifdir ve döngü içermez; en fazla 19 bağlantı, 190000 maliyet vardır.

Rota kurulurken yalnız optimum tamamlaması olan komşular kabul edilir. En küçük sonraki kod, sözlük sırasının ilk farkını belirler. Her adımda kalan bağlantı sayısı 1 azalır; yol sonlanır. Uzaklık hesabı ve geri kurma, doğrulama sonrasında $O(n^2)$ süre, matris $O(n^2)$ bellek kullanır.

1→2:2, 1→3:2, 2→4:3, 3→4:3 ve 1→4:9 için:

Hedefe doğru adım	Kesin düğüm	Yeni uzaklık / bağlantı
1	4	2:(3,1), 3:(3,1), 1:(9,1)
2	2	1:(5,2)
3	3	1 aynı kalır
4	1	Hesap biter

1'den geri kurmada 2 ve 3 optimumu korur; küçük kod 2 seçilir. Matris dizinleri kod değildir; Bu1 bu dönüşümü yapar, EnKısa yalnız bellek sonuçlarını döndürür.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 5) throw new FormatException("5 argüman gerekli.");
    checked
    {
        int[][] dug = Oku(args[0], new[] { 1 }, new[] { 9999 }, 20);
        int[][] bag = Oku(args[1], new[] { 1, 1, 1 },
            new[] { 9999, 9999, 10000 });
        int basKod = Tam("argüman", 0, 3, args[2], 1, 9999);
        int sonKod = Tam("argüman", 0, 4, args[3], 1, 9999);
        Tekil(dug, args[0], 0); Tekil(bag, args[1], 0, 1);
        int n = dug.Length;
        int[,] maliyet = new int[n, n];
        for (int i = 0; i < bag.Length; i++)
        {
            int u = Bul(dug, bag[i][0]), v = Bul(dug, bag[i][1]);
            if (u < 0 || v < 0)
                Hata(args[1], i + 1, 0, "Düğüm bulunamadı.");
            if (u == v) Hata(args[1], i + 1, 0, "Öz bağlantı yasak.");
            maliyet[u, v] = bag[i][2];
        }
        int bas = Bul(dug, basKod), son = Bul(dug, sonKod);
        if (bas < 0 || son < 0)
            throw new FormatException("Başlangıç veya hedef bulunamadı.");

        EnKisa(maliyet, dug, son, out long[] uzak, out int[] adim);
        var rapor = new StringBuilder();
        if (uzak[bas] == long.MaxValue) rapor.Append("ROTA_YOK\n");
        else
        {
            Satir(rapor, "M", uzak[bas]);
            Satir(rapor, "K", adim[bas]);
            int an = bas;
            rapor.Append("Y;").Append(dug[an][0].ToString(
                CultureInfo.InvariantCulture));
            while (an != son)
            {
                int sec = -1;
                for (int v = 0; v < n; v++)
                    if (maliyet[an, v] > 0 && uzak[v] != long.MaxValue &&
                        maliyet[an, v] + uzak[v] == uzak[an] &&
                        1 + adim[v] == adim[an] &&
                        (sec < 0 || dug[v][0] < dug[sec][0])) sec = v;
                if (sec < 0) throw new FormatException("Rota kurulamadı.");
                an = sec;
                rapor.Append(';').Append(dug[an][0].ToString(
                    CultureInfo.InvariantCulture));
            }
            rapor.Append('\n');
        }
        Yaz(args[4], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}
```

```

catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taşttı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            veri[n][j] = Tam(yol, n + 1, j + 1,
                a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

```

```

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sirasira = new int[veri.Length];
    for (int i = 0; i < sirasira.Length; i++) sirasira[i] = i;
    for (int i = 1; i < sirasira.Length; i++)
    {
        int p = sirasira[i], j = i - 1;
        while (j >= 0 && veri[sirasira[j]][0] > veri[p][0])
        { sirasira[j + 1] = sirasira[j]; j--; }
        sirasira[j + 1] = p;
    }
    return sirasira;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

```

```

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

static void EnKisa(int[,] maliyet, int[][] dug, int hedef,
    out long[] uzak, out int[] adim)
{
    int n = dug.Length;
    uzak = new long[n]; adim = new int[n];
    bool[] kesin = new bool[n];
    for (int i = 0; i < n; i++)
    { uzak[i] = long.MaxValue; adim[i] = int.MaxValue; }
    uzak[hedef] = 0; adim[hedef] = 0;
    int[] sirasira = Sira(dug);

    for (int tur = 0; tur < n; tur++)
    {
        int v = -1;
        foreach (int i in sirasira)
            if (!kesin[i] && uzak[i] != long.MaxValue &&
                (v < 0 || uzak[i] < uzak[v] ||
                 (uzak[i] == uzak[v] && adim[i] < adim[v]))) v = i;
        if (v < 0) break;
        kesin[v] = true;
        for (int u = 0; u < n; u++)
        {
            if (kesin[u] || maliyet[u, v] == 0) continue;
            long yeni = uzak[v] + maliyet[u, v];
            int uzunluk = adim[v] + 1;
            if (yeni < uzak[u] ||
                (yeni == uzak[u] && uzunluk < adim[u]))
            { uzak[u] = yeni; adim[u] = uzunluk; }
        }
    }
}

```

Örnek çalıştırmalar

Yukarıdaki komutta dugumler.txt tam içeriği:

```

4
1
3
2

```

baglantilar.txt tam içeriği:

```

1;2;2
1;3;2
2;4;3
3;4;3
1;4;9

```

Yeni rapor.txt:

```

M;5
K;2
Y;1;2;4

```


Konsolun tam çıktısı:

Rapor yazıldı.

Son bağlantıyı 1;4;5 yapınca rapor M;5, K;1, Y;1;4 satırlarıdır. Aynı katalogda başlangıç 4, hedef 1 ise tek ROTA_YOK; başlangıç/hedef 4 ise M;0, K;0, Y;4 satırlarıdır. Bu başarılı durumlarda konsol yine Rapor yazıldı. olur.

Bağlantı dosyası tek 1;4;0 satırına değiştirilirse rapor açılmaz; konsolun tam çıktısı:

Hata: baglantilar.txt: satır 1, alan 3: Geçersiz tamsayı.

Sınır durumları

- Dosyada bulunmayan başlangıç/hedef hatadır; mevcut ama bağlantısız hedef ROTA_YOK sonucudur.
- Döngülü ağ geçerlidir; pozitif maliyet nedeniyle döngü eklemek en iyi yolu iyileştiremez.
- Bir yöndeki yol ters yönde var sayılmaz. Aynı çift tekrar ederse daha ucuzunu sessizce seçmek yerine hata verilir.
- Bağlantı sayısı eşitliği maliyet eşitliğinden sonra, sözlük sırası ikisinden sonra uygulanır.
- Negatif veya sıfır maliyet kabul edilmez; bu çözümün önkoşuludur.

Kazanımlar

- Ağ ilişkilerini matrisle temsil etme; geçici ve kesin uzaklıkları ayırma.
- Optimumu koruyan seçimlerle deterministik yolu geri kurma.
- Düğüm kimliği, yönlü bağlantı ve maliyet doğrulamasının hangi veri/davranış birlikteliklerine karşılık geldiğini açıklama.

Alıştırmalar

1. İki yönde aynı maliyetli bağlantıyı dosyada iki kayıtla gösterin; yönlülük kuralını açıklayın.
2. Her düğümün hedefe uzaklığını ayrıca raporlayın; ulaşılamayan düğümleri sayısal sonsuz yazmadan gösterin.
3. Sıfır maliyet kabul edilseydi sonlanma ve rota geri kurma gerekçelerinde hangi değişikliklerin gerekeceğini tartışın.

099 Bağımlı işlerde kritik yol ve zaman payı

Düzey: İleri Önkoşullar: 078, 090, 098

Kavramlar: Bağımlılık çizgesi, Topolojik sıra, Kritik yol, İleri ve geri geçiş, Zaman payı

Problem tanımı

İş süreleri ve öncelik ilişkilerinden projenin en erken bitişini bulun. Her işin en erken/en geç başlama ve bitişini, zaman payını ve kritik olup olmadığını raporlayın. Bir iş bütün öncülleri bitince başlayabilir. Hazır işler için yeterli kaynak vardır; aynı anda çalıştırılabilecek iş sayısı sınırlandırılmaz. Döngü varsa plan üretmeyin.

Girdi ve çıktı

```
dotnet run -- ornek-099/isler.txt ornek-099/bagimliliklar.txt ornek-099/rapor.txt
```

İş satırı kod;süreDakika: kod 1..9999 benzersiz, süre 1..1000, en fazla 30 iş. Bağımlılık satırı öncekiKod;sonrakiKod: mevcut kodlar, en fazla 100 benzersiz yönlü çift. Öz bağımlılık yasaktır. Bir iş birden çok öncüle sahip olabilir; bu bir ağaç olmak zorunda değildir.

Proje 0. dakikada başlar. Rapor önce T;projeBitişi, ardından kod artan kod;erkenBaşla;erkenBitir;geçBaşla;geçBitir;pay;durum satırlarıdır. Durum sıfır payda KRITİK, diğerinde PAYLI olur. Boş proje için yalnız T;0 yazılır. Birden fazla son iş ve bağımsız bileşen olabilir; hepsi aynı proje bitişine göre değerlendirilir.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Görelî yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtar belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun tam çıktısı Rapor yazıldı. satırdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O

hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taştı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. İşleri ve bağımlılıkları doğrulayın; komşuluk matrisi ile her işin giriş derecesini kurun.
2. Henüz alınmamış giriş derecesi 0 olan en küçük kodlu işi topolojik sıraya ekleyin; ardılarının derecesini azaltın.
3. İşler kalmışken böyle bir aday yoksa döngü hatası verin.
4. Topolojik sırada erken bitiş erken başlangıç + süre yapın; ardıların erken başlangıcını bu bitişle yükseltin.
5. Proje bitişini erken bitişlerin en büyüğü yapın.
6. Ters topolojik sırada geç bitiş ardıların geç başlangıçlarının en küçüğü; ardıl yoksa proje bitiş yapın. Süreyi çıkarıp geç başlangıcı bulun.
7. Geç başlangıç - erken başlangıç farkını zaman payı olarak raporlayın.

Topolojik sıra her öncülü ardılından önce yerleştirir; döngüsüz yönlü çizgede mümkündür. Giriş derecesi, henüz alınmamış öncül sayısıdır. Bir döngüde hiçbir döngü işi derece 0'a inemez. Tüm bileşenler tarandığından bağımsız bir bileşendeki döngü de yakalanır.

Erken başlangıç öncül bitişlerinin en büyüğünü bekler; geri geçişte ardılları geciktirmemek için en küçük geç başlangıç sınırı kullanılır. Pay kadar gecikme, diğer işler uygun planlanırsa proje bitişini uzatmaz. Birden çok işin paylarını aynı anda bağımsız kullanabilecekleri varsayılmaz. Kritik işler sıfır paylıdır; birden çok kritik yol olabilir. Bu, 098'deki en kısa yol hedefinden farklı olarak süreyi belirleyen en uzun bağımlılık zincirini inceler.

Süre toplamı en fazla 30000 dakikadır. Doğrulama sonrası matris taramaları $O(n^2)$ süre ve $O(n^2)$ bellek kullanır. topo yürütme sırasıdır, kod sıralaması yalnız deterministik aday seçimi ve çıktıda kullanılır.

1:3, 2:2, 3:4, 4:1 süreleri ve 1→3, 2→3, 3→4 için:

İş	Erken başla/bitir	Geç başla/bitir	Pay
1	0 / 3	0 / 3	0
2	0 / 2	1 / 3	1
3	3 / 7	3 / 7	0
4	7 / 8	7 / 8	0

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 3) throw new FormatException("3 argüman gerekli.");
    checked
    {
        int[][] isler = Oku(args[0], new[] { 1, 1 },
            new[] { 9999, 1000 }, 30);
        int[][] bag = Oku(args[1], new[] { 1, 1 },
            new[] { 9999, 9999 });
        Tekil(isler, args[0], 0); Tekil(bag, args[1], 0, 1);
        int n = isler.Length;
        bool[,] once = new bool[n, n];
        int[] derece = new int[n];
        for (int i = 0; i < bag.Length; i++)
        {
            int u = Bul(isler, bag[i][0]), v = Bul(isler, bag[i][1]);
            if (u < 0 || v < 0)
                Hata(args[1], i + 1, 0, "İş bulunamadı.");
            if (u == v) Hata(args[1], i + 1, 0, "Öz bağımlılık yasak.");
            once[u, v] = true; derece[v]++;
        }

        int[] kodSirasi = Sira(isler), topo = new int[n];
        bool[] alindi = new bool[n];
        for (int i = 0; i < n; i++)
        {
            int sec = -1;
            foreach (int p in kodSirasi)
                if (!alindi[p] && derece[p] == 0) { sec = p; break; }
            if (sec < 0)
                throw new FormatException("İş bağımlılıkları döngü içeriyor.");
            topo[i] = sec; alindi[sec] = true;
            for (int v = 0; v < n; v++) if (once[sec, v]) derece[v]--;
        }

        int[] eb = new int[n], es = new int[n];
        int[] gb = new int[n], gs = new int[n];
        int bitis = 0;
        foreach (int u in topo)
        {
            es[u] = eb[u] + isler[u][1];
            if (es[u] > bitis) bitis = es[u];
            for (int v = 0; v < n; v++)
                if (once[u, v] && eb[v] < es[u]) eb[v] = es[u];
        }
        for (int i = n - 1; i >= 0; i--)
        {
            int u = topo[i];
            gs[u] = bitis;
            for (int v = 0; v < n; v++)
                if (once[u, v] && gb[v] < gs[u]) gs[u] = gb[v];
            gb[u] = gs[u] - isler[u][1];
        }
    }
}
```

```

        var rapor = new StringBuilder();
        Satir(rapor, "T", bitis);
        foreach (int u in kodSirasi)
        {
            int pay = gb[u] - eb[u];
            Satir(rapor, isler[u][0], eb[u], es[u], gb[u], gs[u], pay,
                pay == 0 ? "KRITIK" : "PAYLI");
        }
        Yaz(args[2], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}

catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            veri[n][j] = Tam(yol, n + 1, j + 1,
                a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

```

```

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satir " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sira = new int[veri.Length];
    for (int i = 0; i < sira.Length; i++) sira[i] = i;
    for (int i = 1; i < sira.Length; i++)
    {
        int p = sira[i], j = i - 1;
        while (j >= 0 && veri[sira[j]][0] > veri[p][0])
        { sira[j + 1] = sira[j]; j--; }
        sira[j + 1] = p;
    }
    return sira;
}

```

```

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

Örnek çalıştırmalar

Yukarıdaki komutta isler.txt tam içeriği:

```

4;1
2;2
3;4
1;3

```

bagimliliklar.txt tam içeriği:

```

1;3
2;3
3;4

```

Yeni rapor.txt:

```

T;8
1;0;3;0;3;0;KRITIK
2;0;2;1;3;1;PAYLI
3;3;7;3;7;0;KRITIK
4;7;8;7;8;0;KRITIK

```

Konsolun tam çıktısı:

```
Rapor yazıldı.
```

İki giriş dosyası sıfır baytken yeni rapor yalnız T;0 satırıdır; konsol yine Rapor yazıldı. olur.

Normal örneğin bağımlılıklarına 4;1 satırı eklenirse rapor açılmaz; konsolun tam çıktısı:

```
Hata: İş bağımlılıkları döngü içeriyor.
```

Sınır durumları

- Öncülsüz işler 0'da başlayabilir; ardılsız işler ortak proje bitişine kadar pay taşıyabilir.
- Birleşen bağımlılıklarda bütün öncüller beklenir; bir önceki işin bitişini kullanmak yeterli değildir.
- Aynı çift iki kez yazılırsa giriş derecesi iki kat sayılmadan hata olur.
- Kısa bağımsız bileşenin işi otomatik kritik sayılmaz; proje bitişine göre hesaplanır.
- Negatif/sıfır süre kabul edilmez. Kaynak veya çalışan sayısı kısıtı eklenirse bu çözüm yeni koşul için yeterli değildir.

Kazanımlar

- Topolojik sıranın hesaplama bağımlılıklarını nasıl sağladığını açıklama.
- İleri en büyük, geri en küçük sınırlarla zaman aralıklarını kurma.
- İş kimliği, süre ve bağımlılık davranışlarının sonraki derste hangi nesneler ve ilişkilerle modellenebileceğini belirleme.

Alıştırmalar

1. Birden fazla kritik yol üreten küçük bir ağ kurun; sıfır paylı bütün işleri gösterin.
2. İş 2'nin payını kullanıp 1'de başlatın; proje bitişinin değişmediğini doğrulayın.
3. Tek çalışan şartı eklendiğinde bu planın neden uygulanamayabileceğini örnekleyin.

100 Uygunluk ve maliyetle en iyi görev ataması

Düzey: İleri Önkoşullar: 074, 079, 095, 097, 098

Kavramlar: Bire bir atama, Uygunluk matrisi, Geri izleme, İyimsen alt sınır, Küresel optimum

Problem tanımı

Eşit sayıda çalışanı ve görevi, her çalışan tam bir görev ve her görev tam bir çalışan alacak şekilde eşleştirin. Yalnız izin verilen çiftleri kullanarak toplam maliyeti en küçük yapan tam atamayı bulun. Eşit maliyetle, çalışan kodu sırasındaki görev kodları listesinin sayısal sözlük sırasına göre ilkinin seçin. Her çalışan için yerel olarak en ucuz görevi seçmek küresel en iyi atamayı garanti etmez.

Girdi ve çıktı

dotnet run -- ornek-100/calisanlar.txt ornek-100/gorevler.txt ornek-100/uygunluk.txt ornek-100/rapor.txt

Çalışan ve görev dosyaları ayrı kod kataloglarıdır: 1..9999 benzersiz kodlar, eşit ve 1..8 kayıt. Uygunluk satırı çalışanKod;görevKod;maliyet biçimindedir: mevcut kodlar, maliyet 0..1000000, çift benzersiz, en fazla 64 satır. Listelenmeyen çift yasaktır; bu bozuk veri değildir. Sıfır maliyet geçerlidir.

Rapor önce T;toplamMaliyet, ardından çalışan kodu artan A;çalışanKod;görevKod;çiftMaliyeti satırlarıdır. Geçerli veriden tam eşleştirme kurulamıyorsa tek ATAMA_YOK satırı yazılır. Katalog boyutu hatası veya eksik kod, bu normal sonuçtan ayrıdır. Sözlük sırası ilk farklı görev kodunu sayısal olarak karşılaştırır; bütün aday listeleri aynı boydadır.

Her çözüm bağımsız .NET 10 konsol projesinin Program.cs dosyasında çalışır. Komut satırı argümanları aşağıdaki sıradadır; konsol girdi istemi yoktur. Göreli yollar çalışma klasörüne göredir. Örnek dosyalarını kendinize ait boş bir klasörde hazırlayın. Son argüman yeni rapor yoludur; üst klasörü önceden oluşturun.

Girişler başlıksız UTF-8 metindir; isteğe bağlı ilk BOM, LF ve CRLF kabul edilir. Ayırıcı noktalı virgüldür; tırnak, çevre boşluğu, yorum veya boş satır kabul edilmez. Dosya başına 32768 bayt, varsayılan 100 fiziksel satır, satır başına 120 UTF-16 kod birimi sınırı vardır; daha küçük problem sınırları ayrıca belirtilir. ReadLine sonucunda null EOF, boş metin gerçek boş satırdır. Son tek satır sonu ek kayıt oluşturmaz. Sıfır baytlık dosya sıfır kayıttır.

Tamsayılar yalnız ASCII rakamlarla yazılır; baştaki sıfırlar kabul edilir ve aynı sayısal anahtar belirtir. İşaret, binlik ayırıcı, üs ve ondalık kabul edilmez. Ayırıştırma ve çıktı CultureInfo.InvariantCulture kullanır. Tarih alanı varsa tam yyyy-MM-dd, 2000–2099 aralığında gerçek takvim tarihidir; kod DateOnly ile doğrulayıp gün numarasına dönüştürür. Ondalıklı sonuçlarda nokta ve problemde belirtilen basamak sayısı kullanılır.

İlk hata işlemi durdurur. Alan hatası Hata: dosya: satır S, alan A: açıklama biçimindedir; satır ve alan 1 tabanlıdır, kayıt/genel satır hatasında alan 0 kullanılır. Bütün dosyalar argüman sırasıyla okunur ve ayrıştırılır; ardından anahtar ve ilişki denetimleri çözümde gösterilen sırayla yapılır. Daha sonraki dosyanın biçim hatası önceki dosyanın ilişki hatasından önce bildirilebilir. Eksik dosya veya I/O sorunu Hata: Dosya işlemi başarısız.; erişim engeli Hata: Dosyaya erişim izni yok.; geçersiz yol veya UTF-8 Hata: Yol veya UTF-8 kodlaması geçersiz.; desteklenmeyen yol Hata: Yol biçimi desteklenmiyor. Üretir. using hata sırasında da kaynakları kapatır.

Rapor yalnız bütün doğrulama ve hesaplamalar başarılıysa açılır. FileMode.CreateNew mevcut dosyayı, girişle aynı yol olsa bile, ezmez; I/O hatası bildirir. Giriş dosyaları yalnız okunur. Çıktı UTF-8 BOM olmadan LF ile yazılır; her kayıt son LF ile biter. Boş rapor sıfır bayttır. Başarıda konsolun

tam çıktısı Rapor yazıldı. satırıdır; bu mesaj akış kapandıktan sonra yazılır. Yazım sırasında I/O hatası yeni dosyada kısmi içerik bırakabilir; başarı bildirilmez, yeni dosyayı denetleyip farklı adla tekrar deneyin. Atomik yayın garantisi yoktur. Giriş hatasında rapor hiç oluşturulmaz. Hesaplar checked bloğundadır; beklenmeyen taşmada Hata: Hesaplama taşıtı. yazılır ve rapor açılmaz. Aşağıdaki sınırlar geçerli girdilerde bu taşmayı önler.

Algoritma

1. Üç dosyayı, katalog boyutlarını, benzersiz anahtarları ve ilişkileri doğrulayın. Yasak çiftleri -1 maliyetle saklayın.
2. Çalışan ve görev dizinlerini kod artan sıralayın.
3. Sıradaki çalışan için kullanılmamış izinli görevleri kod sırasıyla deneyin.
4. Her çağrıda kalan her çalışanın kullanılmamış izinli görevlerdeki en düşük maliyetini mevcut toplama ekleyerek alt sınır kurun.
5. Kalan bir çalışana aday yoksa veya alt sınır mevcut en iyiye eşit/büyükse dalı bırakın.
6. Tam atamada daha düşük toplamı ve seçimi saklayın; geri dönerken görevi serbest bırakın.
7. En iyi tam atamayı veya yok sonucunu raporlayın.

Geri izleme, bir seçimi denedikten sonra ortak durumu eski haline getirerek diğer seçeneğe geçer. kullanıldı[g], aynı görevin iki çalışana verilmesini engeller; dönüşte false yapmak sonraki dallar için gereklidir. Çağrı i+1'e ilerler ve derinlik en fazla n+1'dir.

Alt sınır gerçek bir atama olmayabilir: iki çalışan için aynı ucuz görevi ayrı ayrı sayabilir. Bu iyimserlik sınırı gerçek tamamlamadan büyük yapmaz; dolayısıyla daha iyi olamayacak dallar güvenle budanır. Kod artan derinlik öncelikli tarama tam atamaları sözlük sırasında ziyaret eder. Eşit en iyi toplam güncellenmez; eşit alt sınırdaki sonraki dallar da ilk bulunan eşit optimumu iyileştiremez.

Budamasız en fazla n! tam atama vardır; 8! = 40320. Bir alt sınır hesabı $O(n^2)$ 'dir. Boyut üst sınırı bu öğretim örneğini yönetilebilir tutar; büyük atama problemlerine genel verimlilik iddiası kurulmaz. Toplam en fazla 8000000'dur; hesap ve en iyi işareti long tutulur.

İki çalışan ve iki görev için:

Çalışan	Görev 10 maliyeti	Görev 20 maliyeti
1	1	2
2	1	100

1→10 ardından 2→20 toplam 101 verir. 1→20, 2→10 toplam 3'tür. İlk çağrının alt sınırı 1+1=2 aynı görevi iki kez düşünür; gerçek çözüm diye raporlanmaz. Ata sadece bellek durumlarını değiştirir; dış dosya en iyi sonuç bulununca oluşturulur.

C# çözümü

```
using System;
using System.IO;
using System.Text;
using System.Globalization;

try
{
    if (args.Length != 4) throw new FormatException("4 argüman gerekli.");
    checked
    {
        int[][] cal = Oku(args[0], new[] { 1 }, new[] { 9999 }, 8);
        int[][] gor = Oku(args[1], new[] { 1 }, new[] { 9999 }, 8);
        int[][] uy = Oku(args[2], new[] { 1, 1, 0 },
            new[] { 9999, 9999, 1000000 }, 64);
        Tekil(cal, args[0], 0); Tekil(gor, args[1], 0);
        Tekil(uy, args[2], 0, 1);
        int n = cal.Length;
        if (n == 0 || gor.Length != n)
            throw new FormatException("Eşit ve 1..8 katalog boyutu gerekli.");
        int[,] maliyet = new int[n, n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) maliyet[i, j] = -1;
        for (int i = 0; i < uy.Length; i++)
        {
            int c = Bul(cal, uy[i][0]), g = Bul(gor, uy[i][1]);
            if (c < 0 || g < 0)
                Hata(args[2], i + 1, 0, "Çalışan veya görev bulunamadı.");
            maliyet[c, g] = uy[i][2];
        }

        int[] cs = Sira(cal), gs = Sira(gor);
        int[] an = new int[n], en = new int[n];
        long enIyi = long.MaxValue;
        Ata(maliyet, cs, gs, new bool[n], an, en, 0, 0, ref enIyi);
        var rapor = new StringBuilder();
        if (enIyi == long.MaxValue) rapor.Append("ATAMA_YOK\n");
        else
        {
            Satir(rapor, "T", enIyi);
            for (int i = 0; i < n; i++)
                Satir(rapor, "A", cal[cs[i]][0], gor[en[i]][0],
                    maliyet[cs[i], en[i]]);
        }
        Yaz(args[3], rapor.ToString());
    }
    Console.WriteLine("Rapor yazıldı.");
}

catch (FormatException e) { Console.WriteLine("Hata: " + e.Message); }
catch (IOException) { Console.WriteLine("Hata: Dosya işlemi başarısız."); }
catch (UnauthorizedAccessException)
{ Console.WriteLine("Hata: Dosyaya erişim izni yok."); }
catch (ArgumentException)
{ Console.WriteLine("Hata: Yol veya UTF-8 kodlaması geçersiz."); }
catch (NotSupportedException)
{ Console.WriteLine("Hata: Yol biçimi desteklenmiyor."); }
catch (OverflowException) { Console.WriteLine("Hata: Hesaplama taştı."); }
```

```

static int[][] Oku(string yol, int[] alt, int[] ust,
    int kapasite = 100)
{
    int[][] veri = new int[kapasite][];
    int n = 0;
    using var akis = new FileStream(yol, FileMode.Open,
        FileAccess.Read, FileShare.Read);
    if (akis.Length > 32768)
        throw new FormatException(Path.GetFileName(yol) +
            ": Dosya 32768 baytı aşıyor.");
    using var okuyucu = new StreamReader(akis,
        new UTF8Encoding(false, true), false);
    string? metin;
    while ((metin = okuyucu.ReadLine()) is not null)
    {
        if (n == 0 && metin.StartsWith('\uFEFF'))
            metin = metin.Substring(1);
        if (n == kapasite) Hata(yol, n + 1, 0, "Kapasite aşıldı.");
        if (metin.Length > 120)
            Hata(yol, n + 1, 0, "Satır 120 karakteri aşıyor.");
        string[] a = metin.Split(';');
        if (a.Length != alt.Length)
            Hata(yol, n + 1, 0, "Alan sayısı yanlış.");
        veri[n] = new int[a.Length];
        for (int j = 0; j < a.Length; j++)
        {
            veri[n][j] = Tam(yol, n + 1, j + 1,
                a[j], alt[j], ust[j]);
        }
        n++;
    }
    Array.Resize(ref veri, n);
    return veri;
}

static void Hata(string yol, int satir, int alan, string mesaj)
{
    throw new FormatException(Path.GetFileName(yol) +
        ": satır " + satir.ToString(CultureInfo.InvariantCulture) +
        ", alan " + alan.ToString(CultureInfo.InvariantCulture) +
        ": " + mesaj);
}

static int Tam(string yol, int satir, int alan, string metin,
    int alt, int ust)
{
    if (metin.Length == 0) Hata(yol, satir, alan, "Geçersiz tamsayı.");
    for (int i = 0; i < metin.Length; i++)
        if (metin[i] < '0' || metin[i] > '9')
            Hata(yol, satir, alan, "Geçersiz tamsayı.");
    if (!int.TryParse(metin, NumberStyles.None,
        CultureInfo.InvariantCulture, out int deger) ||
        deger < alt || deger > ust)
        Hata(yol, satir, alan, "Geçersiz tamsayı.");
    return deger;
}

```

```

static void Tekil(int[][] veri, string yol, params int[] alanlar)
{
    for (int i = 0; i < veri.Length; i++)
        for (int j = 0; j < i; j++)
        {
            bool ayni = true;
            for (int k = 0; k < alanlar.Length; k++)
                if (veri[i][alanlar[k]] != veri[j][alanlar[k]])
                    ayni = false;
            if (ayni) Hata(yol, i + 1, alanlar[0] + 1,
                "Yinelenen anahtar.");
        }
}

static int Bul(int[][] veri, int kod)
{
    for (int i = 0; i < veri.Length; i++)
        if (veri[i][0] == kod) return i;
    return -1;
}

static int[] Sira(int[][] veri)
{
    int[] sira = new int[veri.Length];
    for (int i = 0; i < sira.Length; i++) sira[i] = i;
    for (int i = 1; i < sira.Length; i++)
    {
        int p = sira[i], j = i - 1;
        while (j >= 0 && veri[sira[j]][0] > veri[p][0])
            { sira[j + 1] = sira[j]; j--; }
        sira[j + 1] = p;
    }
    return sira;
}

static void Satir(StringBuilder rapor, params object[] alanlar)
{
    for (int i = 0; i < alanlar.Length; i++)
    {
        if (i != 0) rapor.Append(';');
        rapor.Append(Convert.ToString(alanlar[i],
            CultureInfo.InvariantCulture));
    }
    rapor.Append('\n');
}

static void Yaz(string yol, string metin)
{
    using var akis = new FileStream(yol, FileMode.CreateNew,
        FileAccess.Write, FileShare.None);
    using var yazici = new StreamWriter(akis, new UTF8Encoding(false));
    yazici.Write(metin);
}

```

```

static void Ata(int[,] maliyet, int[] cs, int[] gs, bool[] kullanildi,
    int[] an, int[] en, int i, long toplam, ref long enIyi)
{
    int n = cs.Length;
    if (i == n)
    {
        if (toplam < enIyi)
        {
            enIyi = toplam;
            for (int k = 0; k < n; k++) en[k] = an[k];
        }
        return;
    }

    long altSinir = toplam;
    for (int k = i; k < n; k++)
    {
        int ucuz = int.MaxValue;
        foreach (int g in gs)
            if (!kullanildi[g] && maliyet[cs[k], g] >= 0 &&
                maliyet[cs[k], g] < ucuz) ucuz = maliyet[cs[k], g];
        if (ucuz == int.MaxValue) return;
        altSinir += ucuz;
    }
    if (altSinir >= enIyi) return;

    foreach (int g in gs)
    {
        if (kullanildi[g] || maliyet[cs[i], g] < 0) continue;
        kullanildi[g] = true; an[i] = g;
        Ata(maliyet, cs, gs, kullanildi, an, en, i + 1,
            toplam + maliyet[cs[i], g], ref enIyi);
        kullanildi[g] = false;
    }
}

```

Örnek çalıştırmalar

Yukarıdaki komutta calisanlar.txt tam içeriği:

```

2
1

```

gorevler.txt tam içeriği:

```

20
10

```

uygunluk.txt tam içeriği:

```

1;10;1
1;20;2
2;10;1
2;20;100

```

Yeni rapor.txt:

```

T;3
A;1;20;2
A;2;10;1

```

Konsolun tam çıktısı:

Rapor yazıldı.

Aynı kataloglarla dört maliyeti de 0 yapınca rapor T;0, A;1;10;0, A;2;20;0 satırlarıdır. Yalnız 1;10;1 ve 2;10;1 uygunluk satırları varsa rapor tek ATAMA_YOK satırıdır. Her iki durumda konsol yine Rapor yazıldı. olur.

Normal örneğe 3;10;1 uygunluk satırı eklenirse rapor açılmaz; tam konsol çıktısı:

Hata: uygunluk.txt: satır 5, alan 0: Çalışan veya görev bulunamadı.

Sınır durumları

- Her çalışana en az bir aday bulunması tam eşleştirme varlığını garanti etmez; adaylar aynı görev olabilir.
- Sıfır maliyetli çift iznlidir; -1 yalnız bellek içindeki yasak işaretidir ve dosyada kabul edilmez.
- Uygunluk dosyası boşken geçerli kataloglarla ATAMA_YOK yazılır; boş veya farklı boylu kataloglar hata olur.
- Eşit optimum, dosya sırasından bağımsız çalışan ve görev kodu sırasıyla seçilir.
- Bir görev dönüşte serbest bırakılmazsa bazı geçerli atamalar kaybolur; bu durum özel bir testle yakalanmalıdır.

Kazanımlar

- Yerel seçim ile küresel optimumu ayırma; kısıtı her dalda koruma.
- İyimser alt sınırı gerçek sonuçtan ayırıp budamanın güvenliğini açıklama.
- Çalışan/görev kimlikleri, uygunluk, maliyet ve atama davranışlarının nesneye yönelik modelde nasıl ilişkilenebileceğini belirleme.

Alıştırmalar

1. Budamayı kaldırıp aynı en iyi sonucu doğrulayın; çağrı sayısı farkını ölçün.
2. Eşit optimumların tamamını saymak istendiğinde $\geq$ budaması ve eşitlik güncellemesinin nasıl değişeceğini açıklayın.
3. Bu problemdeki Çalışan, Görev ve Atama adaylarının verilerini ve doğrulama davranışlarını ayrı ayrı listeleyin; prosedürel çözümde tekrar eden ilişkileri gösterin.

C# ile Prosedürel Programlama

Problemi anla. Algoritmayı kur. Çözümü geliştir.

Programlama, bir problemi açık ve tutarlı adımlarla çözebilme becerisidir. Bu çalışma kitabı, C# ile çözülen 100 problem üzerinden algoritmik düşünmeyi ve temel programlama becerilerini geliştirir.

Her problem; tanımı, girdi ve çıktı sözleşmesi, algoritması, tam C# çözümü, örnek çalıştırmaları, sınır durumları ve alıştırmalarıyla sunulur. Kolaydan zora ilerleyen yapı, nesneye yönelik programlamaya sağlam bir temel oluşturur.

Kitapta neler var?

- Temel işlemler, karar yapıları ve döngüler
- Diziler, arama, sıralama ve metin işleme
- Metotlar, özyineleme ve dosya işlemleri
- Dinamik programlama, çizgeler ve optimizasyon

100 çözümlü problem • Adım adım algoritmalar • Uygulama ve alıştırmalar

Prof. Dr. Zafer CÖMERT



Samsun Üniversitesi
Bilgisayar ve Bilişim Bilimleri